

The Geometry of Motion

Volume 0: The Mathematical Primer

A Foundational Guide to Kinematics, Configuration, and State
Space

Preface

Before we can begin to analyze the geometry of moving physical bodies or command them to navigate the world via control theory, we must establish a common language. The physics of movement happens in the real world, but our understanding of it happens entirely within mathematical spaces.

This primer, **Volume 0** of *The Geometry of Motion*, is designed to bridge the gap between basic calculus/algebra and the rigorous differential geometry required for modern nonlinear control theory. By intuitively introducing State Space, Configuration Manifolds, $SO(3)$ Rotations, and Screw Theory, this text ensures that the "lay user" possesses the foundational armor needed to attack the subsequent volumes on Tangent Spaces (Volume I) and Trajectory Control (Volume II).

The philosophy of this primer maintains our overarching theme: mathematics should be driven by the physical reality of motion, not abstract algebra for its own sake.

Contents

Preface	i
Nomenclature	xiii
1 A Primer on Linear Algebra	1
1.1 Historical Context	1
1.2 Vectors, Matrices, and Vector Spaces	2
1.2.1 Vectors	2
1.2.2 Matrices	3
1.2.3 The Axioms of a Vector Space	3
1.2.4 Linear Independence and Span	4
1.2.5 Basis and Dimension	5
1.2.6 Linear Transformations	6
1.2.7 Rank, Null Space, and the Rank-Nullity Theorem	6
1.3 The Dot Product, Cross Product, and Inner Product Spaces	8
1.3.1 The Dot Product (Inner Product)	8
1.3.2 The Cross Product and the Skew-Symmetric Matrix	9
1.4 Eigenvalues, Eigenvectors, and Quadratic Forms	11
1.4.1 The Eigenvalue Equation	11
1.4.2 Physical Interpretation of Eigenvalues	12
1.4.3 The Spectral Theorem for Symmetric Matrices	13
1.4.4 Quadratic Forms and Positive Definiteness	14
1.5 The Singular Value Decomposition (SVD)	16
1.5.1 Derivation of the SVD from the Eigendecomposition	16
1.5.2 Geometric Interpretation of the SVD	17
1.5.3 The Condition Number	17
1.5.4 Truncated SVD and Low-Rank Approximation	18
1.6 Dyads, Dyadics, and Tensor Products	18
1.6.1 What Is a Dyad?	19
1.6.2 From Dyads to Dyadics	20
1.6.3 Physical Meaning in Mechanics	20
1.6.4 The Coordinate-Free Perspective	21
1.6.5 Connection to the Rest of This Book	21
1.7 Matrix Norms and Conditioning	22
1.8 Matrix Decompositions for Computation	23
1.8.1 LU Decomposition	23
1.8.2 QR Decomposition	23
1.8.3 Cholesky Decomposition	24
1.9 Worked Example: Eigenvalue and SVD Analysis in Python	24
1.10 Chapter Summary	25
1.11 Further Reading	26
1.12 Exercises	26

2	The Transition to State Space	28
2.1	Introduction: The Physics Abstraction	28
2.2	Historical Context: The Kalman Revolution	29
2.3	Constructing a State Vector	29
2.3.1	Worked Example 1: Simple Pendulum	31
2.3.2	Worked Example 2: DC Motor	31
2.3.3	Worked Example 3: Two-Tank Fluid System	32
2.4	Converting n -th Order ODEs to State Form	32
2.5	State Space Representations	33
2.5.1	Standard Form Differential Equations	33
2.5.2	Linear State Space: The A and B Matrices	34
2.6	Output Equations and Observability	35
2.7	Equilibrium Points and Linearization	35
2.7.1	Equilibrium Point Definition	35
2.7.2	Linearization via Taylor Expansion	35
2.8	Phase Portraits and Stability Classification	36
2.9	Solution of Linear Systems: The Matrix Exponential	37
2.10	Controllability	38
2.11	Observability	39
2.12	The Mass-Spring-Damper System: Complete Analysis	40
2.12.1	Underdamped ($0 < \zeta < 1$)	40
2.12.2	Critically Damped ($\zeta = 1$)	40
2.12.3	Overdamped ($\zeta > 1$)	41
2.12.4	Controllability and Observability	41
2.13	Transfer Function to State Space and Back	41
2.13.1	Discrete-Time State Space	42
2.14	Nonlinear Systems and Phase Portraits	42
2.15	Python Worked Example: State Space Simulation and Phase Portrait	43
2.16	Equilibrium Analysis Summary	44
2.17	Summary	44
2.18	Lyapunov Stability Theory	45
2.19	Further Reading	47
2.20	Exercises	48
3	Configuration Space and Degrees of Freedom	50
3.1	Historical Context: From Lagrange to Riemann	50
3.2	Degrees of Freedom (DOF) and Grübler's Formula	51
3.2.1	The Concept of Degrees of Freedom	51
3.2.2	Grübler's Formula	52
3.2.3	Worked Example 1: Planar 4-Bar Linkage	53
3.2.4	Worked Example 2: Spatial Stewart Platform	53
3.2.5	Worked Example 3: Human Arm Kinematics	54
3.3	The Configuration Space $\mathcal{Q}$ and Topological Manifolds	54
3.3.1	Examples of Manifolds	55
3.4	Charts, Atlases, and Coordinate Patches	55
3.4.1	Worked Example: Charts on the Circle S^1	56
3.5	Transition Maps and Smooth Compatibility	57
3.5.1	Worked Example: Transition Map on S^1	57

3.6	Tangent Vectors, Curves, and Derivations	58
3.6.1	Tangent Vectors as Equivalence Classes of Curves	58
3.6.2	Tangent Vectors as Derivations	59
3.6.3	Equivalence of the Two Definitions	59
3.6.4	The Tangent Bundle TQ	60
3.7	Generalized Coordinates and Forward Kinematics	60
3.7.1	Forward Kinematics	60
3.8	The Jacobian Matrix and the Kinematic Derivative	61
3.8.1	Worked Example: 2-Link Arm Jacobian	62
3.8.2	Worked Example: 3-Link Arm in 3D	62
3.9	Inverse Kinematics	63
3.9.1	Geometric Approach to 2-Link IK	63
3.9.2	Numerical Inverse Kinematics: Newton-Raphson	64
3.10	Workspace Analysis: Reachable and Dexterous Workspace	64
3.11	The Frobenius Theorem and Integrability	65
3.12	Holonomic and Non-Holonomic Constraints	66
3.12.1	More Examples of Non-Holonomic Systems	67
3.12.2	Pfaffian Constraints	68
3.13	Python Worked Example: Forward Kinematics and Workspace of a 2R Arm	69
3.14	Chapter Summary and Key Principles	71
3.15	Further Reading	71
3.16	Exercises	72
4	Rotations, Quaternions, and $SO(3)$	75
4.1	Historical Context: From Euler to Hamilton	75
4.2	The Coordinate Frame	76
4.3	Rotation Matrices: From First Principles	76
4.3.1	What is a Rotation Matrix?	76
4.3.2	Orthonormality Constraints	77
4.3.3	The Three Elementary Rotation Matrices	77
4.4	The Special Orthogonal Group: $SO(3)$	78
4.4.1	$SO(3)$ as a Group	78
4.4.2	$SO(3)$ as a Smooth Manifold	79
4.5	Euler Angles: Gimbal Lock and Beyond	79
4.5.1	Definition of Euler Angles	79
4.5.2	The ZYX Convention	79
4.5.3	The ZXZ Convention	80
4.5.4	Gimbal Lock: Singularity and Proof	80
4.6	Differential Geometry Primer: Lie Groups and Lie Algebras	81
4.6.1	The Tangent Space and Lie Algebra	81
4.7	Rodrigues' Rotation Formula: Full Derivation	82
4.7.1	The Key Identity: $[\hat{\omega}]^3 = -[\hat{\omega}]$	82
4.7.2	Taylor Expansion and Regrouping	83
4.8	Quaternion Algebra	83
4.8.1	Definition and Representation	83
4.8.2	Quaternion Operations	84
4.8.3	Unit Quaternions and Rotation	84
4.8.4	Double Cover: q and $-q$ Represent the Same Rotation	85

4.9	Conversion Between Quaternions and Rotation Matrices	86
4.9.1	Quaternion to Rotation Matrix	86
4.9.2	Rotation Matrix to Quaternion	86
4.9.3	Rotation Representation Comparison	86
4.10	Spherical Linear Interpolation (SLERP)	87
4.11	Angular Velocity and the Matrix Exponential	88
4.11.1	Angular Velocity in the Spatial Frame	88
4.11.2	Angular Velocity in the Body Frame	88
4.11.3	Proof that Angular Velocity Lives in $\mathfrak{so}(3)$	89
4.12	Axis-Angle Representation and Euler's Rotation Theorem	89
4.13	Worked Example: Quaternions and SLERP in Python	89
4.14	Chapter Summary	93
4.15	Further Reading	94
4.15.1	Foundation Tier	94
4.15.2	Intermediate Tier	95
4.15.3	Advanced Tier	95
5	Twists, Wrenches, and the Screw Axis	96
5.1	Historical Context: From Chasles to Modern Screw Theory	97
5.2	The Unified 4×4 Transform $SE(3)$	97
5.2.1	Group Structure of $SE(3)$	99
5.3	Chasles' Theorem and the Screw Axis	100
5.3.1	Geometric Description of the Screw Axis	101
5.4	The Spatial Twist ($\mathcal{V}$)	102
5.4.1	Screw Representation of a Twist	102
5.4.2	Twists for Standard Joint Types	103
5.4.3	Matrix Representation of Twists	103
5.4.4	The Lie Bracket on $\mathfrak{se}(3)$	104
5.5	The Adjoint Representation	104
5.5.1	Changing Reference Frames	104
5.6	The Spatial Wrench ($\mathcal{F}$)	105
5.6.1	Wrenches from Forces and Moments	106
5.6.2	Wrench Transformation Under Changes of Frame	106
5.6.3	Physical Interpretation of Wrenches: Forces and Torques Unified	106
5.7	Proof of Chasles' Theorem	108
5.8	Mechanical Work and the Virtual Power Principle	109
5.8.1	Invariance of Power	110
5.9	Reciprocal Screws and Constraint Analysis	110
5.9.1	Significance in Constraint Analysis	110
5.10	Physical Interpretation of Wrenches	110
5.10.1	Three Fundamental Wrench Types	111
5.10.2	The Wrench-Twist Duality and Mechanical Power	112
5.10.3	Transforming Wrenches Between Frames	112
5.11	Wrench Analysis and Force Polytopes	112
5.12	Worked Example: Wrench Analysis of a Planar 4-Bar Mechanism	113
5.12.1	Setup	113
5.12.2	Twist Space of Each Joint	113
5.12.3	Constraint Wrenches	113

5.12.4	Equilibrium Condition	114
5.12.5	Geometric Interpretation: Force Polygon	114
5.13	Python Implementation	114
5.14	Summary	116
5.15	Further Reading	117
5.16	Exercises	117
6	Exponential Coordinates and Matrix Logarithms	119
6.1	Historical Context: Sophus Lie and the Theory of Continuous Groups	119
6.2	Lie Algebras and the Tangent Space	120
6.2.1	The Lie Bracket: Non-Commutativity of Motions	121
6.3	Exponential Coordinates and Axis-Angle Representation	122
6.3.1	The Exponential Coordinate Vector	122
6.4	Rodrigues' Formula and the Matrix Exponential for $SO(3)$	122
6.4.1	Verification that the Exponential Map Produces Valid Rotations	124
6.5	The Matrix Logarithm on $SO(3)$	124
6.5.1	Computing the Logarithm: All Cases	125
6.6	The Exponential Map on $SE(3)$	126
6.6.1	Derivation of the $SE(3)$ Exponential Formula	127
6.7	The Matrix Logarithm on $SE(3)$	128
6.7.1	Numerical Stability of the Matrix Logarithm	128
6.8	The Baker-Campbell-Hausdorff Formula	129
6.8.1	The Baker-Campbell-Hausdorff Formula: Higher-Order Terms	130
6.9	Numerical Computation of Matrix Exponentials	131
6.9.1	Padé Approximation	131
6.9.2	Scaling and Squaring	131
6.10	Geodesics on $SO(3)$: Shortest Rotation Paths	131
6.10.1	Geodesics on $SO(3)$: Proof via Riemannian Geometry	132
6.11	SLERP: Spherical Linear Interpolation as Geodesic Interpolation	133
6.12	Python Worked Example: Computing Exponentials and Logarithms	134
6.13	Summary	138
6.14	Further Reading	138
6.15	Exercises	139
7	Recursive Algorithms	141
7.1	Historical Context: The Computational Revolution	141
7.2	The Curse of Dimensionality in Dynamics	142
7.3	Kinematic Tree Representation	143
7.3.1	Parent Arrays and Numbering Conventions	143
7.4	The Adjoint Map ($[Ad_T]$)	143
7.4.1	Derivation of the Adjoint Matrix	144
7.4.2	The Adjoint for Forces (Wrenches)	145
7.5	Recursive Forward Kinematics	145
7.5.1	Recursive Velocity Propagation	145
7.5.2	Algorithm: Forward Kinematics in $O(N)$ Time	146
7.6	Spatial Acceleration and Coriolis Terms	146
7.6.1	The Lie Bracket and Centrifugal Forces	146
7.7	Spatial Inertia	147

7.8	The Recursive Newton-Euler Algorithm (RNEA)	148
7.8.1	The Outward Pass: Velocity and Acceleration Propagation	148
7.8.2	The Inward Pass: Force and Torque Propagation	148
7.8.3	Force Recursion and Extraction of Joint Torques	149
7.9	Computational Complexity Analysis	150
7.9.1	Why $O(N)$ and not $O(N^2)$?	150
7.9.2	Proof of Linearity	151
7.10	Branching Kinematic Trees	151
7.10.1	Handling Multiple Children	151
7.11	Contact Forces and External Disturbances	152
7.11.1	Initialization at Leaf Nodes	152
7.12	RNEA Equivalence to the Lagrangian Equations	152
7.13	Contact Forces and Constraint Handling	153
7.13.1	Bilateral Constraints	153
7.13.2	Unilateral Constraints and Friction	153
7.14	Worked Example: 3-Link Planar Arm	154
7.14.1	Setup	154
7.14.2	Outward Pass: Velocities and Accelerations	154
7.14.3	Inward Pass: Forces and Torques	155
7.15	Validation: RNEA vs. Lagrangian	156
7.15.1	2-Link Planar Arm	156
7.16	Adjoint Matrix Derivation from Geometry	157
7.16.1	Infinitesimal Rotations in $SE(3)$	157
7.16.2	Adjoint as Conjugation	157
7.17	Python Implementation of RNEA	157
7.18	Summary	159
7.19	Further Reading	160
7.20	Exercises	161
7.20.1	Tier 1 (Foundation)	161
7.20.2	Tier 2 (Intermediate)	161
7.20.3	Tier 3 (Advanced)	161
8	Spatial Vector Algebra and Inertia	163
8.1	Historical Context: From Plücker to Featherstone	163
8.2	Plücker Coordinates and Lines in 3D	164
8.2.1	Connection to Spatial Vectors	165
8.3	Spatial Vectors: Twists and Wrenches	165
8.4	The Spatial Cross Product	166
8.4.1	Motion Cross Product	166
8.4.2	Force Cross Product	166
8.4.3	Why Two Cross Products?	167
8.5	The Spatial Inertia Matrix	167
8.5.1	Definition and Properties	168
8.5.2	Symmetric and Positive Definite	168
8.6	The Parallel Axis Theorem in 6D	169
8.6.1	Derivation from First Principles	169
8.7	Composite Rigid Body Algorithm	169
8.7.1	Concept: Bottom-Up Inertia Assembly	169

8.7.2	Use in Computing the Mass Matrix	170
8.8	Spatial Algebra for Planar Systems	170
8.8.1	3D Vectors for 2D Motion	170
8.8.2	Computational Advantage	171
8.9	Newton-Euler Equations in Spatial Form	171
8.9.1	Spatial Momentum and the Equation of Motion	171
8.10	Connection to Dual Quaternions	172
8.10.1	Brief Overview	172
8.11	Worked Example: Spatial Inertia of a Multi-Link System	172
8.11.1	Building Link Spatial Inertias	173
8.11.2	Composite Inertia	173
8.12	Physical Interpretation of Spatial Inertia Elements	174
8.12.1	Kinetic Energy and the Origin of Spatial Inertia	174
8.13	Python Implementation	175
8.14	Summary	177
8.15	Further Reading	178
8.16	Exercises	179
8.16.1	Tier 1 (Foundation)	179
8.16.2	Tier 2 (Intermediate)	179
8.16.3	Tier 3 (Advanced)	179
9	The Product of Exponentials Formula	181
9.1	Historical Context and the Evolution of Kinematic Formalisms	181
9.2	Abandoning Denavit-Hartenberg	182
9.3	Screws and Twists: A Geometric Foundation	183
9.4	The Forward Kinematics Equation: Space Frame PoE	183
9.4.1	Derivation from First Principles	184
9.5	Body Frame PoE: Motion Relative to the Tool	184
9.5.1	Conversion Between Space and Body Frame	184
9.6	Spatial Jacobian from the Product of Exponentials	185
9.6.1	Direct Numerical Computation	186
9.7	Body Jacobian and the Adjoint Relationship	186
9.8	Singularity Analysis Using the PoE Jacobian	186
9.8.1	Types of Singularities	187
9.8.2	Detection and Handling	187
9.9	Comparison with D-H: A Worked Example	187
9.9.1	D-H Parametrization	187
9.9.2	PoE Parametrization	188
9.9.3	Computational Advantages of PoE	188
9.10	PoE for Prismatic Joints	189
9.10.1	Example: 3D Cartesian Robot	189
9.11	Velocity Kinematics from PoE Differentiation	189
9.12	Worked Example: Complete PoE for a Spatial 3R Robot	190
9.12.1	Home Configuration	190
9.12.2	Screw Axes	190
9.12.3	Numerical Evaluation at a Test Configuration	191
9.13	Inverse Kinematics using the Jacobian: Newton-Raphson	191
9.13.1	Pose Error Representation	191

9.13.2	Newton-Raphson Update Rule	192
9.13.3	Algorithm and Convergence	192
9.14	Workspace Analysis and Manipulability	192
9.14.1	Redundancy and Null-Space Motion	194
9.15	Python Worked Example	195
9.16	Chapter Summary	196
9.17	Further Reading and References	197
9.18	Exercises	197
10	The Articulated Body Algorithm	199
10.1	Historical Context: The Computational Mechanics Revolution	199
10.2	The Forward Dynamics Problem	200
10.2.1	Problem Formulation	200
10.2.2	Why $O(N^3)$ is Intractable	201
10.3	Articulated Inertia: The Key Insight	201
10.3.1	Intuitive Explanation	201
10.3.2	Mathematical Definition	201
10.4	Derivation from First Principles	202
10.4.1	Recursive Constraint Satisfaction	202
10.5	The Articulated Body Algorithm: Three Passes	203
10.5.1	Pass 1: Outward Pass (Velocity and Bias Forces)	203
10.5.2	Pass 2: Inward Pass (Articulated Inertia Propagation)	204
10.5.3	Pass 3: Outward Pass (Joint Accelerations)	205
10.6	Proof of $O(N)$ Complexity	205
10.7	Floating Base: 6-DOF Root	206
10.7.1	Modification to ABA	206
10.7.2	Contact and Constraints	207
10.8	Floating-Base Dynamics for Humanoid Robots	207
10.8.1	Virtual Joint Formulation	207
10.8.2	Base Acceleration from Articulated Inertia	207
10.8.3	Centroidal Momentum and Whole-Body Dynamics	208
10.9	Joint Friction and Motor Inertia	208
10.9.1	Joint Friction	208
10.9.2	Motor Inertia	208
10.10	ABA vs. CRBA: When to Use Which	209
10.11	Forward Dynamics vs. Inverse Dynamics Decision Criteria	209
10.11.1	When to Use Inverse Dynamics (Recursive Newton-Euler)	209
10.11.2	When to Use Forward Dynamics (ABA)	209
10.12	Constraint Handling: Contact Forces and Joint Limits	210
10.12.1	Unilateral Contact Constraints	210
10.12.2	Joint Limits	210
10.13	Connection to Gaussian Elimination	210
10.14	Worked Example: Full ABA Walkthrough for a 3-Link Arm	210
10.14.1	Configuration at the Test Point	211
10.14.2	Pass 1: Velocity and Bias	211
10.14.3	Pass 2: Articulated Inertia	211
10.14.4	Pass 3: Joint Accelerations	212
10.15	Python Worked Example	212

10.15.1 Connection to Simulation Frameworks	215
10.16 Contact Forces and Impact Dynamics	216
10.17 Contact Dynamics and the Linear Complementarity Problem	216
10.17.1 Non-Penetration and Complementarity	216
10.17.2 Friction Cone Constraint	217
10.17.3 The LCP Formulation	217
10.17.4 Time-Stepping Methods	217
10.18 Chapter Summary	218
10.19 Further Reading and References	218
10.20 Exercises	219

11 Lagrangian Mechanics	221
11.1 Historical Context: From Newton to Lagrange to Hamilton	221
11.2 The Limitations of Newton	222
11.3 Hamilton's Principle and the Calculus of Variations	223
11.4 Lagrangian and Energy	224
11.5 Derivation of the Standard Form: $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{G} = \boldsymbol{\tau}$	225
11.6 Properties of the System Matrices	226
11.6.1 The Mass Matrix $\mathbf{M}(\mathbf{q})$	226
11.6.2 The Coriolis/Centripetal Matrix $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$	226
11.6.3 The Gravity Vector $\mathbf{G}(\mathbf{q})$	227
11.7 Energy Conservation	227
11.8 Hamiltonian Mechanics	228
11.9 Noether's Theorem: Symmetry and Conservation Laws	229
11.9.1 Rigorous Proof of Noether's Theorem	230
11.10 Lagrangian for Constrained Systems	231
11.11 D'Alembert's Principle and Virtual Work	231
11.12 Christoffel Symbols and the Geometry of $\mathbf{M}$	232
11.13 Worked Example 1: Double Pendulum	232
11.13.1 System Setup	232
11.13.2 Kinetic Energy	233
11.13.3 Potential Energy	233
11.13.4 Gravity Vector	234
11.13.5 Coriolis/Centripetal Matrix	234
11.14 Worked Example 2: 2-Link Robot Arm with Gravity	234
11.14.1 System Configuration	234
11.14.2 Kinetic Energy	235
11.14.3 Potential Energy	235
11.15 Comparison: Lagrangian vs. Newton-Euler	235
11.16 Symplectic Integrators	236
11.17 Python Worked Example: Double Pendulum Dynamics	237
11.18 Chapter Summary	239
11.19 Further Reading and References	240
11.20 Exercises	240

12 Machine Learning and Neural Networks	242
12.1 Historical Context: From Perceptron to Deep Reinforcement Learning	242
12.2 The Limits of Analytical Control	243
12.3 Universal Approximation Theorem	244
12.4 Neural Networks as Function Approximators	244
12.4.1 Feedforward Architecture	244
12.5 Activation Functions	245
12.5.1 Rectified Linear Unit (ReLU)	245
12.5.2 Sigmoid	246
12.5.3 Hyperbolic Tangent (Tanh)	246
12.5.4 Gaussian Error Linear Unit (GELU)	246
12.6 Loss Functions	247
12.6.1 Supervised Learning: Mean Squared Error	247
12.6.2 Robustness: Huber Loss	247
12.6.3 Reinforcement Learning: Policy Gradient and Value Loss	247
12.7 Backpropagation: Chain Rule for Neural Networks	247
12.7.1 Computational Graph Example	248
12.7.2 Forward Pass	248
12.7.3 Backward Pass (Backpropagation)	248
12.7.4 General Pattern	249
12.8 Gradient Descent and Optimization	249
12.8.1 Vanilla Stochastic Gradient Descent	249
12.8.2 Momentum	249
12.8.3 Adam Optimizer	250
12.9 Markov Decision Processes (MDPs)	250
12.9.1 Formal Definition	250
12.9.2 Markov Property	250
12.9.3 Return and Value Function	251
12.9.4 Bellman Equation	251
12.10 Policy Gradient Methods	251
12.10.1 REINFORCE Algorithm	251
12.10.2 High Variance Problem	252
12.11 Actor-Critic Architecture	252
12.11.1 Architecture	253
12.11.2 Advantage Estimate	253
12.11.3 Update Rules	253
12.11.4 Advantage	253
12.12 Proximal Policy Optimization (PPO)	253
12.12.1 Problem with Large Policy Updates	253
12.12.2 Policy Clipping	254
12.12.3 Interpretation	254
12.12.4 Full PPO Objective	254
12.13 Model-Based vs. Model-Free RL	254
12.13.1 Model-Free RL	254
12.13.2 Model-Based RL	255
12.13.3 Hybrid Approaches	255
12.14 Physics-Informed Neural Networks (PINNs)	255
12.14.1 Framework	256

12.14.2 Application to Robotics	256
12.15 Neural ODEs and Adjoint Method	256
12.15.1 Neural ODE Formulation	256
12.15.2 Adjoint Method	257
12.16 Sim-to-Real Transfer and Domain Randomization	257
12.16.1 Domain Randomization	257
12.16.2 System Identification	258
12.17 Worked Example: Pendulum Swing-Up Policy	258
12.17.1 Problem Setup	258
12.17.2 Policy Network	258
12.17.3 Training with PPO	258
12.18 Value-Based Methods: DQN and Beyond	259
12.19 Structure-Preserving Neural Networks	260
12.19.1 Exploration Strategies	261
12.20 Python Worked Example: Backpropagation from Scratch	261
12.21 Deep Reinforcement Learning for Robot Control	264
12.21.1 State Representation	265
12.21.2 Action Parameterization	265
12.21.3 Reward Shaping	265
12.21.4 Training Stability	265
12.22 Chapter Summary	266
12.22.1 Connection to Prior Chapters	267
12.23 Further Reading and References	267
12.24 Exercises	268
Glossary of Important Concepts	270
Further Reading and References	272
Index	273

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{n_x}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ or $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta \mathbf{u} = -\mathbf{K}\delta \mathbf{x}$).

γ A trajectory or flow, mapping time and initial conditions to states.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

F_{muscle} Force generated by muscles.

W Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

A Primer on Linear Algebra

You cannot control a machine by staring at the machine. You control it by staring at the mathematical space that completely defines it.

In Plain Language 1.1. Context & Use Case: Why Linear Algebra is the Language of Motion

The Math You Will See: We will cover Vectors ($\mathbf{v}$), Matrices ($\mathbf{M}$), Vector Spaces and their axioms, Linear Independence and Basis, Eigenvalues/Eigenvectors ($\lambda, \mathbf{v}$), the Singular Value Decomposition (SVD), Quadratic Forms ($\mathbf{x}^T \mathbf{M} \mathbf{x}$), Positive Definiteness, and Dyads/Dyadics (the tensor products that underpin inertia, stress, and stiffness).

Why We Need It: A robot's joints or a car's velocity don't exist as single numbers; they exist as large arrays of interacting numbers. Linear algebra is the mathematics of these arrays. If you want to know if a drone will crash into the ground, you don't track every single rotor separately; you assemble the entire system into a single matrix and look at its eigenvalues.

All Variables Defined: Check the **Nomenclature** at the start of the book for standard vector and matrix conventions.

1.1 Historical Context

Linear algebra—the mathematics of vectors, matrices, and linear transformations—is arguably the single most consequential mathematical framework underpinning all of modern engineering. Its roots stretch back to ancient China (the *Nine Chapters on the Mathematical Art*, ca. 200 BCE, contained early row-reduction methods for solving systems of linear equations) and were formalized in the West through the work of Leibniz, Cramer, and Cayley in the 17th–19th centuries.

However, it was not until the mid-20th century that linear algebra assumed its central role in control theory. In 1960, Rudolf Kalman published his landmark paper introducing the *state space representation* of dynamical systems, which recast classical control problems—previously analyzed using frequency-domain transfer functions—into the language of matrices and vectors [15]. Kalman's formulation required engineers to reason about *controllability* and *observability* using rank conditions on matrices, placing linear algebra at the absolute foundation of modern control engineering.

Today, every simulation framework discussed in this text series—from MuJoCo’s articulated-body integrator to Drake’s trajectory optimization engine—performs thousands of matrix operations per millisecond. Understanding the structures, decompositions, and geometric interpretations of linear algebra is not optional; it is the prerequisite upon which every subsequent chapter of *The Geometry of Motion* is built.

1.2 Vectors, Matrices, and Vector Spaces

Before mapping physics, we must understand the fundamental data structures of control theory: vectors and matrices. More importantly, we must understand the abstract *space* in which these objects live, because the geometry of that space dictates everything about how physical systems behave. Readers seeking a comprehensive undergraduate treatment of these ideas will find Strang’s textbook [33] an excellent companion to this chapter.

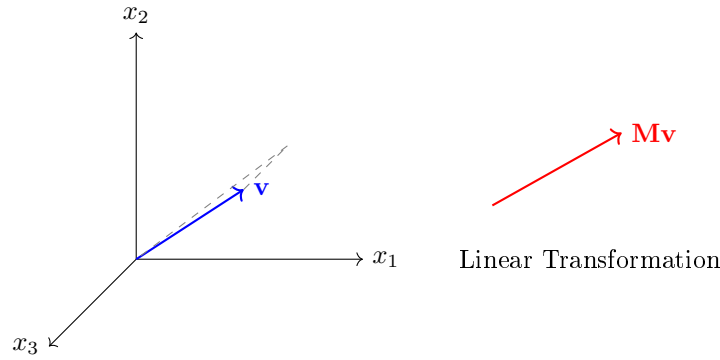


Figure 1.1: Vector in $\mathbb{R}^3$ and its transformation under a linear map $T(\mathbf{v}) = \mathbf{M}\mathbf{v}$.

1.2.1 Vectors

A **vector** $\mathbf{v}$ is an ordered array of numbers. It can represent a position in space, a velocity profile, a list of sensor readings, or the activation levels of every muscle in a golfer’s forearm. In physics, vectors implicitly assume a coordinate frame (e.g., X, Y, Z). A generic column vector in n -dimensional space is denoted:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n \quad (1.1)$$

It is crucial to distinguish two perspectives on what a vector “is.” The *algebraic* perspective treats a vector as a column of numbers. The *geometric* perspective treats a vector as a directed arrow in space, possessing both magnitude and direction. Both perspectives are valid and useful; throughout this text, we will switch between them freely. When we write $\mathbf{v} \in \mathbb{R}^3$, we mean both the triple of numbers (v_1, v_2, v_3) and the arrow pointing from the origin to the point (v_1, v_2, v_3) in three-dimensional Euclidean space.

1.2.2 Matrices

A **matrix** $\mathbf{M}$ is a rectangular grid of numbers arranged in rows and columns. If a vector represents a point or a direction, a matrix acts as a set of instructions on how to *transform* that point—stretching it, rotating it, projecting it, or mapping it into an entirely different dimensional space. An $n \times m$ matrix maps vectors from $\mathbb{R}^m$ to $\mathbb{R}^n$:

$$\mathbf{M} = \begin{bmatrix} m_{11} & m_{12} & \cdots & m_{1m} \\ m_{21} & m_{22} & \cdots & m_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & m_{nm} \end{bmatrix} \in \mathbb{R}^{n \times m} \quad (1.2)$$

When we multiply a matrix by a vector ($\mathbf{M}\mathbf{v}$), the result is a *linear combination of the columns of $\mathbf{M}$* , using the elements of $\mathbf{v}$ as weights. Specifically, if $\mathbf{m}_1, \mathbf{m}_2, \dots, \mathbf{m}_m$ denote the columns of $\mathbf{M}$, then:

$$\mathbf{M}\mathbf{v} = v_1\mathbf{m}_1 + v_2\mathbf{m}_2 + \cdots + v_m\mathbf{m}_m \quad (1.3)$$

This “column view” of matrix-vector multiplication is one of the most important conceptual tools in linear algebra. It means that the *range* (or image) of a matrix—the set of all possible outputs $\mathbf{M}\mathbf{v}$ as $\mathbf{v}$ varies over all of $\mathbb{R}^m$ —is precisely the *span* of the columns of $\mathbf{M}$. Throughout this book, when we rotate a robot arm, we are physically applying a matrix to the vectors extending along its links.

1.2.3 The Axioms of a Vector Space

We have described vectors informally as “arrays of numbers.” But the true power of linear algebra lies in the abstract concept of a **vector space**, which captures the essential algebraic properties that make vectors useful—regardless of whether the “vectors” are columns of numbers, functions, or even infinite-dimensional signals.

Definition 1.1. Vector Space

A *vector space* V over the real numbers $\mathbb{R}$ is a set of objects (called vectors) together with two operations—**addition** ($\mathbf{u} + \mathbf{v}$) and **scalar multiplication** ($\alpha\mathbf{v}$, where $\alpha \in \mathbb{R}$)—satisfying the following eight axioms for all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and all $\alpha, \beta \in \mathbb{R}$:

- (i) **Closure under addition:** $\mathbf{u} + \mathbf{v} \in V$.
- (ii) **Commutativity:** $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$.
- (iii) **Associativity of addition:** $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$.
- (iv) **Existence of a zero vector:** There exists $\mathbf{0} \in V$ such that $\mathbf{v} + \mathbf{0} = \mathbf{v}$.
- (v) **Existence of additive inverses:** For each $\mathbf{v} \in V$, there exists $-\mathbf{v} \in V$ such that $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$.
- (vi) **Closure under scalar multiplication:** $\alpha\mathbf{v} \in V$.
- (vii) **Distributivity:** $\alpha(\mathbf{u} + \mathbf{v}) = \alpha\mathbf{u} + \alpha\mathbf{v}$ and $(\alpha + \beta)\mathbf{v} = \alpha\mathbf{v} + \beta\mathbf{v}$.
- (viii) **Compatibility:** $\alpha(\beta\mathbf{v}) = (\alpha\beta)\mathbf{v}$, and $1 \cdot \mathbf{v} = \mathbf{v}$.

In Plain Language 1.2. Why Do We Care About Abstract Axioms?

You might wonder: why bother with these formal rules? Because the same mathematical structures appear over and over in radically different contexts. The set $\mathbb{R}^n$ of column vectors satisfies these axioms—but so does the set of all continuous functions $f : [0, T] \rightarrow \mathbb{R}$ (where “addition” means adding functions pointwise). In optimal control, the “vectors” we optimize over are often entire *trajectories*, which are functions of time. By proving a theorem about abstract vector spaces, we prove it simultaneously for column vectors, for functions, and for trajectories. The axioms are the price of generality.

Example 1.1. Verifying $\mathbb{R}^n$ is a Vector Space

Let us verify that $\mathbb{R}^n$ with standard component-wise addition and scalar multiplication satisfies the axioms.

Given $\mathbf{u} = (u_1, \dots, u_n)$ and $\mathbf{v} = (v_1, \dots, v_n)$ in $\mathbb{R}^n$, and $\alpha \in \mathbb{R}$:

- *Closure under addition:* $\mathbf{u} + \mathbf{v} = (u_1 + v_1, \dots, u_n + v_n)$. Since each $u_i + v_i \in \mathbb{R}$, the sum is in $\mathbb{R}^n$. ✓
- *Zero vector:* $\mathbf{0} = (0, 0, \dots, 0)$. Clearly $\mathbf{v} + \mathbf{0} = \mathbf{v}$. ✓
- *Additive inverse:* $-\mathbf{v} = (-v_1, \dots, -v_n)$. Then $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$. ✓
- *Closure under scalar multiplication:* $\alpha \mathbf{v} = (\alpha v_1, \dots, \alpha v_n) \in \mathbb{R}^n$. ✓

The remaining axioms (commutativity, associativity, distributivity, compatibility) follow immediately from the corresponding properties of real number arithmetic. Therefore $\mathbb{R}^n$ is a vector space. $\square$

1.2.4 Linear Independence and Span

Definition 1.2. Linear Independence

A set of vectors $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k\}$ in a vector space V is called *linearly independent* if the only solution to the equation

$$\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_k \mathbf{v}_k = \mathbf{0} \quad (1.4)$$

is the trivial solution $\alpha_1 = \alpha_2 = \dots = \alpha_k = 0$. If a nontrivial solution exists (i.e., at least one $\alpha_i \neq 0$), the vectors are *linearly dependent*.

Intuitively, a set of vectors is linearly independent if none of them can be “built” from the others. In $\mathbb{R}^2$, two vectors are linearly independent if and only if they do not point in the same (or exactly opposite) direction. In $\mathbb{R}^3$, three vectors are linearly independent if and only if they do not all lie in the same plane.

Example 1.2. Linear Independence in $\mathbb{R}^3$

Consider the vectors:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{v}_3 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \quad (1.5)$$

Are these linearly independent? We solve $\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \alpha_3 \mathbf{v}_3 = \mathbf{0}$:

$$\alpha_1 + \alpha_3 = 0 \quad (1.6)$$

$$\alpha_2 + \alpha_3 = 0 \quad (1.7)$$

$$0 = 0 \quad (1.8)$$

The third equation provides no constraint. Setting $\alpha_3 = 1$ yields $\alpha_1 = -1$, $\alpha_2 = -1$. A nontrivial solution exists: $-\mathbf{v}_1 - \mathbf{v}_2 + \mathbf{v}_3 = \mathbf{0}$, confirming that $\mathbf{v}_3 = \mathbf{v}_1 + \mathbf{v}_2$. The vectors are linearly *dependent*. Geometrically, all three vectors lie in the xy -plane, and the third is the diagonal of the rectangle formed by the first two.

1.2.5 Basis and Dimension**Definition 1.3. Basis**

A *basis* for a vector space V is a set of vectors $\mathcal{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ that is both linearly independent and spans V (i.e., every vector in V can be written as a linear combination of the basis vectors). The number n of vectors in any basis is called the *dimension* of V , written $\dim(V) = n$.

A fundamental theorem of linear algebra states that *every basis for a given vector space has exactly the same number of elements*. This is why “dimension” is well-defined—it is an intrinsic property of the space, not of any particular choice of basis.

The standard basis for $\mathbb{R}^n$ consists of the unit coordinate vectors $\mathbf{e}_1 = (1, 0, \dots, 0)^T$, $\mathbf{e}_2 = (0, 1, 0, \dots, 0)^T$, through $\mathbf{e}_n = (0, \dots, 0, 1)^T$. Any vector $\mathbf{v} = (v_1, \dots, v_n)^T$ can be written uniquely as $\mathbf{v} = v_1 \mathbf{e}_1 + \dots + v_n \mathbf{e}_n$. The numbers $v_1, \dots, v_n$ are the *coordinates* of $\mathbf{v}$ with respect to this basis.

In Plain Language 1.3. Why Basis Matters for Robotics

When we model a robot arm, the “natural” basis might be the joint angles $(\theta_1, \theta_2, \dots, \theta_n)$. But the task might require us to think in Cartesian coordinates (x, y, z) . Changing basis—expressing the same physical motion in different coordinate systems—is one of the most common operations in robotics. The Jacobian matrix (Chapter 3) is precisely the mathematical tool that translates between these bases.

1.2.6 Linear Transformations

Definition 1.4. Linear Transformation

A function $T : V \rightarrow W$ between two vector spaces is called a *linear transformation* (or *linear map*) if it preserves vector addition and scalar multiplication:

- (i) $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ for all $\mathbf{u}, \mathbf{v} \in V$.
- (ii) $T(\alpha \mathbf{v}) = \alpha T(\mathbf{v})$ for all $\alpha \in \mathbb{R}$ and $\mathbf{v} \in V$.

The fundamental connection between linear transformations and matrices is this: *every linear transformation between finite-dimensional vector spaces can be represented by a matrix, and every matrix represents a linear transformation*. Specifically, if $T : \mathbb{R}^m \rightarrow \mathbb{R}^n$ is linear, then there exists a unique $n \times m$ matrix $\mathbf{M}$ such that $T(\mathbf{v}) = \mathbf{M}\mathbf{v}$ for all $\mathbf{v} \in \mathbb{R}^m$. The columns of $\mathbf{M}$ are simply $T(\mathbf{e}_1), T(\mathbf{e}_2), \dots, T(\mathbf{e}_m)$ —the images of the standard basis vectors.

Theorem 1.1. Matrix Multiplication is Linear

Let $\mathbf{M} \in \mathbb{R}^{n \times m}$. Then the function $T(\mathbf{v}) = \mathbf{M}\mathbf{v}$ is a linear transformation from $\mathbb{R}^m$ to $\mathbb{R}^n$.

Proof. Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ and $\alpha \in \mathbb{R}$. By the distributive property of matrix multiplication:

$$T(\mathbf{u} + \mathbf{v}) = \mathbf{M}(\mathbf{u} + \mathbf{v}) = \mathbf{M}\mathbf{u} + \mathbf{M}\mathbf{v} = T(\mathbf{u}) + T(\mathbf{v}) \quad (1.9)$$

and by the compatibility of matrix multiplication with scalar multiplication:

$$T(\alpha \mathbf{v}) = \mathbf{M}(\alpha \mathbf{v}) = \alpha(\mathbf{M}\mathbf{v}) = \alpha T(\mathbf{v}) \quad (1.10)$$

Both conditions of linearity are satisfied. $\square$

1.2.7 Rank, Null Space, and the Rank-Nullity Theorem

Not all linear transformations preserve full information about the input. Some transformations “collapse” certain directions to zero.

Definition 1.5. Rank

The *rank* of a matrix $\mathbf{M}$, denoted $\text{rank}(\mathbf{M})$, is the number of linearly independent columns (equivalently, the number of linearly independent rows) of $\mathbf{M}$. It equals the dimension of the column space (the range of the corresponding linear transformation).

If a 3×3 matrix has rank 2, it collapses full 3D space into a 2D plane. In robotics, the rank of the Jacobian matrix $\mathbf{J}(\mathbf{q})$ tells you how many independent directions the end-effector can instantaneously move in. If the rank drops below its maximum value, the robot is at a *singularity*—it has lost the ability to move in at least one Cartesian direction, regardless of how its motors are commanded.

Definition 1.6. Null Space (Kernel)

The *null space* (or *kernel*) of a matrix $\mathbf{M}$, denoted $\mathcal{N}(\mathbf{M})$, is the set of all vectors that the matrix maps to zero:

$$\mathcal{N}(\mathbf{M}) = \{\mathbf{x} \in \mathbb{R}^m \mid \mathbf{M}\mathbf{x} = \mathbf{0}\} \quad (1.11)$$

For a humanoid robot, the null space of the arm's Jacobian describes all “internal motions” the arm can execute (like repositioning the elbow) while keeping the hand perfectly stationary holding a cup of coffee. These null-space motions are invisible in task space but consume real joint velocities. Exploiting them is a key strategy in redundant robot control.

Definition 1.7. The Determinant (Volumetric Interpretation)

For a square matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$, the *determinant* $\det(\mathbf{M})$ is a scalar that measures the factor by which $\mathbf{M}$ scales n -dimensional volume. Imagine a unit hypercube $[0, 1]^n$. Applying $\mathbf{M}$ to every point in that cube warps it into a parallelepiped whose signed volume equals $\det(\mathbf{M})$.

- If $\det(\mathbf{M}) = 1$, volume is perfectly preserved (like a pure rotation in space).
- If $\det(\mathbf{M}) = 0$, volume is crushed completely flat—the matrix is *singular* and loses rank. The transformation is not invertible.
- If $\det(\mathbf{M}) < 0$, volume is preserved in magnitude but orientation is flipped (a mirror reflection).

Now we arrive at one of the most important structural results in linear algebra:

Theorem 1.2. Rank-Nullity Theorem

For any $n \times m$ matrix $\mathbf{M}$:

$$\underbrace{\text{rank}(\mathbf{M})}_{\text{dimension of range}} + \underbrace{\dim(\mathcal{N}(\mathbf{M}))}_{\text{dimension of null space}} = m \quad (\text{number of columns}) \quad (1.12)$$

Proof. Let $r = \text{rank}(\mathbf{M})$ and let $\{\mathbf{n}_1, \dots, \mathbf{n}_k\}$ be a basis for $\mathcal{N}(\mathbf{M})$, where $k = \dim(\mathcal{N}(\mathbf{M}))$. Extend this to a basis $\{\mathbf{n}_1, \dots, \mathbf{n}_k, \mathbf{w}_1, \dots, \mathbf{w}_{m-k}\}$ for all of $\mathbb{R}^m$ (this extension is always possible by a standard result in linear algebra).

We claim that $\{\mathbf{M}\mathbf{w}_1, \dots, \mathbf{M}\mathbf{w}_{m-k}\}$ is a basis for the column space of $\mathbf{M}$, which would establish $r = m - k$, i.e., $r + k = m$.

Spanning: Any $\mathbf{y}$ in the range of $\mathbf{M}$ can be written as $\mathbf{y} = \mathbf{M}\mathbf{v}$ for some $\mathbf{v} \in \mathbb{R}^m$. Expanding $\mathbf{v}$ in our basis: $\mathbf{v} = \sum_{i=1}^k \alpha_i \mathbf{n}_i + \sum_{j=1}^{m-k} \beta_j \mathbf{w}_j$. Since $\mathbf{M}\mathbf{n}_i = \mathbf{0}$, we get $\mathbf{y} = \sum_{j=1}^{m-k} \beta_j \mathbf{M}\mathbf{w}_j$. So the $\mathbf{M}\mathbf{w}_j$ span the range.

Independence: Suppose $\sum_{j=1}^{m-k} \beta_j \mathbf{M}\mathbf{w}_j = \mathbf{0}$. Then $\mathbf{M} \left(\sum_j \beta_j \mathbf{w}_j \right) = \mathbf{0}$, meaning $\sum_j \beta_j \mathbf{w}_j \in \mathcal{N}(\mathbf{M})$. So $\sum_j \beta_j \mathbf{w}_j = \sum_{i=1}^k \gamma_i \mathbf{n}_i$ for some scalars γ_i . But $\{\mathbf{n}_1, \dots, \mathbf{n}_k, \mathbf{w}_1, \dots, \mathbf{w}_{m-k}\}$ is a basis for $\mathbb{R}^m$, so these vectors are linearly independent. Therefore all $\beta_j = 0$ and all $\gamma_i = 0$.

Thus $r = m - k$, establishing $\text{rank}(\mathbf{M}) + \dim(\mathcal{N}(\mathbf{M})) = m$. $\square$

In Plain Language 1.4. The Physical Meaning of Rank-Nullity

For a robot with m motors and an end-effector moving in n -dimensional task space:

- The **rank** of the Jacobian tells you how many independent task-space directions the robot can control.
- The **nullity** (dimension of the null space) tells you how many “extra” internal motions remain after the task is specified.

If a 7-DOF arm ($m = 7$) controls a hand in 3D position space ($n = 3$), and the Jacobian has rank 3, then the nullity is $7 - 3 = 4$. The robot has 4 degrees of internal freedom to reposition its elbow, shoulder, and wrist without moving the hand at all.

1.3 The Dot Product, Cross Product, and Inner Product Spaces

Two fundamental operations exist for combining vectors. Understanding them geometrically is essential for everything from computing kinetic energy to defining stability metrics.

1.3.1 The Dot Product (Inner Product)

The dot product (or inner product) takes two vectors and returns a scalar. It measures how much two vectors “align” with each other.

Definition 1.8. Dot Product

For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, the *dot product* is defined as:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \sum_{i=1}^n a_i b_i \quad (1.13)$$

In $\mathbb{R}^3$, this expands to $a_x b_x + a_y b_y + a_z b_z$.

The geometric interpretation connects the algebraic formula to angles:

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos(\theta) \quad (1.14)$$

where θ is the angle between $\mathbf{a}$ and $\mathbf{b}$, and $\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$ is the Euclidean norm (length) of $\mathbf{a}$.

Derivation of Equation (1.14). Consider two vectors $\mathbf{a}$ and $\mathbf{b}$ with an angle θ between them. The vector $\mathbf{a} - \mathbf{b}$ forms the third side of a triangle. By the law of cosines:

$$\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2 \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta \quad (1.15)$$

Expanding the left side using the definition of the norm:

$$\|\mathbf{a} - \mathbf{b}\|^2 = (\mathbf{a} - \mathbf{b})^T(\mathbf{a} - \mathbf{b}) \quad (1.16)$$

$$= \mathbf{a}^T \mathbf{a} - 2\mathbf{a}^T \mathbf{b} + \mathbf{b}^T \mathbf{b} \quad (1.17)$$

$$= \|\mathbf{a}\|^2 - 2\mathbf{a}^T \mathbf{b} + \|\mathbf{b}\|^2 \quad (1.18)$$

Equating the two expressions:

$$\|\mathbf{a}\|^2 - 2\mathbf{a}^T \mathbf{b} + \|\mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\|\mathbf{a}\| \|\mathbf{b}\| \cos \theta \quad (1.19)$$

Canceling $\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2$ from both sides and dividing by -2 :

$$\mathbf{a}^T \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta \quad \square \quad (1.20)$$

$\square$

Key consequences of the dot product formula:

- If $\theta = 0$ (parallel vectors): $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\|$ (maximum positive value).
- If $\theta = 90^\circ$ (perpendicular vectors): $\mathbf{a} \cdot \mathbf{b} = 0$. This is the fundamental test for **orthogonality**.
- If $\theta = 180^\circ$ (antiparallel vectors): $\mathbf{a} \cdot \mathbf{b} = -\|\mathbf{a}\| \|\mathbf{b}\|$ (maximum negative value).

In control theory, the dot product appears constantly: kinetic energy is $E = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$, optimal control costs are $J = \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}$, and projecting forces onto joint axes uses $\tau_i = \mathcal{F}^T \mathcal{S}_i$. In each case, a dot product computes a scalar “alignment” between a vector and a reference direction.

Orthogonal Projection

A common operation is projecting one vector onto another. The **scalar projection** of $\mathbf{b}$ onto $\mathbf{a}$ is:

$$\text{proj}_{\mathbf{a}} \mathbf{b} = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a} \cdot \mathbf{a}} \mathbf{a} = \frac{\mathbf{a}^T \mathbf{b}}{\|\mathbf{a}\|^2} \mathbf{a} \quad (1.21)$$

The residual $\mathbf{b} - \text{proj}_{\mathbf{a}} \mathbf{b}$ is orthogonal to $\mathbf{a}$. This decomposition—splitting a vector into a component along a direction and a component perpendicular to it—is the geometric core of the Gram-Schmidt process and underlies the QR decomposition used in numerical linear algebra.

1.3.2 The Cross Product and the Skew-Symmetric Matrix

The cross product is a vector operation unique to three dimensions. It takes two vectors and generates a third vector that is perfectly perpendicular to both of them.

Definition 1.9. Cross Product

For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$, the *cross product* is:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{bmatrix} \quad (1.22)$$

The direction of $\mathbf{a} \times \mathbf{b}$ is determined by the *right-hand rule*: curl the fingers of your right hand from $\mathbf{a}$ toward $\mathbf{b}$; your thumb points in the direction of the cross product. The magnitude satisfies:

$$\|\mathbf{a} \times \mathbf{b}\| = \|\mathbf{a}\| \|\mathbf{b}\| \sin(\theta) \quad (1.23)$$

which equals the area of the parallelogram spanned by $\mathbf{a}$ and $\mathbf{b}$.

The cross product is *anti-commutative*: $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$. This has direct physical consequences. If you apply a force $\mathbf{F}$ at a displacement $\mathbf{r}$ from a pivot, the resulting torque is $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$. Reversing the order reverses the torque direction.

Crucially, the cross product can be rewritten as a matrix multiplication using the *skew-symmetric matrix*:

$$[\mathbf{a}_\times] = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \implies \mathbf{a} \times \mathbf{b} = [\mathbf{a}_\times] \mathbf{b} \quad (1.24)$$

Proposition 1. *The skew-symmetric matrix $[\mathbf{a}_\times]$ satisfies:*

- (i) $[\mathbf{a}_\times]^T = -[\mathbf{a}_\times]$ (*skew-symmetry; equivalently, all diagonal entries are zero and the off-diagonal entries are negated across the diagonal*).
- (ii) $\mathbf{a}^T [\mathbf{a}_\times] \mathbf{b} = 0$ for all $\mathbf{b}$ (*the cross product is always perpendicular to both input vectors*).
- (iii) *The eigenvalues of $[\mathbf{a}_\times]$ are $\{0, +i\|\mathbf{a}\|, -i\|\mathbf{a}\|\}$ (purely imaginary, reflecting the rotational nature of the cross product).*

Proof. (i) is immediate from inspection of Equation (4.16): transposing negates every off-diagonal element while the diagonal remains zero.

(ii) We compute $\mathbf{a}^T ([\mathbf{a}_\times] \mathbf{b}) = \mathbf{a}^T (\mathbf{a} \times \mathbf{b})$. Since $\mathbf{a} \times \mathbf{b}$ is perpendicular to $\mathbf{a}$ (by the geometric definition of the cross product), the dot product is zero.

(iii) The characteristic polynomial $\det([\mathbf{a}_\times] - \lambda \mathbf{I}) = -\lambda^3 - \|\mathbf{a}\|^2 \lambda = -\lambda(\lambda^2 + \|\mathbf{a}\|^2)$. The roots are $\lambda = 0$ and $\lambda = \pm i\|\mathbf{a}\|$. $\square$

This skew-symmetric matrix representation is absolutely pivotal for Chapters 4 and 6, where we show that the Lie Algebra $\mathfrak{so}(3)$ —the tangent space to the rotation group—consists entirely of 3×3 skew-symmetric matrices. Angular velocities live in this space.

The Scalar Triple Product

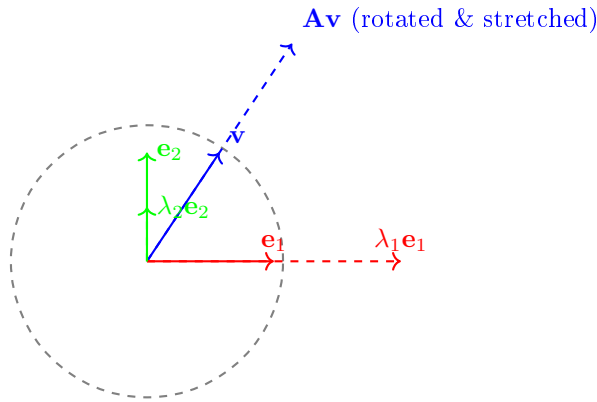
The scalar triple product $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$ computes the signed volume of the parallelepiped formed by three vectors $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$. It can be computed as a determinant:

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \det \begin{bmatrix} a_x & b_x & c_x \\ a_y & b_y & c_y \\ a_z & b_z & c_z \end{bmatrix} \quad (1.25)$$

If this determinant is zero, the three vectors are coplanar and linearly dependent—a direct connection between the cross product and the rank of the matrix formed by stacking the vectors as columns.

1.4 Eigenvalues, Eigenvectors, and Quadratic Forms

When you multiply a general vector by a matrix, the resulting vector typically changes both its magnitude and its direction. However, for almost every square matrix, there exist a few special directions that do *not* change their orientation when the matrix acts upon them. They are only stretched, shrunk, or flipped. These invariant directions reveal the deepest structural properties of the linear transformation.



Eigenvectors are only scaled, not rotated

Figure 1.2: Geometric interpretation of eigenvectors: they are special directions that are only stretched or shrunk by a matrix, never rotated. Dashed arrows show $\mathbf{A}\mathbf{v}$ for various vectors $\mathbf{v}$.

1.4.1 The Eigenvalue Equation

Definition 1.10. Eigenvalues and Eigenvectors

Given a square matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$, a nonzero vector $\mathbf{v} \in \mathbb{R}^n$ is called an *eigenvector* of $\mathbf{M}$ if there exists a scalar λ (possibly complex) such that:

$$\mathbf{M}\mathbf{v} = \lambda\mathbf{v} \quad (1.26)$$

The scalar λ is the corresponding *eigenvalue*. The requirement that $\mathbf{v} \neq \mathbf{0}$ is essential; otherwise the equation would be trivially satisfied for any λ .

Deriving the Characteristic Polynomial

How do we actually find the eigenvalues? We rearrange Equation (1.26):

$$\mathbf{M}\mathbf{v} - \lambda\mathbf{v} = \mathbf{0} \implies (\mathbf{M} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0} \quad (1.27)$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. For a nonzero solution $\mathbf{v}$ to exist, the matrix $(\mathbf{M} - \lambda\mathbf{I})$ must be singular. By our earlier discussion, this means its determinant must vanish:

$$\det(\mathbf{M} - \lambda\mathbf{I}) = 0 \quad (1.28)$$

This is the **characteristic equation**. Expanding the determinant produces a polynomial of degree n in λ , called the **characteristic polynomial**. By the Fundamental Theorem of Algebra, this polynomial has exactly n roots (counted with multiplicity, possibly complex), giving us n eigenvalues.

Example 1.3. Computing Eigenvalues of a 2×2 Matrix

Consider the dynamics matrix of a simple spring system:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -4 & -5 \end{bmatrix} \quad (1.29)$$

Step 1: Form $\mathbf{A} - \lambda\mathbf{I}$:

$$\mathbf{A} - \lambda\mathbf{I} = \begin{bmatrix} -\lambda & 1 \\ -4 & -5 - \lambda \end{bmatrix} \quad (1.30)$$

Step 2: Compute the determinant and set it to zero:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = (-\lambda)(-5 - \lambda) - (1)(-4) \quad (1.31)$$

$$= \lambda^2 + 5\lambda + 4 \quad (1.32)$$

$$= (\lambda + 1)(\lambda + 4) = 0 \quad (1.33)$$

Step 3: The eigenvalues are $\lambda_1 = -1$ and $\lambda_2 = -4$. Both are negative, meaning this system is **asymptotically stable**—any perturbation decays exponentially back to equilibrium. The mode associated with $\lambda_1 = -1$ decays slowly (time constant $\tau_1 = 1$ second), while the mode associated with $\lambda_2 = -4$ decays four times faster ($\tau_2 = 0.25$ seconds).

Step 4: Find the eigenvectors. For $\lambda_1 = -1$, solve $(\mathbf{A} + \mathbf{I})\mathbf{v}_1 = \mathbf{0}$:

$$\begin{bmatrix} 1 & 1 \\ -4 & -4 \end{bmatrix} \mathbf{v}_1 = \mathbf{0} \implies \mathbf{v}_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad (1.34)$$

For $\lambda_2 = -4$, solve $(\mathbf{A} + 4\mathbf{I})\mathbf{v}_2 = \mathbf{0}$:

$$\begin{bmatrix} 4 & 1 \\ -4 & -1 \end{bmatrix} \mathbf{v}_2 = \mathbf{0} \implies \mathbf{v}_2 = \begin{bmatrix} 1 \\ -4 \end{bmatrix} \quad (1.35)$$

1.4.2 Physical Interpretation of Eigenvalues

1. **Inertia tensors and principal axes.** If $\mathbf{M}$ represents the mass-and-inertia tensor of a tumbling satellite, its eigenvectors identify the **Principal Axes**—the natural

directions where the satellite spins stably. The eigenvalues give the moments of inertia about those axes. The famous “tennis racket theorem” (or Dzhanibekov effect) states that rotation about the axis with the *intermediate* eigenvalue is unstable, while rotation about the axes with the largest and smallest eigenvalues is stable.

2. **Stability of dynamical systems.** If $\mathbf{A}$ is the linearized dynamics matrix of a control system ($\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$), the eigenvalues dictate stability:

- $\operatorname{Re}(\lambda_i) < 0$ for all i : the system is **asymptotically stable** (all perturbations decay).
- $\operatorname{Re}(\lambda_i) > 0$ for any i : the system is **unstable** (perturbations grow exponentially).
- $\operatorname{Re}(\lambda_i) = 0$: the system is **marginally stable** (oscillations persist forever without growing or decaying).

Eigenvalues are the fundamental heartbeat of linear dynamics. Every analysis in Chapter 2 builds on this fact.

Example 1.4. Visualizing Eigenvectors: The Stretching Rubber Sheet

Imagine printing a perfect circle onto a rubber square. You grab the edges of the rubber and stretch it wildly in various directions. The circle distorts into an ellipse. Most straight lines drawn on the original circle have been both rotated and elongated. However, exactly two perpendicular lines—the major and minor axes of the resulting ellipse—did not rotate at all; they were only stretched. These non-rotating axes are your **eigenvectors**. The factors by which those specific lines were stretched (perhaps one stretched by $3\times$ and the other shrunk to $0.5\times$) are your **eigenvalues**. The matrix $\mathbf{M}$ is the algebraic instruction set for that stretching operation. In n dimensions, a symmetric matrix transforms a hypersphere into a hyperellipsoid. The eigenvectors point along the principal axes of the ellipsoid, and the eigenvalues are the semi-axis lengths.

1.4.3 The Spectral Theorem for Symmetric Matrices

Among all matrices, *symmetric* matrices ($\mathbf{M} = \mathbf{M}^T$) occupy a privileged position. They arise naturally throughout mechanics (mass matrices, stiffness matrices, inertia tensors) and control theory (cost matrices, Lyapunov functions).

Theorem 1.3. The Spectral Theorem

If $\mathbf{M} \in \mathbb{R}^{n \times n}$ is symmetric ($\mathbf{M} = \mathbf{M}^T$), then:

- All eigenvalues of $\mathbf{M}$ are real.
- Eigenvectors corresponding to distinct eigenvalues are orthogonal.
- $\mathbf{M}$ can be factored as $\mathbf{M} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$, where $\mathbf{Q}$ is an orthogonal matrix ($\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$) whose columns are the eigenvectors, and $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$ is a diagonal matrix of eigenvalues.

Proof that eigenvalues of a real symmetric matrix are real. Let $\mathbf{M} = \mathbf{M}^T$ and suppose $\mathbf{M}\mathbf{v} = \lambda\mathbf{v}$ for some possibly complex eigenvalue λ and eigenvector $\mathbf{v}$. We will show that λ must be real.

Take the complex conjugate transpose (denoted by $*$) of both sides of $\mathbf{M}\mathbf{v} = \lambda\mathbf{v}$:

$$\mathbf{v}^* \mathbf{M}^T = \bar{\lambda} \mathbf{v}^* \quad (1.36)$$

Since $\mathbf{M} = \mathbf{M}^T$ (and $\mathbf{M}$ is real, so $\mathbf{M}^* = \mathbf{M}^T = \mathbf{M}$):

$$\mathbf{v}^* \mathbf{M} = \bar{\lambda} \mathbf{v}^* \quad (1.37)$$

Now multiply the original equation on the left by $\mathbf{v}^*$:

$$\mathbf{v}^* \mathbf{M} \mathbf{v} = \lambda \mathbf{v}^* \mathbf{v} \quad (1.38)$$

And multiply the conjugated equation on the right by $\mathbf{v}$:

$$\mathbf{v}^* \mathbf{M} \mathbf{v} = \bar{\lambda} \mathbf{v}^* \mathbf{v} \quad (1.39)$$

Equating the right-hand sides:

$$\lambda \mathbf{v}^* \mathbf{v} = \bar{\lambda} \mathbf{v}^* \mathbf{v} \quad (1.40)$$

Since $\mathbf{v} \neq \mathbf{0}$, we have $\mathbf{v}^* \mathbf{v} = \sum_i |v_i|^2 > 0$. Dividing both sides by $\mathbf{v}^* \mathbf{v}$ yields $\lambda = \bar{\lambda}$, which is true if and only if λ is real. $\square$

Proof that eigenvectors for distinct eigenvalues are orthogonal. Let $\mathbf{M}\mathbf{v}_1 = \lambda_1 \mathbf{v}_1$ and $\mathbf{M}\mathbf{v}_2 = \lambda_2 \mathbf{v}_2$ with $\lambda_1 \neq \lambda_2$. Compute:

$$\lambda_1 (\mathbf{v}_1^T \mathbf{v}_2) = (\mathbf{M}\mathbf{v}_1)^T \mathbf{v}_2 = \mathbf{v}_1^T \mathbf{M}^T \mathbf{v}_2 = \mathbf{v}_1^T \mathbf{M} \mathbf{v}_2 = \mathbf{v}_1^T (\lambda_2 \mathbf{v}_2) = \lambda_2 (\mathbf{v}_1^T \mathbf{v}_2) \quad (1.41)$$

Therefore $(\lambda_1 - \lambda_2)(\mathbf{v}_1^T \mathbf{v}_2) = 0$. Since $\lambda_1 \neq \lambda_2$, we must have $\mathbf{v}_1^T \mathbf{v}_2 = 0$. The eigenvectors are orthogonal. $\square$

The spectral decomposition $\mathbf{M} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$ has a clear geometric interpretation: the transformation defined by $\mathbf{M}$ can be decomposed into three steps: (1) rotate coordinates to align with the eigenvector axes ($\mathbf{Q}^T$), (2) independently scale each axis by its eigenvalue ($\mathbf{\Lambda}$), (3) rotate back ($\mathbf{Q}$). This is a “rotation-stretch-rotation” decomposition entirely analogous to the SVD, which we discuss next.

1.4.4 Quadratic Forms and Positive Definiteness

In optimization and energy mechanics, we frequently encounter *quadratic forms*—scalar-valued functions that are quadratic in a vector argument:

Definition 1.11. Quadratic Form

Given a symmetric matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$, the associated *quadratic form* is the scalar function $q : \mathbb{R}^n \rightarrow \mathbb{R}$ defined by:

$$q(\mathbf{x}) = \mathbf{x}^T \mathbf{M} \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n M_{ij} x_i x_j \quad (1.42)$$

Quadratic forms appear throughout this text series:

- **Kinetic energy:** $T = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$, where $\mathbf{M}(\mathbf{q})$ is the mass matrix (Chapter 11).
- **Optimal control cost:** $J = \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}$, where $\mathbf{Q}$ penalizes state deviations and $\mathbf{R}$ penalizes control effort.
- **Lyapunov functions:** $V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}$, used to prove stability (Volume I).
- **Contraction metrics:** $\delta \mathbf{x}^T \mathbf{M}(x) \delta \mathbf{x}$, measuring infinitesimal distances on manifolds (Volume I).

Definition 1.12. Positive Definiteness

A symmetric matrix $\mathbf{M}$ is called:

- **Positive definite** ($\mathbf{M} \succ 0$) if $\mathbf{x}^T \mathbf{M} \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$.
- **Positive semi-definite** ($\mathbf{M} \succeq 0$) if $\mathbf{x}^T \mathbf{M} \mathbf{x} \geq 0$ for all $\mathbf{x}$.
- **Negative definite** ($\mathbf{M} \prec 0$) if $\mathbf{x}^T \mathbf{M} \mathbf{x} < 0$ for all $\mathbf{x} \neq \mathbf{0}$.
- **Indefinite** if $\mathbf{x}^T \mathbf{M} \mathbf{x}$ takes both positive and negative values.

Proposition 2. A symmetric matrix $\mathbf{M}$ is positive definite if and only if all of its eigenvalues are strictly positive.

Proof. By the Spectral Theorem (Theorem 1.3), $\mathbf{M} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T$ where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ and $\mathbf{Q}$ is orthogonal. For any nonzero $\mathbf{x}$, define $\mathbf{y} = \mathbf{Q}^T \mathbf{x}$. Since $\mathbf{Q}$ is invertible and $\mathbf{x} \neq \mathbf{0}$, we have $\mathbf{y} \neq \mathbf{0}$. Then:

$$\mathbf{x}^T \mathbf{M} \mathbf{x} = \mathbf{x}^T \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T \mathbf{x} = \mathbf{y}^T \mathbf{\Lambda} \mathbf{y} = \sum_{i=1}^n \lambda_i y_i^2 \quad (1.43)$$

($\Rightarrow$) If $\mathbf{M} \succ 0$, then for each eigenvector $\mathbf{v}_i$ (which is nonzero): $\mathbf{v}_i^T \mathbf{M} \mathbf{v}_i = \lambda_i \|\mathbf{v}_i\|^2 > 0$, so $\lambda_i > 0$.

($\Leftarrow$) If all $\lambda_i > 0$ and $\mathbf{y} \neq \mathbf{0}$, then at least one $y_i \neq 0$, so $\sum_i \lambda_i y_i^2 > 0$. $\square$ $\square$

In Plain Language 1.5. Why Positive Definiteness Matters for Control

In control theory, if you can construct a function $V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}$ with $\mathbf{P} \succ 0$ such that V decreases along every trajectory of your system, you have mathematically

proven that the system is stable. The positive definiteness of $\mathbf{P}$ guarantees that V has a strict minimum at the origin—it forms a “bowl” in state space. If the system’s energy (V) always decreases and the bowl has a definite bottom, the system must converge to rest. This is the essence of **Lyapunov stability theory**, the single most important stability analysis tool in nonlinear control (developed in Volume I).

The Cholesky Decomposition

A practical consequence of positive definiteness: any positive definite matrix $\mathbf{M}$ can be uniquely factored as:

$$\mathbf{M} = \mathbf{L}\mathbf{L}^T \quad (1.44)$$

where $\mathbf{L}$ is a lower triangular matrix with strictly positive diagonal entries. This is the **Cholesky decomposition**. It is numerically efficient (roughly half the cost of a general LU factorization) and is the standard method for solving linear systems involving positive definite matrices in physics engines. When MuJoCo inverts the mass matrix $\mathbf{M}(\mathbf{q})$ to compute forward dynamics, it uses Cholesky decomposition.

The Cholesky factor $\mathbf{L}$ also provides a constructive test for positive definiteness: if the factorization completes without encountering a zero or negative diagonal element, the matrix is positive definite. If the algorithm fails, the matrix is not positive definite.

1.5 The Singular Value Decomposition (SVD)

What if a matrix is not square? What if it does not have a full set of eigenvectors? The **Singular Value Decomposition** (SVD) is the ultimate, universal factorization of *any* matrix $\mathbf{M} \in \mathbb{R}^{n \times m}$, regardless of its shape or rank.

Theorem 1.4. The Singular Value Decomposition

Every matrix $\mathbf{M} \in \mathbb{R}^{n \times m}$ can be factored as:

$$\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \quad (1.45)$$

where:

- $\mathbf{U} \in \mathbb{R}^{n \times n}$ is an orthogonal matrix ($\mathbf{U}^T\mathbf{U} = \mathbf{I}$). Its columns are called the *left singular vectors*.
- $\mathbf{\Sigma} \in \mathbb{R}^{n \times m}$ is a diagonal matrix with non-negative entries $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(n,m)} \geq 0$ on the diagonal. These are the *singular values*.
- $\mathbf{V} \in \mathbb{R}^{m \times m}$ is an orthogonal matrix. Its columns are the *right singular vectors*.

1.5.1 Derivation of the SVD from the Eigendecomposition

The SVD can be derived from the spectral theorem applied to related symmetric matrices. Here is the construction:

Step 1. Form the symmetric matrix $\mathbf{M}^T\mathbf{M} \in \mathbb{R}^{m \times m}$. This matrix is always positive semi-definite because for any $\mathbf{x}$:

$$\mathbf{x}^T(\mathbf{M}^T\mathbf{M})\mathbf{x} = (\mathbf{M}\mathbf{x})^T(\mathbf{M}\mathbf{x}) = \|\mathbf{M}\mathbf{x}\|^2 \geq 0 \quad (1.46)$$

Step 2. By the Spectral Theorem, $\mathbf{M}^T\mathbf{M}$ has an eigendecomposition:

$$\mathbf{M}^T\mathbf{M} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T \quad (1.47)$$

where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_m)$ with $\lambda_i \geq 0$ (since $\mathbf{M}^T\mathbf{M}$ is positive semi-definite). Define the singular values as $\sigma_i = \sqrt{\lambda_i}$.

Step 3. For each nonzero singular value σ_i , define the corresponding left singular vector:

$$\mathbf{u}_i = \frac{1}{\sigma_i} \mathbf{M}\mathbf{v}_i \quad (1.48)$$

These vectors are orthonormal (a straightforward verification using $\mathbf{M}^T\mathbf{M}\mathbf{v}_i = \sigma_i^2\mathbf{v}_i$).

Step 4. Extend $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ to a complete orthonormal basis for $\mathbb{R}^n$ (where $r = \text{rank}(\mathbf{M})$). Assemble these into the columns of $\mathbf{U}$. Then $\mathbf{M} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ follows by construction.

1.5.2 Geometric Interpretation of the SVD

The SVD reveals that *any* linear transformation, no matter how complex, can be decomposed into exactly three simple geometric operations:

1. **Rotate the input space** (apply $\mathbf{V}^T$): align the input with the principal directions.
2. **Scale each axis independently** (apply $\mathbf{\Sigma}$): stretch or shrink along each principal direction.
3. **Rotate the output space** (apply $\mathbf{U}$): reorient the result in the output space.

This geometric decomposition transforms a unit hypersphere in the input space into a hyperellipsoid in the output space. The singular values are the semi-axis lengths of this ellipsoid, and the left singular vectors point along the ellipsoid's principal axes.

1.5.3 The Condition Number

The **condition number** of a matrix $\mathbf{M}$ is the ratio of its largest to its smallest singular value:

$$\kappa(\mathbf{M}) = \frac{\sigma_{\max}}{\sigma_{\min}} \quad (1.49)$$

If $\kappa(\mathbf{M})$ is large (say, 10^6 or more), the matrix is *ill-conditioned*: small perturbations in the input can cause enormous changes in the output. In robotics, the condition number of the Jacobian quantifies how close the robot is to a singularity. As $\sigma_{\min} \rightarrow 0$, the condition number $\rightarrow \infty$, and the numerical solution of $\mathbf{J}\dot{\mathbf{q}} = \mathbf{v}$ becomes catastrophically unreliable.

1.5.4 Truncated SVD and Low-Rank Approximation

One of the most powerful applications of the SVD is data compression. Given a matrix $\mathbf{M}$ with singular values $\sigma_1 \geq \sigma_2 \geq \dots$, the best rank- k approximation (in the Frobenius norm) is:

$$\mathbf{M}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T \quad (1.50)$$

This result (the Eckart-Young-Mirsky theorem) underpins dimensionality reduction techniques like Principal Component Analysis (PCA), which identifies the dominant modes of variation in high-dimensional data such as motion capture recordings of golf swings.

Example 1.5. The SVD of the Jacobian (Manipulability Ellipsoid)

In robotics, the Jacobian matrix $\mathbf{J}(\mathbf{q})$ maps joint velocities to end-effector Cartesian velocity: $\mathbf{v}_{\text{hand}} = \mathbf{J}\dot{\mathbf{q}}$. For a redundant manipulator with 7 joints moving in 3D space, $\mathbf{J} \in \mathbb{R}^{3 \times 7}$. Taking the SVD $\mathbf{J} = \mathbf{U}\Sigma\mathbf{V}^T$:

- $\mathbf{U} \in \mathbb{R}^{3 \times 3}$: The columns define the physical 3D directions the hand can move.
- $\Sigma \in \mathbb{R}^{3 \times 7}$: The singular values $(\sigma_1, \sigma_2, \sigma_3)$ are the semi-axis lengths of the **Velocity Manipulability Ellipsoid**. They quantify how fast the robot can move in each $\mathbf{U}$ -direction. If the arm is stretched perfectly straight, one singular value drops to $\sigma_k = 0$: the ellipsoid collapses to a pancake, and the robot loses a degree of freedom.
- $\mathbf{V}^T \in \mathbb{R}^{7 \times 7}$: The columns define the joint velocity combinations required to produce motion along each principal direction.

Conversely, the **Force Manipulability Ellipsoid** has axes proportional to $1/\sigma_i$. Where the robot is fastest (σ_i large), it is mechanically weakest. Where it is slowest (near singularities, σ_i small), it can exert enormous force—a consequence of the conservation of mechanical power.

1.6 Dyads, Dyadics, and Tensor Products

The matrix decompositions we have studied so far—eigendecomposition, SVD—break a matrix into pieces that reveal its geometric action. In this section we meet a complementary idea: building matrices up from the simplest possible pieces, called *dyads*. This viewpoint is not merely algebraic; it is the language that mechanics has used for over a century to describe inertia, stress, and stiffness without ever choosing a coordinate frame [23].

1.6.1 What Is a Dyad?

Definition 1.13. Dyad (Outer Product)

Given two vectors $\mathbf{a} \in \mathbb{R}^m$ and $\mathbf{b} \in \mathbb{R}^n$, the **dyad** (or **outer product**) is the $m \times n$ matrix

$$\mathbf{D} = \mathbf{a}\mathbf{b}^T.$$

This matrix has rank at most one: every column is a scalar multiple of $\mathbf{a}$, and every row is a scalar multiple of $\mathbf{b}^T$.

A dyad encodes *two* directions simultaneously. When it acts on a vector $\mathbf{v}$,

$$\mathbf{D}\mathbf{v} = (\mathbf{a}\mathbf{b}^T)\mathbf{v} = \mathbf{a} \langle \mathbf{b}, \mathbf{v} \rangle,$$

it first *senses* how much $\mathbf{v}$ aligns with $\mathbf{b}$ (via the inner product $\langle \mathbf{b}, \mathbf{v} \rangle$) and then *responds* along the direction $\mathbf{a}$, scaled by that alignment.

In Plain Language 1.6. Dyads: Direction of Action and Direction of Sensitivity

Imagine a spring that is stiff in one direction but compliant in another—say, a car suspension that resists vertical bumps but allows lateral flex. The relationship “sense displacement in direction $\mathbf{b}$, produce force along direction $\mathbf{a}$ ” is exactly a dyad $\mathbf{a}\mathbf{b}^T$. Think of it as a *cause-and-effect pair*: the vector $\mathbf{b}$ is the “listening direction” (what the spring feels) and $\mathbf{a}$ is the “acting direction” (how the spring pushes back). A single dyad can only do one such pairing, which is why its matrix has rank one—it has only one independent cause-to-effect channel.

Example 1.6. A Rank-1 Force Model

Let $\mathbf{a} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$ (vertical) and $\mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ (horizontal). Then

$$\mathbf{D} = \mathbf{a}\mathbf{b}^T = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Applied to a displacement $\mathbf{v} = \begin{bmatrix} 3 \\ 5 \\ 2 \end{bmatrix}$:

$$\mathbf{D}\mathbf{v} = \begin{bmatrix} 0 \\ 3 \\ 0 \end{bmatrix}.$$

The dyad extracts the horizontal component (3) and produces a purely vertical response—exactly the one-channel behaviour described above.

1.6.2 From Dyads to Dyadics

A single dyad is rank one, but real physical relationships—stiffness, inertia, stress—have full rank. We get there by *summing* dyads.

Definition 1.14. Dyadic (Tensor Product Expansion)

A **dyadic** is a finite sum of dyads:

$$\mathbf{T} = \sum_{k=1}^r \mathbf{a}_k \mathbf{b}_k^T.$$

Every $m \times n$ matrix can be written in this form with $r \leq \min(m, n)$ terms.

This is not a new fact—it is the column–row factorisation you already know from the SVD. If $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, then

$$\mathbf{A} = \sum_{k=1}^r \sigma_k \mathbf{u}_k \mathbf{v}_k^T,$$

which is a dyadic with r terms (the rank of $\mathbf{A}$). The SVD merely chooses the *orthogonal* dyad decomposition; infinitely many other decompositions exist.

The connection to abstract algebra: the space $\mathbb{R}^m \otimes \mathbb{R}^n$ (the **tensor product**) is isomorphic to $\mathbb{R}^{m \times n}$. A dyad $\mathbf{a} \mathbf{b}^T$ corresponds to the elementary tensor $\mathbf{a} \otimes \mathbf{b}$, and a general element of the tensor product space is a sum of such elementary tensors—a dyadic.

1.6.3 Physical Meaning in Mechanics

The power of the dyadic viewpoint is that it names the two “slots” of a matrix: input direction and output direction. Three central objects in mechanics are dyadics.

The inertia tensor. The rotational inertia of a rigid body maps angular velocity to angular momentum:

$$\mathbf{L} = \mathbf{I}\boldsymbol{\omega}. \quad (1.51)$$

Here $\boldsymbol{\omega}$ is the “input” (the axis and rate of spin) and $\mathbf{L}$ is the “output” (the direction and magnitude of angular momentum). Unless the body has special symmetry, $\mathbf{L}$ and $\boldsymbol{\omega}$ point in different directions; the inertia tensor is the dyadic that encodes this directional coupling.

The stress tensor. The Cauchy stress tensor $\boldsymbol{\sigma}$ maps a surface-normal direction $\hat{\mathbf{n}}$ to the traction (force per unit area) $\mathbf{t}$ acting on that surface:

$$\mathbf{t} = \boldsymbol{\sigma} \hat{\mathbf{n}}.$$

Input: the orientation of a small area element. Output: the force that the material on one side exerts on the other.

The stiffness matrix. In structural mechanics, the stiffness matrix $\mathbf{K}$ maps a displacement vector $\mathbf{u}$ to a force vector $\mathbf{f}$: $\mathbf{f} = \mathbf{K} \mathbf{u}$. For a truss with N members, each member contributes a rank-1 dyad (stiffness along its axis), and the global stiffness matrix is the sum:

$$\mathbf{K} = \sum_{e=1}^N k_e \hat{\mathbf{n}}_e \hat{\mathbf{n}}_e^T,$$

a dyadic built from the member directions $\hat{\mathbf{n}}_e$ and axial stiffnesses k_e .

In Plain Language 1.7. Why Engineers Say “Tensor”

When a mechanical engineer refers to the “inertia tensor” or “stress tensor,” they almost always mean a 3×3 symmetric matrix that represents a dyadic. The word *tensor* signals that the object transforms in a specific, predictable way under changes of coordinate frame. It is not just a bag of nine numbers; it is a physical machine that takes one vector in and produces another vector out, regardless of how you choose your axes.

1.6.4 The Coordinate-Free Perspective

A dyadic is a geometric object that lives in the space of linear maps $\mathbb{R}^n \rightarrow \mathbb{R}^m$. Choosing a basis gives it a matrix representation, but the underlying map does not depend on that choice.

Example 1.7. Inertia Tensor of a Point Mass

A point mass m at position $\mathbf{r}$ from the centre of mass has inertia tensor

$$\mathbf{I} = m(\|\mathbf{r}\|^2 \mathbf{1} - \mathbf{r} \mathbf{r}^T), \quad (1.52)$$

where $\mathbf{1}$ is the 3×3 identity. Notice that no basis vectors appear— $\mathbf{r} \mathbf{r}^T$ is a dyad formed from the position vector with itself, and $\mathbf{1}$ is the identity map. This expression is *coordinate-free*: it is valid in every frame simultaneously.

When we change from a body-fixed frame $\{B\}$ to a world frame $\{W\}$ via a rotation $\mathbf{R} \in \text{SO}(3)$, the matrix representation of the inertia tensor transforms by the **congruence transformation**:

$$\mathbf{I}^W = \mathbf{R} \mathbf{I}^B \mathbf{R}^T. \quad (1.53)$$

The congruence form $\mathbf{R}(\cdot)\mathbf{R}^T$ rotates both the input slot and the output slot of the dyadic. It preserves symmetry, positive-definiteness, and—for rotations—eigenvalues (the principal moments of inertia).

1.6.5 Connection to the Rest of This Book

The dyadic point of view recurs throughout our study of rigid-body mechanics:

- **Screw axes and wrenches (Chapter 5).** A wrench combines a force and a moment into a single six-dimensional object. The mapping from a twist (generalised velocity) to a wrench (generalised force) is a 6×6 dyadic—the *spatial inertia*.

- **Spatial algebra (Chapter 8).** The 6×6 spatial inertia matrix is a dyadic on the space of twists. Its coordinate-free formulation uses the same $m(\|\mathbf{r}\|^2 \mathbf{1} - \mathbf{r} \mathbf{r}^T)$ building block we saw in Example 1.7, extended to six dimensions.
- **The mass matrix (Chapter 11).** The joint-space mass matrix $\mathbf{M}(\mathbf{q})$ that appears in the manipulator equation $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{c} = \boldsymbol{\tau}$ is a configuration-dependent dyadic on joint-velocity space. Each link contributes a term built from Jacobians and spatial inertias—a sum of dyads, assembled exactly as we sum member stiffnesses in a truss.
- **The articulated-body algorithm (Chapter 10).** Featherstone’s algorithm propagates *articulated-body inertias* through a kinematic tree. These are 6×6 dyadics that account for the coupling between a link’s motion and the reactions of all its descendants.

Understanding that all these objects are dyadics—linear maps built from sums of outer products—provides a unifying thread: the same algebraic structure appears whether we are analysing a truss, spinning a gyroscope, or computing the forward dynamics of a humanoid robot.

1.7 Matrix Norms and Conditioning

Having decomposed matrices via the eigendecomposition and SVD, we now turn to a fundamental practical question: *how large is a matrix, and how sensitive is a linear system to perturbations?* Matrix norms quantify the first question; the *condition number* answers the second.

Definition 1.15. Frobenius Norm $\|\mathbf{A}\|_F$

The **Frobenius norm** of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(\mathbf{A}^T \mathbf{A})} = \sqrt{\sigma_1^2 + \sigma_2^2 + \cdots + \sigma_{\min(m,n)}^2}. \quad (1.54)$$

The last equality follows directly from the SVD. The Frobenius norm treats the matrix as a long vector and computes its Euclidean length.

Definition 1.16. Spectral Norm $\|\mathbf{A}\|_2$

The **spectral norm** (also called the *operator 2-norm* or *induced 2-norm*) is

$$\|\mathbf{A}\|_2 = \max_{\|\mathbf{x}\|_2=1} \|\mathbf{A}\mathbf{x}\|_2 = \sigma_{\max}(\mathbf{A}). \quad (1.55)$$

This is the maximum factor by which $\mathbf{A}$ can stretch a unit vector—exactly the length of the longest semi-axis of the image ellipsoid from the SVD’s geometric interpretation (Section 1.5).

Definition 1.17. Condition Number $\kappa(\mathbf{A})$

For a matrix $\mathbf{A}$ with $\sigma_{\min} > 0$, the **condition number** with respect to the 2-norm is

$$\kappa(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}}. \quad (1.56)$$

When $\sigma_{\min} = 0$ (i.e., $\mathbf{A}$ is singular), we define $\kappa(\mathbf{A}) = \infty$. A well-conditioned matrix has $\kappa \approx 1$; an ill-conditioned matrix has $\kappa \gg 1$.

Physical interpretation: Jacobian conditioning near singularities. Consider the manipulator Jacobian $\mathbf{J}(\mathbf{q})$ from the SVD example (Example 1.5). When the robot approaches a kinematic singularity, one or more singular values $\sigma_i \rightarrow 0$ while others remain bounded. The condition number $\kappa(\mathbf{J})$ therefore grows without bound. Because the solution to $\dot{\mathbf{q}} = \mathbf{J}^{-1}\mathbf{v}$ amplifies input errors by a factor proportional to κ , even tiny errors in the desired Cartesian velocity $\mathbf{v}$ produce enormous joint velocities. This is why real robots exhibit jerky, unstable motion near singular configurations [33].

In Plain Language 1.8. Why Does Your Robot Arm Shake Near Singularities?

Why does your robot arm shake near singularities? Because the condition number of the Jacobian explodes. When $\kappa(\mathbf{J})$ is huge, the inverse mapping from Cartesian velocity to joint velocity amplifies every small measurement error, rounding error, and servo lag into wild joint commands. Monitoring $\kappa(\mathbf{J})$ in real time is standard practice in industrial controllers: when the condition number exceeds a threshold, the controller switches to a damped pseudo-inverse to keep the motion smooth.

1.8 Matrix Decompositions for Computation

The eigendecomposition and SVD reveal geometric structure. This section surveys three additional decompositions whose primary purpose is *computational efficiency* when solving the linear systems that arise throughout dynamics and control.

1.8.1 LU Decomposition

Any invertible matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ can (with row permutations) be factored as

$$\mathbf{PA} = \mathbf{LU}, \quad (1.57)$$

where $\mathbf{P}$ is a permutation matrix, $\mathbf{L}$ is unit lower triangular, and $\mathbf{U}$ is upper triangular. Solving $\mathbf{Ax} = \mathbf{b}$ then reduces to two triangular solves (forward and back substitution), each costing $O(n^2)$ after the $O(n^3)$ factorization. When multiple right-hand sides share the same $\mathbf{A}$, LU is the workhorse: factor once, solve many times.

1.8.2 QR Decomposition

For any $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$,

$$\mathbf{A} = \mathbf{QR}, \quad (1.58)$$

where $\mathbf{Q} \in \mathbb{R}^{m \times m}$ is orthogonal and $\mathbf{R} \in \mathbb{R}^{m \times n}$ is upper triangular. QR is the preferred method for solving least-squares problems $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2$ because, unlike the normal equations $\mathbf{A}^T \mathbf{Ax} = \mathbf{A}^T \mathbf{b}$, it avoids squaring the condition number ($\kappa(\mathbf{A}^T \mathbf{A}) = \kappa(\mathbf{A})^2$) and is therefore far more numerically stable.

1.8.3 Cholesky Decomposition

If $\mathbf{M} \in \mathbb{R}^{n \times n}$ is symmetric positive definite (SPD), then there exists a unique lower-triangular matrix $\mathbf{L}$ with positive diagonal entries such that

$$\mathbf{M} = \mathbf{LL}^T. \quad (1.59)$$

Cholesky costs roughly half the flops of LU and is always numerically stable for SPD matrices—no pivoting is needed.

Mathematical Principle 1.1. Cholesky and the Mass Matrix

The generalized mass matrix $\mathbf{M}(\mathbf{q})$ that appears in the manipulator equation of motion (Chapter 11) is always symmetric positive definite. Therefore the Cholesky decomposition is the preferred solver for the forward-dynamics equation $\mathbf{M}\ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{C}\dot{\mathbf{q}} - \mathbf{g}$ [24].

1.9 Worked Example: Eigenvalue and SVD Analysis in Python

Let us bring together the theory developed in this chapter with a concrete computational example. The following Python code constructs a 3×3 symmetric matrix (representing a simplified 3D inertia tensor), computes its eigendecomposition and SVD, and verifies the key properties we have proven.

```

1 import numpy as np
2
3 # ---- Define a symmetric positive definite matrix ----
4 # This could represent a simplified 3D inertia tensor
5 M = np.array([
6     [5.0, 1.0, 0.5],
7     [1.0, 4.0, 0.3],
8     [0.5, 0.3, 3.0]
9 ])
10
11 # Verify symmetry
12 assert np.allclose(M, M.T), "Matrix is not symmetric!"
13
14 # ---- Eigendecomposition ----
15 eigenvalues, eigenvectors = np.linalg.eigh(M) # eigh for symmetric matrices
16 print("Eigenvalues:", np.round(eigenvalues, 4))
17 print("Eigenvectors (columns):\n", np.round(eigenvectors, 4))
18
19 # Verify: all eigenvalues positive => positive definite
20 assert all(eigenvalues > 0), "Matrix is not positive definite!"
21 print("Matrix is POSITIVE DEFINITE (all eigenvalues > 0)")
22

```



```

23 # Verify: eigenvectors are orthogonal
24 QTQ = eigenvectors.T @ eigenvectors
25 print("Q^T Q (should be identity):\n", np.round(QTQ, 10))
26
27 # Verify spectral decomposition: M = Q * Lambda * Q^T
28 M_reconstructed = eigenvectors @ np.diag(eigenvalues) @ eigenvectors.T
29 print("Reconstruction error:", np.linalg.norm(M - M_reconstructed))
30
31 # ---- Singular Value Decomposition ----
32 U, sigma, VT = np.linalg.svd(M)
33 print("\nSingular values:", np.round(sigma, 4))
34 print("Eigenvalues (sorted descending):", np.round(sorted(eigenvalues,
    reverse=True), 4))
35 print("For symmetric PD matrices, singular values = eigenvalues (sorted)")
36
37 # ---- Condition Number ----
38 kappa = sigma[0] / sigma[-1]
39 print(f"\nCondition number: {kappa:.4f}")
40 print("(Close to 1 means well-conditioned; >>1 means ill-conditioned)")
41
42 # ---- Cholesky Decomposition ----
43 L = np.linalg.cholesky(M)
44 print("\nCholesky factor L:\n", np.round(L, 4))
45 print("Verify L @ L^T = M, error:", np.linalg.norm(M - L @ L.T))

```

Listing 1.1: Eigendecomposition and SVD of a Symmetric Matrix

1.10 Chapter Summary

Mathematical Principle 1.2. Key Takeaways from Chapter 1

1. A **vector space** is defined by eight axioms guaranteeing closure, commutativity, and compatibility of addition and scalar multiplication. $\mathbb{R}^n$ is the prototypical example.
2. The **rank** of a matrix counts independent columns; the **null space** captures the “lost” dimensions. The **Rank-Nullity Theorem** connects them: rank + nullity = number of columns.
3. The **dot product** measures alignment between vectors and defines orthogonality. The **cross product** in $\mathbb{R}^3$ produces a perpendicular vector and can be written as a *skew-symmetric matrix* multiplication—a representation central to Lie Algebra ($\mathfrak{so}(3)$).
4. **Eigenvalues** of a dynamics matrix dictate stability; eigenvalues of an inertia tensor identify principal axes. The **Spectral Theorem** guarantees that symmetric matrices have real eigenvalues and orthogonal eigenvectors.
5. **Positive definiteness** ($\mathbf{M} \succ 0$) guarantees a quadratic form has a strict minimum—the cornerstone of Lyapunov stability proofs and energy-based control design.
6. The **SVD** decomposes any matrix into rotation-stretch-rotation. It reveals the

manipulability ellipsoid of a robot and quantifies proximity to singularities via the condition number.

1.11 Further Reading

The material in this chapter draws on a long tradition of textbooks at the intersection of linear algebra and applied mathematics. For a comprehensive undergraduate treatment that emphasizes geometric intuition alongside computational technique, Strang [33] remains the standard reference; his “four fundamental subspaces” perspective directly informs the rank-nullity discussion presented here.

In the robotics context, Chapter 2 of Murray, Li, and Sastry [23] develops the specific linear-algebraic tools—rotation matrices, skew-symmetric operators, and the Lie bracket—that recur throughout the rest of this textbook. Readers who wish to pursue the numerical aspects of linear algebra, particularly iterative solvers and the role of condition numbers in optimization, will find Nocedal and Wright [24] an invaluable companion.

The dyadic and tensor-product viewpoint introduced in Section 1.6 connects directly to continuum mechanics, where stress and strain are second-order tensors built from exactly the same outer-product construction. Students interested in these connections should consult any graduate text on continuum mechanics or tensor analysis.

Finally, the eigenvalue structures and matrix exponentials that close this chapter are the first hints of *Lie theory*. Hall [14] provides a mathematically rigorous bridge from linear algebra to Lie groups and Lie algebras, foreshadowing the rotation groups ($\text{SO}(3)$) and rigid-body transformation groups ($\text{SE}(3)$) that appear in Chapters 4 and 5.

1.12 Exercises

Computational Exercises

1. **(Rank and Null Space)** Given the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

compute the rank, determinant, and a basis for the null space. Verify the Rank-Nullity Theorem. *Hint: row reduce to echelon form.*

2. **(Eigenvalue Computation)** For the matrix $\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$, compute the eigenvalues and eigenvectors by hand. Verify that $\mathbf{A}\mathbf{v}_i = \lambda_i\mathbf{v}_i$ for each pair.
3. **(SVD by Hand)** Compute the SVD of $\mathbf{M} = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$. What are the singular values, and how do they relate to the eigenvalues?
4. **(Cholesky)** Verify that $\mathbf{M} = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$ is positive definite by: (a) checking all eigenvalues are positive, (b) computing the Cholesky factorization $\mathbf{LL}^T$.

5. **(Python)** Write a Python script that generates a random 5×5 symmetric matrix $\mathbf{M} = \mathbf{A}^T \mathbf{A} + \epsilon \mathbf{I}$ (guaranteeing positive definiteness), computes its eigendecomposition, SVD, and Cholesky factorization, and verifies all three decompositions reproduce the original matrix within numerical tolerance.

Conceptual Exercises

6. **(Vector Spaces)** Is the set of all 2×2 symmetric matrices a vector space under standard matrix addition and scalar multiplication? Prove or disprove by checking the axioms. What is its dimension?
7. **(Physical Interpretation)** A robot Jacobian $\mathbf{J} \in \mathbb{R}^{3 \times 5}$ has singular values $\sigma_1 = 2.0$, $\sigma_2 = 0.8$, $\sigma_3 = 0.01$. Describe the shape of the velocity manipulability ellipsoid. Is the robot near a singularity? What does the condition number tell you about the reliability of inverse kinematics computations?
8. **(Positive Definiteness in Control)** Explain in your own words why the mass matrix $\mathbf{M}(\mathbf{q})$ of any physical mechanical system must be positive definite. What would a zero eigenvalue of $\mathbf{M}(\mathbf{q})$ physically mean?
9. **(Skew-Symmetric Matrices)** Prove that for any skew-symmetric matrix $\mathbf{S}$ ($\mathbf{S}^T = -\mathbf{S}$), the matrix $e^{\mathbf{S}} = \mathbf{I} + \mathbf{S} + \frac{1}{2!}\mathbf{S}^2 + \cdots$ is an orthogonal matrix (i.e., $(e^{\mathbf{S}})^T e^{\mathbf{S}} = \mathbf{I}$). *Hint: use the fact that $(e^{\mathbf{S}})^T = e^{\mathbf{S}^T} = e^{-\mathbf{S}}$.* This result is the mathematical foundation of Chapter 6.

Proof-Based Exercises

10. **(Rank-Nullity Application)** Prove that if $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m < n$, then $\mathbf{A}$ must have a nontrivial null space (i.e., $\mathcal{N}(\mathbf{A}) \neq \{\mathbf{0}\}$). Interpret this result for an underactuated robot with more joints than task-space dimensions.
11. **(Trace and Eigenvalues)** Prove that the trace of a matrix (sum of diagonal elements) equals the sum of its eigenvalues: $\text{tr}(\mathbf{M}) = \sum_{i=1}^n \lambda_i$. *Hint: use the fact that similar matrices have the same trace.*
12. **(Determinant and Eigenvalues)** Prove that the determinant of a matrix equals the product of its eigenvalues: $\det(\mathbf{M}) = \prod_{i=1}^n \lambda_i$. *Hint: evaluate the characteristic polynomial at $\lambda = 0$.*
13. **(Uniqueness of SVD)** The singular values of a matrix are unique, but the singular vectors are not (when singular values are repeated). Give an example of a 2×2 matrix with a repeated singular value, and show that the corresponding singular vectors can be chosen as any orthonormal pair spanning a two-dimensional subspace.
14. **(Schur Complement and Positive Definiteness)** Let $\mathbf{M} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^T & \mathbf{C} \end{bmatrix}$ be a symmetric block matrix with $\mathbf{A} \succ 0$. Prove that $\mathbf{M} \succ 0$ if and only if $\mathbf{C} - \mathbf{B}^T \mathbf{A}^{-1} \mathbf{B} \succ 0$ (the *Schur complement* condition). This result is used extensively in the Articulated Body Algorithm (Chapter 10).

Chapter 2

The Transition to State Space

You cannot control a machine by staring at the machine. You control it by staring at the mathematical space that completely defines it.

2.1 Introduction: The Physics Abstraction

In Plain Language 2.1. Context & Use Case: Why We Banish 3D Physics

The Math You Will See: We will build the **State Vector** ($\mathbf{x}$), a vertical stack of numbers containing positions ($\mathbf{q}$) and velocities ($\dot{\mathbf{q}}$). We will formulate the universal control equation $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t)$, and its linear cousin $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$.

Why We Need It: When a robotic arm swings a bat, the shoulder, elbow, and wrist all move in weird, overlapping arcs in 3D physics space (X, Y, Z). This is impossibly hard to calculate directly. Instead, we map the entire robot into an abstract mathematical realm called "State Space". In State Space, the complex multi-joint swinging robot is represented by a *single, infinitesimally small dot* moving along a single curved line. This abstraction makes solving incredibly complex control problems possible.

All Variables Defined: $\mathbf{x}$ is the state, $\mathbf{u}$ is the control input (e.g., motor torque), and f is the system dynamics mapping. See the **Nomenclature** at the start of the book for a full reference.

When a layman pictures a robotic arm throwing a ball or a drone correcting its pitch in a crosswind, they visualize physical bodies acting in 3-dimensional Euclidean space.

But control theorists and kinematic engineers almost never look at the physical space. They work in a wholly abstract, higher-dimensional mathematical realm known as **State Space**. State space is arguably the most fundamental translation required in control theory. If you do not understand state space, nothing else in the subsequent volumes of *The Geometry of Motion* will make sense.

2.2 Historical Context: The Kalman Revolution

In Plain Language 2.2. Historical Context: From Transfer Functions to State Space

In the 1950s, control engineers exclusively used **transfer functions**, which relate outputs to inputs in the frequency domain via Laplace transforms. Transfer functions are well suited for single-input single-output (SISO) systems and for analyzing stability via Nyquist diagrams and Bode plots. However, they fail catastrophically for multi-input multi-output (MIMO) systems and provide no insight into what is happening *inside* the system.

In 1960, **Rudolf E. Kalman** introduced the state-space representation of linear dynamical systems and the structural concepts of **controllability** and **observability** [15]. The state-space formulation made the analysis of multi-input multi-output systems systematic, opened the door to optimal linear-quadratic control, and led within a few years to the recursive estimation problem now known as the Kalman filter. Within a decade, state space had become the standard working language of quantitative control engineering.

The fundamental advantage of state-space methods over transfer functions is *transparency* [16]. A transfer function $G(s) = \frac{Y(s)}{U(s)}$ tells you how the input drives the output, but it hides the internal structure. A state-space model $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ exposes the entire flow of energy and information through the system. This transparency makes it possible to:

- Design controllers that regulate internal state, not just final outputs
- Detect when a system is uncontrollable or unobservable
- Handle systems with multiple inputs and multiple outputs naturally
- Compute optimal control laws analytically
- Estimate hidden state variables from measurements (Kalman filter)

The rest of this chapter builds the state-space framework from first principles. By the end, the reader should understand why Kalman's formulation became the standard and why state space is the working representation for almost every modern control problem.

2.3 Constructing a State Vector

Let us build a state space from the ground up using the simplest dynamical system possible: a 1-dimensional point mass (like a train on a straight track).

We want to predict what the train will do. To do this, we need to know its position, q . Is q the "state" of the system?

No. If we only know that the train is at $q = 100$ meters, we cannot predict where it will be 1 second from now. It could be moving right, moving left, or stationary.

Therefore, to complete our understanding, we must also know its velocity, $\dot{q}$ (the time-derivative of position). If we know $(q, \dot{q})$, we know its exact location and its exact speed.

State-Space System Architecture

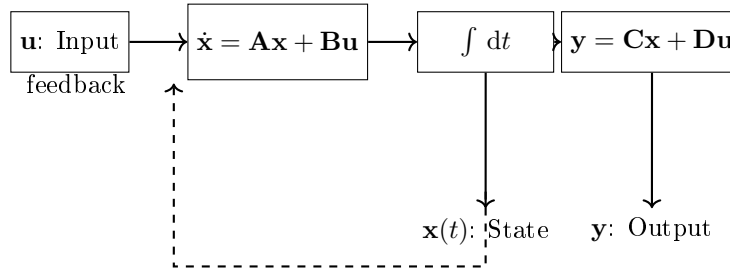


Figure 2.1: Block diagram of a continuous-time linear state-space system. The input $\mathbf{u}$ drives the dynamics, producing state changes. The state $\mathbf{x}$ is obtained by integrating the state derivative. The output $\mathbf{y}$ is a linear combination of state and input.

Do we need to know acceleration ($\ddot{q}$) as part of the state? According to Newton's Second Law: $F = m\ddot{q}$. Acceleration is not an independent property of the train; it is completely driven by the external forces applied at that instant. Therefore, if we know the forces (the Control Inputs, $\mathbf{u}$), we can immediately compute the acceleration. The state itself is fully defined by purely the position and velocity.

Definition 2.1. The State Vector $\mathbf{x}$

The state vector $\mathbf{x}$ is the minimum set of mathematical variables completely describing the condition of a dynamic system at a specific point in time.

For our train, it is a 2-dimensional column vector:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} q \\ \dot{q} \end{bmatrix} = \begin{bmatrix} \text{position} \\ \text{velocity} \end{bmatrix} \quad (2.1)$$

For a general mechanical system with n configuration degrees of freedom $\mathbf{q} \in \mathbb{R}^n$, the state is the pair of configuration and velocity, $\mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}}) \in \mathcal{X} \subset \mathbb{R}^{2n}$. More generally, the state space may be any manifold whose dimension equals the number of first-order ODEs required to predict the future: for a mechanical system that is $2n$; for an electromechanical system it is typically larger (see the DC-motor example below, where the inductor current adds an extra state).

If you take a photograph of a system, you capture its position $\mathbf{q}$. But a state vector $\mathbf{x}$ is the "ultimate photograph"—it captures the position *and* the velocity. With $\mathbf{x}$ and the laws of physics, the entire future of the system is deterministic.

2.3.1 Worked Example 1: Simple Pendulum

Example 2.1. State Vector for a Simple Pendulum

A pendulum of length L and mass m is suspended from a pivot. Let θ denote the angle from vertical, and τ denote an applied torque at the pivot. The equation of motion is:

$$mL^2\ddot{\theta} + mgL \sin(\theta) = \tau \quad (2.2)$$

To predict the pendulum's future, what do we need?

- The angle θ (where is it pointing?)
- The angular velocity $\dot{\theta}$ (is it spinning?)
- The torque τ is the control input (provided at the moment we want to predict)

Thus the state vector is:

$$\mathbf{x} = \begin{bmatrix} \theta \\ \dot{\theta} \end{bmatrix} \quad (2.3)$$

This is a 2-dimensional state space, even though the pendulum physically swings in a 1-dimensional arc in our visual space.

2.3.2 Worked Example 2: DC Motor

Example 2.2. State Vector for a DC Motor

A DC motor is driven by input voltage V_{in} . Let θ be the rotor angle, and let i be the current flowing through the motor windings.

The governing equations are:

$$L \frac{di}{dt} = V_{\text{in}} - Ri - K_e \dot{\theta} \quad (2.4)$$

$$J \frac{d\dot{\theta}}{dt} = K_t i - b\dot{\theta} \quad (2.5)$$

where:

- L is the motor inductance, R is resistance
- K_e is the back-EMF constant, K_t is the torque constant
- J is the rotor inertia, b is viscous friction

The state vector must include all variables whose time-derivatives depend on the inputs. These are:

$$\mathbf{x} = \begin{bmatrix} i \\ \theta \\ \dot{\theta} \end{bmatrix} \quad (2.6)$$

This is a 3-dimensional state space. Notice we include θ (rotor position) even though it does not directly appear in the governing equations—we need it to fully specify the motor's configuration, and its time-derivative $\dot{\theta}$ does appear. As noted in Definition 2.1, the state space dimension (3) exactly matches the number of first-order ODEs needed to predict the system (one electrical + two mechanical), despite there being only 1 mechanical DOF.

2.3.3 Worked Example 3: Two-Tank Fluid System

Example 2.3. State Vector for Two Coupled Tanks

Two tanks, each with cross-sectional area A , are connected by a valve. Let h_1 and h_2 be the fluid levels. Flow into tank 1 is Q_{in} , and flow between tanks is proportional to the height difference:

$$Q_{1 \rightarrow 2} = C \sqrt{|h_1 - h_2|} \quad (2.7)$$

where C is a flow coefficient, and outflow from tank 2 is:

$$Q_{\text{out}} = C_{\text{out}} \sqrt{h_2} \quad (2.8)$$

The height rates of change are:

$$A \frac{dh_1}{dt} = Q_{\text{in}} - Q_{1 \rightarrow 2} \quad (2.9)$$

$$A \frac{dh_2}{dt} = Q_{1 \rightarrow 2} - Q_{\text{out}} \quad (2.10)$$

To predict the system, we need the current heights of both tanks. The state vector is:

$$\mathbf{x} = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} \quad (2.11)$$

This is a 2-dimensional state space. The control input is the inflow rate Q_{in} .

2.4 Converting n -th Order ODEs to State Form

Theorem 2.1. Conversion of n -th Order ODE to First-Order State Form

Any n -th order differential equation of the form

$$y^{(n)} + a_{n-1}y^{(n-1)} + \cdots + a_1\dot{y} + a_0y = u(t) \quad (2.12)$$

can be converted to a system of n first-order ODEs via state-vector substitution.

Proof. Define the state variables as consecutive derivatives of y :

$$x_1 = y \quad (2.13)$$

$$x_2 = \dot{y} \quad (2.14)$$

$$x_3 = \ddot{y} \quad (2.15)$$

$$\vdots \quad (2.16)$$

$$x_n = y^{(n-1)} \quad (2.17)$$

Then:

$$\dot{x}_1 = x_2 \quad (2.18)$$

$$\dot{x}_2 = x_3 \quad (2.19)$$

$$\vdots \quad (2.20)$$

$$\dot{x}_{n-1} = x_n \quad (2.21)$$

$$\dot{x}_n = u(t) - a_0x_1 - a_1x_2 - \cdots - a_{n-1}x_n \quad (2.22)$$

where the final equation is obtained by solving the original ODE for $y^{(n)} = \dot{x}_n$:

$$\dot{x}_n = u(t) - a_0x_1 - a_1x_2 - \cdots - a_{n-1}x_n \quad (2.23)$$

In matrix form, this is $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$ where $\mathbf{A}$ is the companion matrix:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ -a_0 & -a_1 & -a_2 & \cdots & -a_{n-1} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \quad (2.24)$$

This representation is unique up to change of state variables (i.e., state transformation). $\square$

2.5 State Space Representations

Because the state vector $\mathbf{x}$ contains n variables, it naturally lives in an n -dimensional geometry: the State Space, denoted as $\mathcal{X} \subset \mathbb{R}^n$.

An astonishing realization for beginners is that when a dynamic system (like an airplane or a robotic arm) moves through physical space over time, its entire sprawling motion is represented as a single, unified **1-dimensional curve** snaking through n -dimensional state space. This curve is called the system's **Trajectory** $\mathcal{T}$.

2.5.1 Standard Form Differential Equations

The immense power of state space is that it allows us to convert complex, higher-order physical differential equations into simple, first-order vector math.

Consider a simple mass-spring-damper system governed by a second-order differential equation:

$$m\ddot{q} + c\dot{q} + kq = u \quad (2.25)$$

where q is position, m is mass, c is damping, k is stiffness, and u is our control force (e.g., a motor pushing the mass).

By defining our state vector as $\mathbf{x} = [x_1, x_2]^T = [q, \dot{q}]^T$, we can rewrite this physical equation as purely mathematical first-order derivatives. First, isolate the highest derivative ($\ddot{q}$):

$$\ddot{q} = \frac{1}{m}(u - c\dot{q} - kq) \quad (2.26)$$

Now, substitute the state variables ($x_1 = q, x_2 = \dot{q}$):

$$\dot{x}_1 = x_2 \quad (2.27)$$

$$\dot{x}_2 = \frac{1}{m}(u - cx_2 - kx_1) \quad (2.28)$$

This is called the **State Space Representation**. It is universally written in nonlinear control literature as:

Mathematical Principle 2.1. The Fundamental Equation of Control Theory

Every forced control system can be written as a first-order vector differential equation mapping the current state and current control commands to the velocity of the state vector:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t) \quad (2.29)$$

This vector field f dictates how the system "flows" through the abstract geometry of the state space.

2.5.2 Linear State Space: The A and B Matrices

When the system is perfectly linear (like the mass-spring-damper above), the function $f(\mathbf{x}, \mathbf{u}, t)$ can be split into two matrices: the Dynamics Matrix (**A**) and the Input Matrix (**B**).

$$\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu} \quad (2.30)$$

If we package our mass-spring equations into matrices, we get:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u \quad (2.31)$$

- **The A Matrix:** $\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}$. This matrix defines the "drift" of the system.

If you turn off all motors ($u = 0$), the **A** matrix dictates exactly how the system will coast to a stop, oscillate, or explode. Its eigenvalues completely define the system's natural stability.

- **The B Matrix:** $\mathbf{B} = \begin{bmatrix} 0 \\ 1/m \end{bmatrix}$. This matrix defines how your control inputs u actually inject energy into the state. Notice that the top value is 0, meaning our motor cannot instantly teleport the position (x_1); it can only directly push the velocity (x_2).

2.6 Output Equations and Observability

Often, we cannot measure the entire state vector. For example, in a DC motor, we can measure the voltage and current flowing through it, but we may not directly measure the angular position θ . The **output equation** describes what we actually measure:

Definition 2.2. Output Equation

For a linear system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, the output equation is:

$$\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \quad (2.32)$$

where:

- $\mathbf{y} \in \mathbb{R}^p$ is the measurement vector (what we can observe)
- $\mathbf{C} \in \mathbb{R}^{p \times n}$ is the output matrix
- $\mathbf{D} \in \mathbb{R}^{p \times m}$ is the feedthrough matrix (often zero)

The question "Can we infer the full state $\mathbf{x}$ from the measurements $\mathbf{y}$?" is precisely the question of **observability**, which we formalize later in Section 2.11.

2.7 Equilibrium Points and Linearization

2.7.1 Equilibrium Point Definition

Definition 2.3. Equilibrium Point

An **equilibrium point** (or fixed point) $\mathbf{x}^* \in \mathbb{R}^n$ of the system $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}^*)$ is a point where the vector field is zero:

$$f(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0} \quad (2.33)$$

At an equilibrium, the state does not change; the system is in a "steady state."

For linear systems $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, if $\mathbf{A}$ is invertible and $\mathbf{u}^* = \mathbf{0}$, the unique equilibrium is $\mathbf{x}^* = \mathbf{0}$. For nonlinear systems, there may be multiple equilibria.

2.7.2 Linearization via Taylor Expansion

To understand the local behavior near an equilibrium, we linearize via Taylor expansion. Suppose we have an equilibrium $\mathbf{x}^*$ at control input $\mathbf{u}^*$. Consider a small perturbation $\delta\mathbf{x} = \mathbf{x} - \mathbf{x}^*$ and $\delta\mathbf{u} = \mathbf{u} - \mathbf{u}^*$.

Expand f in a Taylor series around $(\mathbf{x}^*, \mathbf{u}^*)$:

$$f(\mathbf{x}^* + \delta\mathbf{x}, \mathbf{u}^* + \delta\mathbf{u}) = f(\mathbf{x}^*, \mathbf{u}^*) + \mathbf{J}_f\delta\mathbf{x} + \mathbf{J}_u\delta\mathbf{u} + O(\|\delta\mathbf{x}\|^2, \|\delta\mathbf{u}\|^2) \quad (2.34)$$

where $\mathbf{J}_f$ is the **Jacobian matrix** of f with respect to the state:

$$\mathbf{J}_f = \frac{\partial f}{\partial \mathbf{x}} \Big|_{(\mathbf{x}^*, \mathbf{u}^*)} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}_{(\mathbf{x}^*, \mathbf{u}^*)} \quad (2.35)$$

and similarly:

$$\mathbf{J}_u = \frac{\partial f}{\partial \mathbf{u}} \Big|_{(\mathbf{x}^*, \mathbf{u}^*)} \quad (2.36)$$

Since $f(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0}$, the linearized dynamics are:

$$\dot{\delta \mathbf{x}} \approx \mathbf{A}_c \delta \mathbf{x} + \mathbf{B}_c \delta \mathbf{u} \quad (2.37)$$

where $\mathbf{A}_c = \mathbf{J}_f$ and $\mathbf{B}_c = \mathbf{J}_u$. This is a linear system describing the small-signal behavior around the equilibrium.

Example 2.4. Pendulum Linearization

Consider the pendulum from Example 2.3.1 with $\tau = 0$ (no control). The equation is:

$$\ddot{\theta} + \frac{g}{L} \sin(\theta) = 0 \quad (2.38)$$

State vector: $\mathbf{x} = [\theta, \dot{\theta}]^T$. The vector field is:

$$f(\mathbf{x}) = \begin{bmatrix} x_2 \\ -\frac{g}{L} \sin(x_1) \end{bmatrix} \quad (2.39)$$

Equilibria: $f(\mathbf{x}^*) = \mathbf{0}$ when $x_2 = 0$ and $\sin(x_1) = 0$, so $\mathbf{x}^* = [k\pi, 0]^T$ for $k \in \mathbb{Z}$.

Bottom equilibrium ($\theta = 0$): The Jacobian is:

$$\mathbf{J}_f = \begin{bmatrix} 0 & 1 \\ -\frac{g}{L} \cos(\theta) & 0 \end{bmatrix} \Big|_{\theta=0} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{L} & 0 \end{bmatrix} \quad (2.40)$$

The linearized system near the bottom is $\ddot{\theta} = -\frac{g}{L}\theta$, a harmonic oscillator.

Top equilibrium ($\theta = \pi$):

$$\mathbf{J}_f \Big|_{\theta=\pi} = \begin{bmatrix} 0 & 1 \\ \frac{g}{L} & 0 \end{bmatrix} \quad (2.41)$$

The linearized system is $\ddot{\theta} = \frac{g}{L}\theta$, which has exponentially growing solutions—an unstable saddle.

2.8 Phase Portraits and Stability Classification

For a 2D system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$, the phase portrait is the collection of all trajectories in the (x_1, x_2) plane. The behavior depends entirely on the eigenvalues of $\mathbf{A}$.

Theorem 2.2. Stability Classification from Eigenvalues

For a 2D linear system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ with eigenvalues λ_1, λ_2 :

- **Stable Node:** Both eigenvalues are real and negative ($\lambda_1, \lambda_2 < 0$). Trajectories spiral into the origin monotonically.
- **Unstable Node:** Both eigenvalues are real and positive ($\lambda_1, \lambda_2 > 0$). Trajectories spiral away from the origin.
- **Saddle Point:** One eigenvalue is positive, one is negative. Trajectories approach along the stable manifold (eigenvector of $\lambda < 0$) and diverge along the unstable manifold (eigenvector of $\lambda > 0$).
- **Stable Spiral (Focus):** Eigenvalues are complex conjugates with negative real part ($\lambda = \alpha \pm i\beta$, $\alpha < 0$). Trajectories spiral inward.
- **Unstable Spiral:** Eigenvalues are complex conjugates with positive real part. Trajectories spiral outward.
- **Center:** Eigenvalues are purely imaginary ($\lambda = \pm i\omega$). Trajectories form closed orbits (periodic motion).

The origin is **asymptotically stable** if and only if all eigenvalues have negative real part. The origin is **marginally stable** if the largest real part is zero. The origin is **unstable** if any eigenvalue has positive real part.

2.9 Solution of Linear Systems: The Matrix Exponential

For a linear time-invariant (LTI) system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ with initial condition $\mathbf{x}(0) = \mathbf{x}_0$, the solution is:

Theorem 2.3. Solution via Matrix Exponential

The unique solution is:

$$\mathbf{x}(t) = e^{\mathbf{A}t} \mathbf{x}_0 \quad (2.42)$$

where $e^{\mathbf{A}t}$ is the **matrix exponential**, defined as:

$$e^{\mathbf{A}t} = \sum_{k=0}^{\infty} \frac{(\mathbf{A}t)^k}{k!} = \mathbf{I} + \mathbf{A}t + \frac{(\mathbf{A}t)^2}{2!} + \frac{(\mathbf{A}t)^3}{3!} + \dots \quad (2.43)$$

Proof. Consider the scalar ODE $\dot{x} = ax$ with initial condition $x(0) = x_0$. The solution is $x(t) = e^{at}x_0$.

By analogy, define $e^{\mathbf{A}t}$ as the solution to:

$$\frac{d}{dt}e^{\mathbf{A}t} = \mathbf{A}e^{\mathbf{A}t}, \quad e^{\mathbf{A} \cdot 0} = \mathbf{I} \quad (2.44)$$

The Taylor series ansatz:

$$e^{\mathbf{A}t} = \sum_{k=0}^{\infty} \frac{(\mathbf{A}t)^k}{k!} \quad (2.45)$$

satisfies this differential equation because:

$$\frac{d}{dt} \sum_{k=0}^{\infty} \frac{(\mathbf{A}t)^k}{k!} = \sum_{k=1}^{\infty} \frac{k(\mathbf{A}t)^{k-1} \mathbf{A}}{k!} = \sum_{k=1}^{\infty} \frac{(\mathbf{A}t)^{k-1} \mathbf{A}}{(k-1)!} = \mathbf{A} \sum_{j=0}^{\infty} \frac{(\mathbf{A}t)^j}{j!} = \mathbf{A} e^{\mathbf{A}t} \quad (2.46)$$

with $e^{\mathbf{A} \cdot 0} = \mathbf{I}$. Now verify that $\mathbf{x}(t) = e^{\mathbf{A}t} \mathbf{x}_0$ solves the original system:

$$\dot{\mathbf{x}}(t) = \frac{d}{dt} (e^{\mathbf{A}t} \mathbf{x}_0) = \mathbf{A} e^{\mathbf{A}t} \mathbf{x}_0 = \mathbf{A} \mathbf{x}(t) \quad \checkmark \quad (2.47)$$

□

Practical Computation: For small-dimensional systems, the matrix exponential can be computed via diagonalization. If $\mathbf{A} = \mathbf{P} \mathbf{\Lambda} \mathbf{P}^{-1}$ where $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$, then:

$$e^{\mathbf{A}t} = \mathbf{P} e^{\mathbf{\Lambda}t} \mathbf{P}^{-1} = \mathbf{P} \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_n t}) \mathbf{P}^{-1} \quad (2.48)$$

For systems with input, $\dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}(t)$:

$$\mathbf{x}(t) = e^{\mathbf{A}t} \mathbf{x}_0 + \int_0^t e^{\mathbf{A}(t-\tau)} \mathbf{B} \mathbf{u}(\tau) d\tau \quad (2.49)$$

This is the superposition of free response (first term) and forced response (second term).

2.10 Controllability

The key question in control is: *Can we steer the system from any initial state to any desired final state using our inputs?*

Definition 2.4. Controllability

A linear system $\dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}$ is **controllable** if for any initial state $\mathbf{x}_0$ and any final state $\mathbf{x}_f$, there exists a finite time $T > 0$ and an input sequence $\mathbf{u}(t)$ for $t \in [0, T]$ that drives the system from $\mathbf{x}_0$ to $\mathbf{x}_f$.

Theorem 2.4. Kalman Controllability Rank Condition

The system $\dot{\mathbf{x}} = \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}$ is controllable if and only if the **controllability matrix**

$$\mathbf{C}_{\text{rank}} = [\mathbf{B} \quad \mathbf{A}\mathbf{B} \quad \mathbf{A}^2\mathbf{B} \quad \dots \quad \mathbf{A}^{n-1}\mathbf{B}] \quad (2.50)$$

has full rank: $\text{rank}(\mathbf{C}_{\text{rank}}) = n$.

Sketch. The controllability matrix $\mathbf{C}_{\text{rank}}$ has dimensions $n \times nm$. Each column $\mathbf{A}^k \mathbf{B}$ represents the "reach" of the input after k applications of the dynamics. The rank condition ensures that the span of these columns covers all n dimensions of state space.

The intuition is from the Cayley-Hamilton theorem: any power $\mathbf{A}^k$ for $k \geq n$ can be expressed as a linear combination of $\mathbf{A}^0, \mathbf{A}^1, \dots, \mathbf{A}^{n-1}$. Thus, checking columns up to $\mathbf{A}^{n-1} \mathbf{B}$ is sufficient.

A rigorous proof uses the Gram matrix approach: controllability is equivalent to the controllability Gram matrix

$$\mathbf{W}_c = \int_0^T e^{\mathbf{A}\tau} \mathbf{B} \mathbf{B}^T e^{\mathbf{A}^T \tau} d\tau \quad (2.51)$$

being positive definite, which is equivalent to $\mathbf{C}_{\text{rank}}$ having full rank. $\square$

Physical Interpretation: If the system is not controllable, it means some mode or degree of freedom cannot be directly influenced by the available inputs. For a DC motor whose position is θ and current is i , if the only measurement is current and there is no direct control over electrical resistance, the position might be uncontrollable.

2.11 Observability

Observability is the dual of controllability: *Can we infer the full state from measurements alone?*

Definition 2.5. Observability

A linear system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, $\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}$ is **observable** if the initial state $\mathbf{x}(0)$ can be uniquely determined from knowledge of the input $\mathbf{u}(t)$ and the output $\mathbf{y}(t)$ over a finite time interval $[0, T]$.

Theorem 2.5. Kalman Observability Rank Condition

The system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, $\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}$ is observable if and only if the **observability matrix**

$$\mathbf{O} = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \mathbf{CA}^2 \\ \vdots \\ \mathbf{CA}^{n-1} \end{bmatrix} \quad (2.52)$$

has full rank: $\text{rank}(\mathbf{O}) = n$.

The observability matrix has dimensions $np \times n$ where p is the number of measurements. Each row $\mathbf{CA}^k$ represents the k -th time derivative of the measurement, weighted by the dynamics. If these rows span all n dimensions, we can reconstruct the state.

2.12 The Mass-Spring-Damper System: Complete Analysis

Let us apply the concepts from this chapter to the mass-spring-damper system with stiffness k , damping c , and mass m :

$$m\ddot{q} + c\dot{q} + kq = u \quad (2.53)$$

State vector: $\mathbf{x} = [q, \dot{q}]^T$.

Matrices:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 1/m \end{bmatrix} \quad (2.54)$$

Characteristic polynomial:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \det \begin{bmatrix} -\lambda & 1 \\ -k/m & -c/m - \lambda \end{bmatrix} = \lambda^2 + \frac{c}{m}\lambda + \frac{k}{m} \quad (2.55)$$

Eigenvalues:

$$\lambda_{1,2} = -\frac{c}{2m} \pm \sqrt{\left(\frac{c}{2m}\right)^2 - \frac{k}{m}} = -\frac{c}{2m} \pm \sqrt{\frac{c^2}{4m^2} - \frac{k}{m}} \quad (2.56)$$

Define the natural frequency $\omega_0 = \sqrt{k/m}$ and damping ratio $\zeta = \frac{c}{2\sqrt{km}}$. Then:

$$\lambda_{1,2} = -\zeta\omega_0 \pm i\omega_0\sqrt{1 - \zeta^2} \quad (2.57)$$

(assuming $\zeta < 1$ for the underdamped case; see below for other cases).

2.12.1 Underdamped ($0 < \zeta < 1$)

The discriminant is negative, so eigenvalues are complex: $\lambda = -\zeta\omega_0 \pm i\omega_d$ where $\omega_d = \omega_0\sqrt{1 - \zeta^2}$ is the damped frequency.

The mass oscillates while decaying. The solution is:

$$q(t) = e^{-\zeta\omega_0 t} \left(q_0 \cos(\omega_d t) + \frac{\dot{q}_0 + \zeta\omega_0 q_0}{\omega_d} \sin(\omega_d t) \right) \quad (2.58)$$

The trajectory in phase space is a **stable spiral**, spiraling into the origin.

2.12.2 Critically Damped ($\zeta = 1$)

The eigenvalues coincide: $\lambda_1 = \lambda_2 = -\omega_0$. The matrix $\mathbf{A}$ is not diagonalizable; it has a Jordan block.

The solution is:

$$q(t) = (q_0 + (\dot{q}_0 + \omega_0 q_0)t)e^{-\omega_0 t} \quad (2.59)$$

This is the fastest non-oscillatory return to equilibrium. The trajectory does *not* spiral; it approaches the origin monotonically.

2.12.3 Overdamped ($\zeta > 1$)

The eigenvalues are real and distinct:

$$\lambda_1 = -\zeta\omega_0 + \omega_0\sqrt{\zeta^2 - 1}, \quad \lambda_2 = -\zeta\omega_0 - \omega_0\sqrt{\zeta^2 - 1} \quad (2.60)$$

Both are negative (since $\zeta > 0$). The solution is:

$$q(t) = C_1 e^{\lambda_1 t} + C_2 e^{\lambda_2 t} \quad (2.61)$$

The slower eigenvalue λ_1 dominates for large times. The trajectory is a **stable node**—no oscillation, just exponential decay. The system returns to rest more slowly than the critically damped case.

2.12.4 Controllability and Observability

The controllability matrix is:

$$\mathbf{C}_{\text{rank}} = [\mathbf{B} \quad \mathbf{AB}] = \begin{bmatrix} 0 & 1/m \\ 1/m & -c/m^2 \end{bmatrix} \quad (2.62)$$

Its determinant is $\det(\mathbf{C}_{\text{rank}}) = 0 - \frac{1}{m^2} = -\frac{1}{m^2} \neq 0$. Thus $\mathbf{C}_{\text{rank}}$ has full rank and the system is **controllable**. We can steer position and velocity to any desired values using the input force u .

If we measure position only, $\mathbf{C} = [1, 0]$, the observability matrix is:

$$\mathbf{O} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (2.63)$$

which has rank 2. The system is **observable**. We can infer both position and velocity from position measurements alone (by differentiating or using a Kalman filter).

2.13 Transfer Function to State Space and Back

The transfer function $G(s) = \frac{Y(s)}{U(s)}$ relates input and output in the Laplace domain. For a linear system with zero initial conditions, the transfer function is:

$$G(s) = \mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} + \mathbf{D} \quad (2.64)$$

This is derived from the state-space equations by taking Laplace transforms:

$$s\mathbf{X}(s) = \mathbf{A}\mathbf{X}(s) + \mathbf{B}U(s) \quad (2.65)$$

$$\mathbf{Y}(s) = \mathbf{C}\mathbf{X}(s) + \mathbf{D}U(s) \quad (2.66)$$

Solve for $\mathbf{X}(s)$:

$$(s\mathbf{I} - \mathbf{A})\mathbf{X}(s) = \mathbf{B}U(s) \quad \Rightarrow \quad \mathbf{X}(s) = (s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B}U(s) \quad (2.67)$$

Substitute into the output equation:

$$\mathbf{Y}(s) = [\mathbf{C}(s\mathbf{I} - \mathbf{A})^{-1}\mathbf{B} + \mathbf{D}] U(s) = G(s)U(s) \quad (2.68)$$

Conversion from transfer function to state space is not unique—there are many valid state representations. The most common choice is the controllable canonical form (the companion matrix form from Section 2.4).

2.13.1 Discrete-Time State Space

Every modern robot controller is implemented on a digital processor that reads sensors and writes actuator commands at a fixed sample rate $f_s = 1/T_s$. Between samples the controller holds its output constant (zero-order hold). We therefore need a *discrete-time* counterpart of the continuous state-space model.

Definition 2.6. Discrete-Time State-Space Model

A linear discrete-time system is

$$\mathbf{x}[k+1] = \mathbf{A}_d \mathbf{x}[k] + \mathbf{B}_d \mathbf{u}[k], \quad (2.69)$$

$$\mathbf{y}[k] = \mathbf{C} \mathbf{x}[k] + \mathbf{D} \mathbf{u}[k], \quad (2.70)$$

where $k \in \{0, 1, 2, \dots\}$ indexes the sample instants $t_k = k T_s$.

Exact discretization (zero-order hold). Given the continuous system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ and a sampling period T_s , the exact discrete-time matrices are

$$\mathbf{A}_d = e^{\mathbf{A}T_s}, \quad \mathbf{B}_d = \int_0^{T_s} e^{\mathbf{A}\tau} \mathbf{B} d\tau = \mathbf{A}^{-1}(e^{\mathbf{A}T_s} - \mathbf{I})\mathbf{B}, \quad (2.71)$$

where the closed-form expression for $\mathbf{B}_d$ holds when $\mathbf{A}$ is invertible. The matrix exponential $e^{\mathbf{A}T_s}$ connects directly to Theorem 2.3: the free response over one sample period is simply propagated by $\mathbf{A}_d$.

Theorem 2.6. Discrete-Time Stability Criterion

The equilibrium $\mathbf{x}^* = \mathbf{0}$ of the autonomous discrete-time system $\mathbf{x}[k+1] = \mathbf{A}_d \mathbf{x}[k]$ is asymptotically stable if and only if every eigenvalue of $\mathbf{A}_d$ lies strictly inside the unit circle:

$$|\lambda_i(\mathbf{A}_d)| < 1 \quad \forall i. \quad (2.72)$$

This is the discrete analogue of requiring all eigenvalues of $\mathbf{A}$ to have negative real parts in continuous time (Theorem 2.2).

In Plain Language 2.3. All Robot Controllers Are Discrete

All modern robot controllers run in discrete time at sample rates of 1–10 kHz. The continuous-time model $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ is the physicist's description of reality; the discrete-time model $\mathbf{x}[k+1] = \mathbf{A}_d \mathbf{x}[k] + \mathbf{B}_d \mathbf{u}[k]$ is what actually executes on the real-time processor. A control design that ignores this distinction—for example, by using too low a sample rate so that eigenvalues migrate outside the unit circle—will destabilise an otherwise stable system.

2.14 Nonlinear Systems and Phase Portraits

While we have focused on linear systems, real physical systems are often nonlinear. For a nonlinear system $\dot{\mathbf{x}} = f(\mathbf{x})$, the phase portrait can be qualitatively understood by:

1. Finding equilibrium points by solving $f(\mathbf{x}^*) = \mathbf{0}$
2. Linearizing around each equilibrium: $\delta \dot{\mathbf{x}} \approx \mathbf{J}_f(\mathbf{x}^*)\delta \mathbf{x}$
3. Classifying the equilibrium by its eigenvalues (Theorem 2.8)
4. Plotting the vector field $\dot{\mathbf{x}} = f(\mathbf{x})$ to visualize global behavior

The linearization is valid in a neighborhood of the equilibrium. For large perturbations, we must rely on numerical simulation or other techniques (Lyapunov theory, bifurcation analysis).

2.15 Python Worked Example: State Space Simulation and Phase Portrait

Consider the underdamped mass-spring-damper system with *illustrative numerical values*: $m = 1$ kg, $c = 0.5$ N · s/m, $k = 2$ N/m, with initial conditions $q_0 = 1$ m, $\dot{q}_0 = 0$.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import odeint

# Parameters
m, c, k = 1.0, 0.5, 2.0
A = np.array([[0, 1], [-k/m, -c/m]])
B = np.array([[0], [1/m]])

# System dynamics
def dynamics(x, t, u=0):
    return A @ x + B.flatten() * u

# Initial condition
x0 = np.array([1.0, 0.0])
t = np.linspace(0, 10, 1000)

# Simulate
x = odeint(dynamics, x0, t)

# Phase portrait
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Time evolution
ax1.plot(t, x[:, 0], label='Position q(t)')
ax1.plot(t, x[:, 1], label='Velocity dq/dt(t)')
ax1.set_xlabel('Time (s)')
ax1.set_ylabel('State')
ax1.legend()
ax1.grid()
```

```

ax1.set_title('Time Evolution')

# Phase portrait
ax2.plot(x[:, 0], x[:, 1], 'b-', linewidth=2)
ax2.plot(x[0, 0], x[0, 1], 'go', markersize=10, label='Initial')
ax2.plot(x[-1, 0], x[-1, 1], 'ro', markersize=10, label='Final')
ax2.set_xlabel('Position q')
ax2.set_ylabel('Velocity dq/dt')
ax2.grid()
ax2.legend()
ax2.set_title('Phase Portrait (Stable Spiral)')

plt.tight_layout()
plt.show()

# Eigenvalues
eigenvalues, eigenvectors = np.linalg.eig(A)
print(f"Eigenvalues: {eigenvalues}")
print(f"Damping ratio zeta = {c / (2*np.sqrt(k*m)):.3f}")
print(f"Natural freq omega_0 = {np.sqrt(k/m):.3f} rad/s")

```

This code simulates the system and plots both the time evolution and the phase portrait. For the underdamped case ($\zeta = 0.177$), you should see a spiral converging to the origin with oscillation.

2.16 Equilibrium Analysis Summary

For any equilibrium point $\mathbf{x}^*$:

- Compute the Jacobian: $\mathbf{A} = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}^*}$
- Find eigenvalues of $\mathbf{A}$
- Classify the equilibrium according to Theorem 2.8
- If all eigenvalues have negative real part: **asymptotically stable**
- If any eigenvalue has positive real part: **unstable**
- If the largest real part is zero (center or boundary case): **marginally stable**

2.17 Summary

Mathematical Principle 2.2. Key Takeaways: State Space Fundamentals

1. **State Vector:** The state $\mathbf{x}$ is the minimum information needed to predict the system's future. For an n -th order ODE, the state is n -dimensional.
2. **Kalman's Revolution (1960):** State-space representations replace transfer

functions, enabling control of MIMO systems, design of optimal controllers, and systematic analysis of internal system structure.

3. Standard Forms: Any system can be written as $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, t)$ (nonlinear) or $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ (linear). The $\mathbf{A}$ matrix encodes natural stability; the $\mathbf{B}$ matrix encodes control authority.

4. ODE Reduction: Any n -th order ODE reduces to n first-order state equations via companion form.

5. Equilibrium and Linearization: Equilibria are found by solving $f(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0}$. Local behavior is determined by the Jacobian $\mathbf{J}_f = \partial f / \partial \mathbf{x}$.

6. Stability from Eigenvalues: Eigenvalues of $\mathbf{A}$ determine phase-portrait structure (node, spiral, saddle, center). Negative real parts mean stability.

7. Matrix Exponential: The solution $\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}_0 + \int_0^t e^{\mathbf{A}(t-\tau)}\mathbf{B}\mathbf{u}(\tau) d\tau$ decomposes into free response and forced response.

8. Controllability (Kalman Rank Test): The system is controllable iff $\text{rank}([\mathbf{B}, \mathbf{A}\mathbf{B}, \dots, \mathbf{A}^{n-1}\mathbf{B}]) = n$. If not controllable, some state variables cannot be influenced by inputs.

9. Observability (Dual of Controllability): The system is observable iff $\text{rank}([\mathbf{C}^T, \mathbf{A}^T\mathbf{C}^T, \dots, (\mathbf{A}^T)^{n-1}\mathbf{C}^T]^T) = n$. If not observable, some state variables cannot be inferred from measurements.

10. Duality: A system is controllable iff its dual (with $\mathbf{A}^T$, $\mathbf{C}^T$, $\mathbf{B}^T$) is observable. This duality underpins Kalman filter design.

2.18 Lyapunov Stability Theory

The eigenvalue-based stability analysis of Theorem 2.2 is powerful but inherently *local*: it applies only in a neighbourhood of an equilibrium where the linearization is valid. For nonlinear systems, we need a theory that can certify stability over large—possibly global—regions of state space. **Lyapunov's direct method** provides exactly this by replacing spectral analysis with an energy-like argument.

Linearization tells us about local stability. Lyapunov theory works globally.

Definition 2.7. Lyapunov Function $V(\mathbf{x})$

Consider the autonomous system $\dot{\mathbf{x}} = f(\mathbf{x})$ with an equilibrium at $\mathbf{x}^* = \mathbf{0}$. A continuously differentiable function $V : \mathbb{R}^n \rightarrow \mathbb{R}$ is called a **Lyapunov function candidate** if

1. $V(\mathbf{0}) = 0$ (zero at the equilibrium),
2. $V(\mathbf{x}) > 0$ for all $\mathbf{x} \neq \mathbf{0}$ in a neighbourhood of the origin (*positive definite*).

Its time derivative along trajectories of the system is

$$\dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}) = \nabla V(\mathbf{x})^T f(\mathbf{x}). \quad (2.73)$$

Theorem 2.7. Lyapunov's Direct Method (Second Method)

Let $\mathbf{x}^* = \mathbf{0}$ be an equilibrium of $\dot{\mathbf{x}} = f(\mathbf{x})$ and let $V(\mathbf{x})$ be a Lyapunov function candidate in a domain $\mathcal{D}$ containing the origin.

1. If $\dot{V}(\mathbf{x}) \leq 0$ in $\mathcal{D}$, the equilibrium is **stable** (in the sense of Lyapunov).
2. If $\dot{V}(\mathbf{x}) < 0$ for all $\mathbf{x} \neq \mathbf{0}$ in $\mathcal{D}$, the equilibrium is **asymptotically stable**.
3. If the conditions above hold with $\mathcal{D} = \mathbb{R}^n$ and, in addition, $V(\mathbf{x}) \rightarrow \infty$ as $\|\mathbf{x}\| \rightarrow \infty$ (*radially unbounded*), the equilibrium is **globally asymptotically stable**.

The power of the theorem lies in the fact that we never need to solve the differential equation—we only need to find a single scalar function V with the right properties.

Example 2.5. Pendulum with Damping

Consider a simple pendulum of mass m and length ℓ with viscous damping $b > 0$:

$$m\ell^2\ddot{\theta} + b\dot{\theta} + mg\ell \sin \theta = 0. \quad (2.74)$$

The state is $\mathbf{x} = (\theta, \dot{\theta})^T$ and the downward equilibrium is $\mathbf{x}^* = \mathbf{0}$. Choose the total mechanical energy as a Lyapunov candidate:

$$V(\theta, \dot{\theta}) = \underbrace{\frac{1}{2} m\ell^2 \dot{\theta}^2}_{\text{kinetic}} + \underbrace{mg\ell (1 - \cos \theta)}_{\text{potential}}. \quad (2.75)$$

Clearly $V(\mathbf{0}) = 0$ and $V > 0$ for $(\theta, \dot{\theta}) \neq \mathbf{0}$ with $|\theta| < \pi$. Differentiating along trajectories:

$$\dot{V} = m\ell^2 \dot{\theta} \ddot{\theta} + mg\ell \sin \theta \dot{\theta} = \dot{\theta}(m\ell^2 \ddot{\theta} + mg\ell \sin \theta) = \dot{\theta}(-b\dot{\theta}) = -b\dot{\theta}^2 \leq 0. \quad (2.76)$$

Since $\dot{V} \leq 0$, the equilibrium is stable. Because $\dot{V} = 0$ only on the set $\{\dot{\theta} = 0\}$ and no trajectory other than the equilibrium can stay there forever (by LaSalle's invariance principle), the equilibrium is in fact **asymptotically stable**.

Physically, the damper dissipates energy at rate $b\dot{\theta}^2$, and the total energy V is a non-increasing “altitude” that the system rolls down until it reaches the bottom of the energy landscape [16].

In Plain Language 2.4. Lyapunov Functions Are Like Altitude

Lyapunov functions are like altitude—if you can prove you are always going downhill, you must eventually reach the valley. The function $V(\mathbf{x})$ measures how “high” the state is above the equilibrium, and the condition $\dot{V} \leq 0$ guarantees that the system never climbs. Finding a good Lyapunov function is an art: energy is often the first candidate, but there are systematic construction methods for certain system classes [31].

Mathematical Principle 2.3. Lyapunov Theory as Foundation

Lyapunov stability theory is the foundation of virtually all stability proofs in modern control. Every adaptive controller, every robust controller, and every learning-based stability certificate in Chapters 11–12 ultimately relies on constructing a Lyapunov (or Lyapunov-like) function and showing that its derivative is non-positive along closed-loop trajectories.

2.19 Further Reading

The state-space framework presented in this chapter traces its origins to Kalman’s foundational 1960 paper [15], which introduced the concepts of controllability and observability that remain central to modern control engineering. That single paper launched an entire field; readers who wish to appreciate its full scope should read the original.

For nonlinear extensions of the state-space ideas developed here, Slotine and Li [31] provide an accessible treatment oriented toward robotic systems, covering sliding-mode control, adaptive control, and Lyapunov-based design—all formulated in state space. A more mathematically rigorous treatment of nonlinear systems theory, including stability in the sense of Lyapunov, input-output stability, and singular perturbation methods, can be found in Khalil [16].

Spong, Hutchinson, and Vidyasagar [32] develop state-space models specifically for robotic manipulators and mobile robots, making explicit the connection between the equations of motion derived from Lagrangian or Newton–Euler methods and the $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ formalism of this chapter.

Finally, readers should note the deep connection between this chapter and Chapter 11 (Lagrangian Mechanics): the Euler–Lagrange equations provide the equations of motion that, once cast in first-order form, populate the state-space models studied here. Understanding both perspectives—the energy-based derivation of dynamics and the state-space representation of those dynamics—is essential for modern robotics.

2.20 Exercises

Computational Exercises

1. For the DC motor from Example 2.3.2, suppose $L = 0.01$ H, $R = 1$ Ω , $K_e = 0.02$ V·s/rad, $K_t = 0.02$ N·m/A, $J = 0.001$ kg·m², $b = 0.001$ N·m·s/rad. Write the system in the form $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}V_{\text{in}}$. Compute the eigenvalues of $\mathbf{A}$ numerically and classify the open-loop stability.
2. For the two-tank system (Example 2.3.3), assume $A = 0.1$ m², $C = 0.05$, $C_{\text{out}} = 0.03$. Starting from $h_1(0) = 0.5$ m, $h_2(0) = 0.2$ m, with constant inflow $Q_{\text{in}} = 0.01$ m³/s, simulate the system for 100 seconds and plot $h_1(t)$ and $h_2(t)$.
3. Consider the mass-spring-damper with $m = 2$ kg, $c = 4$ N·s/m, $k = 3$ N/m. Compute the damping ratio ζ . Is the system underdamped, critically damped, or overdamped? Plot the phase portrait for initial conditions $(q, \dot{q}) = (1, 0)$ and $(0.5, 1)$.
4. For the pendulum system (Example 2.3.1) with $L = 1$ m, $g = 9.81$ m/s², write the nonlinear state equations. Linearize around both the upright ($\theta = 0$) and inverted ($\theta = \pi$) equilibria. Classify each equilibrium.

Conceptual Exercises

5. Explain in your own words why knowledge of position alone is insufficient to specify the state of a system. Use a simple example (e.g., a mass on a spring) to illustrate why velocity must also be included.
6. Why did Rudolf Kalman's 1960 paper on state-space representations represent a breakthrough compared to classical transfer-function methods? What specific limitations of transfer functions does state space overcome?
7. Consider a 3-dimensional system with eigenvalues $\lambda_1 = -2$, $\lambda_2 = 1 + 2i$, $\lambda_3 = 1 - 2i$. Sketch a qualitative phase portrait. What will happen to a trajectory starting near the origin?
8. A system is controllable but not observable. What does this mean physically? Can you still design a feedback controller, and if so, what information would you need?
9. The Kalman rank condition for controllability checks that $\text{rank}([\mathbf{B}, \mathbf{A}\mathbf{B}, \dots, \mathbf{A}^{n-1}\mathbf{B}]) = n$. Why is it sufficient to check only up to $\mathbf{A}^{n-1}\mathbf{B}$ and not higher powers?

Proof-Based Exercises

10. Prove that if $\mathbf{A}$ is diagonalizable with eigenvalues $\lambda_1, \dots, \lambda_n$, then the matrix exponential is:

$$e^{\mathbf{A}t} = \mathbf{P} \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_n t}) \mathbf{P}^{-1} \quad (2.77)$$

where $\mathbf{P}$ is the matrix of eigenvectors.

11. Derive the Jacobian matrix for the pendulum system $\ddot{\theta} + (g/L)\sin(\theta) = \tau$ at the bottom equilibrium ($\theta = 0$). Show that the linearized system is $\ddot{\theta} = -(g/L)\theta$ and classify the equilibrium's stability.

12. Show that for a 2D system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ with $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the eigenvalues are $\lambda = \frac{(a+d) \pm \sqrt{(a-d)^2 + 4bc}}{2}$. Under what conditions are the eigenvalues (i) both real, (ii) complex conjugates?
13. Prove that the controllability matrix $\mathbf{C}_{\text{rank}} = [\mathbf{B}, \mathbf{A}\mathbf{B}, \dots, \mathbf{A}^{n-1}\mathbf{B}]$ has full rank if and only if there is no eigenvector of $\mathbf{A}$ orthogonal to $\mathbf{B}^T$.
14. Using the variation of constants formula, show that the solution to $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}(t)$ with $\mathbf{x}(0) = \mathbf{x}_0$ is:

$$\mathbf{x}(t) = e^{\mathbf{A}t}\mathbf{x}_0 + \int_0^t e^{\mathbf{A}(t-\tau)}\mathbf{B}\mathbf{u}(\tau) d\tau \quad (2.78)$$

15. (Challenge) Prove that a system is observable (can reconstruct initial state from output) if and only if the observability matrix $\mathbf{O} = [\mathbf{C}^T, \mathbf{A}^T\mathbf{C}^T, \dots, (\mathbf{A}^T)^{n-1}\mathbf{C}^T]^T$ has full rank. Hint: Use the dual system.

Chapter 3

Configuration Space and Degrees of Freedom

To command a body to move, we must first establish exactly where it is. We do not use the raw atoms of the universe to map a body; we use its independent coordinates—its configuration.

In Plain Language 3.1. Context & Use Case: Mapping the Possible

The Math You Will See: We define the **Configuration Space** ($\mathcal{Q}$), count Degrees of Freedom (DOF) using Grübler's formula, establish the mathematics of **differentiable manifolds**, define **Forward Kinematics** maps, analyze reachable workspace, and distinguish **holonomic** from **non-holonomic** constraints.

Why We Need It: If you want a robot arm to pick up a coffee cup, you know where the cup is in physical 3D space (X, Y, Z). But you cannot command the robot's X, Y, Z position directly! You can only command the motors at its joints (its "configuration"). We must build mathematical maps that translate between the physical space where the coffee cup lives, and the abstract angular "configuration space" where the robot's motors live. The dimension of this space is determined by DOF; its topology and geometry are governed by differential geometry.

All Variables Defined: $\mathbf{q}$ represents joint angles, $\mathcal{Q}$ is the configuration manifold, $\mathbf{p}_{end}$ is the end-effector position, and m, n, c denote the number of bodies, DOF, and independent constraints in Grübler's formula.

3.1 Historical Context: From Lagrange to Riemann

The mathematical framework of Configuration Space owes its development to three foundational ideas.

Lagrange's Generalized Coordinates (1788): Joseph-Louis Lagrange introduced the concept that one need not track every particle's Cartesian coordinates. Instead, for a constrained mechanical system, one could choose a minimal set of independent variables $q_1, q_2, \dots, q_n$ whose values completely specify the system's configuration. This abstraction—choosing coordinates that *already respect* the system's constraints—transformed mechanics from a coordinate-fixing discipline into a geometric one. The Lagrangian formulation of

mechanics, summarized in the principle of least action, operates naturally in generalized coordinates.

Riemann's Manifold Concept (1854): Bernhard Riemann's Habilitationsvortrag introduced the idea of manifolds: spaces that are locally Euclidean but globally curved. Riemann showed that curvature itself is intrinsic to a space, not imposed by an embedding in a higher-dimensional flat space. This insight enables us to treat Configuration Space not as a rigid subset of $\mathbb{R}^n$, but as an independent geometric object with its own metric and topology.

Modern Differential Geometry: The 20th-century formalization by Élie Cartan, Charles Ehresmann, and others crystallized these ideas into a rigorous calculus on manifolds. Today, Configuration Space is understood as a smooth differentiable manifold, the tangent bundle TQ forms the state space, and the Jacobian matrix encodes the instantaneous kinematic relationship between joint velocities and end-effector velocities.

3.2 Degrees of Freedom (DOF) and Grübler's Formula

In Chapter 2, we learned that the State Space $\mathcal{X}$ consists of both position coordinates ($\mathbf{q}$) and velocity coordinates ($\dot{\mathbf{q}}$). But how many position coordinates does our system actually have?

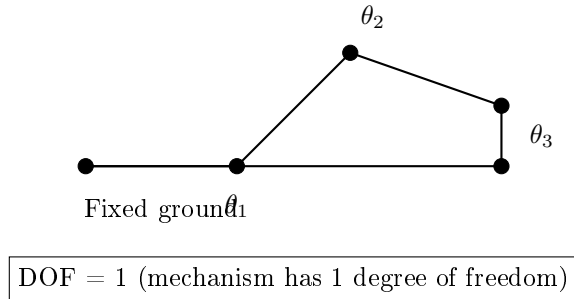


Figure 3.1: A four-bar linkage mechanism. Although there are three moving links, the geometric constraints from the four revolute joints reduce the mechanism to 1 degree of freedom. Specifying θ_1 uniquely determines θ_2 and θ_3 .

3.2.1 The Concept of Degrees of Freedom

Consider a single speck of dust floating completely unconstrained in a room. To perfectly define where it is, you need 3 coordinates: x, y, z . If it is instead a rigid body floating in the room, knowing x, y, z only fixes its center of mass. It could be pitched entirely backward, or rolled on its side. It therefore requires three additional **angles** (roll, pitch, yaw) defining its rotation.

This unconstrained rigid body has exactly **6 Degrees of Freedom (DOF)**. It requires a minimum of 6 independent parameters (or coordinates) to fully define its posture in 3D Euclidean space.

Definition 3.1. Degrees of Freedom (DOF)

The Degrees of Freedom of a system represents the absolute minimum number of independent scalar coordinates required to perfectly and unambiguously define the position and orientation of every single particle in that system. Equivalently, DOF is the dimension of the Configuration Space $\mathcal{Q}$.

If you connect two floating rigid bodies together with a perfectly rigid bar (welded so they cannot move relative to each other), you do not have 12 DOF. Since the distance and orientation between them are perfectly locked by the weld, you still only have 6 DOF for the combined assembly. The weld acts as a geometric *constraint*. Every independent constraint you add mathematically subtracts one DOF. A hinge joint locking two 6-DOF bodies together removes 5 DOF, leaving exactly 1 DOF of relative rotation between them.

3.2.2 Grübler's Formula

For planar mechanisms, we can count DOF systematically using **Grübler's formula**, developed by Martin Grübler in 1883. This formula gives the degrees of freedom of a kinematic chain.

Theorem 3.1. Grübler's Formula for Planar Mechanisms

For a planar mechanism with m rigid bodies (including ground as one body), n movable links, and c independent geometric constraints, the degrees of freedom are given by:

$$\text{DOF} = 3n - \sum_i c_i \quad (3.1)$$

where the sum is over all c independent constraints imposed by joints.

Equivalently, if there are j joints with degrees of freedom $d_1, d_2, \dots, d_j$:

$$\text{DOF} = 3(n) - \sum_{i=1}^j (3 - d_i) = 3n - 3j + \sum_{i=1}^j d_i \quad (3.2)$$

Proof. In a planar mechanism, each of n movable bodies independently has 3 DOF (2 translational + 1 rotational). If there were no constraints, the total would be $3n$. Each joint imposes geometric constraints: a revolute joint (pin joint) constrains 2 DOF of relative motion, a prismatic joint constrains 2 DOF, and higher-order joints constrain accordingly. The net DOF equals the free DOF minus the sum of imposed constraint DOF. $\square$

For mechanisms with spatial (3D) linkages, the formula becomes:

Theorem 3.2. Grübler's Formula for Spatial Mechanisms

For a spatial mechanism with n movable links and j joints where joint i permits d_i relative DOF:

$$\text{DOF} = 6n - \sum_{i=1}^j (6 - d_i) \quad (3.3)$$

3.2.3 Worked Example 1: Planar 4-Bar Linkage

Example 3.1. Four-Bar Mechanism DOF

Consider the classic planar 4-bar linkage: a mechanism with 4 rigid links (one of which is the ground frame) connected by 4 revolute joints.

We have:

- $n = 3$ movable links (link 1, 2, 3; ground is fixed)
- $j = 4$ revolute joints
- Each revolute joint has $d_i = 1$ (one DOF: rotation about the pin)

Applying Grübler's formula:

$$\text{DOF} = 3n - 3j + \sum_{i=1}^4 d_i \quad (3.4)$$

$$= 3(3) - 3(4) + 4(1) \quad (3.5)$$

$$= 9 - 12 + 4 = 1 \quad (3.6)$$

A four-bar linkage has exactly **1 DOF**. Once you specify the angle of one input link (the crank), the positions and angles of all other links are completely determined by the geometric constraints. This is why a four-bar can be driven by a single motor to produce a controlled path motion.

3.2.4 Worked Example 2: Spatial Stewart Platform

Example 3.2. Stewart Platform DOF

The Stewart platform (also called a Gough-Stewart platform) is a parallel manipulator consisting of a base platform, a movable top platform, and six variable-length actuators connecting them.

We apply the Grübler–Kutzbach formula for spatial mechanisms. Each leg connects the base to the top platform through a universal joint (2 DOF) at the base, a prismatic actuator (1 DOF), and a spherical joint (3 DOF) at the top:

- $n = 14$ total bodies: 1 top platform + 6 upper leg segments + 6 lower leg segments + ground
- $j = 18$ joints: 6 universal (2 DOF each) + 6 prismatic (1 DOF each) + 6 spherical (3 DOF each)
- Total joint freedoms: $\sum d_i = 6 \times 2 + 6 \times 1 + 6 \times 3 = 36$

Applying the spatial Grübler formula ($\lambda = 6$ for spatial mechanisms):

$$\text{DOF} = \lambda(n - 1 - j) + \sum_{i=1}^j d_i = 6(14 - 1 - 18) + 36 = 6(-5) + 36 = 6. \quad (3.7)$$

A non-degenerate Stewart platform therefore has exactly **6 DOF** in its end-effector pose and requires 6 actuators (the prismatic joints) to control all 6 spatial DOF. This makes it a fully parallel manipulator: all actuators work in parallel to position and orient the top platform.

3.2.5 Worked Example 3: Human Arm Kinematics

Example 3.3. Human Arm DOF

Consider a simplified model of the human arm: shoulder, elbow, and wrist.

- Shoulder: ball-and-socket joint ≈ 3 DOF (roll, pitch, yaw about the shoulder center)
- Elbow: hinge joint ≈ 1 DOF (flexion/extension)
- Wrist: ball-and-socket joint ≈ 3 DOF (pronation/supination, flexion/extension, radial/ulnar deviation)

Total: $3 + 1 + 3 = 7$ DOF in a gross anatomical model. (The human arm actually has 7 DOF, which is why we can reach and reorient objects in multiple ways—we have redundancy.)

The hand position can be specified by 3 Cartesian coordinates (x, y, z) and the wrist orientation by 3 angles (roll-pitch-yaw). Thus the end-effector pose has 6 DOF. With 7 joint angles, the arm is *redundant*: there are infinitely many joint configurations $\mathbf{q} \in \mathbb{R}^7$ that achieve the same end-effector pose. This redundancy is exploited in human motor control to avoid obstacles and optimize reach.

3.3 The Configuration Space $\mathcal{Q}$ and Topological Manifolds

The collection of all valid configurations a system can possibly achieve—accounting for all of its physical constraints, linkages, pins, and bearings—is called its **Configuration Space**, mathematically denoted by the symbol $\mathcal{Q}$.

Like state space, configuration space is an abstract, multi-dimensional geometric surface. It is, strictly speaking, a mathematical **manifold** [23]. The number of dimensions of this space is exactly equal to the system's Degrees of Freedom.

Definition 3.2. Topological Manifold

A *topological manifold* of dimension n is a topological space $\mathcal{M}$ such that:

1. $\mathcal{M}$ is Hausdorff (any two distinct points have disjoint open neighborhoods),
2. $\mathcal{M}$ is second-countable (has a countable basis for its topology),
3. Every point $p \in \mathcal{M}$ has an open neighborhood U such that U is homeomorphic

to an open subset of $\mathbb{R}^n$.

In less formal terms: a manifold *locally looks like Euclidean space*, but globally may have a curved or non-trivial topology.

3.3.1 Examples of Manifolds

Example 3.4. The Circle S^1

The circle $S^1 = \{(x, y) \in \mathbb{R}^2 : x^2 + y^2 = 1\}$ is a 1-dimensional manifold. Globally, it closes on itself and is topologically distinct from a line. However, locally, any small arc of the circle looks like a line segment from $\mathbb{R}^1$.

Example 3.5. The 2-Sphere S^2

The 2-sphere $S^2 = \{(x, y, z) \in \mathbb{R}^3 : x^2 + y^2 + z^2 = 1\}$ is a 2-dimensional manifold (the Earth's surface). Globally, it is closed and without boundary. Locally, near any point (except the poles in certain coordinates), it looks like a flat 2D plane.

Example 3.6. The Torus T^2

The torus $T^2 = S^1 \times S^1$ is the Cartesian product of two circles. It is a 2-dimensional manifold. Topologically, it can be visualized as the surface of a doughnut: closed without boundary, genus 1.

In Plain Language 3.2. Configuration Space vs. State Space

A system with n Degrees of Freedom has a Configuration Space $\mathcal{Q}$ of dimension n , defined completely by the vector $\mathbf{q} = [q_1, q_2, \dots, q_n]^T$.

The corresponding **State Space** $\mathcal{X}$ (from Chapter 2) is the configuration space multiplied by the velocity of those coordinates, meaning state space always has $2n$ dimensions.

$$\text{State } \mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T \in \mathcal{X} = T\mathcal{Q}$$

(Where $T\mathcal{Q}$ represents the Tangent Bundle of the configuration manifold).

3.4 Charts, Atlases, and Coordinate Patches

While the configuration manifold $\mathcal{Q}$ is curved globally, our primary computational tool is always Euclidean coordinates. We use local maps called *charts* to cover the manifold piecewise with flat coordinate systems.

Definition 3.3. Chart

A *chart* on a manifold $\mathcal{Q}$ is a pair (U, φ) where:

- $U \subseteq \mathcal{Q}$ is an open set,
- $\varphi : U \rightarrow \mathbb{R}^n$ is a homeomorphism (continuous bijection with continuous inverse) onto an open subset of $\mathbb{R}^n$.

The map φ assigns local coordinates (real numbers) to each point in U .

Definition 3.4. Atlas

An *atlas* on a manifold $\mathcal{Q}$ is a collection of charts $\{(U_i, \varphi_i)\}_{i \in I}$ such that:

- The sets U_i cover the entire manifold: $\mathcal{Q} = \bigcup_{i \in I} U_i$,
- The charts are pairwise compatible in a sense we clarify below.

3.4.1 Worked Example: Charts on the Circle S^1 **Example 3.7. Two Charts Are Necessary for S^1**

Consider the unit circle $S^1 = \{(x, y) : x^2 + y^2 = 1\}$. Can we use a single chart $\varphi : S^1 \rightarrow \mathbb{R}$ that maps every point on the circle to a real number?

Attempt 1: Naive Angle Parameterization

Define $\varphi(\theta) = \theta$ where $\theta \in [0, 2\pi)$ is the angle from the positive x -axis. This gives a bijection between most of S^1 and $[0, 2\pi)$. However, this map has two problems:

- The interval $[0, 2\pi)$ is not homeomorphic to $\mathbb{R}$ (it is half-open and bounded).
- The map is not continuous at the boundary where $\theta = 0$ and $\theta = 2\pi$ identify the same point.

A single chart cannot cover all of S^1 continuously.

Solution: Two Overlapping Charts

Define two open sets:

$$U_1 = \{(x, y) \in S^1 : y > -\epsilon\} \quad (\text{excludes a small arc at the bottom}) \quad (3.8)$$

$$U_2 = \{(x, y) \in S^1 : y < \epsilon\} \quad (\text{excludes a small arc at the top}) \quad (3.9)$$

where ϵ is small.

On U_1 , use the chart (U_1, φ_1) where $\varphi_1(x, y) = x$. Since $y > -\epsilon$, the x -coordinate alone determines the point (by the Pythagorean theorem, $x \in (-1, 1)$), and φ_1 maps homeomorphically onto the interval $(-1, 1) \subset \mathbb{R}$.

On U_2 , use the chart (U_2, φ_2) where $\varphi_2(x, y) = -x$. This also maps homeomorphically onto $(-1, 1)$.

The two charts overlap on $U_1 \cap U_2 = \{(x, y) \in S^1 : -\epsilon < y < \epsilon\}$. This overlap is essential: we have covered the entire circle with two overlapping charts, each mapping to Euclidean space $\mathbb{R}$.

Key Insight: The necessity of two charts reflects the topological fact that the circle is not contractible: you cannot continuously shrink it to a point. This global topology cannot be captured by a single local coordinate system.

3.5 Transition Maps and Smooth Compatibility

When two charts overlap, we must ensure they agree on their shared region. This is formalized by *transition maps*.

Definition 3.5. Transition Map

If (U_i, φ_i) and (U_j, φ_j) are two charts whose domains overlap, the *transition map* is defined as:

$$\tau_{ij} = \varphi_j \circ \varphi_i^{-1} : \varphi_i(U_i \cap U_j) \rightarrow \varphi_j(U_i \cap U_j) \quad (3.10)$$

This map takes coordinates from one chart and converts them to coordinates in another.

For a *smooth* (or *differentiable*) manifold, we require all transition maps to be smooth (infinitely differentiable).

Definition 3.6. Differentiable Manifold

A differentiable manifold is a topological manifold equipped with a differentiable structure: an atlas where every transition map is C^∞ (infinitely differentiable).

3.5.1 Worked Example: Transition Map on S^1

Example 3.8. Transition Map on the Circle

Using the two charts from Example 3.4.1:

- Chart 1: (U_1, φ_1) where $\varphi_1(x, y) = x$ for $y > -\epsilon$.
- Chart 2: (U_2, φ_2) where $\varphi_2(x, y) = -x$ for $y < \epsilon$.

On the overlap $U_1 \cap U_2$, we have points (x, y) with $-\epsilon < y < \epsilon$.

In chart 1 coordinates, a point is labeled by $x_1 = x \in (-1, 1)$.

In chart 2 coordinates, the same point is labeled by $x_2 = -x \in (-1, 1)$.

The transition map is:

$$\tau_{12}(x_1) = -x_1 \quad (3.11)$$

Equivalently, going the other direction:

$$\tau_{21}(x_2) = -x_2 \quad (3.12)$$

Both are clearly C^∞ (linear functions, hence infinitely differentiable). This smoothness ensures that the calculus we do with one chart's coordinates yields the same results when computed in another chart's coordinates.

3.6 Tangent Vectors, Curves, and Derivations

While the manifold $\mathcal{Q}$ represents all valid positions (configurations), motion requires us to talk about velocities. Because the manifold is curved, a velocity vector cannot "live" on the surface itself—it would point off the surface tangentially!

Definition 3.7. Tangent Space $T_q\mathcal{Q}$

At any specific configuration $q \in \mathcal{Q}$, the *Tangent Space* $T_q\mathcal{Q}$ is a vector space whose elements are called *tangent vectors*. It is the space of all possible velocities at q .

There are two rigorous ways to define tangent vectors: as equivalence classes of curves, and as derivations on function algebras. Both are equivalent, and understanding both perspectives is essential.

3.6.1 Tangent Vectors as Equivalence Classes of Curves

Definition 3.8. Tangent Vector via Curves

Let $q \in \mathcal{Q}$ be a point on the manifold. Two smooth curves $\gamma_1, \gamma_2 : (-\epsilon, \epsilon) \rightarrow \mathcal{Q}$ with $\gamma_1(0) = \gamma_2(0) = q$ are said to be *tangent at q in a chart (U, φ)* if:

$$\left. \frac{d}{dt} \right|_{t=0} \varphi(\gamma_1(t)) = \left. \frac{d}{dt} \right|_{t=0} \varphi(\gamma_2(t)) \quad (3.13)$$

(The derivatives of their coordinate expressions agree at $t = 0$.)

A *tangent vector* at q is an equivalence class $[\gamma]$ of curves all tangent to each other at

q . The value $\mathbf{v} = \frac{d}{dt}\big|_{t=0} \varphi(\gamma(t)) \in \mathbb{R}^n$ is the coordinate representation of the tangent vector in the chart.

This definition is independent of the chart chosen: if two tangent vectors agree in one chart's coordinates, they agree in all charts' coordinates (by properties of transition maps).

3.6.2 Tangent Vectors as Derivations

An alternative algebraic viewpoint: a tangent vector is a linear operator on smooth functions.

Definition 3.9. Tangent Vector via Derivations

A *derivation at q* is a linear map $v : C^\infty(\mathcal{Q}) \rightarrow \mathbb{R}$ (where $C^\infty(\mathcal{Q})$ is the algebra of smooth real-valued functions on $\mathcal{Q}$) such that:

$$v(fg) = v(f)g(q) + f(q)v(g) \quad (\text{Leibniz rule}) \quad (3.14)$$

A *tangent vector at q* can equivalently be defined as a derivation at q .

The Leibniz rule (or product rule) encodes the essential algebraic property of derivatives. A tangent vector is "an infinitesimal displacement" in the sense that it measures the first-order change in functions along infinitesimal directions.

3.6.3 Equivalence of the Two Definitions

Theorem 3.3. Equivalence of Tangent Vector Definitions

The space of equivalence classes of curves passing through q (Definition 3.8) is isomorphic, as a vector space, to the space of derivations at q (Definition 3.9). Both have dimension n (the dimension of $\mathcal{Q}$).

Proof. Given a curve γ with $\gamma(0) = q$, define a derivation by:

$$v_\gamma(f) = \frac{d}{dt}\bigg|_{t=0} f(\gamma(t)) \quad (3.15)$$

This satisfies the Leibniz rule by the product rule of calculus. Conversely, given a derivation v , one can construct curves tangent to that direction. The maps between these spaces are linear isomorphisms, preserving dimension. $\square$

The intuition: curves give a geometric picture of infinitesimal motion; derivations give an algebraic picture of how functions change. Both capture the same mathematical object.

3.6.4 The Tangent Bundle $T\mathcal{Q}$

Definition 3.10. Tangent Bundle

The *Tangent Bundle* $T\mathcal{Q}$ is the disjoint union of all tangent spaces:

$$T\mathcal{Q} = \bigcup_{q \in \mathcal{Q}} T_q\mathcal{Q} \quad (3.16)$$

Equipped with a natural smooth structure, $T\mathcal{Q}$ is itself a differentiable manifold of dimension $2n$ (if $\mathcal{Q}$ has dimension n).

An element of $T\mathcal{Q}$ is a pair $(q, \mathbf{v})$ where $q \in \mathcal{Q}$ is a configuration and $\mathbf{v} \in T_q\mathcal{Q}$ is a velocity at that configuration.

In Plain Language 3.3. Mathematical Reminder: Tangent Bundle vs. State Space

Whenever you see the State Vector $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$, recall its geometric meaning. The upper half ($\mathbf{q}$) is a point on the curved manifold $\mathcal{Q}$. The lower half ($\dot{\mathbf{q}}$) is a vector living in the flat tangent space $T_q\mathcal{Q}$ attached specifically to that point $\mathbf{q}$. Together, they coordinatize the Tangent Bundle $T\mathcal{Q}$, which is exactly the State Space $\mathcal{X}$.

3.7 Generalized Coordinates and Forward Kinematics

When we model a complex physical system (such as the 15-DOF human golfer in Volume III), we do not track the 3D (x, y, z) position of the golfer's wrist, elbow, and shoulder separately, and then painstakingly write huge algebraic constraint equations ensuring their bones don't stretch or compress during simulation.

Instead, we use **Generalized Coordinates**. Since an elbow is a pin joint allowing only 1 DOF of rotation, we abandon (x, y, z) mapping entirely and instead just track the single angle θ_{elbow} . If we define the angle of the shoulder θ_{shoulder} , and the fixed length of the humerus bone L_1 , the Cartesian position of the elbow is instantly known.

Definition 3.11. Generalized Coordinates

Generalized coordinates are the minimal set of independent variables $q_1, q_2, \dots, q_n$ that perfectly span the Configuration Space, such that every point in $\mathcal{Q}$ is uniquely represented by some choice of $(q_1, \dots, q_n)$. They are the most mathematically efficient map of the mechanism's geometry. In robotics, $\mathbf{q}$ almost always refers to the physical joint angles of the motors.

3.7.1 Forward Kinematics

The mathematical function that translates generalized coordinates $\mathbf{q}$ back into the physical 3D Euclidean space of the real world is called **Forward Kinematics**.

Definition 3.12. Forward Kinematics Map

If $\mathbf{p}_{end} = [x, y, z]^T \in \mathbb{R}^3$ is the Cartesian workspace position of the end-effector of a robotic arm, Forward Kinematics is the generally nonlinear function $f_{kin} : \mathcal{Q} \rightarrow \mathbb{R}^3$ defined by:

$$\mathbf{p}_{end} = f_{kin}(\mathbf{q}) \quad (3.17)$$

By using trigonometry and the chain rule of coordinate composition, we stack the lengths of the links and the angles of the joints $\mathbf{q}$ to compute exactly where the end-effector sits in physical space.

Example: 2-Link Planar Arm

For a simple 2-link robotic arm rotating on a flat 2D plane with fixed link lengths L_1, L_2 and variable joint angles q_1, q_2 :

$$x = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) \quad (3.18)$$

$$y = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) \quad (3.19)$$

The forward kinematics map is $f_{kin}(q_1, q_2) = [x(q_1, q_2), y(q_1, q_2)]^T$.

3.8 The Jacobian Matrix and the Kinematic Derivative

To study the instantaneous kinematic relationship between joint velocities $\dot{\mathbf{q}}$ and end-effector velocities $\dot{\mathbf{p}}$, we differentiate the forward kinematics [30].

Definition 3.13. Jacobian Matrix of Forward Kinematics

The Jacobian matrix of $f_{kin} : \mathcal{Q} \rightarrow \mathbb{R}^m$ at a configuration $\mathbf{q}$ is:

$$J_{kin}(\mathbf{q}) = \left. \frac{\partial f_{kin}}{\partial \mathbf{q}} \right|_{\mathbf{q}} = \begin{bmatrix} \frac{\partial f_1}{\partial q_1} & \dots & \frac{\partial f_1}{\partial q_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial q_1} & \dots & \frac{\partial f_m}{\partial q_n} \end{bmatrix} \quad (3.20)$$

where m is the dimension of the workspace (usually $m = 3$ for position in 3D space, or $m = 6$ if orientation is included).

By the chain rule, the end-effector velocity is related to joint velocity by:

$$\dot{\mathbf{p}} = J_{kin}(\mathbf{q})\dot{\mathbf{q}} \quad (3.21)$$

3.8.1 Worked Example: 2-Link Arm Jacobian

Example 3.9. Jacobian of 2-Link Planar Arm

For the 2-link planar arm from Section 3.7:

$$x(q_1, q_2) = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) \quad (3.22)$$

$$y(q_1, q_2) = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) \quad (3.23)$$

Compute the partial derivatives:

$$\frac{\partial x}{\partial q_1} = -L_1 \sin(q_1) - L_2 \sin(q_1 + q_2) \quad (3.24)$$

$$\frac{\partial x}{\partial q_2} = -L_2 \sin(q_1 + q_2) \quad (3.25)$$

$$\frac{\partial y}{\partial q_1} = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) \quad (3.26)$$

$$\frac{\partial y}{\partial q_2} = L_2 \cos(q_1 + q_2) \quad (3.27)$$

Thus, the Jacobian is:

$$J_{kin}(q_1, q_2) = \begin{bmatrix} -L_1 \sin(q_1) - L_2 \sin(q_1 + q_2) & -L_2 \sin(q_1 + q_2) \\ L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) & L_2 \cos(q_1 + q_2) \end{bmatrix} \quad (3.28)$$

This is a 2×2 matrix (workspace dimension 2, joint DOF 2). For any joint velocity $[\dot{q}_1, \dot{q}_2]^T$, the end-effector velocity is:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = J_{kin} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \quad (3.29)$$

The Jacobian depends on the configuration: it changes as the arm moves through $\mathcal{Q}$.

3.8.2 Worked Example: 3-Link Arm in 3D

Example 3.10. Jacobian of 3-Link Arm with Orientation

Consider a 3-link planar arm (same plane), but now we also track the orientation of the end-effector (the angle of the end link).

Forward kinematics:

$$x = L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) + L_3 \cos(q_1 + q_2 + q_3) \quad (3.30)$$

$$y = L_1 \sin(q_1) + L_2 \sin(q_1 + q_2) + L_3 \sin(q_1 + q_2 + q_3) \quad (3.31)$$

$$\theta = q_1 + q_2 + q_3 \quad (3.32)$$

The task space is now 3-dimensional: (x, y, θ) . The Jacobian is 3×3 :

$$J_{kin} = \begin{bmatrix} -L_1 \sin(q_1) - L_2 \sin(q_1 + q_2) - L_3 \sin(q_1 + q_2 + q_3) & -L_2 \sin(q_1 + q_2) - L_3 \sin(q_1 + q_2 + q_3) & -L_3 \sin(q_1 + q_2 + q_3) \\ L_1 \cos(q_1) + L_2 \cos(q_1 + q_2) + L_3 \cos(q_1 + q_2 + q_3) & L_2 \cos(q_1 + q_2) + L_3 \cos(q_1 + q_2 + q_3) & L_3 \cos(q_1 + q_2 + q_3) \\ 1 & 1 & 1 \end{bmatrix} \quad (3.33)$$

When the Jacobian is square and full rank, the arm is in a non-singular configuration: there is a unique relationship between joint velocities and task velocities. When the determinant is zero, the arm enters a singularity, and some task velocities become unreachable.

3.9 Inverse Kinematics

The forward kinematics problem is: given joint angles $\mathbf{q}$, find the end-effector position $\mathbf{p} = f_{kin}(\mathbf{q})$.

The inverse problem is fundamentally harder: given a desired end-effector position $\mathbf{p}_d$, find the joint angles $\mathbf{q}$ such that $f_{kin}(\mathbf{q}) = \mathbf{p}_d$.

Definition 3.14. Inverse Kinematics Problem

The Inverse Kinematics problem is: solve for $\mathbf{q}$ given $\mathbf{p}_d$:

$$f_{kin}(\mathbf{q}) = \mathbf{p}_d \quad (3.34)$$

Inverse kinematics is harder for three reasons:

1. *Non-uniqueness*: Multiple joint configurations may reach the same end-effector position. For a 2-link arm, you can reach a target by bending the elbow forward (elbow-up) or backward (elbow-down).
2. *Non-existence*: Some end-effector positions may be unreachable (outside the workspace).
3. *Nonlinearity*: The forward kinematics is nonlinear (contains sines and cosines), making the inverse algebraically intractable in general.

3.9.1 Geometric Approach to 2-Link IK

For the 2-link planar arm, we can solve geometrically.

Example 3.11. 2-Link Arm Inverse Kinematics (Geometric)

Given $\mathbf{p}_d = [x_d, y_d]^T$ and link lengths L_1, L_2 , find $[q_1, q_2]^T$.

Step 1: Use the law of cosines to find the distance $r = \sqrt{x_d^2 + y_d^2}$ from the base to the target. The angle q_2 (elbow angle) satisfies:

$$r^2 = L_1^2 + L_2^2 - 2L_1L_2 \cos(\pi - q_2) \quad (3.35)$$

(Here $\pi - q_2$ is the interior angle at the elbow.)

Solving:

$$\cos(q_2) = \frac{L_1^2 + L_2^2 - r^2}{2L_1L_2} \quad (3.36)$$

Step 2: Once q_2 is known, the elbow position is constrained. The shoulder angle q_1 can be found from:

$$q_1 = \text{atan2}(y_d, x_d) - \text{atan2}(L_2 \sin(q_2), L_1 + L_2 \cos(q_2)) \quad (3.37)$$

This geometric approach yields, generically, two solutions (elbow-up and elbow-down), corresponding to the two square roots in solving the cosine equation.

3.9.2 Numerical Inverse Kinematics: Newton-Raphson

For higher DOF arms or when geometric solutions don't exist, we use numerical optimization.

Theorem 3.4. Newton-Raphson for Inverse Kinematics

Define the error function:

$$\mathbf{e}(\mathbf{q}) = \mathbf{p}_d - f_{kin}(\mathbf{q}) \quad (3.38)$$

The Newton-Raphson iteration is:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \alpha_k J_{kin}(\mathbf{q}_k)^\dagger \mathbf{e}(\mathbf{q}_k) \quad (3.39)$$

where $J_{kin}^\dagger$ is the pseudo-inverse of the Jacobian and α_k is a step size (often $\alpha_k = 1$ initially, or line search if convergence is slow).

When J_{kin} is square and full rank, $J_{kin}^\dagger = J_{kin}^{-1}$.

This iterative method starts from an initial guess $\mathbf{q}_0$ and updates it repeatedly until $\|\mathbf{e}(\mathbf{q}_k)\|$ is small. Convergence is fast (quadratic) near a solution, but global convergence is not guaranteed.

3.10 Workspace Analysis: Reachable and Dexterous Workspace

Once we have a forward kinematics function, a key question is: what end-effector positions can the robot actually reach?

Definition 3.15. Reachable Workspace

The *reachable workspace* of a robot arm is the set of all end-effector positions $\mathbf{p}$ that can be reached by at least one configuration $\mathbf{q}$:

$$\mathcal{W}_{reach} = \{\mathbf{p} \in \mathbb{R}^m : \exists \mathbf{q} \in \mathcal{Q}, f_{kin}(\mathbf{q}) = \mathbf{p}\} \quad (3.40)$$

It is the image of f_{kin} .

Definition 3.16. Dexterous Workspace

The *dexterous workspace* is the subset of reachable workspace where the robot can orient the end-effector in all directions (or at least a specified set of directions):

$$\mathcal{W}_{dex} = \{\mathbf{p} : \exists \mathbf{q} \text{ reaching } \mathbf{p} \text{ with } \det(J_{kin}(\mathbf{q})) \neq 0\} \quad (3.41)$$

(Requires the Jacobian to be full rank in at least one configuration.)

For a 2-link arm with $L_1 = L_2 = 1$:

- The reachable workspace is an annulus (ring) with inner radius $|L_1 - L_2| = 0$ and outer radius $L_1 + L_2 = 2$. Any point (x, y) with $0 \leq \sqrt{x^2 + y^2} \leq 2$ can be reached.
- The dexterous workspace is smaller: a subset of the annulus where configurations exist with full-rank Jacobian.

3.11 The Frobenius Theorem and Integrability

A central question in constraint analysis is: *when does a velocity constraint actually reduce the reachable configuration space?* The answer lies in the **Frobenius Theorem**, which provides an algebraic test for *integrability* of a set of velocity constraints.

Definition 3.17. Smooth Distribution

A *smooth distribution* Δ on a manifold $\mathcal{Q}$ assigns to each point $\mathbf{q} \in \mathcal{Q}$ a linear subspace $\Delta(\mathbf{q}) \subseteq T_{\mathbf{q}}\mathcal{Q}$ of the tangent space. If $\dim \Delta(\mathbf{q}) = k$ for all $\mathbf{q}$, the distribution has *rank* k . A distribution is *integrable* if there exists a foliation of $\mathcal{Q}$ into k -dimensional submanifolds (leaves) whose tangent spaces coincide with Δ .

Intuitively, a distribution collects the “allowed velocity directions” at every configuration. If the distribution is integrable, those allowed velocities confine the system to a lower-dimensional surface—the constraint genuinely eliminates degrees of freedom. If the distribution is *not* integrable, the system can eventually reach any configuration despite the instantaneous velocity restriction.

Theorem 3.5. Frobenius Theorem

A smooth distribution Δ is integrable if and only if it is *involutive*: for every pair of smooth vector fields X, Y belonging to Δ , their Lie bracket also belongs to Δ :

$$X, Y \in \Delta \implies [X, Y] \in \Delta. \quad (3.42)$$

When the Frobenius condition *fails*—that is, when there exist vector fields $X, Y \in \Delta$ whose Lie bracket $[X, Y] \notin \Delta$ —the distribution is non-integrable. In mechanical terms, this means the velocity constraint is genuinely **non-holonomic**: it restricts instantaneous motion but does *not* reduce the reachable set of configurations.

Example 3.12. The Car: A Non-Integrable Constraint

Consider a car on the plane with configuration $\mathbf{q} = (x, y, \theta) \in \mathbb{R}^2 \times S^1$. The no-lateral-slip constraint is:

$$\dot{y} \cos \theta - \dot{x} \sin \theta = 0. \quad (3.43)$$

This defines a rank-2 distribution Δ spanned by the vector fields:

$$X_1 = \cos \theta \frac{\partial}{\partial x} + \sin \theta \frac{\partial}{\partial y}, \quad X_2 = \frac{\partial}{\partial \theta}. \quad (3.44)$$

Computing the Lie bracket:

$$[X_1, X_2] = -\sin \theta \frac{\partial}{\partial x} + \cos \theta \frac{\partial}{\partial y}. \quad (3.45)$$

Since $[X_1, X_2]$ is *not* a linear combination of X_1 and X_2 , the Frobenius condition fails. Therefore the constraint is **non-integrable**: despite the velocity restriction, the car can reach *any* configuration (x, y, θ) in $\mathbb{R}^2 \times S^1$.

Example 3.13. A Rigid Rod: An Integrable Constraint

Consider two particles in $\mathbb{R}^3$ connected by a rigid rod of length L . The constraint $\|\mathbf{r}_1 - \mathbf{r}_2\|^2 = L^2$ depends only on positions (not velocities). Differentiating yields a velocity constraint, but this velocity constraint *is* integrable—the original position-level equation defines a submanifold of the full configuration space. The Frobenius condition is satisfied, and the constraint permanently reduces the DOF from 6 to 5.

In Plain Language 3.4. Parallel Parking and Reachability

A car can parallel park into any spot despite only being able to drive forward/backward and turn—non-holonomic constraints restrict velocity but not reachability. The Frobenius theorem makes this precise: because the car's velocity constraint is non-integrable, repeated manoeuvres (sequences of steering and driving) generate motion in *every* direction of configuration space. This is why parallel parking works, even though you cannot slide the car sideways.

The Frobenius theorem thus provides the rigorous dividing line between constraints that reduce configuration space dimension (holonomic, integrable) and those that merely shape the geometry of feasible paths (non-holonomic, non-integrable). For a comprehensive treatment, see [23] Chapter 8 and [5].

3.12 Holonomic and Non-Holonomic Constraints

It is critical to distinguish between constraints that restrict position versus those that restrict velocity.

Definition 3.18. Holonomic Constraint

A *holonomic constraint* is a geometric constraint that restricts the actual position of the system. It can be expressed as an equation $\phi(\mathbf{q}) = 0$ involving only the generalized coordinates (not their derivatives). Holonomic constraints permanently reduce the dimension of the Configuration Space.

Example: A train locked to a physical track cannot deviate sideways; if $x(t)$ is its position along the track and $y(t)$ is its lateral deviation, the constraint $y(t) = 0$ for all time is holonomic. This is a geometric constraint on position.

Definition 3.19. Non-Holonomic Constraint

A *non-holonomic constraint* is a constraint on the system's velocity but not its position. It cannot be integrated to a geometric constraint. Non-holonomic constraints do not reduce the dimension of the Configuration Space, but they severely restrict instantaneous motion.

Example: A car's wheels have a no-slip condition: the car cannot move purely sideways. If (X, Y, Θ) are the car's position and heading, the constraint is:

$$\dot{X} \sin(\Theta) - \dot{Y} \cos(\Theta) = 0 \quad (3.46)$$

This is a differential constraint on velocities. However, by parallel parking (going forward, turning, reversing), a car can eventually reach any (X, Y, Θ) on the plane. The Configuration Space remains 3-dimensional, but instantaneous motion is constrained to 1 DOF (forward speed, with heading changing as a function of steering angle).

3.12.1 More Examples of Non-Holonomic Systems**Example 3.14. Rolling Coin Without Slipping**

A coin rolling without slipping on a table has constraints:

- Position: (X, Y, Θ) (center location and heading)
- Rotation: ϕ (angle rotated)

No-slip condition: the coin's rotation is proportional to distance traveled:

$$\frac{d\phi}{dt} = \frac{R}{r} \sqrt{\dot{X}^2 + \dot{Y}^2} \quad (3.47)$$

where R is the coin's radius and r is some reference length.

This is a velocity constraint; it does not reduce the position configuration space. But it is very restrictive: the coin cannot instantaneously move sideways while rotating.

Example 3.15. Unicycle Model

A unicycle has configuration (X, Y, Θ) and one input (pedal). The kinematics are:

$$\dot{X} = v \cos(\Theta) \quad (3.48)$$

$$\dot{Y} = v \sin(\Theta) \quad (3.49)$$

$$\dot{\Theta} = \frac{v}{L} \tan(\delta) \quad (3.50)$$

where v is velocity, L is wheelbase, and δ is steering angle.

The non-holonomic constraint is $\dot{X} \sin(\Theta) - \dot{Y} \cos(\Theta) = 0$. The system has 3 DOF position space but only 2 inputs (v, δ) , yielding a 2-dimensional input-reachable set in velocity space at each configuration.

Example 3.16. Ball on a Tilted Plate

A ball rolling (without slipping) on an inclined or moving rigid plate in 3D. The ball's position is (x, y) and orientation is a rotation matrix $R \in SO(3)$.

The no-slip constraint couples the ball's linear velocity to its angular velocity:

$$\dot{\mathbf{r}} = \boldsymbol{\omega} \times \mathbf{c} \quad (3.51)$$

where $\mathbf{c}$ is the contact point and $\boldsymbol{\omega}$ is the angular velocity.

This is again a velocity constraint. The 5-dimensional position space (3 for position, 3 for orientation minus 1 for the fixed contact constraint) is not reduced to lower dimension, but instantaneous motion is restricted to a lower-dimensional subset.

3.12.2 Pfaffian Constraints

Non-holonomic constraints are often written in Pfaffian form.

Definition 3.20. Pfaffian Constraint

A *Pfaffian constraint* is a linear constraint on velocity of the form:

$$\mathbf{A}(\mathbf{q})d\mathbf{q} = \mathbf{0} \quad (3.52)$$

or equivalently:

$$\mathbf{A}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0} \quad (3.53)$$

where $\mathbf{A}(\mathbf{q}) \in \mathbb{R}^{k \times n}$ is a matrix of constraint coefficients (typically $k < n$), and the rows are assumed to be linearly independent.

Interpretation: The constraint eliminates k directions of velocity from the tangent space. The system can only move in directions orthogonal to the rows of $\mathbf{A}$.

Example 3.17. Car Non-Holonomic Constraint in Pfaffian Form

For a car with configuration $\mathbf{q} = [X, Y, \Theta]^T$, the no-slip constraint is:

$$\dot{X} \sin(\Theta) - \dot{Y} \cos(\Theta) = 0 \quad (3.54)$$

In Pfaffian form:

$$\begin{bmatrix} \sin(\Theta) & -\cos(\Theta) & 0 \end{bmatrix} \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{\Theta} \end{bmatrix} = 0 \quad (3.55)$$

This single row eliminates one velocity direction. The car can move in any direction tangent to the constraint (i.e., perpendicular to $[\sin(\Theta), -\cos(\Theta), 0]^T$).

3.13 Python Worked Example: Forward Kinematics and Workspace of a 2R Arm

Example 3.18. Computing Workspace of 2-Link Arm

Below is a complete Python script that computes and visualizes the workspace of a 2-link planar robot arm.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
L1, L2 = 1.0, 1.0 # link lengths
n_samples = 100   # samples per joint angle

# Create a grid of joint angles
q1 = np.linspace(0, 2*np.pi, n_samples)
q2 = np.linspace(0, 2*np.pi, n_samples)
Q1, Q2 = np.meshgrid(q1, q2)

# Forward kinematics
X = L1 * np.cos(Q1) + L2 * np.cos(Q1 + Q2)
Y = L1 * np.sin(Q1) + L2 * np.sin(Q1 + Q2)

# Plot workspace
plt.figure(figsize=(10, 5))

# Left: scatter plot of all reachable points
plt.subplot(1, 2, 1)
plt.scatter(X, Y, s=1, alpha=0.5, c='blue')
plt.axis('equal')
plt.xlabel('x')
```

```

plt.ylabel('y')
plt.title('Reachable Workspace (2R Arm)')
plt.grid(True)

# Right: compute Jacobian determinant at each configuration
# J = [[-L1*sin(q1) - L2*sin(q1+q2), -L2*sin(q1+q2)]
#      [ L1*cos(q1) + L2*cos(q1+q2),  L2*cos(q1+q2)]]
J11 = -L1*np.sin(Q1) - L2*np.sin(Q1+Q2)
J12 = -L2*np.sin(Q1+Q2)
J21 = L1*np.cos(Q1) + L2*np.cos(Q1+Q2)
J22 = L2*np.cos(Q1+Q2)

det_J = J11*J22 - J12*J21

# Plot dexterous workspace (where det(J) != 0)
plt.subplot(1, 2, 2)
# Color points by Jacobian determinant
plt.scatter(X, Y, s=1, c=np.abs(det_J), cmap='viridis', alpha=0.6)
plt.colorbar(label='|det(J)|')
plt.axis('equal')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Workspace colored by Singularity Distance')
plt.grid(True)

plt.tight_layout()
plt.show()

# Demonstrate forward kinematics at a specific configuration
q = np.array([np.pi/4, np.pi/3]) # example configuration
p = np.array([L1*np.cos(q[0]) + L2*np.cos(q[0]+q[1]),
              L1*np.sin(q[0]) + L2*np.sin(q[0]+q[1])])
print(f"Configuration q = {q}")
print(f"End-effector position p = {p}")

```

This script:

1. Defines link lengths $L_1 = L_2 = 1$.
2. Evaluates forward kinematics at $100 \times 100 = 10,000$ configurations in the joint space $[0, 2\pi]^2$.
3. Scatters the resulting end-effector positions to visualize the reachable workspace (should be an annulus).
4. Computes the Jacobian determinant at each configuration and colors points by singularity distance (higher determinant = further from singularities = more dexterous).

5. Demonstrates kinematics at a specific configuration.

The output shows that the 2-link arm can reach any point in the annulus $0 \leq r \leq L_1 + L_2 = 2$, and the dexterous workspace (colored region) is concentrated in the interior where the Jacobian is well-conditioned.

3.14 Chapter Summary and Key Principles

Mathematical Principle 3.1. Configuration Space Essentials

Core Concepts:

- The **Configuration Space** $\mathcal{Q}$ is a manifold of dimension equal to the system's DOF. Every point represents a unique configuration.
- **Degrees of Freedom** are counted via Grübler's formula and represent the minimal number of independent coordinates needed.
- **Generalized coordinates** $\mathbf{q}$ span $\mathcal{Q}$ efficiently, avoiding explicit constraint equations.
- **Forward Kinematics** $\mathbf{p} = f_{kin}(\mathbf{q})$ maps configurations to workspace positions.
- The **Jacobian** $J_{kin} = \frac{\partial f_{kin}}{\partial \mathbf{q}}$ relates joint and task-space velocities.
- The **Tangent Bundle** $T\mathcal{Q}$ (pairs of configurations and velocities) is the state space.

Practical Insights:

- **Charts and atlases** provide local Euclidean coordinates for curved manifolds.
- **Inverse Kinematics** is generally nonlinear and may have multiple (or no) solutions.
- **Workspace** divides into reachable and dexterous regions; singularities mark the boundary.
- **Holonomic constraints** reduce dimension; **non-holonomic constraints** restrict velocity but not position.
- Pfaffian constraints encode velocity restrictions algebraically: $\mathbf{A}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0}$.

3.15 Further Reading

The configuration-space perspective developed in this chapter has its roots in differential geometry. Chapters 2–3 of Murray, Li, and Sastry [23] provide a rigorous treatment of configuration manifolds, tangent spaces, and the kinematic maps that connect joint space

to task space, all within the robotics context. Their development of Grübler's formula and the topology of common joint types directly complements the material presented here.

Lynch and Park [20] offer a modern, self-contained treatment of configuration-space analysis with extensive worked examples for serial and parallel mechanisms. Their textbook is particularly strong on workspace computation and kinematic singularities.

For Jacobian analysis, velocity kinematics, and workspace characterization, Siciliano et al. [30] provide detailed algorithms and case studies drawn from industrial manipulators. Their treatment of singularity classification is especially useful for practical applications.

Bullo and Lewis [5] develop geometric control theory on configuration manifolds, extending the ideas of this chapter into the realm of controllability and motion planning on curved spaces. Their framework unifies the holonomic and non-holonomic constraint analysis introduced in Section 10.12 with modern tools from differential geometry.

More broadly, the configuration manifolds studied here are special cases of the smooth manifolds treated in any graduate text on differential geometry. Readers seeking mathematical depth beyond the robotics context will find that the manifold, chart, and atlas concepts introduced in this chapter transfer directly to the general theory.

3.16 Exercises

Foundational Tier

1. A system consisting of a single rigid body in 3D space (not subject to constraints) has how many degrees of freedom? Explain why.
2. For the planar 4-bar linkage, verify Grübler's formula:
 - (a) Identify n (movable links), j (joints), and d_i (DOF per joint).
 - (b) Compute $\text{DOF} = 3n - 3j + \sum d_i$.
 - (c) Explain why the result matches your intuition.
3. The circle S^1 is a 1-dimensional manifold. Why can you not cover S^1 with a single chart (U, φ) where $\varphi : U \rightarrow \mathbb{R}$? (Hint: think about continuity at the wraparound point.)
4. For a point $q \in S^1$ (the circle), what is the dimension of the tangent space $T_q S^1$? Why?
5. A 2-link planar arm has link lengths $L_1 = 1, L_2 = 1$. At the configuration $q_1 = 0, q_2 = 0$, what is the end-effector position $\mathbf{p}_{\text{end}}$? What is the Jacobian at this configuration?

Intermediate Tier

1. Grübler's formula for planar mechanisms: $\text{DOF} = 3n - 3j + \sum d_i$.
 - (a) A planar slider-crank mechanism has 3 moving links, 4 joints (one revolute, three other). What must the other three joints be to have $\text{DOF} = 1$? Show your calculation.
 - (b) Sketch such a mechanism.

2. For the 2-link arm from Example 3.8.1, show that at configuration $q_1 = \pi/2, q_2 = 0$, the Jacobian is singular ($\det J = 0$). What does this singularity represent geometrically? (Hint: what is the end-effector orientation?)
3. Consider the reachable workspace of a 2-link arm with $L_1 = 2, L_2 = 1$.
 - (a) What is the inner radius (minimum distance from origin)?
 - (b) What is the outer radius (maximum distance)?
 - (c) What is the area of the annular workspace?
4. Transition maps on S^1 : Using the two charts from Example 3.4.1, show that the transition map τ_{12} is smooth. What would happen if you tried to use a chart (U, φ) with $\varphi(\theta) = \theta$ (angle in radians) covering the entire circle? Why does this fail to be a valid chart?
5. For a unicycle model (Example 3.15), write the non-holonomic constraint in Pfaffian form $\mathbf{A}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{0}$. How many velocity constraints are there? What is the dimension of the allowed velocity subspace?

Advanced Tier

1. Prove that for a 2-link arm, the workspace is indeed an annulus (not a disk or other shape).
 - (a) Show that the reachable set is $\{(x, y) : |L_1 - L_2| \leq r \leq L_1 + L_2\}$ where $r = \sqrt{x^2 + y^2}$.
 - (b) Hint: parameterize by $q_1, q_2 \in [0, 2\pi)$ and use the constraint that the sum of two vectors of fixed length L_1, L_2 has magnitude bounded by $L_1 + L_2$ and at least $|L_1 - L_2|$.
2. Inverse Kinematics for a 3-link planar arm: Suppose we augment the 2-link arm with a third link of length $L_3 = 0.5$.
 - (a) Write the forward kinematics: x, y, θ in terms of q_1, q_2, q_3 .
 - (b) If the target is $\mathbf{p}_d = [2.0, 0.5, \pi/4]^T$ (position and orientation), set up the Newton-Raphson iteration $\mathbf{q}_{k+1} = \mathbf{q}_k + J_{kin}^{-1}(\mathbf{q}_k)\mathbf{e}(\mathbf{q}_k)$.
 - (c) Numerically solve (using Python or Matlab) starting from an initial guess $\mathbf{q}_0 = [\pi/6, \pi/6, \pi/6]^T$.
3. For a rolling ball on a 2D plane with no-slip condition (Example 3.14), write the full state (configuration + velocity) and state the non-holonomic constraint in Pfaffian form. Does this constraint reduce the dimension of the configuration space?
4. Prove the equivalence (Theorem 3.3) by constructing explicit isomorphisms:
 - (a) Given a curve $\gamma(t)$ with $\gamma(0) = q$ and local coordinates $\varphi(\gamma(t))$, define a derivation v_γ on smooth functions.
 - (b) Conversely, given a derivation v , show you can construct a family of curves whose tangent vectors represent v .

- (c) Verify that these maps are mutual inverses (up to equivalence).
5. Stewart Platform Inverse Kinematics: A Stewart platform has a desired end-effector pose (position $\mathbf{p} \in \mathbb{R}^3$ and orientation $R \in SO(3)$). Given 6 attachment points on the base and 6 on the platform, the inverse kinematics problem is to find the 6 actuator lengths $\ell_1, \dots, \ell_6$.
- (a) Write the constraint equations (distance constraints).
 - (b) Discuss why this nonlinear system is easier to solve than general 6-DOF arm IK (Hint: it is a set of independent distance constraints, not a chain of rotations).
 - (c) Sketch an algorithm (e.g., Newton-Raphson or geometric approach) and discuss convergence.

Chapter 4

Rotations, Quaternions, and $SO(3)$

The tragedy of Euclidean space is that it works perfectly until you spin.

In Plain Language 4.1. Context & Use Case: The Problem with Spinning

The Math You Will See: We introduce **Rotation Matrices** ($\mathbf{R}$), the **Special Orthogonal Group** $SO(3)$, and **Quaternions** ($\mathbf{q}$). **Why We Need It:** Moving a robot arm up, down, left, or right is easy; you just add numbers to X, Y, Z . But calculating what happens when a drone performs a backflip while spinning like a top is a mathematical nightmare. The most intuitive way to track rotation (Euler angles like Roll, Pitch, Yaw) literally breaks the laws of math in certain positions, causing flight controllers to crash. To solve this, we must map rotations using completely different objects: 3×3 matrices and 4-dimensional Quaternions. **All Variables Defined:** $\mathbf{R} \in SO(3)$ is a rotation matrix, and $\mathbf{q} \in \mathbb{R}^4$ is a quaternion.

4.1 Historical Context: From Euler to Hamilton

The mathematics of 3D rotation has been refined over centuries by some of history's greatest mathematicians. In **1775**, Leonhard Euler published his groundbreaking *Euler's Rotation Theorem*, establishing that any finite rotation in 3D space can be described as a single rotation about a fixed axis by some angle—a foundational geometric insight that remains central to modern rotation calculus.

However, Euler's representation (axis-angle form) requires solving nonlinear equations and computing trigonometric functions. In **1843**, the Irish mathematician **William Rowan Hamilton** invented **Quaternions** ($\mathcal{H}$), an extension of complex numbers to four dimensions. Quaternions elegantly encode rotations as multiplication operations on 4-tuples, bypassing the singularities that plague other representations. Almost simultaneously, **Olinde Rodrigues** (1795–1853) derived the famous **Rodrigues rotation formula**, expressing how a vector rotates about an axis—a result that connects Euler's axis-angle theorem to matrix algebra.

These three frameworks—rotation matrices, quaternions, and axis-angle representations—are *mathematically equivalent* but offer different computational and conceptual advantages. Modern robotics and computer graphics employ all three, often switching between them fluidly.

4.2 The Coordinate Frame

When we model movement or control robotic systems, we are not looking at the physical object itself; we are tracking a **Coordinate Frame** rigidly attached to the object, often denoted as the Body Frame ($\{b\}$).

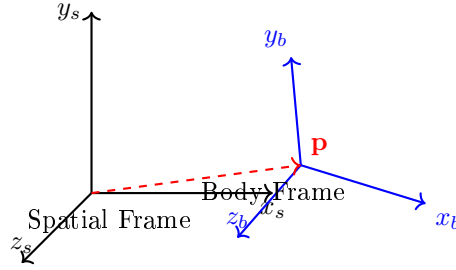


Figure 4.1: Spatial and body frames: The spatial frame is fixed in the world; the body frame is rigidly attached to a moving object. The relationship between them is encoded by a rotation matrix $\mathbf{R}$ and a position vector $\mathbf{p}$.

To know where the Body Frame $\{b\}$ is relative to the absolute stationary universe—the Spatial (or World) Frame ($\{s\}$)—we must define two mathematical properties:

1. The **translation** vector $\mathbf{p} \in \mathbb{R}^3$ pointing from the origin of $\{s\}$ to the origin of $\{b\}$.
2. The **rotation** of the axes of $\{b\}$ relative to the axes of $\{s\}$.

Unlike the straightforward (X, Y, Z) translation vector, rotation is mathematically subtle and surprisingly difficult to parameterize.

4.3 Rotation Matrices: From First Principles

4.3.1 What is a Rotation Matrix?

To represent a rotation, we examine the three orthonormal basis vectors (axes) of the body frame $\{b\}$: the unit vectors $\hat{\mathbf{x}}_b$, $\hat{\mathbf{y}}_b$, and $\hat{\mathbf{z}}_b$, each expressed in the coordinates of the spatial frame $\{s\}$. Stack these three column vectors side-by-side:

$$\mathbf{R} = \begin{bmatrix} | & | & | \\ \hat{\mathbf{x}}_b & \hat{\mathbf{y}}_b & \hat{\mathbf{z}}_b \\ | & | & | \end{bmatrix} \in \mathbb{R}^{3 \times 3} \quad (4.1)$$

This is a **Rotation Matrix**. Each column is a unit vector, and any two columns are orthogonal to each other. This orthonormality is the key property.

4.3.2 Orthonormality Constraints

Theorem 4.1. Orthonormality Conditions

A matrix $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ is a valid rotation matrix if and only if:

1. **Orthogonality:** $\mathbf{R}^T \mathbf{R} = \mathbf{I}$, which means $\mathbf{R}^{-1} = \mathbf{R}^T$.
2. **Determinant Constraint:** $\det(\mathbf{R}) = +1$.

Proof. Let $\mathbf{R} = [\mathbf{c}_1 \mid \mathbf{c}_2 \mid \mathbf{c}_3]$, where $\mathbf{c}_i$ are the columns. The product $\mathbf{R}^T \mathbf{R}$ yields a 3×3 matrix whose (i, j) entry is $\mathbf{c}_i^T \mathbf{c}_j$. For this to equal $\mathbf{I}$:

- **Diagonal:** $\mathbf{c}_i^T \mathbf{c}_i = 1$ for each i (each column is unit length).
- **Off-diagonal:** $\mathbf{c}_i^T \mathbf{c}_j = 0$ for $i \neq j$ (columns are mutually orthogonal).

These are exactly 6 independent constraints on the 9 matrix entries, leaving $9 - 6 = 3$ degrees of freedom.

The determinant condition $\det(\mathbf{R}) = 1$ distinguishes *proper rotations* (right-handed orientation preserving) from reflections, which would have $\det(\mathbf{R}) = -1$. $\square$

4.3.3 The Three Elementary Rotation Matrices

Any rotation in 3D can be decomposed into rotations about the principal axes. We derive the three elementary rotation matrices.

Theorem 4.2. Rotation About the x -Axis

A rotation by angle θ about the x -axis, with the rotation viewed as right-hand rule (counterclockwise when looking from positive x toward the origin), is given by:

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix} \quad (4.2)$$

Geometric Interpretation:

- **Column 1:** $[1, 0, 0]^T$ is the x -axis, unchanged by rotation about itself.
- **Column 2:** $[0, \cos \theta, \sin \theta]^T$ is where the original y -axis rotates to.
- **Column 3:** $[0, -\sin \theta, \cos \theta]^T$ is where the original z -axis rotates to.

Derivation. The x -axis unit vector $\hat{\mathbf{e}}_x = [1, 0, 0]^T$ is invariant. The y and z axes lie in the yz -plane, which undergoes a 2D rotation:

$$\begin{bmatrix} y' \\ z' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} y \\ z \end{bmatrix}.$$

Embedding this in 3D yields equation (4.2). $\square$

Theorem 4.3. Rotation About the y -Axis

A rotation by angle θ about the y -axis is:

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \quad (4.3)$$

Theorem 4.4. Rotation About the z -Axis

A rotation by angle θ about the z -axis is:

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (4.4)$$

Each elementary matrix satisfies $\mathbf{R}^T \mathbf{R} = \mathbf{I}$ and $\det(\mathbf{R}) = 1$ by construction (they encode orthonormal bases).

4.4 The Special Orthogonal Group: $\text{SO}(3)$

4.4.1 $\text{SO}(3)$ as a Group

Definition 4.1. The Group $\text{SO}(3)(3)$

The **Special Orthogonal Group** $\text{SO}(3)(3)$ [23] is the set of all 3×3 real matrices satisfying:

$$\text{SO}(3)(3) = \{\mathbf{R} \in \mathbb{R}^{3 \times 3} : \mathbf{R}^T \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = 1\} \quad (4.5)$$

Theorem 4.5. $\text{SO}(3)(3)$ is a Group

The set $\text{SO}(3)(3)$ forms a mathematical group under matrix multiplication.

Proof. We verify the four group axioms:

Closure: If $\mathbf{R}_1, \mathbf{R}_2 \in \text{SO}(3)(3)$, then:

$$(\mathbf{R}_1 \mathbf{R}_2)^T (\mathbf{R}_1 \mathbf{R}_2) = \mathbf{R}_2^T \mathbf{R}_1^T \mathbf{R}_1 \mathbf{R}_2 = \mathbf{R}_2^T \mathbf{I} \mathbf{R}_2 = \mathbf{I}, \quad (4.6)$$

$$\det(\mathbf{R}_1 \mathbf{R}_2) = \det(\mathbf{R}_1) \det(\mathbf{R}_2) = 1 \cdot 1 = 1. \quad (4.7)$$

Thus $\mathbf{R}_1 \mathbf{R}_2 \in \text{SO}(3)(3)$.

Associativity: Matrix multiplication is associative: $(\mathbf{R}_1 \mathbf{R}_2) \mathbf{R}_3 = \mathbf{R}_1 (\mathbf{R}_2 \mathbf{R}_3)$.

Identity: The identity matrix $\mathbf{I}$ satisfies $\mathbf{I}^T \mathbf{I} = \mathbf{I}$ and $\det(\mathbf{I}) = 1$, so $\mathbf{I} \in \text{SO}(3)(3)$.

Inverses: For $\mathbf{R} \in \text{SO}(3)(3)$, we have $\mathbf{R}^T \mathbf{R} = \mathbf{I}$, so $\mathbf{R}^{-1} = \mathbf{R}^T$. Moreover:

$$\det(\mathbf{R}^T) = \det(\mathbf{R}) = 1, \quad (4.8)$$

so $\mathbf{R}^{-1} \in \text{SO}(3)(3)$. □

4.4.2 $SO(3)$ as a Smooth Manifold

Theorem 4.6. $SO(3)(3)$ is a 3-Dimensional Smooth Manifold

The set $SO(3)(3)$ is a smooth, compact manifold of dimension 3.

Argument. The orthogonality constraint $\mathbf{R}^T \mathbf{R} = \mathbf{I}$ imposes 6 independent scalar equations (the symmetric matrix $\mathbf{I}$ has 6 independent entries) on the 9 entries of $\mathbf{R}$. Additionally, $\det(\mathbf{R}) = 1$ is one equation, but it is redundant: for orthogonal matrices with real entries, $\det(\mathbf{R}) = \pm 1$ automatically, and we select the component with determinant +1 by continuity and connectedness.

Thus, the effective number of independent constraints is 6, leaving:

$$\dim(SO(3)(3)) = 9 - 6 = 3. \quad (4.9)$$

Each point in $SO(3)(3)$ admits a local parameterization by 3 parameters (e.g., Euler angles in a suitable domain), confirming that $SO(3)(3)$ is a smooth 3-dimensional manifold. $\square$

Mathematical Principle 4.1. The $SO(3)$ Manifold

The continuous, infinite geometric space of all possible valid 3×3 rotation matrices satisfying $\mathbf{R}^T \mathbf{R} = \mathbf{I}$ and $\det(\mathbf{R}) = 1$ is a mathematical **manifold** called the **Special Orthogonal Group in 3 Dimensions**, denoted as $SO(3)(3)$.

If a Configuration Space $\mathcal{Q}$ is simply a rigid body freely rotating around a fixed point, we say $\mathcal{Q} = SO(3)(3)$.

4.5 Euler Angles: Gimbal Lock and Beyond

4.5.1 Definition of Euler Angles

The most intuitive way for a human to describe the orientation of an aircraft or robotic arm is to split rotation into three successive rotations about primary axes. This is known as an **Euler Angle** representation: $\boldsymbol{\theta} = [\phi, \theta, \psi]^T$.

While highly intuitive for human pilots, Euler Angles harbor a fatal mathematical flaw known as **Gimbal Lock**.

4.5.2 The ZYX Convention

One common convention is **ZYX Euler angles** (also called *yaw-pitch-roll*):

1. Rotate by ψ (yaw) about the original z -axis.
2. Rotate by θ (pitch) about the *current* y -axis (after yaw).
3. Rotate by ϕ (roll) about the *current* x -axis (after yaw and pitch).

The composite rotation matrix is:

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_x(\phi)\mathbf{R}_y(\theta)\mathbf{R}_z(\psi). \quad (4.10)$$

Computing this product explicitly:

$$\mathbf{R}(\phi, \theta, \psi) = \begin{bmatrix} \cos \psi \cos \theta & -\sin \psi \cos \phi + \cos \psi \sin \theta \sin \phi & \sin \psi \sin \phi + \cos \psi \sin \theta \cos \phi \\ \sin \psi \cos \theta & \cos \psi \cos \phi + \sin \psi \sin \theta \sin \phi & -\cos \psi \sin \phi + \sin \psi \sin \theta \cos \phi \\ -\sin \theta & \cos \theta \sin \phi & \cos \theta \cos \phi \end{bmatrix} \quad (4.11)$$

4.5.3 The ZXX Convention

Another standard convention is **ZXX Euler angles** (symmetric, used in physics and astronomy):

1. Rotate by ϕ about the original z -axis.
2. Rotate by θ about the *current* x -axis.
3. Rotate by ψ about the *current* z -axis (after the first two rotations).

The composite rotation is:

$$\mathbf{R}(\phi, \theta, \psi) = \mathbf{R}_z(\psi)\mathbf{R}_x(\theta)\mathbf{R}_z(\phi). \quad (4.12)$$

4.5.4 Gimbal Lock: Singularity and Proof

In Plain Language 4.2. Gimbal Lock: Physical Intuition

If an airplane pitches perfectly 90° straight up into the sky, its Roll axis (pointing out its nose) now perfectly aligns with its original Yaw axis (pointing perfectly down towards the Earth).

Mathematically, two independent degrees of freedom have become linearly dependent—they merged into the identical physical rotation axis. A degree of freedom is literally lost during the calculation. If you attempt to control the plane passing through this singularity, the Jacobian mapping Euler angle rates to angular velocity becomes singular, producing undefined or numerically unstable results (NaN values, wildly large rate commands).

Theorem 4.7. Gimbal Lock at $\theta = \pm 90^\circ$

For ZYX Euler angles, the Jacobian matrix relating angular velocity components to Euler angle rate components becomes singular when $\theta = \pm 90^\circ$.

Derivation. The relationship between angular velocity $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ in the spatial frame and Euler angle rates $\dot{\boldsymbol{\theta}} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^T$ is given by:

$$\boldsymbol{\omega} = \mathbf{J}(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}}, \quad (4.13)$$

where $\mathbf{J}(\boldsymbol{\theta})$ is the Jacobian. For ZYX angles, it can be shown that:

$$\mathbf{J}(\phi, \theta, \psi) = \begin{bmatrix} 0 & -\sin \psi & \cos \psi \cos \theta \\ 0 & \cos \psi & \sin \psi \cos \theta \\ 1 & 0 & -\sin \theta \end{bmatrix} \quad (4.14)$$

When $\theta = 90^\circ$, we have $\cos \theta = 0$, so:

$$\mathbf{J}(\phi, 90^\circ, \psi) = \begin{bmatrix} 0 & -\sin \psi & 0 \\ 0 & \cos \psi & 0 \\ 1 & 0 & -1 \end{bmatrix}. \quad (4.15)$$

The third column is linearly dependent on the first two, so $\text{rank}(\mathbf{J}) < 3$ and $\det(\mathbf{J}) = 0$. The Jacobian is singular, making it impossible to invert: given $\boldsymbol{\omega}$, we cannot solve uniquely for $\dot{\boldsymbol{\theta}}$. Physically, the yaw and roll rates become indistinguishable. $\square$

Since our control theory and geometric motion algorithms fundamentally rely on continuous trajectories twisting through all possible spatial angles (like chaotic golf club arcs or throwing motions), we must completely abandon Euler angles for internal calculations. We cannot afford mathematical singularities.

4.6 Differential Geometry Primer: Lie Groups and Lie Algebras

We have established that $SO(3)(3)$ is a manifold. However, it is not just any manifold. Because you can combine two rotations to get a third rotation (via matrix multiplication), and every rotation can be smoothly inverted, $SO(3)(3)$ possesses both algebraic structure (it is a mathematical *group*) and geometric structure (it is a *smooth manifold*).

Definition 4.2. Lie Group

A *Lie Group* is a smooth manifold that is also a mathematical group, where the group operations (multiplication and inversion) are smooth functions. As a result, we can perform calculus on them. $SO(3)(3)$ is the preeminent example of a Lie Group in mechanics.

4.6.1 The Tangent Space and Lie Algebra

If $SO(3)(3)$ is a curved manifold of positions (rotations), what is its velocity space? By our earlier definition in Chapter 3, the velocities must live in the Tangent Space.

If we examine the tangent space at the identity rotation (the "zero" rotation, matrix $\mathbf{I}$), we discover an incredibly special mathematical object called the **Lie Algebra**, denoted by lowercase $\mathfrak{so}(3)(3)$.

Definition 4.3. Lie Algebra $\mathfrak{so}(3)(3)$

The *Lie Algebra* $\mathfrak{so}(3)(3)$ is the tangent space to the Lie Group $SO(3)(3)$ at the identity element. Physically, it represents the space of all possible angular velocities

(spin rates) when the object is currently unrotated. Mathematically, it consists of all 3×3 *skew-symmetric* matrices: matrices $[\boldsymbol{\omega}]_{\times}$ satisfying $[\boldsymbol{\omega}]_{\times}^T = -[\boldsymbol{\omega}]_{\times}$.

A skew-symmetric matrix representing a vector $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ is:

$$[\boldsymbol{\omega}]_{\times} = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \quad (4.16)$$

In Plain Language 4.3. Mathematical Reminder: Group vs. Algebra

Whenever you hear **Lie Group** (like $\text{SO}(3)$ or $\text{SE}(3)$), think of the curved, global manifold of *Positions/Configurations*. Whenever you hear **Lie Algebra** (like $\mathfrak{so}(3)$ or $\mathfrak{se}(3)$), think of the perfectly flat, linear tangent space of *Velocities*. We can do standard linear control math in the Lie Algebra, but we map the results back onto the curved Lie Group using the *Exponential Map* (covered next) to actually rotate the physical robot.

4.7 Rodrigues' Rotation Formula: Full Derivation

We now prove the celebrated **Rodrigues' rotation formula**, which gives a closed-form expression for the matrix exponential of a skew-symmetric matrix. This result, first obtained by Olinde Rodrigues in 1840 [26], is the computational backbone of the exponential map for $\text{SO}(3)$.

4.7.1 The Key Identity: $[\hat{\boldsymbol{\omega}}]^3 = -[\hat{\boldsymbol{\omega}}]$

Let $\hat{\boldsymbol{\omega}} \in \mathbb{R}^3$ be a unit vector ($\|\hat{\boldsymbol{\omega}}\| = 1$) and $[\hat{\boldsymbol{\omega}}] \in \mathfrak{so}(3)$ the corresponding skew-symmetric matrix. We first establish a crucial algebraic identity.

Theorem 4.8. Cyclic Property of Skew-Symmetric Matrices

For any unit vector $\hat{\boldsymbol{\omega}}$ with skew-symmetric matrix $[\hat{\boldsymbol{\omega}}]$:

$$[\hat{\boldsymbol{\omega}}]^3 = -[\hat{\boldsymbol{\omega}}]. \quad (4.17)$$

Proof. Recall that for a skew-symmetric matrix, the action $[\hat{\boldsymbol{\omega}}]\mathbf{v} = \hat{\boldsymbol{\omega}} \times \mathbf{v}$. Thus:

$$[\hat{\boldsymbol{\omega}}]^2 \mathbf{v} = \hat{\boldsymbol{\omega}} \times (\hat{\boldsymbol{\omega}} \times \mathbf{v}). \quad (4.18)$$

By the vector triple product identity $\mathbf{a} \times (\mathbf{a} \times \mathbf{v}) = \mathbf{a}(\mathbf{a} \cdot \mathbf{v}) - \mathbf{v}(\mathbf{a} \cdot \mathbf{a})$ with $\|\hat{\boldsymbol{\omega}}\| = 1$:

$$[\hat{\boldsymbol{\omega}}]^2 = \hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I}. \quad (4.19)$$

Therefore:

$$[\hat{\boldsymbol{\omega}}]^3 = [\hat{\boldsymbol{\omega}}](\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - \mathbf{I}) = [\hat{\boldsymbol{\omega}}]\hat{\boldsymbol{\omega}}\hat{\boldsymbol{\omega}}^T - [\hat{\boldsymbol{\omega}}]. \quad (4.20)$$

Since $[\hat{\boldsymbol{\omega}}]\hat{\boldsymbol{\omega}} = \hat{\boldsymbol{\omega}} \times \hat{\boldsymbol{\omega}} = \mathbf{0}$, we obtain $[\hat{\boldsymbol{\omega}}]^3 = -[\hat{\boldsymbol{\omega}}]$. $\square$

This identity implies that all higher powers of $[\hat{\omega}]$ reduce to either $[\hat{\omega}]$ or $[\hat{\omega}]^2$:

$$[\hat{\omega}]^4 = -[\hat{\omega}]^2, \quad [\hat{\omega}]^5 = [\hat{\omega}], \quad [\hat{\omega}]^6 = [\hat{\omega}]^2, \quad \dots \quad (4.21)$$

4.7.2 Taylor Expansion and Regrouping

The matrix exponential is defined by its Taylor series:

$$e^{[\hat{\omega}]\theta} = \mathbf{I} + \theta[\hat{\omega}] + \frac{\theta^2}{2!}[\hat{\omega}]^2 + \frac{\theta^3}{3!}[\hat{\omega}]^3 + \frac{\theta^4}{4!}[\hat{\omega}]^4 + \dots \quad (4.22)$$

Substituting $[\hat{\omega}]^3 = -[\hat{\omega}]$, $[\hat{\omega}]^4 = -[\hat{\omega}]^2$, $[\hat{\omega}]^5 = [\hat{\omega}]$, etc., and grouping by powers of $[\hat{\omega}]$:

$$e^{[\hat{\omega}]\theta} = \mathbf{I} + \underbrace{\left(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \dots\right)}_{\sin \theta} [\hat{\omega}] + \underbrace{\left(\frac{\theta^2}{2!} - \frac{\theta^4}{4!} + \frac{\theta^6}{6!} - \dots\right)}_{1 - \cos \theta} [\hat{\omega}]^2. \quad (4.23)$$

The odd-power series is exactly $\sin \theta$ and the even-power series is exactly $1 - \cos \theta$.

Theorem 4.9. Rodrigues' Rotation Formula

For a unit axis $\hat{\omega} \in \mathbb{R}^3$ and angle $\theta \in \mathbb{R}$, the rotation matrix is:

$$e^{[\hat{\omega}]\theta} = \mathbf{I} + \sin \theta [\hat{\omega}] + (1 - \cos \theta) [\hat{\omega}]^2. \quad (4.24)$$

Equivalently, using (4.19), the action on a vector $\mathbf{v}$ is:

$$\mathbf{R}\mathbf{v} = \mathbf{v} \cos \theta + (\hat{\omega} \times \mathbf{v}) \sin \theta + \hat{\omega}(\hat{\omega} \cdot \mathbf{v})(1 - \cos \theta). \quad (4.25)$$

The vector form (4.25) has a geometric interpretation: decompose $\mathbf{v}$ into a component along $\hat{\omega}$ (unchanged by rotation) and a component perpendicular to $\hat{\omega}$ (rotated by θ in the plane normal to $\hat{\omega}$). See [26] for the original derivation and [23] for the modern Lie-theoretic perspective.

4.8 Quaternion Algebra

4.8.1 Definition and Representation

A **quaternion** $\mathbf{q} \in \mathbb{H}$ is a 4-tuple written as:

$$\mathbf{q} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix} = q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k} \quad (4.26)$$

where $\mathbf{i}, \mathbf{j}, \mathbf{k}$ are the three *imaginary units* satisfying:

$$\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1, \quad \mathbf{ij} = \mathbf{k}, \quad \mathbf{jk} = \mathbf{i}, \quad \mathbf{ki} = \mathbf{j}. \quad (4.27)$$

Note that quaternion multiplication is *non-commutative*: $\mathbf{ij} = \mathbf{k}$ but $\mathbf{ji} = -\mathbf{k}$.

4.8.2 Quaternion Operations

Addition

Quaternion addition is component-wise:

$$(q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k}) + (r_w + r_x \mathbf{i} + r_y \mathbf{j} + r_z \mathbf{k}) = (q_w + r_w) + (q_x + r_x) \mathbf{i} + (q_y + r_y) \mathbf{j} + (q_z + r_z) \mathbf{k}. \quad (4.28)$$

Hamilton Product (Multiplication)

The product of two quaternions $\mathbf{q} = q_w + \mathbf{q}_v$ and $\mathbf{r} = r_w + \mathbf{r}_v$, where $\mathbf{q}_v = [q_x, q_y, q_z]^T$ and $\mathbf{r}_v = [r_x, r_y, r_z]^T$, is:

$$\mathbf{q} \otimes \mathbf{r} = \begin{bmatrix} q_w r_w - \mathbf{q}_v^T \mathbf{r}_v \\ q_w \mathbf{r}_v + r_w \mathbf{q}_v + \mathbf{q}_v \times \mathbf{r}_v \end{bmatrix} \quad (4.29)$$

where $\times$ denotes the 3D cross product. In component form:

$$\mathbf{q} \otimes \mathbf{r} = \begin{bmatrix} q_w r_w - q_x r_x - q_y r_y - q_z r_z \\ q_w r_x + q_x r_w + q_y r_z - q_z r_y \\ q_w r_y - q_x r_z + q_y r_w + q_z r_x \\ q_w r_z + q_x r_y - q_y r_x + q_z r_w \end{bmatrix} \quad (4.30)$$

Conjugate

The **conjugate** of $\mathbf{q} = q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k}$ is:

$$\mathbf{q}^* = q_w - q_x \mathbf{i} - q_y \mathbf{j} - q_z \mathbf{k} = \begin{bmatrix} q_w \\ -q_x \\ -q_y \\ -q_z \end{bmatrix} \quad (4.31)$$

Norm and Magnitude

The **norm** (or magnitude) of a quaternion is:

$$\|\mathbf{q}\| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2} \quad (4.32)$$

A **unit quaternion** satisfies $\|\mathbf{q}\| = 1$.

Inverse

For a unit quaternion $\mathbf{q}$, the inverse is:

$$\mathbf{q}^{-1} = \mathbf{q}^* \quad (4.33)$$

since $\mathbf{q} \otimes \mathbf{q}^* = \mathbf{q}^* \otimes \mathbf{q} = 1$ (the multiplicative identity quaternion).

For a non-unit quaternion, the inverse is $\mathbf{q}^{-1} = \mathbf{q}^* / \|\mathbf{q}\|^2$.

4.8.3 Unit Quaternions and Rotation

Theorem 4.10. Unit Quaternions Represent Rotations

Every unit quaternion $\mathbf{q}$ can be written as:

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) \\ \hat{\omega} \sin(\theta/2) \end{bmatrix} = \begin{bmatrix} \cos(\theta/2) \\ \hat{\omega}_x \sin(\theta/2) \\ \hat{\omega}_y \sin(\theta/2) \\ \hat{\omega}_z \sin(\theta/2) \end{bmatrix} \quad (4.34)$$

where $\hat{\omega}$ is a unit vector (the rotation axis) and $\theta \in [0, 2\pi)$ is the rotation angle. This quaternion represents a rotation of angle θ about the axis $\hat{\omega}$.

In Plain Language 4.4. Why the Half-Angles?

You might notice $\theta/2$ in the formula instead of just θ . This is because applying a quaternion rotation to a vector $\mathbf{v}$ requires multiplying it from the front *and* the back ($\mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^{-1}$, where $\mathbf{v}$ is viewed as the quaternion $[0, v_x, v_y, v_z]^T$). The half-angles combine during this double-multiplication to give the perfect full-angle θ rotation! Note also that the quaternions $\mathbf{q}$ and $-\mathbf{q}$ represent the exact same physical rotation in 3D space.

4.8.4 Double Cover: q and $-q$ Represent the Same Rotation**Theorem 4.11. Quaternion Double Cover**

The unit quaternions $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation in 3D space. Thus, the map from unit quaternions to $SO(3)(\mathbb{R})$ is a 2-to-1 covering map, and we say that S^3 (the unit sphere in $\mathbb{H}$) is a *double cover* of $SO(3)(\mathbb{R})$.

Proof. The rotation action of a unit quaternion $\mathbf{q}$ on a pure-imaginary quaternion $\mathbf{v} = [0, \vec{v}]$ is:

$$\mathbf{v}' = \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*. \quad (4.35)$$

For $-\mathbf{q}$, we compute:

$$(-\mathbf{q}) \otimes \mathbf{v} \otimes (-\mathbf{q})^* = (-\mathbf{q}) \otimes \mathbf{v} \otimes (-\mathbf{q}^*) \quad (4.36)$$

$$= (-1)(\mathbf{q}) \otimes \mathbf{v} \otimes (-1)(\mathbf{q}^*) \quad (4.37)$$

$$= (-1)(-1) \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^* \quad (4.38)$$

$$= \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^*, \quad (4.39)$$

where we used the fact that $(-\mathbf{q})^* = -\mathbf{q}^*$ and that the scalar factor (-1) commutes with quaternion multiplication and appears twice, yielding $(-1)^2 = 1$.

Thus $\mathbf{q}$ and $-\mathbf{q}$ induce the identical rotation on every vector $\vec{v} \in \mathbb{R}^3$. $\square$

4.9 Conversion Between Quaternions and Rotation Matrices

4.9.1 Quaternion to Rotation Matrix

Theorem 4.12. Quaternion-to-Matrix Conversion

A unit quaternion $\mathbf{q} = [q_w, q_x, q_y, q_z]^T$ with $q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1$ corresponds to the rotation matrix:

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix} \quad (4.40)$$

Derivation. A rotation by angle θ about unit axis $\hat{\omega} = [\omega_x, \omega_y, \omega_z]^T$ can be expressed via the Rodrigues formula [26]:

$$\mathbf{R} = \mathbf{I} + \sin \theta [\hat{\omega}]_{\times} + (1 - \cos \theta) [\hat{\omega}]_{\times}^2. \quad (4.41)$$

With $q_w = \cos(\theta/2)$ and $\hat{\omega} \sin(\theta/2) = [q_x, q_y, q_z]^T$, one can expand using the identities $\sin \theta = 2 \sin(\theta/2) \cos(\theta/2) = 2q_w \sqrt{q_x^2 + q_y^2 + q_z^2}$ and $(1 - \cos \theta) = 2 \sin^2(\theta/2) = 2(q_x^2 + q_y^2 + q_z^2)$ to obtain equation (4.40). $\square$

4.9.2 Rotation Matrix to Quaternion

Theorem 4.13. Matrix-to-Quaternion Conversion

From a rotation matrix $\mathbf{R}$, the unit quaternion can be recovered (up to sign) via:

$$q_w = \frac{1}{2} \sqrt{1 + R_{11} + R_{22} + R_{33}}, \quad (4.42)$$

with q_x, q_y, q_z determined by:

$$\begin{aligned} q_x &= \frac{R_{32} - R_{23}}{4q_w}, \\ q_y &= \frac{R_{13} - R_{31}}{4q_w}, \\ q_z &= \frac{R_{21} - R_{12}}{4q_w}. \end{aligned} \quad (4.43)$$

For numerical stability, a more robust algorithm selects the largest diagonal element to compute first, then back-solves for the others.

4.9.3 Rotation Representation Comparison

Having introduced four rotation representations, we now compare their properties systematically. The choice of representation has significant practical consequences for storage,

computation, and numerical stability.

Table 4.1: Comparison of rotation representations in $\mathbb{R}^3$.

Property	Rotation Matrix	Euler Angles	Axis-Angle	Quaternion
Parameters	9	3	3	4
Constraints	6 (orthonormality)	0	0	1 (unit norm)
Singularities	None	Gimbal lock	$\theta = \pi$	None
Composition	Matrix multiply	Complex	Via exp map	Quaternion multiply
Interpolation	Difficult	Nonuniform	Via exp map	SLERP
Storage cost	High	Low	Low	Low
Unique	Yes	No (periodic)	No ($\theta > \pi$)	No ($\pm \mathbf{q}$)

When to use each representation:

- **Rotation matrices** are the natural choice for *composing* transformations (chain of rigid-body transforms) and for direct application to vectors. Their 9-parameter redundancy is the cost of a singularity-free, algebraically closed representation.
- **Euler angles** remain useful for *human interpretation* (roll, pitch, yaw) and low-DOF joints, but gimbal lock (Section 4.5.4) makes them unsuitable for general-purpose computation.
- **Axis-angle** (equivalently, exponential coordinates) is the natural parameterisation of the Lie algebra $\mathfrak{so}(3)(3)$ and is essential for optimisation on $SO(3)(3)$, perturbation analysis, and control.
- **Quaternions** offer the best trade-off for *interpolation* (via SLERP, Section 6.11), *storage* (4 parameters, 1 constraint), and *numerical stability* (no singularities, cheap renormalisation). Modern simulation frameworks—including MuJoCo, Drake, and Bullet—typically store orientations as unit quaternions internally and convert to rotation matrices only when matrix operations are required.

4.10 Spherical Linear Interpolation (SLERP)

In computer graphics and animation, we often need to smoothly interpolate between two rotations. For quaternions, the natural interpolation is **Spherical Linear Interpolation (SLERP)**.

Definition 4.4. SLERP Formula

The SLERP between two unit quaternions $\mathbf{q}_0$ and $\mathbf{q}_1$ at parameter $t \in [0, 1]$ is:

$$\text{SLERP}(\mathbf{q}_0, \mathbf{q}_1; t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} \mathbf{q}_0 + \frac{\sin(t\Omega)}{\sin \Omega} \mathbf{q}_1 \quad (4.44)$$

where $\Omega = \arccos(|\mathbf{q}_0 \cdot \mathbf{q}_1|)$ is the (acute) angle between the quaternions on the unit sphere.

Derivation. Two unit quaternions $\mathbf{q}_0$ and $\mathbf{q}_1$ lie on the unit sphere $S^3 \subset \mathbb{H}$. The angle between them is $\Omega = \arccos(\mathbf{q}_0 \cdot \mathbf{q}_1)$ (taking absolute value ensures we pick the shorter arc).

A geodesic (great circle arc) on S^3 is the shortest path between two points. Parameterizing the geodesic by arc length with $t \in [0, 1]$ (where $t = 0$ at $\mathbf{q}_0$ and $t = 1$ at $\mathbf{q}_1$) gives the SLERP formula.

If $\Omega \approx 0$ (very close quaternions), use linear interpolation $\text{LERP}(\mathbf{q}_0, \mathbf{q}_1; t) = (1 - t)\mathbf{q}_0 + t\mathbf{q}_1$ followed by normalization to avoid numerical division by zero. $\square$

SLERP is constant-speed interpolation and is the standard for smooth rotational animations in robotics and graphics.

4.11 Angular Velocity and the Matrix Exponential

4.11.1 Angular Velocity in the Spatial Frame

When a rigid body rotates, its instantaneous rotational motion is characterized by the **angular velocity** vector $\boldsymbol{\omega} \in \mathbb{R}^3$. The magnitude $\|\boldsymbol{\omega}\|$ is the rate of rotation (rad/s), and the direction of $\boldsymbol{\omega}$ is the instantaneous axis of rotation.

The rate of change of the rotation matrix $\mathbf{R}(t)$ with respect to time is related to angular velocity by:

Theorem 4.14. Matrix Derivative in Spatial Frame

$$\dot{\mathbf{R}}(t) = [\boldsymbol{\omega}]_{\times} \mathbf{R}(t) \quad (4.45)$$

where $\boldsymbol{\omega}$ is expressed in the spatial frame and $[\boldsymbol{\omega}]_{\times}$ is the skew-symmetric matrix defined in equation (4.16).

Proof. Differentiating the orthogonality constraint $\mathbf{R}^T(t)\mathbf{R}(t) = \mathbf{I}$ with respect to time:

$$\frac{d}{dt}[\mathbf{R}^T \mathbf{R}] = \dot{\mathbf{R}}^T \mathbf{R} + \mathbf{R}^T \dot{\mathbf{R}} = \mathbf{0}. \quad (4.46)$$

Rearranging:

$$\mathbf{R}^T \dot{\mathbf{R}} = -(\mathbf{R}^T \dot{\mathbf{R}})^T, \quad (4.47)$$

so $\mathbf{R}^T \dot{\mathbf{R}}$ is skew-symmetric. Define $[\boldsymbol{\omega}]_{\times} := \mathbf{R}^T \dot{\mathbf{R}}$. Then:

$$\dot{\mathbf{R}} = \mathbf{R}[\boldsymbol{\omega}]_{\times}. \quad (4.48)$$

However, by convention in robotics, we often write $\dot{\mathbf{R}} = [\boldsymbol{\omega}]_{\times} \mathbf{R}$ where $\boldsymbol{\omega}$ is the spatial frame angular velocity. Both are correct; the difference is in the choice of frame. $\square$

4.11.2 Angular Velocity in the Body Frame

When the angular velocity is expressed in the body-fixed frame, the differential equation becomes:

$$\dot{\mathbf{R}}(t) = \mathbf{R}(t)[\boldsymbol{\omega}_b]_{\times} \quad (4.49)$$

where $\boldsymbol{\omega}_b$ is the angular velocity in the body frame.

4.11.3 Proof that Angular Velocity Lives in $\mathfrak{so}(3)(3)$

Theorem 4.15. Angular Velocity is a Lie Algebra Element

For any rotation matrix $\mathbf{R}(t) \in SO(3)(3)$, the quantity $\mathbf{R}^T \dot{\mathbf{R}} \in \mathfrak{so}(3)(3)$ is a skew-symmetric matrix, i.e., an element of the Lie algebra $\mathfrak{so}(3)(3)$.

Proof. From $\mathbf{R}^T \mathbf{R} = \mathbf{I}$, differentiate:

$$\dot{\mathbf{R}}^T \mathbf{R} + \mathbf{R}^T \dot{\mathbf{R}} = \mathbf{0}. \quad (4.50)$$

Taking the transpose of the second term:

$$(\mathbf{R}^T \dot{\mathbf{R}})^T = \dot{\mathbf{R}}^T \mathbf{R}. \quad (4.51)$$

From the differentiated constraint, $\dot{\mathbf{R}}^T \mathbf{R} = -\mathbf{R}^T \dot{\mathbf{R}}$, so:

$$(\mathbf{R}^T \dot{\mathbf{R}})^T = -\mathbf{R}^T \dot{\mathbf{R}}. \quad (4.52)$$

Thus $\mathbf{R}^T \dot{\mathbf{R}}$ is skew-symmetric, confirming it belongs to $\mathfrak{so}(3)(3)$. $\square$

4.12 Axis-Angle Representation and Euler's Rotation Theorem

Theorem 4.16. Euler's Rotation Theorem

Every rotation in 3D can be expressed as a rotation about a single axis by some angle. That is, for any $\mathbf{R} \in SO(3)(3)$, there exist a unit vector $\hat{\omega} \in \mathbb{R}^3$ (the rotation axis) and an angle $\theta \in [0, \pi]$ such that the rotation is completely described by this pair $(\hat{\omega}, \theta)$.

Proof. The rotation axis $\hat{\omega}$ is an eigenvector of $\mathbf{R}$ with eigenvalue 1:

$$\mathbf{R} \hat{\omega} = \hat{\omega}. \quad (4.53)$$

Such an eigenvector exists because $\det(\mathbf{R} - \mathbf{I}) = 0$ for any rotation (a rotation always leaves one axis invariant). The angle θ can be recovered from $\text{trace}(\mathbf{R}) = 1 + 2 \cos \theta$. $\square$

The axis-angle representation is compact and intuitive but computationally less efficient than quaternions for repeated operations.

4.13 Worked Example: Quaternions and SLERP in Python

Below is a complete Python implementation of quaternion operations and rotation interpolation.

```

1  """Quaternion operations and SLERP, as developed in Volume 0 Chapter
   4.
2
3  Self-contained by design: this file is typeset into the chapter as a
   worked
4  example, so it depends only on NumPy and repeats nothing the chapter
   has not
5  already derived. Equation numbers in the comments refer to Chapter 4.
6
7  Correctness is pinned by tests/test_affine_control/
   test_quaternion_demo.py --
8  the properties asserted there (unit norm, orthogonality of the
   rotation matrix,
9  agreement between quaternion rotation and matrix rotation, SLERP
   endpoints and
10 constant angular velocity) are what make this an example rather than a
   claim.
11 """
12
13 from __future__ import annotations
14
15 import numpy as np
16 from numpy.typing import NDArray
17
18 type Array = NDArray[np.float64]
19
20 # Below this angle the small-angle expansion of SLERP is better
   conditioned
21 # than the closed form: sin(0omega) appears in a denominator, so the
   closed form
22 # loses precision as the two quaternions approach each other.
23 SLERP_LINEAR_THRESHOLD = 1.0 - 1.0e-9
24
25
26 def hamilton_product(p: Array, q: Array) -> Array:
27     """Quaternion product, equation (eq:hamilton_product).
28
29     Both arguments are ordered [w, x, y, z]. The product is not
   commutative;
30     ‘‘hamilton_product(p, q)’‘ applies ‘‘q’’ first, then ‘‘p’’.
31     """
32     pw, px, py, pz = p
33     qw, qx, qy, qz = q
34     return np.array(
35         [
36             pw * qw - px * qx - py * qy - pz * qz,
37             pw * qx + px * qw + py * qz - pz * qy,
38             pw * qy - px * qz + py * qw + pz * qx,
39             pw * qz + px * qy - py * qx + pz * qw,
40         ]
41     )
42

```

```

43
44 def normalize(q: Array) -> Array:
45     """Project onto the unit sphere, which is where rotations live."""
46     norm = float(np.linalg.norm(q))
47     if norm == 0.0:
48         msg = "the zero quaternion represents no rotation and cannot
49             be normalised"
50         raise ValueError(msg)
51     return q / norm
52
53 def conjugate(q: Array) -> Array:
54     """For a unit quaternion this is also the inverse."""
55     w, x, y, z = q
56     return np.array([w, -x, -y, -z])
57
58
59 def from_axis_angle(axis: Array, angle: float) -> Array:
60     """Unit quaternion for a rotation of 'angle' about 'axis'.SO(3) twice, so  $q$  and  $-q$ 
63     are the
64     same rotation.
65     """
66     unit = normalize(np.asarray(axis, dtype=float))
67     return np.concatenate([[np.cos(angle / 2.0)], np.sin(angle / 2.0)
68         * unit])
69
70 def to_matrix(q: Array) -> Array:
71     """Rotation matrix of a unit quaternion, equation (eq:
72     quat_to_matrix)."""
73     w, x, y, z = normalize(q)
74     return np.array(
75         [
76             [1 - 2 * (y * y + z * z), 2 * (x * y - w * z), 2 * (x * z
77             + w * y)],
78             [2 * (x * y + w * z), 1 - 2 * (x * x + z * z), 2 * (y * z
79             - w * x)],
80             [2 * (x * z - w * y), 2 * (y * z + w * x), 1 - 2 * (x * x
81             + y * y)],
82             ]
83     )
84
85 def rotate(q: Array, v: Array) -> Array:
86     """Rotate a vector by the sandwich product  $q v q^*$ , without forming
87     a matrix."""
88     pure = np.concatenate([[0.0], np.asarray(v, dtype=float)])
89     return hamilton_product(hamilton_product(q, pure), conjugate(q))
90     [1:]

```

```

86
87 def slerp(q0: Array, q1: Array, t: float) -> Array:
88     """Spherical linear interpolation, equation (eq:slerp).
89
90     Two details matter and neither is cosmetic:
91
92     * If the dot product is negative the interpolation would take the
93     long way
94     round the sphere -- 360 degrees minus the short arc. Negating
95     one input
96     selects the short arc, which is legitimate because q and -q
97     denote the
98     same rotation.
99     * As the quaternions approach each other sin(0omega) approaches
100     zero and the
101     closed form loses precision, so below a threshold we fall back
102     to
103     normalised linear interpolation, which agrees to first order.
104     """
105     a = normalize(np.asarray(q0, dtype=float))
106     b = normalize(np.asarray(q1, dtype=float))
107
108     dot = float(np.dot(a, b))
109     if dot < 0.0:
110         b, dot = -b, -dot
111
112     if dot > SLERP_LINEAR_THRESHOLD:
113         return normalize(a + t * (b - a))
114
115     omega = float(np.arccos(np.clip(dot, -1.0, 1.0)))
116     sin_omega = np.sin(omega)
117     return (np.sin((1.0 - t) * omega) / sin_omega) * a + (np.sin(t *
118     omega) / sin_omega) * b
119
120
121 def interpolate_point_cloud(points: Array, q0: Array, q1: Array,
122 frames: int) -> list[Array]:
123     """Carry a point cloud through the interpolated frames.
124
125     Returns one rotated copy of ‘points’ per frame, which is what
126     the
127     chapter’s visualisation plots.
128     """
129     return [
130         (to_matrix(slerp(q0, q1, t)) @ np.asarray(points, dtype=float)
131         .T).T
132         for t in np.linspace(0.0, 1.0, frames)
133     ]

```

Key points from the code:

1. **Quaternion representation:** We store $\mathbf{q} = [q_w, q_x, q_y, q_z]$ as a NumPy array.

2. **Hamilton product:** Custom function implements the quaternion multiplication rule (equation (4.29)).
3. **Normalization:** Unit quaternions are enforced by dividing by norm.
4. **Quaternion-to-matrix:** Uses equation (4.40).
5. **SLERP:** Implements equation (4.44) with fallback to linear interpolation for nearby quaternions.
6. **Visualization:** The example rotates a point cloud through interpolated frames.

4.14 Chapter Summary

This chapter has developed the complete mathematical framework for 3D rotations:

- **Rotation Matrices ($SO(3)(3)$):** We defined the Special Orthogonal Group as the set of 3×3 orthogonal matrices with determinant $+1$, proved it is both a group and a 3-dimensional smooth manifold, and derived the three elementary rotations $\mathbf{R}_x(\theta)$, $\mathbf{R}_y(\theta)$, $\mathbf{R}_z(\theta)$.
- **Euler Angles and Gimbal Lock:** We presented the ZYX and ZXZ conventions and rigorously proved that the Jacobian relating angular velocities to Euler angle rates becomes singular at $\theta = \pm 90^\circ$, making Euler angles unsuitable for continuous control.
- **Lie Groups and Algebras:** We identified $SO(3)(3)$ as a Lie Group (smooth manifold + group structure) and its Lie algebra $\mathfrak{so}(3)(3)$ as the space of skew-symmetric matrices, which represent angular velocities.
- **Quaternion Algebra:** We introduced the quaternion representation of rotations, defined all quaternion operations (addition, Hamilton product, conjugate, norm, inverse), and proved that $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation (the double cover property).
- **Conversions:** We derived explicit formulas for converting between quaternions and rotation matrices in both directions.
- **SLERP:** We presented spherical linear interpolation as the natural way to smoothly interpolate between rotations.
- **Angular Velocity:** We connected the time-derivative of rotation matrices to angular velocity vectors, showing that $\dot{\mathbf{R}} = [\boldsymbol{\omega}]_{\times} \mathbf{R}$ and that $\mathbf{R}^T \dot{\mathbf{R}} \in \mathfrak{so}(3)(3)$.
- **Euler's Theorem:** We stated and sketched the proof that every 3D rotation can be expressed as a rotation about a single axis by a single angle.

These tools eliminate the singularities of Euler angles while providing efficient computational representations. The next chapter combines rotation matrices with translation vectors into the full rigid body transformation group $SE(3)(3)$.

4.15 Further Reading

The rotation group $\text{SO}(3)(3)$ and its extension to rigid-body transformations $\text{SE}(3)(3)$ are treated in depth by Murray, Li, and Sastry [23], whose Chapter 2 develops the group-theoretic and differential-geometric foundations that underpin this chapter. Lynch and Park [20], Chapters 3–4, provide a complementary modern treatment with numerous worked examples of rotation matrices, homogeneous transformation matrices, and their application to serial kinematic chains.

The Lie group structure of $\text{SO}(3)(3)$ and its Lie algebra $\mathfrak{so}(3)(3)$ are part of a much larger mathematical framework. Hall [14] offers a self-contained introduction to Lie groups and Lie algebras accessible to engineers with the linear algebra background developed in Chapter 1 of this text. For the geometric and symplectic perspective on rotation groups and their role in mechanics, Marsden and Ratiu [22] provide the definitive reference.

The Rodrigues rotation formula derived in this chapter has a distinguished history: it was first published by Olinde Rodrigues in 1840 [26], predating the modern matrix formalism by nearly a century. Rodrigues' original derivation used purely geometric reasoning; the matrix exponential form $e^{[\omega]_{\times}\theta}$ that we derive in Chapter 6 provides the modern algebraic counterpart.

Finally, readers implementing rotation representations in software should be aware that quaternion arithmetic is the *de facto* standard in modern robotics and game-engine frameworks (e.g., Eigen, ROS tf2, Unity) precisely because it avoids the gimbal-lock singularity and requires only four parameters with a single unit-norm constraint. The SLERP algorithm presented in this chapter is the core interpolation primitive in all of these libraries.

Exercises 4.1. Rotations, Quaternions, and $\text{SO}(3)$

4.15.1 Foundation Tier

Exercise. Verify that $\mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$ satisfies $\mathbf{R}_z^T \mathbf{R}_z = \mathbf{I}$ and $\det(\mathbf{R}_z) = 1$. Interpret each column as a rotated basis vector.

Exercise. Show that the composition of two rotations $\mathbf{R}_1$ and $\mathbf{R}_2$ (applied as $\mathbf{R}_1 \mathbf{R}_2 \mathbf{v}$) is itself a rotation by verifying closure in $\text{SO}(3)(3)$.

Exercise. Compute the full ZYX Euler angle rotation matrix for $\phi = 45^\circ$, $\theta = 30^\circ$, $\psi = 60^\circ$ by multiplying $\mathbf{R}_x(\phi)\mathbf{R}_y(\theta)\mathbf{R}_z(\psi)$. Verify orthogonality numerically.

Exercise. At $\theta = 90^\circ$ in ZYX Euler angles, show that $\dot{\phi}$ and $\dot{\psi}$ are indistinguishable in their effect on the body frame by examining which rows of the Jacobian become parallel.

Exercise. For $\omega = [1, 2, 3]^T$, construct the skew-symmetric matrix $[\omega]_{\times}$ and verify that it is skew-symmetric by checking $[\omega]_{\times}^T = -[\omega]_{\times}$.

4.15.2 Intermediate Tier

Exercise. Compute the Hamilton product $\mathbf{q}_1 \otimes \mathbf{q}_2$ for $\mathbf{q}_1 = [0.7071, 0.7071, 0, 0]^T$ and $\mathbf{q}_2 = [0.7071, 0, 0.7071, 0]^T$ by hand and verify that the result is unit norm.

Exercise. Show that $\mathbf{q} = [0.7071, 0.7071, 0, 0]^T$ and $-\mathbf{q} = [-0.7071, -0.7071, 0, 0]^T$ represent the same rotation by computing the axis and angle for each and verifying they are identical (modulo 2π).

Exercise. Convert the quaternion $\mathbf{q} = [\cos(45^\circ/2), \sin(45^\circ/2), 0, 0]^T$ (rotation about the x -axis by 45°) to a rotation matrix using equation (4.40) and verify it matches $\mathbf{R}_x(45^\circ)$.

Exercise. Interpolate between $\mathbf{q}_0 = [1, 0, 0, 0]^T$ (identity) and $\mathbf{q}_1 = [\cos(45^\circ/2), \sin(45^\circ/2), 0, 0]^T$ at $t = 0.5$ using SLERP and verify the result represents a 22.5° rotation about the x -axis.

Exercise. If $\mathbf{R}(t) = \mathbf{R}_z(t)$ (rotation about z -axis with unit angular velocity), compute $\dot{\mathbf{R}}$ and verify that $[\boldsymbol{\omega}]_\times = \mathbf{R}^T \dot{\mathbf{R}}$ with $\boldsymbol{\omega} = [0, 0, 1]^T$.

Exercise. From the rotation matrix $\mathbf{R}_x(60^\circ)$, extract the rotation axis (should be $[1, 0, 0]^T$) and angle (should be 60°) using the eigenvector method and $\text{trace}(\mathbf{R})$ formula.

4.15.3 Advanced Tier

Exercise. Prove rigorously that $SO(3)$ is a 3-dimensional manifold by counting independent constraints and showing a 3-parameter local chart. Consider both the orthogonality constraints and determinant condition.

Exercise. Show that the set of pure quaternions $\{\mathbf{q} : q_w = 0\}$ forms a 3-dimensional vector space that is isomorphic to $\mathfrak{so}(3)$. Under what operation does this isomorphism hold?

Exercise. For small t , the matrix exponential provides $\mathbf{R}(t) = e^{t[\boldsymbol{\omega}]_\times}$ where $\boldsymbol{\omega} = [0, 0, 1]^T$ (unit rotation about z). Expand the series $e^{t[\boldsymbol{\omega}]_\times} = \mathbf{I} + t[\boldsymbol{\omega}]_\times + \frac{t^2}{2}[\boldsymbol{\omega}]_\times^2 + \dots$ and verify it matches $\mathbf{R}_z(t)$ to second order in t .

Exercise. Derive the Rodrigues rotation formula $\mathbf{R} = \mathbf{I} + \sin \theta [\hat{\boldsymbol{\omega}}]_\times + (1 - \cos \theta) [\hat{\boldsymbol{\omega}}]_\times^2$ by expanding the rotation of an arbitrary vector about an axis, using the decomposition $\mathbf{v} = (\hat{\boldsymbol{\omega}} \cdot \mathbf{v})\hat{\boldsymbol{\omega}} + \mathbf{v}_\perp$.

Exercise. Prove that SLERP produces constant-speed interpolation: the quaternion at parameter t moves along the geodesic at constant angular velocity. Hint: compute the derivative $\frac{d}{dt} \text{SLERP}(\mathbf{q}_0, \mathbf{q}_1; t)$ and show its norm is constant.

Chapter 5

Twists, Wrenches, and the Screw Axis

Chasles' Theorem mathematically ensures that all motion of a rigid body in the universe, no matter how chaotic, is equivalent to spinning a screw on a single perfect axis. Motion and force are unified at the deepest level of spatial mechanics.

In Plain Language 5.1. Context & Use Case: Unifying Translation and Rotation

The Math You Will See: We will merge 3D translation vectors ($\mathbf{p}$) and 3×3 rotation matrices ($\mathbf{R}$) together into a massive 4×4 **Homogeneous Transformation Matrix** ($\mathbf{T}$). Furthermore, we will combine linear and angular velocities into a 6D **Twist** ($\mathcal{S}$), and linear forces and torques into a 6D **Wrench** ($\mathcal{F}$). We will prove Chasles' theorem rigorously, derive the group and Lie algebra structure, and develop the full machinery of screw theory including reciprocal screws and virtual work.

Why We Need It: In code, doing math on rotation and translation separately is messy, prone to bugs, and algebraically slow. If we pack everything into 4×4 matrices and 6D vectors, the equations for moving a robot arm, calculating gravity, and applying motor torques all collapse down into concise, single-line matrix multiplications. This is exactly how industrial physics engines (like MuJoCo and Drake) do their internal calculations.

All Variables Defined: $\mathbf{T} \in \text{SE}(3)$ describes a full pose, $\mathcal{S} \in \mathbb{R}^6$ describes 6D spatial velocity, and $\mathcal{F} \in \mathbb{R}^6$ describes 6D spatial force. See the **Nomenclature** at the start of the book.

Mathematical Principle 5.1. Twist and Wrench Ordering Convention

Throughout this text, we adopt the **Murray–Li–Sastry / Lynch–Park convention** [23, 20] for spatial vectors:

- **Twist** (spatial velocity): $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})^T \in \mathbb{R}^6$, with angular velocity $\boldsymbol{\omega}$ in the *top* three components and linear velocity $\mathbf{v}$ in the *bottom* three.

- **Wrench** (spatial force): $\mathcal{F} = (\boldsymbol{\tau}, \mathbf{f})^T \in \mathbb{R}^6$, with torque $\boldsymbol{\tau}$ in the top three and linear force $\mathbf{f}$ in the bottom three.

Note that Featherstone [10] uses the reversed ordering $(\mathbf{v}, \boldsymbol{\omega})$. Readers consulting Featherstone’s *Rigid Body Dynamics Algorithms* should be mindful of this transposition when comparing equations.

5.1 Historical Context: From Chasles to Modern Screw Theory

The mathematical formalism of screws has a rich history spanning over two centuries. Italian mathematician **Giulio Mozzi** first showed in 1763 that any rigid body displacement can be decomposed into a rotation and a parallel translation, though his work received little attention at the time. French mathematician **Michel Chasles** independently published the result in 1830 in a more widely disseminated form, establishing that any rigid body displacement in 3D space can be described as a rotation about a fixed axis combined with a translation parallel to that axis. This unifying insight—that even the most complex rigid body motion is geometrically equivalent to a single screw—became the cornerstone of spatial mechanics.

The theory lay relatively dormant until the late 19th century, when **Robert Stawell Ball** formalized screw calculus in his monograph *A Treatise on the Theory of Screws* [2]. Ball’s framework unified the representation of twists (infinitesimal motions) and wrenches (spatial forces) into a coherent algebraic system. He introduced the concept of **reciprocal screws**—pairs of screws satisfying a special orthogonality condition—which became essential for constraint analysis in mechanisms. The modern Lie-algebraic treatment of twists and wrenches was developed systematically by Murray, Li, and Sastry [23].

In the late 20th century, the theory experienced a modern renaissance through the work of mathematicians and roboticists including **Richard Paul** (in industrial manipulator kinematics), **John Duffy** (who systematized screw theory for mechanism analysis), and **Hunt** (through the Lie-algebra formulation). The emergence of **Lie group** methods, which view $SE(3)$ as a manifold with algebraic structure, revealed that screw theory is not merely a computational convenience but rather a fundamental geometric language for spatial mechanics.

Today, screw theory forms the mathematical backbone of robotics, mechanism design, and physics simulation. The computational efficiency of 6D spatial vectors and the elegance of group-theoretic manipulations have made screw-based algorithms the industry standard in systems like Featherstone’s Rigid Body Dynamics Algorithm [10] and modern geometry-aware physics engines.

5.2 The Unified 4×4 Transform $SE(3)$

We established in previous chapters that tracking a physical object requires two elements: its **translation vector** $\mathbf{p} \in \mathbb{R}^3$ and its **Rotation Matrix** $\mathbf{R} \in SO(3)$.

If a robot moves its arm to a new location, the new coordinates of a point attached to

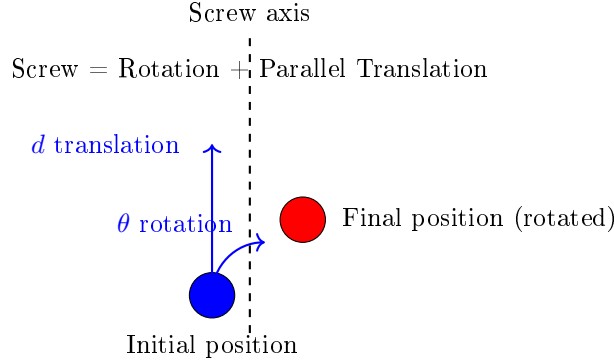


Figure 5.1: Visualization of Chasles' theorem: Any rigid body motion can be decomposed as rotation about a screw axis combined with translation parallel to that axis. This unified motion is more efficient to represent and compute with than separate rotation and translation.

that arm are computed by a sequence of operations that must be carefully ordered:

$$\mathbf{x}_{\text{mapped}} = \mathbf{R}\mathbf{x}_{\text{body}} + \mathbf{p} \quad (5.1)$$

This operation involves both matrix multiplication and vector addition, which destroys the fundamental principle of linear algebra: every transformation should be expressible as a single linear map. Having to handle addition and multiplication entirely separately also prevents us from composing transformations in a unified way. If two transformations $(\mathbf{R}_1, \mathbf{p}_1)$ and $(\mathbf{R}_2, \mathbf{p}_2)$ are applied in sequence, we must separately compute:

$$\mathbf{R}_{\text{combined}} = \mathbf{R}_2\mathbf{R}_1 \quad \text{and} \quad \mathbf{p}_{\text{combined}} = \mathbf{R}_2\mathbf{p}_1 + \mathbf{p}_2 \quad (5.2)$$

To unify translation and rotation into a single, flawless algebraic operation, mathematicians embed the 3×3 matrix and the 3×1 vector into a single 4×4 matrix using the machinery of **homogeneous coordinates**.

Definition 5.1. Homogeneous Transformation Matrix (T)

By appending a row of zeros and a single '1' to the bottom, we define the **Homogeneous Transformation Matrix T**:

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & p_x \\ R_{21} & R_{22} & R_{23} & p_y \\ R_{31} & R_{32} & R_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 4} \quad (5.3)$$

The continuous manifold of all valid Transformation Matrices mapping rigid body movement is called the **Special Euclidean Group in 3 Dimensions**, denoted as $\text{SE}(3)$.

Mathematical Principle 5.2. Composability of SE(3) Transformations

A crucial property of the homogeneous embedding is that transformation composition becomes simple matrix multiplication. If we append a 1 to the bottom of our 3D position vectors (defining points as *homogeneous coordinates* $\mathbf{x}_{\text{homo}} = [x, y, z, 1]^T$), the cumbersome addition and multiplication collapse into a single unified matrix multiplication:

$$\begin{bmatrix} \mathbf{x}_{\text{mapped}} \\ 1 \end{bmatrix} = \mathbf{T} \begin{bmatrix} \mathbf{x}_{\text{body}} \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}\mathbf{x}_{\text{body}} + \mathbf{p} \\ 1 \end{bmatrix} \quad (5.4)$$

Moreover, composing two transformations $\mathbf{T}_1, \mathbf{T}_2 \in \text{SE}(3)(3)$ is now a single matrix product:

$$\mathbf{T}_{\text{combined}} = \mathbf{T}_2 \mathbf{T}_1 \quad (5.5)$$

This is the central nervous system of modern physics simulations: algorithms like Rigid Body Dynamics (RBDA), game engines (Unreal/Unity), and geometric control theory map absolutely everything—cameras, limbs, drones, and sensors—using these 4×4 transformations in $\text{SE}(3)(3)$.

5.2.1 Group Structure of SE(3)

The set $\text{SE}(3)(3)$ is more than just a collection of matrices; it forms a **Lie group**, meaning it has both group-theoretic and smooth manifold properties. Let us verify the group axioms explicitly.

Theorem 5.1. SE(3) is a Lie Group

The set $\text{SE}(3)(3) = \{\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} : \mathbf{R} \in \text{SO}(3)(3), \mathbf{p} \in \mathbb{R}^3\}$ forms a Lie group under matrix multiplication.

Proof. We verify the group axioms:

Closure: If $\mathbf{T}_1 = \begin{bmatrix} \mathbf{R}_1 & \mathbf{p}_1 \\ 0 & 1 \end{bmatrix}$ and $\mathbf{T}_2 = \begin{bmatrix} \mathbf{R}_2 & \mathbf{p}_2 \\ 0 & 1 \end{bmatrix}$, then:

$$\mathbf{T}_1 \mathbf{T}_2 = \begin{bmatrix} \mathbf{R}_1 \mathbf{R}_2 & \mathbf{R}_1 \mathbf{p}_2 + \mathbf{p}_1 \\ 0 & 1 \end{bmatrix} \quad (5.6)$$

Since $\mathbf{R}_1 \mathbf{R}_2 \in \text{SO}(3)(3)$ (closure of $\text{SO}(3)(3)$) and $\mathbf{R}_1 \mathbf{p}_2 + \mathbf{p}_1 \in \mathbb{R}^3$, we have $\mathbf{T}_1 \mathbf{T}_2 \in \text{SE}(3)(3)$.

Identity: The identity element is $\mathbf{I} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0} \\ 0 & 1 \end{bmatrix}$. For any $\mathbf{T} \in \text{SE}(3)(3)$, we have $\mathbf{T} \mathbf{I} = \mathbf{T}$.

Inverses: For $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix}$, define:

$$\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T \mathbf{p} \\ 0 & 1 \end{bmatrix} \quad (5.7)$$

Verification:

$$\mathbf{T}\mathbf{T}^{-1} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T \mathbf{p} \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}\mathbf{R}^T & -\mathbf{R}\mathbf{R}^T \mathbf{p} + \mathbf{p} \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_3 & \mathbf{0} \\ 0 & 1 \end{bmatrix} \quad (5.8)$$

Associativity: Matrix multiplication is associative, so $(\mathbf{T}_1\mathbf{T}_2)\mathbf{T}_3 = \mathbf{T}_1(\mathbf{T}_2\mathbf{T}_3)$.

The group is smooth because the entries of $\mathbf{T}$ depend smoothly on the parameters of $\mathbf{R}$ and $\mathbf{p}$, making it a Lie group. $\square$

The dimension of $\text{SE}(3)(3)$ is $\dim(\text{SO}(3)(3)) + \dim(\mathbb{R}^3) = 3 + 3 = 6$. This reflects the fact that a rigid body pose has 6 degrees of freedom: 3 for rotation and 3 for translation.

5.3 Chasles' Theorem and the Screw Axis

If $\mathbf{T} \in \text{SE}(3)(3)$ defines the complete layout of a rigid body in space, what is the *velocity* of that body as it moves? And more fundamentally, what is the geometry of any displacement $\mathbf{T}$?

The answer is provided by one of the central theorems in mechanics:

Theorem 5.2. Chasles–Mozzi Theorem (Mozzi 1763; Chasles 1830)

Any rigid body displacement in 3D space can be expressed as a rotation about a unique axis combined with a translation parallel to that axis. In other words, every element $\mathbf{T} \in \text{SE}(3)(3)$ corresponds to a **screw motion** along a specific geometric axis.

Proof Sketch. Consider an arbitrary displacement $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \in \text{SE}(3)(3)$.

Case 1: Pure Rotation ($\mathbf{p} = \mathbf{0}$)

If $\mathbf{p} = \mathbf{0}$, then $\mathbf{T}$ represents a pure rotation $\mathbf{R} \in \text{SO}(3)(3)$. By the Rodrigues formula from Chapter 4, this rotation is equivalent to a rotation by angle θ about some axis $\hat{\mathbf{s}}$ passing through the origin. The pitch is $h = 0$ (pure rotation).

Case 2: Pure Translation ($\mathbf{R} = \mathbf{I}$)

If $\mathbf{R} = \mathbf{I}$, then $\mathbf{T}$ represents a pure translation by vector $\mathbf{p}$. This can be viewed as a screw motion with infinite pitch: the axis passes through any point and points in the direction of $\mathbf{p}$, but the rotation angle θ is infinitesimal.

Case 3: General Rotation and Translation

For arbitrary $\mathbf{R}$ and $\mathbf{p}$, decompose the translation into components parallel and perpendicular to the rotation axis. Let $\hat{\mathbf{s}}$ be the unit axis of rotation (obtained from the eigenvector of $\mathbf{R}$ corresponding to eigenvalue 1, which exists for any rotation in $\text{SO}(3)$), and let θ be the rotation angle.

The translation can be written as:

$$\mathbf{p} = h\hat{\mathbf{s}} + \mathbf{p}_\perp \quad (5.9)$$

where $\mathbf{p}_\perp \perp \hat{\mathbf{s}}$. The key observation is that $\mathbf{p}_\perp$ arises from the rotation offset: there exists a unique point $\mathbf{q}$ on the screw axis such that rotating $\mathbf{q}$ by angle θ about $\hat{\mathbf{s}}$ produces the same perpendicular displacement.

Specifically, if the screw axis passes through $\mathbf{q}$ in the direction $\hat{\mathbf{s}}$, then the motion can be expressed as: rotate by θ about the line passing through $\mathbf{q}$ parallel to $\hat{\mathbf{s}}$, then translate by $h\hat{\mathbf{s}}$ along that axis.

This axis and pitch are uniquely determined by the displacement $\mathbf{T}$. $\square$

In Plain Language 5.2. The Screw Axis Intuition

Think of a literal metal screw going into a block of wood. As the screw turns along an axis (rotational velocity), it simultaneously plunges forward along that exact identical axis (linear translation).

Mathematically, a chaotic tumbling drone or spacecraft is just spinning around a single invisible straight line in space (Rotation) while simultaneously sliding down that very same line (Translation). This line is the **screw axis**.

Example 5.1. Visualizing the Screw: Football vs. Frisbee

To visualize how the Screw Axis unifies motion, consider two classic throws:

1. **A Perfect Football Spiral:** When a quarterback throws a tight spiral, the football translates rapidly down the field (the linear velocity $\mathbf{v}$). Simultaneously, it spins rapidly around its long axis (angular velocity $\boldsymbol{\omega}$). Because the spin axis and the travel direction are perfectly parallel, this is a pure screw motion. The *Pitch* h is large because it travels far for every rotation.
2. **A Frisbee Throw:** A frisbee flies horizontally across a field ($\mathbf{v}$ points forward horizontally). However, it spins like a top around its vertical axis ($\boldsymbol{\omega}$ points straight up). The translation and rotation vectors are perfectly perpendicular (90° apart). This is *still* a screw motion! Chasles' Theorem guarantees we can find a mathematical vertical axis floating somewhere to the side of the frisbee where evaluating purely the spin around that distant axis flawlessly replicates the combined forward flight and local spin.
3. **A Tumbling Robot:** A robot falls and rotates about some chaotic axis while moving through the air. Chasles' theorem tells us that there exists a single unique line in space (the screw axis) such that the entire motion can be viewed as pure rotation about that line combined with pure translation along it.

5.3.1 Geometric Description of the Screw Axis

A **Screw Axis** $\mathcal{S}$ is fully defined by three geometric parameters:

Definition 5.2. Screw Axis Parameters

The screw axis is characterized by:

1. A specific **origin point** $\mathbf{q} \in \mathbb{R}^3$ that the line passes through (the closest point to the origin, or any point on the line).
2. A specific **direction unit vector** $\hat{\mathbf{s}} \in \mathbb{R}^3$ with $\|\hat{\mathbf{s}}\| = 1$ pointing exactly along

the line.

3. The **Pitch** $h \in \mathbb{R}$, which defines the ratio of translation speed to rotation speed (how "steep" the threads of the virtual screw are).
 - A pitch of $h = 0$ means pure spinning like a hinge, with no sliding.
 - A pitch of $h > 0$ means the screw advances along its axis as it rotates.
 - A pitch of $h \rightarrow \infty$ means pure sliding with no spinning.

The screw axis line itself is the set of all points $\mathbf{r}(t) = \mathbf{q} + t\hat{\mathbf{s}}$ for $t \in \mathbb{R}$. Any two representations with the same axis but different choices of $\mathbf{q}$ on the line give the same geometric motion.

5.4 The Spatial Twist ($\mathcal{V}$)

The Screw Axis $\mathcal{S}$ is the geometric "thread"; the **Twist** $\mathcal{V} \in \mathbb{R}^6$ is the actual physical velocity at which the body slides along and spins around that thread.

Definition 5.3. Spatial Twist

A **Spatial Twist** or **Twist** is a 6-dimensional vector that unifies angular velocity $\boldsymbol{\omega} \in \mathbb{R}^3$ and linear velocity $\mathbf{v} \in \mathbb{R}^3$:

$$\mathcal{V} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} \in \mathbb{R}^6 \quad (5.10)$$

If the body is moving along a screw axis with normalized direction $\hat{\mathbf{s}}$ at speed $\dot{\theta}$ (the rate of change along the screw), then:

$$\mathcal{V} = \mathcal{S} \dot{\theta} \quad (5.11)$$

where $\mathcal{S}$ is the normalized screw axis vector. The space of all spatial twists is denoted $\mathfrak{se}(3)(3)$, the Lie algebra of $\text{SE}(3)(3)$.

5.4.1 Screw Representation of a Twist

A twist can be expressed in terms of the screw axis and the rate of motion along it. If the screw axis has direction $\hat{\mathbf{s}}$, passes through point $\mathbf{q}$, and has pitch h , then the corresponding unit screw is:

$$\mathcal{S} = \begin{bmatrix} \hat{\mathbf{s}} \\ h\hat{\mathbf{s}} + \mathbf{q} \times \hat{\mathbf{s}} \end{bmatrix} \quad (5.12)$$

The bottom three components $h\hat{\mathbf{s}} + \mathbf{q} \times \hat{\mathbf{s}}$ encode both the pitch and the location of the axis. This representation makes it possible to work with screws algebraically as 6D vectors.

5.4.2 Twists for Standard Joint Types

Different mechanical joints produce twists with specific geometric properties:

Example 5.2. Twists for Revolute, Prismatic, and Helical Joints

Revolute Joint (Hinge): A joint that rotates about a fixed axis passing through point $\mathbf{q}$ with direction $\hat{\mathbf{s}}$. The pitch is zero (pure rotation). At angular velocity $\dot{\theta}$:

$$\mathcal{V}_{\text{rev}} = \dot{\theta} \begin{bmatrix} \hat{\mathbf{s}} \\ \mathbf{q} \times \hat{\mathbf{s}} \end{bmatrix} \quad (5.13)$$

The linear velocity at point $\mathbf{q}$ is zero; at other points, $\mathbf{v} = \boldsymbol{\omega} \times (\mathbf{r} - \mathbf{q}) = \mathbf{q} \times \hat{\mathbf{s}} \cdot \dot{\theta}$ for points $\mathbf{r}$ on the axis.

Prismatic Joint (Slider): A joint that translates along a direction $\hat{\mathbf{s}}$ with no rotation. The pitch is infinite. At velocity $\dot{d}$ along the axis:

$$\mathcal{V}_{\text{pris}} = \dot{d} \begin{bmatrix} \mathbf{0} \\ \hat{\mathbf{s}} \end{bmatrix} \quad (5.14)$$

The same linear velocity appears at all points of the body.

Helical Joint (Screw): A joint that rotates and translates together along a fixed axis. With pitch $h > 0$, the relationship $\mathbf{v}_{\text{axial}} = h\boldsymbol{\omega}$ holds. At angular velocity $\dot{\theta}$:

$$\mathcal{V}_{\text{hel}} = \dot{\theta} \begin{bmatrix} \hat{\mathbf{s}} \\ h\hat{\mathbf{s}} + \mathbf{q} \times \hat{\mathbf{s}} \end{bmatrix} \quad (5.15)$$

This single screw type unifies all three standard joint classes geometrically.

5.4.3 Matrix Representation of Twists

In numerical algorithms, twists are often represented as 4×4 skew-symmetric matrices in $\mathfrak{se}(3)(3)$:

Definition 5.4. The $\mathfrak{se}(3)$ Lie Algebra

Given a twist $\mathcal{V} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix}$, its matrix representation in $\mathfrak{se}(3)(3)$ is:

$$[\mathcal{V}] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -\omega_z & \omega_y & v_x \\ \omega_z & 0 & -\omega_x & v_y \\ -\omega_y & \omega_x & 0 & v_z \\ 0 & 0 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{4 \times 4} \quad (5.16)$$

where $[\boldsymbol{\omega}]$ is the skew-symmetric matrix of $\boldsymbol{\omega}$ (as defined in Chapter 4).

5.4.4 The Lie Bracket on $\mathfrak{se}(3)$

The Lie algebra $\mathfrak{se}(3)(3)$ inherits a Lie bracket structure from the matrix commutator. For two twists $\mathcal{V}_1, \mathcal{V}_2 \in \mathfrak{se}(3)(3)$ represented as matrices, their bracket is:

$$[\mathcal{V}_1, \mathcal{V}_2] = [\mathcal{V}_1][\mathcal{V}_2] - [\mathcal{V}_2][\mathcal{V}_1] \quad (5.17)$$

This bracket measures the "failure to commute" and has deep significance in control theory and dynamics. Geometrically, the bracket of two twists is another twist representing the relative motion between the two motions.

5.5 The Adjoint Representation

One of the most powerful tools in screw theory is the **Adjoint representation**, which describes how twists and wrenches transform when viewed from different reference frames.

5.5.1 Changing Reference Frames

Suppose we have a twist $\mathcal{V}$ expressed in the base (inertial) frame. Now we want to express it in a different frame whose pose relative to the base frame is $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \in \text{SE}(3)(3)$.

The transformation of twists is not a simple application of $\mathbf{R}$; instead, it involves both the rotation and the translation of the frame. This is captured by the **Adjoint map** $\text{Ad}_{\mathbf{T}}$.

Definition 5.5. Adjoint Representation

For $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \in \text{SE}(3)(3)$, the Adjoint map is the 6×6 matrix:

$$\text{Ad}_{\mathbf{T}} = \begin{bmatrix} \mathbf{R} & 0 \\ [\mathbf{p}]\mathbf{R} & \mathbf{R} \end{bmatrix} \quad (5.18)$$

where $[\mathbf{p}]$ is the skew-symmetric matrix of $\mathbf{p}$.

A twist $\mathcal{V}$ expressed in frame A is related to its expression in frame B (whose pose is $\mathbf{T}_{AB}$) by:

$$\mathcal{V}_B = \text{Ad}_{\mathbf{T}_{AB}} \mathcal{V}_A \quad (5.19)$$

Derivation of the Adjoint. Consider a twist generating a small displacement in the body frame. If the body has pose $\mathbf{T}$ in the base frame, and we apply a twist $\mathcal{V}$ in the body frame, the resulting infinitesimal change in the pose is:

$$\mathbf{T} \exp([\mathcal{V}]dt) \quad (5.20)$$

But from the perspective of the base frame, this same motion should be expressible using a base-frame twist $\mathcal{V}_{\text{base}}$:

$$\exp([\mathcal{V}_{\text{base}}]dt)\mathbf{T} \quad (5.21)$$

Setting these equal (to first order in dt):

$$\mathbf{T}[\mathcal{V}]dt = [\mathcal{V}_{\text{base}}]\mathbf{T}dt \quad (5.22)$$

Therefore:

$$[\mathcal{V}_{\text{base}}] = \mathbf{T}[\mathcal{V}]\mathbf{T}^{-1} \quad (5.23)$$

Substituting $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix}$ and $[\mathcal{V}] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}$:

$$\begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T \mathbf{p} \\ 0 & 1 \end{bmatrix} \quad (5.24)$$

Computing this product step by step:

$$\begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}^T & -\mathbf{R}^T \mathbf{p} \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} [\boldsymbol{\omega}]\mathbf{R}^T & -[\boldsymbol{\omega}]\mathbf{R}^T \mathbf{p} + \mathbf{v} \\ 0 & 0 \end{bmatrix} \quad (5.25)$$

$$\begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} [\boldsymbol{\omega}]\mathbf{R}^T & -[\boldsymbol{\omega}]\mathbf{R}^T \mathbf{p} + \mathbf{v} \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{R}[\boldsymbol{\omega}]\mathbf{R}^T & \mathbf{R}(-[\boldsymbol{\omega}]\mathbf{R}^T \mathbf{p} + \mathbf{v}) \\ 0 & 0 \end{bmatrix} \quad (5.26)$$

Using the identity $\mathbf{R}[\boldsymbol{\omega}]\mathbf{R}^T = [\mathbf{R}\boldsymbol{\omega}]$:

$$\mathbf{R}(-[\boldsymbol{\omega}]\mathbf{R}^T \mathbf{p} + \mathbf{v}) = -\mathbf{R}[\boldsymbol{\omega}]\mathbf{R}^T \mathbf{p} + \mathbf{R}\mathbf{v} = -[\mathbf{R}\boldsymbol{\omega}]\mathbf{p} + \mathbf{R}\mathbf{v} \quad (5.27)$$

Using the cyclic property of cross product: $-[\mathbf{R}\boldsymbol{\omega}]\mathbf{p} = [\mathbf{p}]\mathbf{R}\boldsymbol{\omega}$:

$$[\mathcal{V}_{\text{base}}] = \begin{bmatrix} [\mathbf{R}\boldsymbol{\omega}] & [\mathbf{p}]\mathbf{R}\boldsymbol{\omega} + \mathbf{R}\mathbf{v} \\ 0 & 0 \end{bmatrix} \quad (5.28)$$

Extracting the 6D vector form:

$$\mathcal{V}_{\text{base}} = \begin{bmatrix} \mathbf{R}\boldsymbol{\omega} \\ [\mathbf{p}]\mathbf{R}\boldsymbol{\omega} + \mathbf{R}\mathbf{v} \end{bmatrix} = \begin{bmatrix} \mathbf{R} & 0 \\ [\mathbf{p}]\mathbf{R} & \mathbf{R} \end{bmatrix} \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} = \text{Ad}_{\mathbf{T}}\mathcal{V} \quad (5.29)$$

□

5.6 The Spatial Wrench ($\mathcal{F}$)

Just as velocity is unified into Twists ($\mathcal{V} \in \mathbb{R}^6$), Newton's physical forces are unified by the exact same geometry.

Definition 5.6. Spatial Wrench

Torques ($\boldsymbol{\tau} \in \mathbb{R}^3$, expressed in the top three components) and linear push forces ($\mathbf{f} \in \mathbb{R}^3$, expressed in the bottom three components) are merged into a 6-Dimensional **Wrench** $\mathcal{F}$:

$$\mathcal{F} = \begin{bmatrix} \boldsymbol{\tau} \\ \mathbf{f} \end{bmatrix} \in \mathbb{R}^6 \quad (5.30)$$

Note the convention: unlike twists where $\boldsymbol{\omega}$ is on top, wrenches have torque on top. This convention ensures that the power calculation (the inner product of wrench and twist) has the correct sign.

5.6.1 Wrenches from Forces and Moments

A wrench can arise from a force $\mathbf{f}$ applied at a point $\mathbf{r}$, which also generates a moment (torque) about the origin:

$$\mathcal{F} = \begin{bmatrix} \mathbf{r} \times \mathbf{f} \\ \mathbf{f} \end{bmatrix} \quad (5.31)$$

This representation is fundamental in mechanics: a force applied at a distance from the origin creates both a linear force and a rotational moment.

5.6.2 Wrench Transformation Under Changes of Frame

Just as twists transform via the Adjoint map, wrenches transform via the **dual** or **co-adjoint** map:

$$\mathcal{F}_B = (\text{Ad}_{\mathbf{T}_{AB}})^T \mathcal{F}_A \quad (5.32)$$

This relationship ensures that mechanical power is invariant under coordinate transformations, a principle we will use next.

5.6.3 Physical Interpretation of Wrenches: Forces and Torques Unified

The wrench $\mathcal{F} = [\boldsymbol{\tau}^T, \mathbf{f}^T]^T \in \mathbb{R}^6$ is far more than a convenient notational shorthand. It is a single geometric object that captures the *complete* force system acting on a rigid body—both the linear push and the rotational tendency—in one 6D vector. Just as the twist unified angular and linear velocity, the wrench unifies torque and force.

Definition 5.7. Wrench as a Dual Vector on $\mathfrak{se}(3)(3)$

A wrench $\mathcal{F} \in \mathbb{R}^6$ is an element of the **dual space** $\mathfrak{se}(3)(3)^*$. It acts on twists (elements of $\mathfrak{se}(3)(3)$) through the natural pairing:

$$\langle \mathcal{F}, \mathcal{V} \rangle = \mathcal{F}^T \mathcal{V} = \boldsymbol{\tau} \cdot \boldsymbol{\omega} + \mathbf{f} \cdot \mathbf{v} \quad (5.33)$$

This pairing yields a scalar: mechanical power. The wrench “measures” the twist by returning the instantaneous rate of work.

To build physical intuition, consider three canonical cases:

Example 5.3. Three Canonical Wrenches

Case 1: Pure force at the origin. A force $\mathbf{f} = [0, 0, f_z]^T$ applied directly at the

origin produces no moment:

$$\mathcal{F}_{\text{pure force}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{f} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ f_z \end{bmatrix} \quad (5.34)$$

This wrench has zero pitch ($h = 0$): it is a pure “pushing” screw.

Case 2: Pure couple (torque without net force). A matched pair of equal-and-opposite forces separated by a moment arm produces a torque $\boldsymbol{\tau}$ but zero net force:

$$\mathcal{F}_{\text{couple}} = \begin{bmatrix} \boldsymbol{\tau} \\ \mathbf{0} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \tau_z \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (5.35)$$

This wrench has infinite pitch ($h = \infty$): it is a pure “twisting” screw.

Case 3: Force applied at an offset. A force $\mathbf{f} = [f_x, 0, 0]^T$ applied at position $\mathbf{r} = [0, 0, d]^T$ (a distance d along the z -axis from the origin) creates both a force and a moment:

$$\mathcal{F}_{\text{offset}} = \begin{bmatrix} \mathbf{r} \times \mathbf{f} \\ \mathbf{f} \end{bmatrix} = \begin{bmatrix} 0 \\ -d f_x \\ 0 \\ f_x \\ 0 \\ 0 \end{bmatrix} \quad (5.36)$$

The offset generates a moment about the y -axis with magnitude $d f_x$ —the familiar “lever arm” effect, now packaged automatically in the wrench representation.

The deep power of the wrench formalism lies in its *duality* with twists. Twists describe how a body moves; wrenches describe how forces act on it. Their pairing through the inner product gives physical power:

$$P = \mathcal{F}^T \mathcal{V} = \underbrace{\boldsymbol{\tau} \cdot \boldsymbol{\omega}}_{\text{rotational power}} + \underbrace{\mathbf{f} \cdot \mathbf{v}}_{\text{translational power}} \quad (5.37)$$

This relationship is not merely convenient—it is the reason the wrench convention places torque on top (matching angular velocity in the twist) and force on the bottom (matching linear velocity). The ordering is dictated by the requirement that the inner product yields the physically correct power.

Furthermore, this duality is **frame-invariant**: if we transform both the wrench and twist to a new frame using the co-adjoint and adjoint maps respectively, the power remains unchanged. This is the mathematical expression of a physical fact—energy transfer does not depend on the observer’s viewpoint.

In Plain Language 5.3. Why One 6D Object Instead of Two 3D Ones?

In introductory physics, force $\mathbf{f}$ and torque $\boldsymbol{\tau}$ are taught as separate concepts with separate equations: $\mathbf{F} = m\mathbf{a}$ for translation and $\boldsymbol{\tau} = I\boldsymbol{\alpha}$ for rotation. This separation works fine for simple problems—a block on a ramp, a spinning wheel—but breaks down for robots.

A robot joint motor produces a torque that simultaneously creates forces at the end-effector and torques at every downstream joint. Tracking forces and torques separately requires careful bookkeeping of lever arms, reference points, and sign conventions. The wrench eliminates this bookkeeping: a single 6D vector captures the entire mechanical effect, transforms cleanly between frames via the co-adjoint map, and yields power through a single dot product with the twist.

For a 6-DOF robot arm, the alternative is juggling 12 separate 3D equations (force and torque for each link). The wrench formalism reduces this to 6 equations in 6D—a conceptual and computational simplification that becomes essential for real-time control.

5.7 Proof of Chasles' Theorem

The informal proof sketch given in Section 5.3 established the geometric plausibility of Chasles' theorem. Here we present a rigorous constructive proof that extracts the screw parameters from an arbitrary rigid body displacement [2, 23].

Theorem 5.3. Chasles' Theorem (Formal Statement)

Every rigid body displacement $\mathbf{T} = (\mathbf{R}, \mathbf{p}) \in \text{SE}(3)(3)$ can be uniquely represented as a screw motion: a rotation by angle θ about a line ℓ in space combined with a translation $d = h\theta$ along that same line, where h is the pitch of the screw.

Proof. Let $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ \mathbf{0}^T & 1 \end{bmatrix} \in \text{SE}(3)(3)$.

Case 1: $\mathbf{R} = \mathbf{I}$ (Pure Translation). When $\mathbf{R} = \mathbf{I}$ and $\mathbf{p} \neq \mathbf{0}$, the displacement is a pure translation along $\hat{\mathbf{s}} = \mathbf{p} / \|\mathbf{p}\|$. This corresponds to a screw motion with rotation angle $\theta = 0$ and pitch $h \rightarrow \infty$; the screw axis is a line at infinity in the direction $\hat{\mathbf{s}}$. The displacement is $d = \|\mathbf{p}\|$.

Case 2: $\mathbf{R} \neq \mathbf{I}$ (General Displacement). By Euler's rotation theorem, every rotation $\mathbf{R} \in \text{SO}(3)(3)$, $\mathbf{R} \neq \mathbf{I}$, possesses an eigenvector $\hat{\boldsymbol{\omega}}$ with eigenvalue 1:

$$\mathbf{R}\hat{\boldsymbol{\omega}} = \hat{\boldsymbol{\omega}}, \quad \|\hat{\boldsymbol{\omega}}\| = 1. \quad (5.38)$$

The associated rotation angle $\theta \in (0, \pi]$ satisfies $\text{tr}(\mathbf{R}) = 1 + 2\cos\theta$.

Step 1 — Decompose the translation. Project $\mathbf{p}$ onto $\hat{\boldsymbol{\omega}}$ and its orthogonal complement:

$$\mathbf{p}_{\parallel} = (\hat{\boldsymbol{\omega}}^T \mathbf{p}) \hat{\boldsymbol{\omega}}, \quad \mathbf{p}_{\perp} = \mathbf{p} - \mathbf{p}_{\parallel}. \quad (5.39)$$

The pitch is determined by the parallel component:

$$h = \frac{\hat{\omega}^T \mathbf{p}}{\theta}. \quad (5.40)$$

Step 2 — Locate the screw axis. We seek a point $\mathbf{q}$ on the screw axis such that the perpendicular displacement $\mathbf{p}_\perp$ arises purely from the rotation about the line through $\mathbf{q}$ parallel to $\hat{\omega}$. The fixed-point condition for the perpendicular motion gives:

$$\mathbf{p}_\perp = (\mathbf{I} - \mathbf{R}) \mathbf{q}. \quad (5.41)$$

Because $(\mathbf{I} - \mathbf{R})$ is singular along $\hat{\omega}$ (precisely the eigenvector with eigenvalue 1), we restrict to the plane perpendicular to $\hat{\omega}$ and solve:

$$\mathbf{q} = \hat{\omega} \times \frac{(\mathbf{I} - \mathbf{R})_\perp^{-1} \mathbf{p}_\perp}{1} = \frac{1}{2} \mathbf{p}_\perp + \frac{1}{2 \tan(\theta/2)} \hat{\omega} \times \mathbf{p}_\perp, \quad (5.42)$$

where we used the identity $(\mathbf{I} - \mathbf{R})_\perp^{-1} = \frac{1}{2} \mathbf{I}_\perp + \frac{1}{2 \tan(\theta/2)} [\hat{\omega}]$, valid on the subspace perpendicular to $\hat{\omega}$.

Step 3 — Uniqueness. The screw axis direction $\hat{\omega}$ is unique up to sign (fixed by the convention $\theta > 0$). The point $\mathbf{q}$ is unique modulo translation along $\hat{\omega}$; choosing $\mathbf{q}$ as the point closest to the origin removes this ambiguity. The pitch h is uniquely determined by Eq. (5.40). Therefore the screw axis, angle, and pitch are unique. $\square$

In Plain Language 5.4. Chasles' Theorem in One Sentence

Every rigid body motion—no matter how complex—is equivalent to twisting a screw. Finding that screw is Chasles' theorem.

5.8 Mechanical Work and the Virtual Power Principle

Mathematical Principle 5.3. Virtual Work / Power Duality

One of the most useful consequences of the unified 6D formalism is that **Mechanical Power** (P)—the rate of work done by a wrench on a moving body—can be computed as a single dot-product calculation:

$$P = \mathcal{F}^T \mathcal{V} = [\boldsymbol{\tau}^T, \mathbf{f}^T] \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} = \boldsymbol{\tau} \cdot \boldsymbol{\omega} + \mathbf{f} \cdot \mathbf{v} \quad (5.43)$$

This formula states that power has two contributions: the rotational power $\boldsymbol{\tau} \cdot \boldsymbol{\omega}$ (torque times angular velocity) and the translational power $\mathbf{f} \cdot \mathbf{v}$ (force times linear velocity). Both are united in a single inner product.

Proof. The power delivered by a force $\mathbf{f}$ acting at a point with velocity $\mathbf{v}$ is $P_{\text{force}} = \mathbf{f} \cdot \mathbf{v}$. The power delivered by a torque $\boldsymbol{\tau}$ causing angular velocity $\boldsymbol{\omega}$ is $P_{\text{torque}} = \boldsymbol{\tau} \cdot \boldsymbol{\omega}$. The total mechanical power is their sum. By using the 6D twist and wrench representations, these two powers are automatically combined through the inner product formula. $\square$

5.8.1 Invariance of Power

A notable property of the spatial formalism is that power is invariant under changes of reference frame. If we express both the twist and the wrench in any frame, the inner product $\mathcal{F}^T \mathcal{V}$ yields the same value. This is the mathematical embodiment of energy conservation: the rate of work done on a body does not depend on which observer calculates it.

5.9 Reciprocal Screws and Constraint Analysis

Definition 5.8. Reciprocal Screws

Two screws (or spatial twists/wrenches viewed as screws) are **reciprocal** if they satisfy:

$$\mathcal{S}_1^T \mathcal{S}_2 = 0 \quad (5.44)$$

That is, their inner product (as 6D vectors) is zero.

Geometrically, two screws are reciprocal if and only if their lines in 3D space are perpendicular (in the sense of the generalized Plücker metric) and satisfy a specific relationship between their pitches.

5.9.1 Significance in Constraint Analysis

Reciprocal screws play a crucial role in mechanism analysis:

Mathematical Principle 5.4. Constraints from Reciprocal Screws

If a mechanism joint constrains the motion to a twist $\mathcal{V}_{\text{joint}}$, then any wrench that does zero virtual work on that joint must be reciprocal to $\mathcal{V}_{\text{joint}}$. In other words, the constraint wrench lies in the space reciprocal to the allowed twist.

For example, a revolute joint constrains motion to pure rotation about a fixed axis. The allowed twist is $\mathcal{V}_{\text{rev}} = \dot{\theta} \begin{bmatrix} \hat{\mathbf{s}} \\ \mathbf{q} \times \hat{\mathbf{s}} \end{bmatrix}$. Any wrench that the joint can exert (its constraint forces and torques) must be reciprocal to this twist, meaning they cannot deliver power along the allowed motion direction.

5.10 Physical Interpretation of Wrenches

The wrench $\mathcal{F} = [\boldsymbol{\tau}^T, \mathbf{f}^T]^T \in \mathbb{R}^6$ is not merely a convenient notation—it is the natural language for describing how forces and torques act on rigid bodies in three-dimensional space. To build physical intuition, we examine three fundamental loading scenarios.

In Plain Language 5.5. Why Combine Force and Torque into One Object?

In everyday life, we think of forces (pushes) and torques (twists) as separate things. But in robotics, they are inseparable. When you push a door handle, you are simultaneously applying a force (at the handle) and a torque (about the hinge). The wrench captures both in a single 6D vector, just as the twist captures both linear and angular velocity. This unification is not merely convenient—it is *computationally essential* for algorithms that must compute thousands of force-motion interactions per second.

5.10.1 Three Fundamental Wrench Types

Pure force at the origin. A force $\mathbf{f}$ applied directly at the origin of the reference frame produces no torque about that origin. The wrench is:

$$\mathcal{F}_{\text{pure force}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{f} \end{bmatrix} \quad (5.45)$$

Pure couple (torque with no net force). A couple is a pair of equal and opposite forces separated by a distance, producing a net torque $\boldsymbol{\tau}$ but zero net force. The wrench is:

$$\mathcal{F}_{\text{couple}} = \begin{bmatrix} \boldsymbol{\tau} \\ \mathbf{0} \end{bmatrix} \quad (5.46)$$

Examples include the twisting torque applied by a screwdriver or the reaction torque from a motor.

Force at an offset point. The most common case in robotics: a force $\mathbf{f}$ applied at a point $\mathbf{r}$ away from the reference frame origin. By the moment-arm principle, this creates a torque $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{f}$:

$$\mathcal{F}_{\text{offset}} = \begin{bmatrix} \mathbf{r} \times \mathbf{f} \\ \mathbf{f} \end{bmatrix} \quad (5.47)$$

This is the wrench that gravity produces on every link of a robot—the gravitational force $m\mathbf{g}$ acts at the centre of mass, offset from the joint frame.

Example 5.4. Gravity Wrench on a Robot Link

Consider a robot link of mass $m = 2$ kg with its centre of mass at $\mathbf{r} = [0.3, 0, 0]^T$ m from the joint frame. Gravity acts as $\mathbf{f} = [0, 0, -mg]^T = [0, 0, -19.62]^T$ N. The

wrench at the joint frame is:

$$\boldsymbol{\tau} = \mathbf{r} \times \mathbf{f} = \begin{bmatrix} 0.3 \\ 0 \\ 0 \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ -19.62 \end{bmatrix} = \begin{bmatrix} 0 \\ 5.886 \\ 0 \end{bmatrix} \text{ N}\cdot\text{m} \quad (5.48)$$

$$\mathcal{F}_{\text{gravity}} = \begin{bmatrix} 0 \\ 5.886 \\ 0 \\ 0 \\ 0 \\ -19.62 \end{bmatrix} \quad (5.49)$$

The torque component tells us gravity tries to rotate the link about the y -axis (bending the arm down). The force component tells us gravity pulls the link straight down. Both are captured in one wrench vector.

5.10.2 The Wrench-Twist Duality and Mechanical Power

The deep relationship between wrenches and twists is *duality*: they are elements of dual vector spaces. The mechanical power delivered by a wrench $\mathcal{F}$ acting on a body moving with twist $\mathcal{V}$ is:

$$P = \mathcal{F}^T \mathcal{V} = \boldsymbol{\tau} \cdot \boldsymbol{\omega} + \mathbf{f} \cdot \mathbf{v} \quad (5.50)$$

This is the sum of rotational power (torque times angular velocity) and translational power (force times linear velocity). The fact that this is a simple inner product—not requiring separate calculations for rotation and translation—is the central payoff of the spatial algebra formalism.

5.10.3 Transforming Wrenches Between Frames

When we move a wrench from frame $\{A\}$ to frame $\{B\}$, we use the *adjoint transpose*:

$$\mathcal{F}_B = [\text{Ad}_{\mathbf{T}_{BA}}]^T \mathcal{F}_A \quad (5.51)$$

This is *not* the same transformation as for twists (which use $[\text{Ad}]$ directly). The distinction arises because wrenches are *covectors*—they transform contravariantly. Physically, forces and torques measured in one frame must be re-expressed to account for both the rotation and the lever-arm effect of the frame offset.

5.11 Wrench Analysis and Force Polytopes

Example 5.5. Graphical Statics and Wrench Space

Consider a planar mechanism with several joint constraints. In classical mechanics, determining whether a mechanism can be in equilibrium under external loads required drawing force diagrams and balancing force triangles. The 6D wrench formalism unifies this classical approach into an algebraic framework.

Each joint constraint produces a set of *allowed* wrenches—the forces and torques the joint can transmit. In wrench space $\mathbb{R}^6$, these form a convex polytope (a multi-dimensional polyhedron). For the mechanism to support an external wrench $\mathcal{F}_{\text{ext}}$, that wrench must lie within the convex cone generated by the joint wrenches. For planar mechanisms, the wrench space is 3-dimensional (two force components and one torque). The convex hull of all wrench combinations the mechanism can produce is called the *force polytope* or *wrench polytope*. External wrenches that fall outside this polytope cannot be equilibrated—the mechanism will not support them. This geometric analysis extends classical graphical statics to 3D and to mechanisms with more complex constraints.

5.12 Worked Example: Wrench Analysis of a Planar 4-Bar Mechanism

Consider a planar 4-bar linkage: four rigid bars connected by revolute joints at their ends, forming a closed chain. Let's analyze what wrenches can be transmitted through this mechanism.

5.12.1 Setup

Label the four joints as A, B, C, D , arranged in a quadrilateral. Each joint is a revolute hinge (rotates about a point in the plane). Suppose an external wrench $\mathcal{F}_{\text{ext}} = [\tau_z, f_x, f_y]^T$ (two force components and one moment) is applied to link BC .

5.12.2 Twist Space of Each Joint

Each revolute joint constrains motion to a 1-degree-of-freedom rotation about the joint axis (perpendicular to the plane, i.e., along the z -axis). The allowed twist at joint i is:

$$\mathcal{V}_i = \dot{\theta}_i \begin{bmatrix} 0 \\ 0 \\ 1 \\ q_{i,x} \\ q_{i,y} \\ 0 \end{bmatrix} \quad (5.52)$$

where $q_{i,x}, q_{i,y}$ are the coordinates of joint i .

5.12.3 Constraint Wrenches

The wrench that a revolute joint can exert is any wrench reciprocal to its allowed twist. Algebraically, a wrench $\mathcal{F}_i = [\tau_x, \tau_y, \tau_z, f_x, f_y, f_z]^T$ is reciprocal to the revolute joint twist if:

$$\begin{bmatrix} 0 & 0 & 1 \\ q_{i,x} & q_{i,y} & 0 \end{bmatrix} \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} + \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix} = 0 \implies f_z = 0, \quad \tau_x = -q_{i,y}f_z = 0, \quad \tau_y = q_{i,x}f_z = 0 \quad (5.53)$$

This simplifies to: the wrench must be a pure force (no moment component perpendicular to the plane) and the force must pass through the joint point (so the moment arm is zero). In planar mechanics, this corresponds to the classical result that revolute joints can only transmit forces along the bars they connect.

5.12.4 Equilibrium Condition

For the mechanism to be in equilibrium under the external wrench $\mathcal{F}_{\text{ext}}$, there must exist wrenches $\mathcal{F}_A, \mathcal{F}_B, \mathcal{F}_C, \mathcal{F}_D$ from each joint such that:

$$\mathcal{F}_A + \mathcal{F}_B + \mathcal{F}_C + \mathcal{F}_D = -\mathcal{F}_{\text{ext}} \quad (5.54)$$

and each $\mathcal{F}_i$ is a valid constraint wrench for joint i .

This is a linear algebra problem: solve for the joint wrenches. The mechanism is capable of supporting $\mathcal{F}_{\text{ext}}$ if and only if this system has a solution.

5.12.5 Geometric Interpretation: Force Polygon

In classical graphical statics, the solution is found by drawing a force polygon: each edge of the polygon represents one of the joint forces, and the polygon closes if equilibrium is achieved. The wrench formalism recovers this construction automatically through the linear system. The "force polytope" in this case is the convex hull of all possible combinations of joint wrenches, and the external wrench must lie within this polytope for equilibrium to exist.

5.13 Python Implementation

```

1 import numpy as np
2 from scipy.spatial.transform import Rotation as R
3
4 def skew(v):
5     """Compute the 3x3 skew-symmetric matrix of a 3D vector."""
6     return np.array([
7         [0, -v[2], v[1]],
8         [v[2], 0, -v[0]],
9         [-v[1], v[0], 0]
10    ])
11
12 def unskew(S):
13     """Extract the 3D vector from a skew-symmetric matrix."""
14     return np.array([S[2,1], S[0,2], S[1,0]])
15
16 def twist_to_matrix(twist):
17     """Convert a 6D twist to its 4x4 matrix representation in se(3)."""
18     omega = twist[:3]
19     v = twist[3:]
20     T = np.zeros((4, 4))
21     T[:3, :3] = skew(omega)
22     T[:3, 3] = v
23     return T
24
25 def wrench_from_force(force, position):
26     """Create a wrench from a force applied at a position."""

```

```

27     tau = np.cross(position, force)
28     return np.hstack([tau, force])
29
30 def power(wrench, twist):
31     """Compute mechanical power as  $W^T * V$ ."""
32     return np.dot(wrench, twist)
33
34 def adjoint_matrix(T):
35     """
36     Compute the 6x6 adjoint matrix  $Ad_T$  for transforming twists.
37     T is a 4x4 homogeneous transformation matrix.
38     """
39     R = T[:3, :3]
40     p = T[:3, 3]
41
42     Ad = np.zeros((6, 6))
43     Ad[:3, :3] = R
44     Ad[3:, :3] = skew(p) @ R
45     Ad[3:, 3:] = R
46     return Ad
47
48 def transform_twist(twist, T):
49     """Transform a twist using adjoint representation."""
50     Ad = adjoint_matrix(T)
51     return Ad @ twist
52
53 def transform_wrench(wrench, T):
54     """Transform a wrench using co-adjoint representation."""
55     Ad = adjoint_matrix(T)
56     return Ad.T @ wrench
57
58 def reciprocal_check(screw1, screw2):
59     """Check if two screws (as 6D vectors) are reciprocal."""
60     return np.abs(np.dot(screw1, screw2)) < 1e-10
61
62 # Example: revolute joint twist
63 def revolute_twist(axis, point, omega_rate):
64     """
65     Create the twist for a revolute joint.
66     axis: unit direction vector
67     point: point on the axis
68     omega_rate: angular velocity magnitude
69     """
70     omega = omega_rate * axis
71     v = omega_rate * np.cross(point, axis)
72     return np.hstack([omega, v])
73
74 # Example usage
75 if __name__ == "__main__":
76     # Define a homogeneous transformation (identity + translation)
77     T = np.eye(4)
78     T[:3, 3] = np.array([1, 0, 0])
79
80     # Define a revolute joint on the z-axis through the origin
81     z_axis = np.array([0, 0, 1])
82     origin = np.array([0, 0, 0])
83     twist_rev = revolute_twist(z_axis, origin, 1.0)
84
85     # Apply a force at position (1, 0, 0)
86     force = np.array([0, 1, 0])

```

```

87 wrench = wrench_from_force(force, np.array([1, 0, 0]))
88
89 # Compute power
90 P = power(wrench, twist_rev)
91 print(f"Power: {P}")
92
93 # Check if wrench is reciprocal to twist
94 is_recip = reciprocal_check(wrench, twist_rev)
95 print(f"Reciprocal: {is_recip}")
96
97 # Transform twist to a different frame
98 twist_transformed = transform_twist(twist_rev, T)
99 print(f"Original twist: {twist_rev}")
100 print(f"Transformed twist: {twist_transformed}")

```

Listing 5.1: Screw Theory Computations in Python

5.14 Summary

In this chapter, we unified the separate worlds of translation and rotation into a single 4×4 matrix formalism $\text{SE}(3)(3)$. We proved **Chasles' Theorem**, which states that any rigid body displacement is equivalent to a screw motion—a rotation about an axis combined with a translation along that axis.

The key insights are:

1. **$\text{SE}(3)$ is a Lie Group:** The set of 4×4 homogeneous transformation matrices forms a mathematical group under matrix multiplication, with well-defined group operations and inverses.
2. **Twists ($\mathfrak{se}(3)(3)$ Lie Algebra):** The velocity of a rigid body is represented as a 6D vector combining angular and linear velocity. This is the Lie algebra, the tangent space to $\text{SE}(3)(3)$.
3. **Wrenches:** Forces and torques are similarly unified into 6D vectors, dual to twists.
4. **Power is Inner Product:** Mechanical power is the inner product of wrench and twist, $P = \mathcal{F}^T \mathcal{V}$, providing a compact, unified formula for energy calculations.
5. **Adjoint Representation:** The 6×6 Adjoint matrix transforms twists between reference frames, accounting for both rotation and translation of the frame.
6. **Reciprocal Screws:** Constraint wrenches are reciprocal to allowed twists, providing an algebraic framework for analyzing mechanism constraints.
7. **Wrench Polytopes:** The convex hull of transmissible wrenches defines the mechanical capability of a mechanism—a powerful tool for force/torque analysis.

These mathematical tools are now fully developed. In Chapter 6, we extend this framework by introducing exponential coordinates and the matrix logarithm, which enable smooth interpolation between poses and the solution of differential equations governing rigid body motion.

5.15 Further Reading

The foundational treatise on screw theory is Ball's *A Treatise on the Theory of Screws* [2], which remains remarkable for its geometric insight and systematic development of reciprocal screw systems. Readers interested in the original source will find Ball's treatment of screw systems of various orders particularly illuminating, as it laid the groundwork for modern constraint analysis in mechanism design.

For the modern Lie-algebraic formulation of twists and wrenches used throughout this chapter, the definitive robotics reference is Murray, Li, and Sastry [23], whose Chapter 2 develops the full SE(3) framework with mathematical rigor. Lynch and Park [20] provide an exceptionally accessible treatment of spatial velocities, body and space frame conventions, and the geometric meaning of the adjoint representation, making it an ideal companion text for readers encountering these ideas for the first time.

Featherstone [10] develops spatial vectors from a computational perspective, showing how the 6D twist and wrench formalism leads to efficient recursive algorithms for multi-body dynamics. His notation differs slightly from the Lie-algebraic convention but is widely used in the physics engine and simulation communities.

The connection between reciprocal screws and constraint analysis, briefly introduced in Section 5.9, has deep implications for mechanism design. The reciprocal screw framework provides an algebraic method for determining the degrees of freedom and constraint wrenches of complex spatial mechanisms, a topic treated extensively in the mechanism synthesis literature. Readers pursuing this direction will find that the duality between twist spaces and wrench spaces is the key structural insight underlying mobility analysis.

5.16 Exercises

1. **SE(3) Inverse Computation:** Given $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix}$ with $\mathbf{R} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ and $\mathbf{p} = [1, 2, 3]^T$, compute $\mathbf{T}^{-1}$.
2. **Twist from Screw Parameters:** A screw axis passes through $\mathbf{q} = [1, 0, 0]$ with direction $\hat{\mathbf{s}} = [0, 0, 1]$ (vertical) and pitch $h = 2$. At motion rate $\dot{\theta} = 0.5$ rad/s, compute the twist vector.
3. **Power Calculation:** A wrench $\mathcal{F} = [1, 0, 0, 2, 0, 0]^T$ (moment plus force) acts on a body with twist $\mathcal{V} = [0, 0, 1, 0, 1, 0]^T$ (angular plus linear velocity). Compute the mechanical power.
4. **Adjoint Transformation:** For the transformation $\mathbf{T}$ from Exercise 1, construct the 6×6 adjoint matrix $\text{Ad}_{\mathbf{T}}$ and verify that it correctly transforms the twist from Exercise 2.
5. **Reciprocal Screw:** Given the revolute joint twist $\mathcal{V}_{\text{rev}} = [0, 0, 1, 1, 0, 0]^T$, find a wrench that is reciprocal to it.
6. **Constraint Analysis:** For a mechanism with two revolute joints at positions $(0, 0, 0)$ and $(1, 0, 0)$ on the z -axis, what wrenches can they jointly support? Sketch the wrench polytope in 2D force space.

7. **Frame-Dependent Twist:** A body rotates about the x -axis with twist $\mathcal{V}_{\text{base}} = [1, 0, 0, 0, 0, 0]^T$. The body frame is displaced by $[0, 1, 0]$ from the base frame. Compute the twist as expressed in the body frame.
8. **Pitch Extraction:** From a screw axis with direction $\hat{\mathbf{s}} = [0, 0, 1]$ passing through $\mathbf{q} = [1, 1, 0]$ and pitch $h = 3$, write out the $\mathcal{S}$ vector and verify that the top and bottom components have the correct relationship.
9. **Planar Mechanism Force Path:** For a 4-bar linkage in the xy -plane, show that if an external vertical force is applied, the joint constraint forces must all lie in the plane (no z -components).
10. **Helical Joint Twist:** A helical joint rotates and translates together along the y -axis with pitch $h = 0.1$ m/rad. At angular velocity $\dot{\theta} = 2$ rad/s and passing through point $\mathbf{q} = [0, 0, 0]$, compute the full twist vector.
11. **Co-Adjoint Wrench Transformation:** Show that the co-adjoint transformation $(\text{Ad}_{\mathbf{T}})^T$ preserves the power inner product: $\mathcal{F}_{\text{base}}^T \mathcal{V}_{\text{base}} = \mathcal{F}_{\text{body}}^T \mathcal{V}_{\text{body}}$ when the wrench and twist are transformed consistently.
12. **Moment Arm Verification:** For a force $\mathbf{f} = [1, 0, 0]^T$ applied at position $\mathbf{r} = [0, 2, 0]^T$, compute the wrench and show that a revolute joint at the origin constrained to rotate about the z -axis can transmit this wrench (i.e., the wrench is reciprocal to the joint twist).
13. **Screw Axis from Transformation:** Given a small displacement $\mathbf{T} = \mathbf{I} + [\mathcal{V}]\epsilon$ with
$$[\mathcal{V}] = \begin{bmatrix} 0 & -0.1 & 0 & 1 \\ 0.1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$
 (in $\mathfrak{se}(3)$ form), extract the screw axis parameters.
14. **Reciprocal Screw Pair:** For a revolute joint with twist $\mathcal{V}_1 = [0, 0, 1, 1, 1, 0]^T$, find a prismatic joint twist $\mathcal{V}_2 = [0, 0, 0, a, b, c]^T$ such that the two twists are reciprocal.
15. **Virtual Work Principle:** Prove algebraically that if a wrench $\mathcal{F}$ is reciprocal to a twist $\mathcal{V}$, then the mechanical power $P = \mathcal{F}^T \mathcal{V} = 0$.

Chapter 6

Exponential Coordinates and Matrix Logarithms

To move smoothly through the universe, we do not jump from matrix to matrix. We integrate continuously along the exponent of a physical axis. The exponential map wraps velocity vectors onto the curved surface of rotation space; the logarithm unfolds curved poses back into flat, linear velocity directions.

In Plain Language 6.1. Context & Use Case: Smooth Interpolation and Geodesics

The Math You Will See: We will introduce **Lie Algebras** ($\mathfrak{so}(3)$ and $\mathfrak{se}(3)$), the **Matrix Exponential** ($\exp$), the **Matrix Logarithm** ($\log$), and Rodrigues' formula. We will develop exponential coordinates, the Baker-Campbell-Hausdorff formula, and understand geodesics (shortest paths) on rotation space.

Why We Need It: Imagine a robotic arm needs to move from Pose A to Pose B. You cannot just average the 4×4 matrices of Pose A and Pose B ($(\mathbf{T}_A + \mathbf{T}_B)/2$); the resulting matrix is garbage and physically means nothing. Because 3D space is curved (due to rotation), moving smoothly requires calculating the exact "velocity" of motion and sliding along it. The Logarithm extracts the direction and speed of motion, and the Exponential walks along the geodesic path.

All Variables Defined: $\mathfrak{so}(3)$ represents the tangent space of infinitesimal rotations (Lie algebra), $\text{SE}(3)$ represents the curved position manifold (Lie group), $\hat{\omega}$ is the unit rotation axis, and θ is the rotation angle.

6.1 Historical Context: Sophus Lie and the Theory of Continuous Groups

The modern theory of Lie groups emerged in the 1870s-1890s from the groundbreaking work of Norwegian mathematician **Sophus Lie**. While studying differential equations, Lie discovered that continuous families of symmetries—transformations that preserve the

structure of an equation—form special algebraic objects called **continuous groups**. These are now known as Lie groups in his honor.

Lie's central insight was that every Lie group has a **Lie algebra**—an associated linear (flat) vector space at the identity element that completely encodes the local structure of the group. The exponential map serves as the bridge: it exponentiates elements of the Lie algebra to produce group elements. Conversely, the logarithm inverts this operation.

For the rotation group $SO(3)$, Lie's theory predicts that the Lie algebra $\mathfrak{so}(3)$ should consist of 3×3 skew-symmetric matrices. The exponential map then converts these skew-symmetric matrices into rotation matrices—a fact that was rediscovered and formalized by **Olinde Rodrigues** in the context of the rotation formula.

In the early 20th century, the theory was extended and generalized. By the latter half of the century, computer scientists and roboticists (notably **Richard Murray**, **Zexiang Li**, and **S. Shankar Sastry** [23], as well as **John Hauser**) recognized that Lie group and Lie algebra techniques provide the natural language for parametrizing rigid body motions and solving differential equations in robotics. A rigorous modern treatment of Lie groups and their algebras can be found in Hall [14].

The exponential-logarithm formalism is now foundational to modern control theory, computer graphics (SLERP interpolation), and numerical algorithms for rigid body dynamics. Understanding it is essential for anyone working with continuous motion in space.

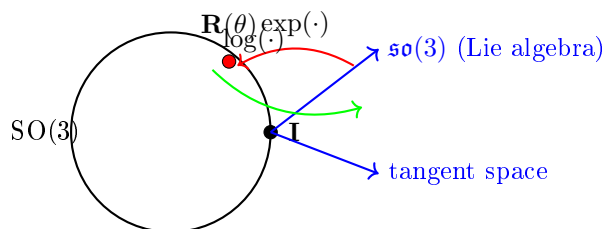


Figure 6.1: The exponential and logarithmic maps bridge the curved Lie group $SO(3)$ and its flat tangent space (Lie algebra $\mathfrak{so}(3)$). The exponential map converts infinitesimal rotations to finite rotations; the logarithm extracts the rotation axis and angle from a rotation matrix.

6.2 Lie Algebras and the Tangent Space

Recall from Chapter 5 that $SE(3)$ and $SO(3)$ are Lie groups—they have both algebraic and smooth geometric structure. But they are curved, non-linear manifolds: you cannot simply add two rotation matrices and get a valid rotation matrix.

However, at each point on a Lie group, there exists a **tangent space**—the space of infinitesimal "velocity directions" at that point. For a Lie group, the tangent space at the identity element is particularly special: it inherits a vector space structure and can be identified with the Lie algebra.

Definition 6.1. The Lie Algebra $\mathfrak{so}(3)$

The **Lie algebra** $\mathfrak{so}(3)$ is the set of all 3×3 skew-symmetric matrices:

$$\mathfrak{so}(3) = \{\mathbf{A} \in \mathbb{R}^{3 \times 3} : \mathbf{A}^T = -\mathbf{A}\} \quad (6.1)$$

Equivalently, $\mathfrak{so}(3)$ can be identified with $\mathbb{R}^3$ by the isomorphism between vectors and their skew-symmetric matrix representations. A vector $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T$ corresponds to the matrix:

$$[\boldsymbol{\omega}] = \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix} \quad (6.2)$$

The dimension of $\mathfrak{so}(3)$ is 3 (the same as $\text{SO}(3)$), and it is a **vector space**: sums and scalar multiples of skew-symmetric matrices are skew-symmetric.

Definition 6.2. The Lie Algebra $\mathfrak{se}(3)$

The **Lie algebra** $\mathfrak{se}(3)$ is the set of 4×4 matrices of the form:

$$\mathfrak{se}(3) = \left\{ \begin{bmatrix} \mathbf{A} & \mathbf{v} \\ 0 & 0 \end{bmatrix} : \mathbf{A} \in \mathfrak{so}(3), \mathbf{v} \in \mathbb{R}^3 \right\} \quad (6.3)$$

Equivalently, $\mathfrak{se}(3)$ can be identified with $\mathbb{R}^6$ by the isomorphism between 6D twist vectors $[\boldsymbol{\omega}^T, \mathbf{v}^T]^T$ and their matrix representation. The dimension is 6.

6.2.1 The Lie Bracket: Non-Commutativity of Motions

The Lie algebra structure includes a **Lie bracket** operation that measures the "failure to commute":

$$[\mathbf{A}, \mathbf{B}] = \mathbf{AB} - \mathbf{BA} \quad (6.4)$$

For skew-symmetric matrices $\mathbf{A} = [\boldsymbol{\omega}_1]$ and $\mathbf{B} = [\boldsymbol{\omega}_2]$, the bracket is itself skew-symmetric:

$$[[\boldsymbol{\omega}_1], [\boldsymbol{\omega}_2]] = [\boldsymbol{\omega}_1 \times \boldsymbol{\omega}_2] \quad (6.5)$$

Geometrically, the Lie bracket of two rotations is another rotation—a reflection of the fact that two sequential rotations about different axes can be rearranged in a non-commutative way.

6.3 Exponential Coordinates and Axis-Angle Representation

6.3.1 The Exponential Coordinate Vector

Let us begin by revisiting the concept of rotating around a single fixed unit axis $\hat{\omega} \in \mathbb{R}^3$ (with $\|\hat{\omega}\| = 1$) by a certain angle $\theta \in \mathbb{R}$ (measured in radians).

If we multiply the physical axis unit vector $\hat{\omega}$ by the total angle θ , we obtain a new 3-dimensional vector:

$$\mathbf{r} = \hat{\omega}\theta \in \mathbb{R}^3 \quad (6.6)$$

This incredibly compact vector $\mathbf{r}$ contains all the information needed to describe any 3D rotation: the direction of $\mathbf{r}$ specifies the axis of rotation, and the magnitude $\|\mathbf{r}\| = \theta$ specifies the angle. This representation is known as the **Exponential Coordinate** representation of a rotation, or the **axis-angle** representation.

In Plain Language 6.2. Why is it called "Exponential"?

In basic 1D calculus, the solution to the differential equation governing continuous exponential growth ($\dot{x} = ax$) is the scalar exponential function: $x(t) = e^{at}x(0)$. Similarly, the differential equation governing a rigid body continuously rotating around a fixed angular velocity matrix $[\hat{\omega}]$ at speed θ is:

$$\dot{\mathbf{R}} = [\hat{\omega}]\dot{\theta}\mathbf{R} \quad (6.7)$$

The solution uses exactly the same mathematical structure, just upgraded for matrices: the **Matrix Exponential**:

$$\mathbf{R}(\theta) = e^{[\hat{\omega}]\theta}\mathbf{R}(0) \quad (6.8)$$

The matrix exponential e^X is defined through its Taylor series (just like the scalar exponential):

$$e^X = \sum_{n=0}^{\infty} \frac{X^n}{n!} = I + X + \frac{X^2}{2!} + \frac{X^3}{3!} + \cdots \quad (6.9)$$

6.4 Rodrigues' Formula and the Matrix Exponential for SO(3)

Computing the matrix exponential $e^{[\hat{\omega}]\theta}$ by summing the infinite Taylor series would be prohibitively slow for computer calculations. Fortunately, there is a closed-form formula.

Theorem 6.1. Rodrigues' Formula (Exponential Map on SO(3))

For any unit vector $\hat{\omega} \in \mathbb{R}^3$ with $\|\hat{\omega}\| = 1$ and angle $\theta \in [0, \pi]$, the matrix exponential is:

$$\mathbf{R} = \exp([\hat{\omega}]\theta) = \mathbf{I} + \sin \theta [\hat{\omega}] + (1 - \cos \theta) [\hat{\omega}]^2 \quad (6.10)$$

Derivation of Rodrigues' Formula. We begin with the Taylor series expansion:

$$e^{[\hat{\omega}]\theta} = \sum_{n=0}^{\infty} \frac{([\hat{\omega}]\theta)^n}{n!} \quad (6.11)$$

The key insight is to find a pattern in the powers of $[\hat{\omega}]$. Since $\hat{\omega}$ is a unit vector, its skew-symmetric matrix satisfies a special property. Using the vector triple product identity, we can show:

$$[\hat{\omega}]^2 = \hat{\omega}\hat{\omega}^T - \mathbf{I} \quad (6.12)$$

(This follows from the identity $\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a} \cdot \mathbf{c}) - \mathbf{c}(\mathbf{a} \cdot \mathbf{b})$ applied to skew matrices.) Crucially, we also have:

$$[\hat{\omega}]^3 = -[\hat{\omega}] \quad (6.13)$$

This can be verified directly: $[\hat{\omega}]^3 = [\hat{\omega}] \cdot [\hat{\omega}]^2 = [\hat{\omega}](\hat{\omega}\hat{\omega}^T - \mathbf{I}) = \mathbf{0} - [\hat{\omega}] = -[\hat{\omega}]$ (using $[\hat{\omega}]\hat{\omega} = \mathbf{0}$).

The pattern repeats:

$$[\hat{\omega}]^4 = [\hat{\omega}]^2, \quad [\hat{\omega}]^5 = [\hat{\omega}]^3 = -[\hat{\omega}], \quad [\hat{\omega}]^6 = [\hat{\omega}]^2, \quad [\hat{\omega}]^7 = [\hat{\omega}]^3 = -[\hat{\omega}], \quad \dots \quad (6.14)$$

Therefore, the Taylor series can be reorganized by grouping terms with the same power modulo 3:

$$e^{[\hat{\omega}]\theta} = \sum_{n=0}^{\infty} \frac{([\hat{\omega}]\theta)^n}{n!} \quad (6.15)$$

$$= \sum_{k=0}^{\infty} \frac{[\hat{\omega}]^{2k}\theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{(-1)^k[\hat{\omega}]\theta^{2k+1}}{(2k+1)!} \quad (6.16)$$

$$= \sum_{k=0}^{\infty} \frac{[\hat{\omega}]^2\theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{(-1)^k[\hat{\omega}]\theta^{2k+1}}{(2k+1)!} \quad (6.17)$$

(using $[\hat{\omega}]^0 = \mathbf{I}$, $[\hat{\omega}]^2 = [\hat{\omega}]^2$, and $[\hat{\omega}]^{2k+1} = (-1)^k[\hat{\omega}]$).

Now separate the constant terms:

$$e^{[\hat{\omega}]\theta} = \mathbf{I} + \sum_{k=1}^{\infty} \frac{[\hat{\omega}]^2\theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{(-1)^k[\hat{\omega}]\theta^{2k+1}}{(2k+1)!} \quad (6.18)$$

$$= \mathbf{I} + [\hat{\omega}]^2 \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(2k)!} + [\hat{\omega}] \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k+1}}{(2k+1)!} \quad (6.19)$$

$$= \mathbf{I} + [\hat{\omega}]^2 \left(\sum_{k=0}^{\infty} \frac{\theta^{2k}}{(2k)!} - 1 \right) + [\hat{\omega}] \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k+1}}{(2k+1)!} \quad (6.20)$$

Recognizing the Taylor series for $\cos \theta$ and $\sin \theta$:

$$\cos \theta = \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k}}{(2k)!}, \quad \sin \theta = \sum_{k=0}^{\infty} \frac{(-1)^k\theta^{2k+1}}{(2k+1)!} \quad (6.21)$$

we get:

$$e^{[\hat{\omega}]^\theta} = \mathbf{I} + [\hat{\omega}]^2(\cos \theta - 1) + [\hat{\omega}] \sin \theta \quad (6.22)$$

Rearranging:

$$e^{[\hat{\omega}]^\theta} = \mathbf{I} + \sin \theta [\hat{\omega}] + (1 - \cos \theta) [\hat{\omega}]^2 \quad (6.23)$$

which is Rodrigues' formula. $\square$

6.4.1 Verification that the Exponential Map Produces Valid Rotations

We must verify that Rodrigues' formula actually produces a matrix in $\text{SO}(3)(3)$ (i.e., $\mathbf{R}^T \mathbf{R} = \mathbf{I}$ and $\det(\mathbf{R}) = 1$).

Theorem 6.2. The Exponential Map $\mathfrak{so}(3) \rightarrow \text{SO}(3)$ is Surjective

The map $\exp : \mathfrak{so}(3)(3) \rightarrow \text{SO}(3)(3)$ defined by Rodrigues' formula is surjective (onto). That is, every rotation matrix can be expressed as the exponential of some skew-symmetric matrix.

Proof. Orthogonality: For any skew-symmetric matrix $[\hat{\omega}]$, we have $[\hat{\omega}]^T = -[\hat{\omega}]$. The matrix exponential of a skew-symmetric matrix is orthogonal:

$$(e^{[\hat{\omega}]^\theta})^T e^{[\hat{\omega}]^\theta} = e^{-[\hat{\omega}]^\theta} e^{[\hat{\omega}]^\theta} = e^0 = \mathbf{I} \quad (6.24)$$

(using the property that $e^{A^T} = (e^A)^T$ for any matrix.)

Determinant: The determinant of a matrix exponential is $\det(e^A) = e^{\text{tr}(A)}$. For skew-symmetric matrices, $\text{tr}([\hat{\omega}]^\theta) = 0$, so $\det(e^{[\hat{\omega}]^\theta}) = e^0 = 1$.

Surjectivity: Any rotation matrix $\mathbf{R} \in \text{SO}(3)(3)$ can be decomposed using the axis-angle representation: there exist a unit vector $\hat{\omega}$ and angle $\theta \in [0, \pi]$ such that $\mathbf{R}$ represents rotation by angle θ about axis $\hat{\omega}$. For this rotation, Rodrigues' formula shows that $\mathbf{R} = e^{[\hat{\omega}]^\theta}$. $\square$

6.5 The Matrix Logarithm on $\text{SO}(3)$

The exponential map is not injective (one-to-one) on $\text{SO}(3)(3)$: multiple axis-angle pairs can produce the same rotation (e.g., rotating by θ about $\hat{\omega}$ is the same as rotating by $-\theta$ about $-\hat{\omega}$, and rotations by θ and $\theta + 2\pi$ are identical). However, if we restrict to $\theta \in [0, \pi]$, the exponential map becomes bijective, and the logarithm is its inverse.

Definition 6.3. Matrix Logarithm on $\text{SO}(3)$

The **Matrix Logarithm** on $\text{SO}(3)(3)$ is the inverse of the exponential map, mapping from $\text{SO}(3)(3)$ back to $\mathfrak{so}(3)(3)$:

$$\log : \text{SO}(3)(3) \rightarrow \mathfrak{so}(3)(3) \quad (6.25)$$

Given a rotation matrix $\mathbf{R}$, the logarithm extracts the axis-angle representation.

6.5.1 Computing the Logarithm: All Cases

The logarithm of a rotation matrix depends on the rotation angle and must be handled in multiple cases:

Theorem 6.3. Logarithm of $\text{SO}(3)$ Rotations

For a rotation matrix $\mathbf{R} \in \text{SO}(3)$, the logarithm $[\hat{\omega}]\theta = \log(\mathbf{R})$ can be computed as follows:

Case 1: Identity ($\theta = 0$)

If $\mathbf{R} = \mathbf{I}$, then:

$$\log(\mathbf{R}) = 0 \in \mathfrak{so}(3) \quad (6.26)$$

Case 2: Small Angle ($\theta \ll 1$)

For small rotations, $\sin \theta \approx \theta$ and $1 - \cos \theta \approx \theta^2/2$, so Rodrigues' formula becomes:

$$\mathbf{R} \approx \mathbf{I} + \theta[\hat{\omega}] + O(\theta^2) \quad (6.27)$$

Inverting this approximation (solving for $[\hat{\omega}]\theta$):

$$[\hat{\omega}]\theta \approx \mathbf{R} - \mathbf{R}^T \quad (\text{skew-symmetric part of } \mathbf{R}) \quad (6.28)$$

More precisely:

$$\log(\mathbf{R}) = \frac{1}{2}(\mathbf{R} - \mathbf{R}^T) \quad \text{for small angles} \quad (6.29)$$

Case 3: General Case

For arbitrary $\theta \in (0, \pi]$, the angle θ can be extracted from the trace of $\mathbf{R}$:

$$\text{tr}(\mathbf{R}) = 1 + 2 \cos \theta \quad (6.30)$$

Solving:

$$\theta = \arccos\left(\frac{\text{tr}(\mathbf{R}) - 1}{2}\right) \quad (6.31)$$

Once θ is known, the axis $\hat{\omega}$ is extracted from the skew-symmetric part:

$$\hat{\omega} = \frac{1}{2 \sin \theta} \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix} \quad (6.32)$$

The skew-symmetric matrix logarithm is:

$$\log(\mathbf{R}) = [\hat{\omega}]\theta \quad (6.33)$$

Case 4: π Rotation

When $\theta = \pi$ (180-degree rotation), $\sin \theta = 0$, so equation (6.32) becomes singular.

In this case, use:

$$\mathbf{R} = 2\hat{\omega}\hat{\omega}^T - \mathbf{I} \quad (6.34)$$

The axis can be found from the eigenvector of $\mathbf{R}$ corresponding to eigenvalue 1:

$$\hat{\omega} = \text{unit eigenvector of } (\mathbf{R} + \mathbf{I}) \text{ with eigenvalue } 2 \quad (6.35)$$

Example 6.1. Computing Logarithm: Example

Suppose we have a rotation matrix representing a 60° rotation about the z -axis:

$$\mathbf{R} = \begin{bmatrix} \cos(60^\circ) & -\sin(60^\circ) & 0 \\ \sin(60^\circ) & \cos(60^\circ) & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1/2 & -\sqrt{3}/2 & 0 \\ \sqrt{3}/2 & 1/2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (6.36)$$

Step 1: Compute $\text{tr}(\mathbf{R}) = 1/2 + 1/2 + 1 = 2$.

Step 2: Extract angle: $\theta = \arccos((2 - 1)/2) = \arccos(1/2) = 60^\circ = \pi/3$ rad.

Step 3: Extract axis from skew part:

$$\hat{\omega} = \frac{1}{2 \sin(60^\circ)} \begin{bmatrix} 0 - 0 & 0 - 0 \\ \sqrt{3}/2 - (-\sqrt{3}/2) \end{bmatrix} = \frac{1}{2 \cdot \sqrt{3}/2} \begin{bmatrix} 0 \\ 0 \\ \sqrt{3} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad (6.37)$$

Step 4: Construct logarithm:

$$\log(\mathbf{R}) = \begin{bmatrix} 0 \\ 0 \\ \pi/3 \end{bmatrix} \quad (6.38)$$

As a skew matrix: $[\hat{\omega}]\theta = \begin{bmatrix} 0 & -\pi/3 & 0 \\ \pi/3 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$

Mathematical Principle 6.1. Geometric Distance via Logarithms

The Matrix Logarithm is computationally critical in optimal control and error correction. If your robotic hand is at orientation $\mathbf{R}_{\text{current}}$ and your target is $\mathbf{R}_{\text{target}}$, you *absolutely cannot* compute an "error" by subtracting them ($\mathbf{R}_{\text{target}} - \mathbf{R}_{\text{current}}$) because subtracting curved matrices yields garbage that does not belong to $\text{SO}(3)(3)$. The true geometric error—the precise, shortest-path arc of rotation needed to correct the deviation—is found using the Logarithm of their relative transform:

$$\mathbf{e}_{\text{error}} = \log(\mathbf{R}_{\text{target}} \mathbf{R}_{\text{current}}^T) \quad (6.39)$$

The resulting axis-angle vector $\mathbf{e}_{\text{error}} \in \mathfrak{so}(3)(3)$ lives in the tangent space and directly tells you the minimal rotation needed to align the two frames. This is the mathematical foundation for proportional-derivative (PD) control in robotics.

6.6 The Exponential Map on $\text{SE}(3)$

We now extend the exponential map from rotations alone ($\text{SO}(3)(3)$) to full spatial rigid body transformations ($\text{SE}(3)(3)$).

Definition 6.4. The SE(3) Exponential Map

Given a twist $\mathcal{V} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} \in \mathfrak{se}(3)(3)$ and a scalar θ (the parameter along the motion), the exponential map is:

$$\mathbf{T} = \exp([\mathcal{V}]\theta) \in \text{SE}(3)(3) \quad (6.40)$$

where $[\mathcal{V}] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}$ is the 4×4 matrix representation of the twist.

6.6.1 Derivation of the SE(3) Exponential Formula

To evaluate $\exp([\mathcal{V}]\theta)$, we use the structure of the matrix. Let $[\boldsymbol{\omega}]$ denote the skew-symmetric matrix of $\boldsymbol{\omega}$, which satisfies $[\boldsymbol{\omega}]^3 = -[\boldsymbol{\omega}]$ as before.

The Taylor series is:

$$\exp([\mathcal{V}]\theta) = \sum_{n=0}^{\infty} \frac{1}{n!} \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}^n \theta^n \quad (6.41)$$

Computing powers:

$$\begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}^2 = \begin{bmatrix} [\boldsymbol{\omega}]^2 & [\boldsymbol{\omega}]\mathbf{v} \\ 0 & 0 \end{bmatrix} \quad (6.42)$$

$$\begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ 0 & 0 \end{bmatrix}^3 = \begin{bmatrix} [\boldsymbol{\omega}]^3 & [\boldsymbol{\omega}]^2\mathbf{v} \\ 0 & 0 \end{bmatrix} \quad (6.43)$$

The pattern is similar to the $SO(3)$ case, but now we must also track the translation vectors. Using the fact that $[\boldsymbol{\omega}]^3 = -[\boldsymbol{\omega}]$:

$$\exp([\mathcal{V}]\theta) = \begin{bmatrix} e^{[\boldsymbol{\omega}]\theta} & \mathbf{V}(\theta)\mathbf{v} \\ 0 & 1 \end{bmatrix} \quad (6.44)$$

where the **V matrix** (or screw Jacobian) is:

Theorem 6.4. The V Matrix Formula

The 3×3 matrix that relates the translation component of the twist to the spatial translation is:

$$\mathbf{V}(\theta) = \mathbf{I} + (1 - \cos \theta) \frac{[\boldsymbol{\omega}]}{\|\boldsymbol{\omega}\|^2} + (\theta - \sin \theta) \frac{[\boldsymbol{\omega}]^2}{\|\boldsymbol{\omega}\|^3} \quad (6.45)$$

For unit angular velocity $\hat{\boldsymbol{\omega}}$, this simplifies to:

$$\mathbf{V}(\theta) = \mathbf{I} + (1 - \cos \theta)[\hat{\boldsymbol{\omega}}] + (\theta - \sin \theta)[\hat{\boldsymbol{\omega}}]^2 \quad (6.46)$$

For a prismatic (pure translation) twist with $\boldsymbol{\omega} = 0$, we have $\mathbf{V} = \mathbf{I}$.

Proof. Separating the Taylor series into even and odd powers (as we did for $SO(3)$), and

using the cyclic property $[\omega]^3 = -[\omega]$:

$$\exp([\mathcal{V}]\theta) = \sum_{n=0}^{\infty} \frac{1}{n!} \begin{bmatrix} [\omega] & \mathbf{v} \\ 0 & 0 \end{bmatrix}^n \theta^n \quad (6.47)$$

$$= \begin{bmatrix} \sum_{k=0}^{\infty} \frac{[\omega]^{2k} \theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{[\omega]^{2k+1} \theta^{2k+1}}{(2k+1)!} & \mathbf{V}(\theta)\mathbf{v} \\ 0 & 1 \end{bmatrix} \quad (6.48)$$

The rotation part gives Rodrigues' formula. For the translation part, we have:

$$\mathbf{V}(\theta)\mathbf{v} = \left(\mathbf{I} + \sum_{k=1}^{\infty} \frac{[\omega]^2 \theta^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{[\omega] \theta^{2k+1}}{(2k+1)!} \right) \mathbf{v} \quad (6.49)$$

$$= \left(\mathbf{I} + [\omega] \sum_{k=0}^{\infty} \frac{\theta^{2k+1}}{(2k+1)!} + [\omega]^2 \sum_{k=1}^{\infty} \frac{\theta^{2k}}{(2k)!} \right) \mathbf{v} \quad (6.50)$$

$$= (\mathbf{I} + [\omega] \sin \theta + [\omega]^2 (\cos \theta - 1)) \mathbf{v} \quad (6.51)$$

This is the $\mathbf{V}$ matrix. □

6.7 The Matrix Logarithm on SE(3)

The inverse of the exponential map is the logarithm:

Definition 6.5. Matrix Logarithm on SE(3)

Given a transformation $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \in \text{SE}(3)$, the logarithm extracts the twist:

$$\log(\mathbf{T}) = [\mathcal{V}] \quad (6.52)$$

where $\mathcal{V} = \begin{bmatrix} \omega \\ \mathbf{v} \end{bmatrix}$ is reconstructed as:

1. Extract the angular part: $[\omega] = \log(\mathbf{R})$ using the $SO(3)$ logarithm.
2. Extract the linear part: $\mathbf{v} = \mathbf{V}^{-1}(\theta)\mathbf{p}$, where $\mathbf{V}^{-1}$ is the inverse of the $\mathbf{V}$ matrix.

For numerical implementation, the inverse of the $\mathbf{V}$ matrix is:

$$\mathbf{V}^{-1}(\theta) = \mathbf{I} - \frac{1}{2}[\omega] + \frac{1}{\theta^2} \left(\frac{1}{2} - \frac{\sin(2\theta)}{4 \sin \theta} \right) [\omega]^2 \quad (6.53)$$

(and for $\theta \rightarrow 0$, use L'Hôpital's rule or series approximations to avoid numerical issues).

6.7.1 Numerical Stability of the Matrix Logarithm

The closed-form logarithm derived in Section 6.5 is exact in infinite precision but requires care in floating-point arithmetic at two boundary regimes.

Near $\theta = 0$ (identity rotation). For $\theta \ll 1$, the general formula $[\hat{\omega}] = \frac{\theta}{2 \sin \theta}(\mathbf{R} - \mathbf{R}^T)$ suffers because both θ and $\sin \theta$ approach zero. However their ratio is well-behaved: $\theta / \sin \theta \rightarrow 1$. In practice, use the first-order Taylor expansion

$$\log(\mathbf{R}) \approx \mathbf{R} - \mathbf{I} \quad (\theta \ll 1), \quad (6.54)$$

which is the skew-symmetric part of $\mathbf{R}$ and avoids all division by small quantities.

Near $\theta = \pi$ (180° rotation). The standard axis-extraction formula

$$\hat{\omega} = \frac{1}{2 \sin \theta} \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix} \quad (6.55)$$

has a $1/\sin \theta$ singularity as $\theta \rightarrow \pi$, because $\sin \pi = 0$. Near this boundary the axis must instead be extracted from the *eigenvector* of $\mathbf{R}$ corresponding to eigenvalue +1. A robust approach is to compute $\mathbf{S} = \mathbf{R} + \mathbf{I}$ and take the column of $\mathbf{S}$ with largest norm as $\hat{\omega}$ (after normalization).

Condition number. The condition number of the logarithm map at angle θ scales as

$$\kappa(\theta) \approx \frac{\theta}{\sin \theta}, \quad (6.56)$$

which equals 1 at $\theta = 0$ and diverges as $\theta \rightarrow \pi$. Consequently, the logarithm is inherently ill-conditioned near half-turns: small perturbations in $\mathbf{R}$ can produce large changes in the recovered axis.

In Plain Language 6.3. Practical Recommendation

For maximum robustness, use an `atan2`-based extraction of θ (avoiding the arccos whose derivative diverges at ± 1) and switch to eigenvector-based axis recovery when $\sin \theta$ drops below a threshold (e.g. 10^{-6}). Modern libraries such as `scipy.spatial.transform` implement these guards automatically.

6.8 The Baker-Campbell-Hausdorff Formula

One of the most subtle facts about Lie algebras and Lie groups is that they do not commute under composition. That is, if $\mathbf{A}$ and $\mathbf{B}$ are two elements of a Lie algebra, then:

$$e^{\mathbf{A}}e^{\mathbf{B}} \neq e^{\mathbf{A}+\mathbf{B}} \quad (6.57)$$

in general. The difference is captured by the **Baker-Campbell-Hausdorff (BCH) formula**, which expresses the "product" of two exponentials as a single exponential:

Theorem 6.5. Baker-Campbell-Hausdorff Formula (First-Order)

For small perturbations $\mathbf{A}, \mathbf{B} \in \mathfrak{g}$ (the Lie algebra) with $\|\mathbf{A}\|, \|\mathbf{B}\| \ll 1$:

$$\log(e^{\mathbf{A}}e^{\mathbf{B}}) = \mathbf{A} + \mathbf{B} + \frac{1}{2}[\mathbf{A}, \mathbf{B}] + O(\|\mathbf{A}\|^3, \|\mathbf{B}\|^3) \quad (6.58)$$

where $[\mathbf{A}, \mathbf{B}] = \mathbf{AB} - \mathbf{BA}$ is the Lie bracket (commutator).

Mathematical Principle 6.2. Geometric Interpretation of BCH

The BCH formula reveals that when you compose two small rotations, the result is not simply the sum of the rotations, but the sum plus a correction term proportional to their bracket (commutator). This correction term vanishes only if the two motions commute (i.e., $[\mathbf{A}, \mathbf{B}] = 0$).

For rotations, the bracket of two twists is another twist:

$$[\mathcal{V}_1, \mathcal{V}_2] = \text{Ad}_{\exp(\mathcal{V}_1)} \mathcal{V}_2 - \mathcal{V}_2 + \text{higher order} \quad (6.59)$$

This is the mathematical reason why the order of rotations matters: it is not a "software bug" but a fundamental property of 3D rotations.

The full BCH formula (for arbitrary elements) is an infinite series:

$$\log(e^{\mathbf{A}}e^{\mathbf{B}}) = \mathbf{A} + \mathbf{B} + \frac{1}{2}[\mathbf{A}, \mathbf{B}] + \frac{1}{12}([\mathbf{A}, [\mathbf{A}, \mathbf{B}]] + [\mathbf{B}, [\mathbf{B}, \mathbf{A}]]) + \cdots \quad (6.60)$$

In practice, the first-order approximation is sufficient for small perturbations. A thorough development of the BCH formula and its convergence properties can be found in Hall [14].

6.8.1 The Baker-Campbell-Hausdorff Formula: Higher-Order Terms

The first-order BCH approximation $\log(e^{\mathbf{A}}e^{\mathbf{B}}) \approx \mathbf{A} + \mathbf{B} + \frac{1}{2}[\mathbf{A}, \mathbf{B}]$ is adequate when both $\mathbf{A}$ and $\mathbf{B}$ are small. For larger rotations the higher-order Lie-bracket terms become significant. The full expansion to third order reads [14]:

Theorem 6.6. BCH Formula to Third Order

For $\mathbf{A}, \mathbf{B}$ in a Lie algebra $\mathfrak{g}$,

$$\log(e^{\mathbf{A}}e^{\mathbf{B}}) = \mathbf{A} + \mathbf{B} + \frac{1}{2}[\mathbf{A}, \mathbf{B}] + \frac{1}{12}([\mathbf{A}, [\mathbf{A}, \mathbf{B}]] + [\mathbf{B}, [\mathbf{B}, \mathbf{A}]]) + O(\|\cdot\|^4), \quad (6.61)$$

where $[\cdot, \cdot]$ denotes the Lie bracket and the remainder collects terms involving four or more nested brackets.

Convergence. For the rotation group $\text{SO}(3)(3)$, the BCH series converges absolutely whenever $\|\mathbf{A}\|, \|\mathbf{B}\| < \pi$, i.e. whenever both rotation angles are less than 180° . Outside this radius the series may diverge because the exponential map on $\text{SO}(3)(3)$ is not globally injective—rotations by θ and $\theta + 2\pi$ are indistinguishable. More precisely, the convergence radius is set by the injectivity radius of the exponential map, which equals π for $\text{SO}(3)(3)$.

Physical meaning. The BCH formula quantifies how much two sequential rotations about different axes *interact*. The first-order bracket term $\frac{1}{2}[\mathbf{A}, \mathbf{B}]$ captures the leading coupling between the two rotation axes. The second-order terms $\frac{1}{12}[\mathbf{A}, [\mathbf{A}, \mathbf{B}]]$ and $\frac{1}{12}[\mathbf{B}, [\mathbf{B}, \mathbf{A}]]$ describe how each rotation axis “precesses” the other—an effect that grows with the magnitude of the rotations.

Mathematical Principle 6.3. Small-Angle Approximation from BCH

For small angles ($\|\mathbf{A}\|, \|\mathbf{B}\| \ll 1$), all bracket terms are negligible and

$$e^{\mathbf{A}} e^{\mathbf{B}} \approx e^{\mathbf{A}+\mathbf{B}}. \quad (6.62)$$

Rotations therefore “add like vectors” in the small-angle regime. For large angles the Lie bracket terms become significant: a 90° rotation about x followed by a 90° rotation about y differs from the reverse order by a 90° rotation about z —a dramatic non-commutativity fully predicted by the BCH series.

6.9 Numerical Computation of Matrix Exponentials

Computing the matrix exponential e^A numerically is a classic problem in scientific computing. For large matrices or large exponents, directly summing the Taylor series is inefficient and can be numerically unstable.

6.9.1 Padé Approximation

One robust approach is the **Padé approximation**, which approximates e^A as a ratio of two polynomials:

$$e^A \approx \frac{N(A)}{D(A)} = \left(I + \frac{A}{2} + \frac{A^2}{12} + \cdots \right) \left(I - \frac{A}{2} + \frac{A^2}{12} - \cdots \right)^{-1} \quad (6.63)$$

The advantage is that rational functions converge faster than power series and are more numerically stable.

6.9.2 Scaling and Squaring

For very large matrices, the **scaling and squaring** method is effective: compute $e^{A/2^k}$ (which has small norm and is easy to compute), then square the result k times:

$$e^A = (e^{A/2^k})^{2^k} \quad (6.64)$$

In Plain Language 6.4. Practical Implementation Note

Modern libraries like Eigen (C++), NumPy (Python), and MATLAB provide optimized `matrix_exp` functions that combine multiple techniques (scaling, Padé approximation, eigenvalue decomposition if applicable) to ensure numerical accuracy and efficiency. For robotics applications, it is almost always better to use these library functions than to implement your own.

6.10 Geodesics on SO(3): Shortest Rotation Paths

A **geodesic** is the shortest path between two points on a curved surface. On the surface of a sphere, geodesics are great circles. On the manifold $\text{SO}(3)$, geodesics are paths parameterized by the exponential map.

Mathematical Principle 6.4. Geodesics via the Exponential Map

The shortest path (in a Riemannian sense) from the identity rotation to a rotation $\mathbf{R} \in \text{SO}(3)(3)$ is:

$$\mathbf{R}(t) = \exp(t \cdot \log(\mathbf{R})), \quad t \in [0, 1] \quad (6.65)$$

where $t = 0$ gives the identity and $t = 1$ gives $\mathbf{R}$.

Geometrically, this path rotates at constant angular velocity along the axis extracted by the logarithm. The path length is the rotation angle $\theta = \|\log(\mathbf{R})\|$.

Proof. The geodesic distance on $\text{SO}(3)(3)$ is defined by the Riemannian metric inherited from the tangent space. The exponential map is a local isometry, meaning it preserves distances: the Euclidean distance in the tangent space $\mathfrak{so}(3)(3)$ equals the geodesic distance on the manifold $\text{SO}(3)(3)$ (at least locally).

A straight line in the tangent space—parameterized by $t \cdot \mathbf{v}$ for a direction $\mathbf{v}$ —has minimal length. Exponentiating this line gives the geodesic on the manifold. $\square$

6.10.1 Geodesics on $\text{SO}(3)(3)$: Proof via Riemannian Geometry

The informal argument above appeals to intuition; we now give a more rigorous derivation grounded in Riemannian geometry [22].

Definition 6.6. Bi-Invariant Metric on $\text{SO}(3)(3)$

The **bi-invariant Riemannian metric** on $\text{SO}(3)(3)$ is defined at the identity by the inner product on $\mathfrak{so}(3)(3)$:

$$\langle \mathbf{A}, \mathbf{B} \rangle = -\frac{1}{2} \text{tr}(\mathbf{AB}), \quad \mathbf{A}, \mathbf{B} \in \mathfrak{so}(3)(3). \quad (6.66)$$

This metric is extended to the entire group by left (or right) translation: for tangent vectors $\mathbf{V}, \mathbf{W}$ at $\mathbf{R} \in \text{SO}(3)(3)$,

$$\langle \mathbf{V}, \mathbf{W} \rangle_{\mathbf{R}} = -\frac{1}{2} \text{tr}((\mathbf{R}^{-1}\mathbf{V})(\mathbf{R}^{-1}\mathbf{W})). \quad (6.67)$$

The metric is *bi-invariant*: unchanged under both left and right multiplication by any group element.

Theorem 6.7. One-Parameter Subgroups Are Geodesics

With respect to the bi-invariant metric (6.66), the geodesics through the identity $\mathbf{I} \in \text{SO}(3)(3)$ are exactly the one-parameter subgroups

$$\gamma(t) = e^{t[\hat{\omega}]}, \quad [\hat{\omega}] \in \mathfrak{so}(3)(3). \quad (6.68)$$

Proof. A curve $\gamma(t)$ on $\text{SO}(3)(3)$ is a geodesic if and only if it satisfies the Euler–Lagrange equation for length-minimizing curves. For a bi-invariant metric this equation reduces to

the condition that the **body velocity** is constant:

$$\dot{\gamma}(t) \gamma(t)^{-1} = \text{const} \in \mathfrak{so}(3)(3). \quad (6.69)$$

For the one-parameter subgroup $\gamma(t) = e^{t\mathbf{A}}$ with $\mathbf{A} \in \mathfrak{so}(3)(3)$, we compute

$$\dot{\gamma}(t) = \mathbf{A} e^{t\mathbf{A}}, \quad \dot{\gamma}(t) \gamma(t)^{-1} = \mathbf{A} e^{t\mathbf{A}} e^{-t\mathbf{A}} = \mathbf{A}, \quad (6.70)$$

which is manifestly constant. Hence every one-parameter subgroup is a geodesic. Conversely, any solution of (6.69) with $\gamma(0) = \mathbf{I}$ and $\dot{\gamma}(0) = \mathbf{A}$ must satisfy $\gamma(t) = e^{t\mathbf{A}}$ by uniqueness of ODE solutions. Therefore the geodesics through $\mathbf{I}$ are *precisely* the one-parameter subgroups. $\square$

Theorem 6.8. Geodesic Distance on $\text{SO}(3)(3)$

The geodesic distance between two rotations $\mathbf{R}_1, \mathbf{R}_2 \in \text{SO}(3)(3)$ under the bi-invariant metric is

$$d(\mathbf{R}_1, \mathbf{R}_2) = \|\log(\mathbf{R}_1^T \mathbf{R}_2)\| = \theta, \quad (6.71)$$

where $\theta \in [0, \pi]$ is the angle of the relative rotation $\mathbf{R}_1^T \mathbf{R}_2$ and the norm is $\|\mathbf{A}\| = \sqrt{\langle \mathbf{A}, \mathbf{A} \rangle} = \sqrt{-\frac{1}{2} \text{tr}(\mathbf{A}^2)}$.

Proof. By left-invariance of the metric, $d(\mathbf{R}_1, \mathbf{R}_2) = d(\mathbf{I}, \mathbf{R}_1^T \mathbf{R}_2)$. The geodesic from $\mathbf{I}$ to $\mathbf{R}_1^T \mathbf{R}_2$ is $\gamma(t) = e^{t \log(\mathbf{R}_1^T \mathbf{R}_2)}$, whose length is $\|\log(\mathbf{R}_1^T \mathbf{R}_2)\|$. For $\mathbf{A} = [\hat{\omega}] \theta \in \mathfrak{so}(3)(3)$, we have $\|\mathbf{A}\| = \theta$, confirming that the geodesic distance is the rotation angle. $\square$

6.11 SLERP: Spherical Linear Interpolation as Geodesic Interpolation

In Chapter 4, we introduced **SLERP (Spherical Linear Interpolation)**, which smoothly interpolates between two quaternions. We can now understand SLERP as a discretization of the geodesic on $\text{SO}(3)(3)$ [20].

Theorem 6.9. SLERP as Exponential Interpolation

Given two rotation matrices $\mathbf{R}_0$ and $\mathbf{R}_1$, the SLERP interpolation:

$$\mathbf{R}(t) = \mathbf{R}_0 \exp(t \log(\mathbf{R}_0^T \mathbf{R}_1)), \quad t \in [0, 1] \quad (6.72)$$

traces a geodesic from $\mathbf{R}_0$ to $\mathbf{R}_1$. This path:

1. Is the shortest distance (in terms of rotation angle) between the two orientations.
2. Rotates at constant angular velocity.
3. Is independent of the order (left-invariant parameterization) or dependent on the parameterization (right-invariant), depending on whether we multiply on the left or right.

Proof. At $t = 0$: $\mathbf{R}(0) = \mathbf{R}_0 \exp(0) = \mathbf{R}_0$.

At $t = 1$: $\mathbf{R}(1) = \mathbf{R}_0 \exp(\log(\mathbf{R}_0^T \mathbf{R}_1)) = \mathbf{R}_0(\mathbf{R}_0^T \mathbf{R}_1) = \mathbf{R}_1$.

The rotation angle between $\mathbf{R}(t)$ and $\mathbf{R}(0)$ is $t\theta_{\max}$ where $\theta_{\max} = \|\log(\mathbf{R}_0^T \mathbf{R}_1)\|$, so the angular velocity is constant.

For quaternions, the same path can be parameterized as:

$$\mathbf{q}(t) = \frac{\sin((1-t)\theta)}{\sin\theta} \mathbf{q}_0 + \frac{\sin(t\theta)}{\sin\theta} \mathbf{q}_1 \quad (6.73)$$

where $\theta = \arccos(\mathbf{q}_0 \cdot \mathbf{q}_1)$ is the angle between the quaternions. This is the classical SLERP formula. $\square$

Example 6.2. SLERP vs. Linear Interpolation

Consider interpolating between two rotations: - $\mathbf{R}_0 = I$ (identity, no rotation) - $\mathbf{R}_1 = e^{[0,0,1] \cdot \pi/2}$ (90-degree rotation about z -axis)

Linear Interpolation (Wrong):

$$\mathbf{R}_{\text{wrong}}(t) = (1-t)\mathbf{R}_0 + t\mathbf{R}_1 = (1-t)\mathbf{I} + t\mathbf{R}_1 \quad (6.74)$$

At $t = 0.5$, this yields a matrix that is neither orthogonal nor a valid rotation—it is geometrically meaningless.

SLERP (Correct):

$$\mathbf{R}_{\text{correct}}(t) = \mathbf{R}_0 \exp(t \log(\mathbf{R}_0^T \mathbf{R}_1)) = \exp(t[0, 0, 1] \cdot \pi/2) \quad (6.75)$$

At $t = 0.5$, this yields a rotation by 45 degrees about the z -axis, which is geometrically correct and lies on the shortest path between the two orientations.

6.12 Python Worked Example: Computing Exponentials and Logarithms

```

1 import numpy as np
2 from scipy.spatial.transform import Rotation as R
3 from scipy.linalg import expm, logm
4 import warnings
5
6 def skew(v):
7     """Compute the skew-symmetric matrix of a 3D vector."""
8     return np.array([
9         [0, -v[2], v[1]],
10        [v[2], 0, -v[0]],
11        [-v[1], v[0], 0]
12    ])
13
14 def unskew(S):
15     """Extract a 3D vector from a skew-symmetric matrix."""
16     return np.array([S[2, 1], S[0, 2], S[1, 0]])
17
18 def rodrigues(omega, theta):
19     """

```

```

20     Compute rotation matrix using Rodrigues' formula.
21     omega: unit axis vector (3D)
22     theta: rotation angle in radians
23     """
24     if np.linalg.norm(omega) < 1e-10:
25         return np.eye(3)
26
27     omega_hat = omega / np.linalg.norm(omega)
28     omega_skew = skew(omega_hat)
29
30     R = (np.eye(3) +
31          np.sin(theta) * omega_skew +
32          (1 - np.cos(theta)) * omega_skew @ omega_skew)
33
34     return R
35
36 def log_SO3(R):
37     """
38     Compute the logarithm of a rotation matrix.
39     Returns the axis-angle vector w = omega_hat * theta.
40     """
41     # Clamp to avoid numerical errors with arccos
42     trace = np.trace(R)
43     theta = np.arccos(np.clip((trace - 1) / 2, -1, 1))
44
45     if theta < 1e-10: # Identity rotation
46         return np.zeros(3)
47     elif np.abs(theta - np.pi) < 1e-10: # 180-degree rotation
48         # Find the axis from the eigenvector with eigenvalue 1
49         eigvals, eigvecs = np.linalg.eig((R + np.eye(3)))
50         idx = np.argmax(np.real(eigvals))
51         omega_hat = np.real(eigvecs[:, idx])
52         omega_hat = omega_hat / np.linalg.norm(omega_hat)
53         return omega_hat * theta
54     else:
55         omega_skew = (R - R.T) / (2 * np.sin(theta))
56         omega_hat = unskew(omega_skew)
57         return omega_hat * theta
58
59 def exp_SE3(twist, theta):
60     """
61     Compute the exponential map from se(3) to SE(3).
62     twist: 6D twist vector [omega; v]
63     theta: parameter (typically 1.0 for unit time step)
64     """
65     omega = twist[:3]
66     v = twist[3:]
67
68     if np.linalg.norm(omega) < 1e-10: # Pure translation
69         R = np.eye(3)
70         p = v * theta
71     else:
72         omega_hat = omega / np.linalg.norm(omega)
73         w_magnitude = np.linalg.norm(omega)
74         theta_total = w_magnitude * theta
75
76         # Rotation part
77         R = rodrigues(omega_hat, theta_total)
78
79         # Translation part via V matrix

```

```

80     V = (np.eye(3) +
81          (1 - np.cos(theta_total)) * skew(omega_hat) / w_magnitude +
82          (theta_total - np.sin(theta_total)) * skew(omega_hat)**2 / (
w_magnitude**2))
83
84     p = V @ v * theta
85
86     T = np.eye(4)
87     T[:3, :3] = R
88     T[:3, 3] = p
89     return T
90
91 def log_SE3(T):
92     """
93     Compute the logarithm of a transformation matrix.
94     Returns the twist vector [omega; v].
95     """
96     R = T[:3, :3]
97     p = T[:3, 3]
98
99     omega = log_SO3(R)
100    w_magnitude = np.linalg.norm(omega)
101
102    if w_magnitude < 1e-10: # Pure translation
103        v = p
104    else:
105        omega_hat = omega / w_magnitude
106        theta = w_magnitude
107
108        # Compute V^{-1}
109        V_inv = (np.eye(3) -
110                0.5 * skew(omega_hat) +
111                (1.0/theta**2) * (0.5 - np.sin(2*theta)/(4*np.sin(theta))) *
skew(omega_hat)**2)
112
113        v = V_inv @ p
114
115    twist = np.hstack([omega, v])
116    return twist
117
118 def slerp(R0, R1, t):
119     """
120     Spherical linear interpolation between two rotations.
121     Uses exponential coordinates for geodesic interpolation.
122     """
123     # Relative rotation from R0 to R1
124     R_rel = R0.T @ R1
125     omega = log_SO3(R_rel)
126
127     # Interpolated relative rotation
128     R_interp_rel = rodrigues(omega / np.linalg.norm(omega), t * np.linalg.
norm(omega)) if np.linalg.norm(omega) > 1e-10 else np.eye(3)
129
130     # Final rotation
131     R_interp = R0 @ R_interp_rel
132     return R_interp
133
134 # Example usage
135 if __name__ == "__main__":
136     # 1. Create a rotation: 60 degrees about the z-axis

```



```

137 theta = np.pi / 3 # 60 degrees
138 omega = np.array([0, 0, 1]) # z-axis
139 R = rodrigues(omega, theta)
140 print("Rotation matrix (60 deg about z):")
141 print(R)
142 print()
143
144 # 2. Compute logarithm
145 omega_recovered = log_S03(R)
146 print(f"Recovered axis-angle: {omega_recovered}")
147 print(f"Expected: {omega * theta}")
148 print()
149
150 # 3. Verify round-trip
151 R_recovered = rodrigues(omega_recovered / np.linalg.norm(omega_recovered)
152 ,
153                        np.linalg.norm(omega_recovered))
154 print("Round-trip error (should be ~0):")
155 print(np.linalg.norm(R - R_recovered))
156 print()
157
158 # 4. Exponential map on SE(3)
159 twist = np.array([0, 0, 1, 1, 0, 0]) # rotate about z, translate in x
160 T = exp_SE3(twist, 1.0)
161 print("SE(3) exponential:")
162 print(T)
163 print()
164
165 # 5. Log of SE(3)
166 twist_recovered = log_SE3(T)
167 print(f"Recovered twist: {twist_recovered}")
168 print(f"Expected twist: {twist}")
169 print()
170
171 # 6. SLERP interpolation
172 R0 = np.eye(3)
173 theta1 = np.pi / 2 # 90 degrees
174 R1 = rodrigues(np.array([0, 0, 1]), theta1)
175
176 print("SLERP interpolation (0, 0.5, 1.0):")
177 for t in [0, 0.5, 1.0]:
178     R_t = slerp(R0, R1, t)
179     omega_t = log_S03(R_t)
180     angle_t = np.linalg.norm(omega_t)
181     print(f"t={t}: rotation angle = {np.degrees(angle_t):.1f} deg")
182 print()
183
184 # 7. Geodesic distance
185 R_test = rodrigues(np.array([1, 1, 1]) / np.sqrt(3), np.pi / 4)
186 omega_log = log_S03(R_test)
187 geodesic_distance = np.linalg.norm(omega_log)
188 print(f"Geodesic distance from identity to test rotation: {
189 geodesic_distance:.4f} rad = {np.degrees(geodesic_distance):.2f} deg")

```

Listing 6.1: Exponential and Logarithm Computations

6.13 Summary

In this chapter, we developed the exponential and logarithmic maps that govern motion on curved manifolds like $\text{SO}(3)(3)$ and $\text{SE}(3)(3)$.

1. **Lie Algebras:** The tangent spaces $\mathfrak{so}(3)(3)$ and $\mathfrak{se}(3)(3)$ are flat vector spaces where standard calculus operations work perfectly. They are identified with skew-symmetric matrices (and 6D vectors in the $\text{SE}(3)$ case).
2. **Rodrigues' Formula:** The exponential map from $\mathfrak{so}(3)(3)$ to $\text{SO}(3)(3)$ is given in closed form by Rodrigues' formula:

$$e^{[\hat{\omega}]\theta} = \mathbf{I} + \sin \theta [\hat{\omega}] + (1 - \cos \theta) [\hat{\omega}]^2 \quad (6.76)$$

3. **Matrix Logarithm:** The inverse operation extracts axis-angle coordinates from rotation matrices. It handles multiple cases (identity, small angles, general angles, 180-degree rotations) carefully to avoid numerical singularities.
4. **SE(3) Exponential:** Extension to full spatial transformations introduces the $\mathbf{V}$ matrix, which mediates between translation and rotation.
5. **Baker-Campbell-Hausdorff:** The composition of two exponentials is not simply the exponential of the sum; the Lie bracket (commutator) introduces correction terms. This reflects the non-commutativity of rotations.
6. **Geodesics:** The exponential map traces out geodesics (shortest paths) on the manifold. SLERP interpolation of quaternions is mathematically equivalent to geodesic interpolation on $\text{SO}(3)(3)$.
7. **Numerical Computation:** Modern libraries implement robust algorithms (scaling-and-squaring, Padé approximation) for computing exponentials and logarithms numerically.

These tools enable us to: - Smoothly interpolate between poses without averaging invalid matrices. - Extract meaningful "error vectors" from the difference between two poses (via the logarithm). - Solve differential equations governing rigid body motion. - Parameterize smooth trajectories and control laws.

With exponential coordinates and the matrix logarithm in hand, we possess the complete mathematical toolkit for describing, analyzing, and controlling rigid body motion in 3D space. We are now ready to move into Volume I, where we develop the geometric properties of $\text{SO}(3)(3)$ and $\text{SE}(3)(3)$ in depth.

6.14 Further Reading

For a rigorous mathematical treatment of Lie groups, Lie algebras, and the exponential map, Hall [14] is an excellent graduate-level reference that develops the theory from first principles with careful attention to the analytic subtleties. Readers seeking a deeper understanding of the algebraic structures underlying this chapter will find Hall's treatment of matrix Lie groups particularly relevant.

The robotics-specific development of exponential coordinates is best found in Murray, Li, and Sastry [23], whose Chapter 2 constructs the exponential map on $\text{SO}(3)(3)$ and $\text{SE}(3)(3)$ with an emphasis on applications to manipulator kinematics. Their treatment of the product of exponentials formula, which we develop in Chapter 9, builds directly on the material presented here.

Marsden and Ratiu [22] offer a geometric mechanics perspective on the exponential map, connecting it to Hamiltonian systems, symplectic structures, and momentum maps. Their framework reveals that Rodrigues' formula and the $\text{SE}(3)(3)$ exponential are specific instances of a much broader theory governing the geometry of mechanical systems on Lie groups.

Lynch and Park [20] provide an accessible and application-oriented treatment of SLERP interpolation, geodesics on $\text{SO}(3)(3)$, and the practical use of exponential coordinates in trajectory planning and motion control. Their worked examples are particularly valuable for building geometric intuition.

It is worth noting that the Baker-Campbell-Hausdorff formula developed in Section 6.8 has deep connections beyond robotics. In quantum mechanics, the BCH formula governs the composition of unitary operators and appears centrally in the theory of quantum gates. In differential geometry, it encodes the local structure of Lie groups and is intimately related to the curvature of the group manifold.

6.15 Exercises

1. **Rodrigues' Formula Verification:** Use Rodrigues' formula to compute the rotation matrix for a 45° rotation about the axis $\hat{\omega} = [1, 1, 1]^T/\sqrt{3}$. Verify that the result is orthogonal ($\mathbf{R}^T \mathbf{R} = \mathbf{I}$) and has determinant 1.
2. **Logarithm of a 90-Degree Rotation:** Given the rotation matrix:

$$\mathbf{R} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (6.77)$$

compute $\log(\mathbf{R})$ and extract the axis and angle.

3. **Trace-Based Angle Extraction:** Show that for any rotation matrix $\mathbf{R}$, the trace $\text{tr}(\mathbf{R}) = 1 + 2\cos\theta$ where θ is the rotation angle. Explain why this formula breaks down when $\theta = \pi$ and how to handle this case.
4. **Skew Matrix Pattern:** Prove that for any unit vector $\hat{\omega}$, the skew-symmetric matrix $[\hat{\omega}]$ satisfies $[\hat{\omega}]^3 = -[\hat{\omega}]$.
5. **Small Angle Approximation:** For small rotations with $\theta \ll 1$, show that $e^{[\hat{\omega}]\theta} \approx \mathbf{I} + [\hat{\omega}]\theta$ and derive the error term (to second order in θ).

6. **SE(3) Exponential:** Given a twist $\mathcal{V} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$ (rotation about z , translation in x),

compute the exponential map to produce the corresponding 4×4 transformation $\mathbf{T} = \exp([\mathcal{V}])$.

7. **V Matrix for Pure Translation:** For a prismatic (pure translation) joint with twist

$$\mathcal{V} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \text{ show that } \mathbf{V} = \mathbf{I} \text{ and thus } \exp([\mathcal{V}]) = \begin{bmatrix} \mathbf{I} & [1, 0, 0]^T \\ 0 & 1 \end{bmatrix}.$$

8. **Logarithm Inversion:** If $\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix}$, show that $\log(\mathbf{T}^{-1}) = -\log(\mathbf{T})$ (as a property of the logarithm).

9. **Baker-Campbell-Hausdorff (First Order):** For two small rotations with twists $\mathcal{V}_1 = \epsilon \mathbf{a}$ and $\mathcal{V}_2 = \epsilon \mathbf{b}$ (where $\epsilon \ll 1$), show that:

$$\log(e^{\mathcal{V}_1} e^{\mathcal{V}_2}) \approx \mathcal{V}_1 + \mathcal{V}_2 + \frac{1}{2}[\mathcal{V}_1, \mathcal{V}_2] \quad (6.78)$$

where $[\cdot, \cdot]$ is the Lie bracket.

10. **SLERP Geodesic Property:** Show algebraically that SLERP between two rotations $\mathbf{R}_0$ and $\mathbf{R}_1$ rotates at constant angular velocity. Specifically, verify that the angular velocity $\boldsymbol{\omega}(t) = \frac{d}{dt} \log(\mathbf{R}(t) \mathbf{R}_0^T)$ is constant (independent of t).

11. **Geodesic Distance:** Compute the geodesic distance (rotation angle) between the identity and a rotation by 120° about $[1, 1, 1]^T / \sqrt{3}$.

12. **Numerical Stability:** For $\theta = 0.001$ rad and $\hat{\boldsymbol{\omega}} = [0, 0, 1]^T$, compare the Rodrigues formula result with the first-order Taylor approximation $\mathbf{R} \approx \mathbf{I} + [\hat{\boldsymbol{\omega}}]\theta$. How large is the error?

13. **Matrix Exponential via scipy:** Using scipy's `linalg.expm`, compute $\exp([\mathcal{V}]t)$ for

$$\mathcal{V} = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \text{ and } t = \pi/4. \text{ Verify the result matches Rodrigues.}$$

14. **Interpolation Verification:** Generate two random rotation matrices $\mathbf{R}_0$ and $\mathbf{R}_1$. Interpolate between them using SLERP at $t = 0.3$ and $t = 0.7$. Verify that the rotations are on a geodesic by checking that $\theta(0.3) \approx 0.3 \cdot \theta_{\max}$ and $\theta(0.7) \approx 0.7 \cdot \theta_{\max}$.

15. **Exponential Trajectory:** A robot arm follows a screw axis with twist $\mathcal{V} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$ (rotation about x , translation in y). Compute its pose at times $t = 0, \pi/4, \pi/2$ by evaluating $\mathbf{T}(t) = \exp([\mathcal{V}]t)$.

Chapter 7

Recursive Algorithms

You do not solve the equations of motion for a 15-DOF robot in a single monolithic formula. You walk from the base to the tip, and then from the tip back to the base.

In Plain Language 7.1. Context & Use Case: Beating the Math Explosion

The Math You Will See: We will introduce the **Adjoint Matrix** ($[\text{Ad}_T]$), the **Spatial Inertia Matrix** ($\mathcal{I}$), the formal representation of kinematic trees, and the two-pass **Recursive Newton-Euler Algorithm (RNEA)**.

Why We Need It: If you try to write down the exact algebra connecting the shoulder motor of a robot to the tip of its finger, the equation takes up a whole page. If you try to write the equation for a 15-joint humanoid robot, the trigonometric math expands to fill *tens of thousands* of pages of dense algebra. No computer can solve that in real-time. Instead, we use "Recursive Algorithms" that only ever look at two bones at a time. By chaining these tiny, fast calculations together sequentially, we can simulate complex physics at 1,000 frames per second.

All Variables Defined: $\mathcal{V}_i$ is the velocity of link i , $\mathcal{F}_i$ is the force on link i , $\mathbf{T}_{i,i-1}$ is the relative pose between them, and $\dot{\mathcal{V}}_i$ is the spatial acceleration.

7.1 Historical Context: The Computational Revolution

The computational efficiency crisis in robotics was born in the 1970s. As robotic arms grew from simple planar mechanisms to fully 6-DOF industrial manipulators, researchers faced a brutal truth: the Lagrangian equations of motion, when expanded symbolically, produced expressions so massive they literally filled computers' memory banks.

In the early 1980s, **Orin**, **McGhee**, and **Luh** [19] independently developed recursive formulations of robot dynamics. Instead of solving the global inverse dynamics problem as a monolithic matrix equation, they recognized that the sequential tree structure of a kinematic chain could be exploited: each joint only directly interacts with its immediate neighbors. Their key insight was that by organizing calculations to respect this local structure, one could reduce the computational complexity from $O(N^3)$ (matrix inversion) or even $O(N^4)$ (symbolic algebra) down to $O(N)$ (linear in the number of joints).

In 1987, **Roy Featherstone** completed this revolution [10]. His reformulation using

spatial vector algebra (Chapter 8) and adjoint transformations made recursive dynamics not merely efficient, but *concise*. The Recursive Newton-Euler Algorithm (RNEA) became the standard approach used in every modern physics engine: MuJoCo, PyBullet, Drake, Gazebo, and UpstreamDrift all rely fundamentally on Featherstone’s recursive framework, executed as purely numerical matrix operations rather than symbolic manipulation.

7.2 The Curse of Dimensionality in Dynamics

As systems scale in complexity—from a simple pendulum (1 DOF) to a fully modeled anatomical golf swing (15+ DOF) or a humanoid Atlas robot (30+ DOF)—the monolithic, closed-form equations of physics become mathematically intractable.

In standard Lagrangian mechanics (which we review in a later chapter), the equations of motion are written as a massive matrix equation:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau} \quad (7.1)$$

Here, $\mathbf{M}$ is the $N \times N$ mass matrix, $\mathbf{C}$ contains Coriolis and centrifugal forces, $\mathbf{G}$ is gravity, and $\boldsymbol{\tau}$ are the motor torques.

If you attempt to write the analytical Coriolis matrix explicitly for a 15-DOF human model on a piece of paper, the resulting trigonometric expansion ($\sin(\theta_1 + \theta_2) \cos(\theta_3) \dots$) will literally fill thousands of pages. Each entry in $\mathbf{C}$ is a sum of products of sines, cosines, and Denavit-Hartenberg parameters; for even a moderately complex chain, these expressions become unmaintainable.

To control complex mechanisms in the real world at thousands of Hertz, modern engines (such as MuJoCo, Drake, Pinocchio, and UpstreamDrift) completely abandon building these massive equations explicitly. Instead, they rely entirely on **Recursive Algorithms** executing purely numerical matrix multiplications in $O(N)$ time.

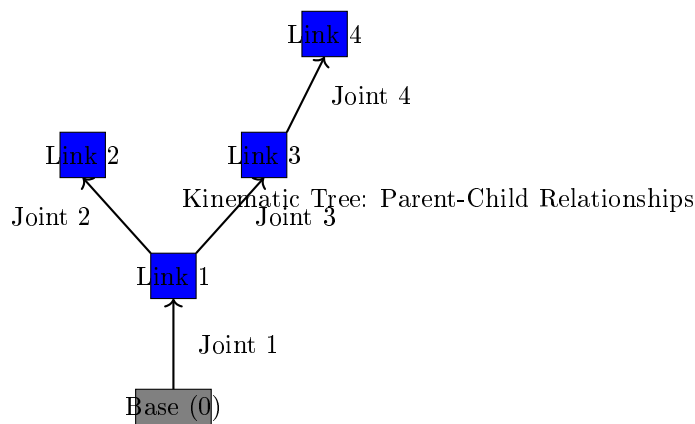


Figure 7.1: A kinematic tree representation of a robot with a base link and three movable links forming a branching structure. Each link connects to its parent through a joint. Recursive algorithms exploit this tree structure to compute dynamics in linear time $O(N)$ rather than cubic time.

7.3 Kinematic Tree Representation

Before we can implement recursive algorithms, we must formalize how to represent a kinematic chain as a tree structure. Unlike the Lagrangian approach, which treats the system globally, the recursive approach exploits the *local* structure: each link i has at most one parent (the link it is directly connected to by a joint) and zero or more children.

Definition 7.1. Kinematic Tree

A kinematic tree is a directed, acyclic graph in which:

- Each link (node) i has at most one parent link, denoted $\text{parent}(i)$ or $\lambda(i)$.
- Link 0 is the **base link** (the root of the tree), and has no parent.
- A joint i connects link i to its parent link $\lambda(i)$ via a transformation $\mathbf{T}_{i,\lambda(i)}(q_i)$.
- The motion of link i is determined entirely by the motion of its parent, plus the motion of joint i itself.

7.3.1 Parent Arrays and Numbering Conventions

To implement this structure efficiently in code, we maintain a simple **parent array** $\lambda : \{1, 2, \dots, N\} \rightarrow \{0, 1, \dots, N-1\}$, where $\lambda(i)$ is the index of link i 's parent. For the base link, $\lambda(0) = -1$ or is undefined.

Example 7.1. Parent Array for a 3-Link Arm

Consider a planar robotic arm: Base (link 0) $\rightarrow$ Shoulder (link 1) $\rightarrow$ Elbow (link 2) $\rightarrow$ Wrist (link 3).

The parent array is:

$$\lambda = [-1, 0, 1, 2]$$

Read as: Link 0 has no parent, link 1's parent is link 0, link 2's parent is link 1, link 3's parent is link 2.

For a humanoid with two legs, the parent array might be:

$$\lambda = [-1, 0, 1, 0, 3, 0, \dots]$$

with link 0 as the pelvis, links 1-2 the left leg, links 3-4 the right leg, link 5 the torso, etc.

Once the kinematic tree structure is defined via λ , the recursive algorithms simply iterate: for each link $i = 1, 2, \dots, N$, they access data from the parent link $\lambda(i)$ and compute updated values for link i .

7.4 The Adjoint Map ($[\text{Ad}_T]$)

Before we can compute recursive mechanics, we need to know how to shift a 6D spatial Twist ($\mathcal{V}$) or 6D Wrench ($\mathcal{F}$) from one reference frame to another. This is done using the

Adjoint Representation of $SE(3)$.

Mathematical Principle 7.1. Change of Frame for Spatial Vectors

If a physical motion (e.g., a velocity) is the same regardless of which frame we measure it from, but its *numerical representation* changes because the frame itself has rotated and translated, we need a matrix to shift the numbers without changing the physics.

7.4.1 Derivation of the Adjoint Matrix

Suppose Frame B is related to Frame A by a homogeneous transformation $\mathbf{T}_{BA} \in SE(3)$, decomposed as:

$$\mathbf{T}_{BA} = \begin{bmatrix} \mathbf{R}_{BA} & \mathbf{p}_{BA} \\ \mathbf{0} & 1 \end{bmatrix}$$

where $\mathbf{R}_{BA} \in SO(3)$ and $\mathbf{p}_{BA} \in \mathbb{R}^3$.

A spatial velocity $\mathcal{V}_A = (\boldsymbol{\omega}_A, \mathbf{v}_A) \in \mathfrak{se}(3)(3)$ describes the motion of a point in Frame A . When measured from Frame B , the same physical motion is represented numerically as $\mathcal{V}_B = (\boldsymbol{\omega}_B, \mathbf{v}_B)$.

The angular velocity transforms purely by rotation:

$$\boldsymbol{\omega}_B = \mathbf{R}_{BA} \boldsymbol{\omega}_A$$

The linear velocity, however, must account for the fact that the origin of Frame A is moving relative to Frame B . If Frame A 's origin has velocity $\mathbf{v}_{o,A}$ and the point is at offset $\mathbf{r}$ from Frame A 's origin, then in Frame B 's coordinates:

$$\mathbf{v}_B = \mathbf{R}_{BA}(\mathbf{v}_A + \boldsymbol{\omega}_A \times \mathbf{r})$$

For the origin of Frame A itself (where $\mathbf{r} = \mathbf{0}$), this becomes:

$$\mathbf{v}_B = \mathbf{R}_{BA} \mathbf{v}_A - \mathbf{R}_{BA}(\boldsymbol{\omega}_A \times \mathbf{p}_{BA})$$

Rewriting the cross product as a matrix multiplication and using the property $\mathbf{a} \times \mathbf{b} = -[\mathbf{b}]\mathbf{a}$:

$$\mathbf{v}_B = \mathbf{R}_{BA} \mathbf{v}_A + \mathbf{R}_{BA}[\mathbf{p}_{BA}]\boldsymbol{\omega}_A = \mathbf{R}_{BA} \mathbf{v}_A + [\mathbf{R}_{BA} \mathbf{p}_{BA}](\mathbf{R}_{BA} \boldsymbol{\omega}_A)$$

But we can group this as a matrix acting on the 6D vector:

$$\begin{bmatrix} \boldsymbol{\omega}_B \\ \mathbf{v}_B \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{BA} & \mathbf{0} \\ [\mathbf{p}_{BA}]\mathbf{R}_{BA} & \mathbf{R}_{BA} \end{bmatrix} \begin{bmatrix} \boldsymbol{\omega}_A \\ \mathbf{v}_A \end{bmatrix}$$

This is the **Adjoint Matrix**:

Definition 7.2. Adjoint Representation

The Adjoint Matrix $[\text{Ad}_{\mathbf{T}}] \in \mathbb{R}^{6 \times 6}$ for a transformation $\mathbf{T} = (\mathbf{R}, \mathbf{p})$ is:

$$[\text{Ad}_{\mathbf{T}}] = \begin{bmatrix} \mathbf{R} & \mathbf{0}_{3 \times 3} \\ [\mathbf{p}]\mathbf{R} & \mathbf{R} \end{bmatrix} \quad (7.2)$$

where $[\mathbf{p}]$ is the 3×3 skew-symmetric cross product matrix of $\mathbf{p}$.

If Frame A has spatial velocity $\mathcal{V}_A$, the numerically equivalent velocity as measured by Frame B is:

$$\mathcal{V}_B = [\text{Ad}_{\mathbf{T}_{BA}}] \mathcal{V}_A \quad (7.3)$$

7.4.2 The Adjoint for Forces (Wrenches)

For forces, the situation is dual. A wrench $\mathcal{F} = (\mathbf{m}, \mathbf{f})$ (moment and force) transforms using the **coadjoint** representation, which is the transpose-inverse:

$$\mathcal{F}_B = [\text{Ad}_{\mathbf{T}_{BA}}]^{-T} \mathcal{F}_A = \begin{bmatrix} \mathbf{R}^T & [\mathbf{p}]^T \mathbf{R}^T \\ \mathbf{0} & \mathbf{R}^T \end{bmatrix} \mathcal{F}_A \quad (7.4)$$

This ensures that the power (work rate) $\mathcal{F}^T \mathcal{V}$ is invariant under frame changes, as it must be for physical consistency.

7.5 Recursive Forward Kinematics

In Plain Language 7.2. The Breadcrumb Strategy

To find the absolute speed of the end-effector relative to the ground, we don't jump straight from the ground to the end-effector.

We ask: How fast is the pelvis moving relative to the feet? Then, we add the speed of the torso relative to the pelvis. Then we add the speed of the shoulder relative to the torso. Chain by chain, link by link, we compose the velocities.

7.5.1 Recursive Velocity Propagation

Suppose we want to find the exact spatial position and velocity of the golf clubhead at the very end of a 15-link robotic golfer. Instead of deriving a massive equation relating the base of the feet directly to the clubhead, we use our 4×4 Transforms and 6×6 Adjoints, and we propagate them sequentially outwards.

The fundamental recursion for spatial velocity is:

Theorem 7.1. Recursive Velocity Propagation

For a kinematic tree, the spatial velocity of link i is given by:

$$\mathcal{V}_i = [\text{Ad}_{\mathbf{T}_{i,\lambda(i)}}] \mathcal{V}_{\lambda(i)} + \mathcal{S}_i \dot{q}_i \quad (7.5)$$

where:

- $\mathcal{V}_{\lambda(i)}$ is the velocity of the parent link (already computed).
- $[\text{Ad}_{\mathbf{T}_{i,\lambda(i)}}]$ transforms the parent's velocity into the coordinate frame of link i .
- $\mathcal{S}_i \in \mathbb{R}^6$ is the motion subspace (axis of joint i) expressed in link i 's frame.
- $\dot{q}_i$ is the scalar joint velocity.

Derivation: The velocity of link i can be decomposed as the velocity it inherits from its parent (carried through the transformation $\mathbf{T}_{i,\lambda(i)}$), plus the additional velocity generated by joint i spinning.

For a revolute joint about axis $\mathbf{z}$ in the joint frame, the screw axis is:

$$\mathcal{S}_i = \begin{bmatrix} \mathbf{z} \\ \mathbf{0} \end{bmatrix} \in \mathbb{R}^6$$

For a prismatic joint along axis $\mathbf{z}$:

$$\mathcal{S}_i = \begin{bmatrix} \mathbf{0} \\ \mathbf{z} \end{bmatrix} \in \mathbb{R}^6$$

7.5.2 Algorithm: Forward Kinematics in $O(N)$ Time

Algorithm 7.1. Forward Kinematics Recursion

```

1 function ForwardKinematics(q, dq, T_rel, S, lambda):
2   // Input: joint positions q, velocities dq, relative transforms
   T_rel,
3   //          screw axes S, parent array lambda
4   // Output: link transforms T, spatial velocities V
5
6   T[0] <- Identity           // Base link is at the origin
7   V[0] <- Zero 6D vector     // Base link is stationary (or has
   gravity/base motion)
8
9   for i <- 1 to N:
10    // Compose transforms sequentially
11    T[i] <- T[lambda[i]] * T_rel[i]
12
13    // Propagate velocity using Adjoint
14    V_parent_in_i <- Ad[T_rel[i]] * V[lambda[i]]
15    V[i] <- V_parent_in_i + S[i] * dq[i]
16
17   return (T, V)

```

7.6 Spatial Acceleration and Coriolis Terms

Once velocities are propagated outward, we need to compute accelerations. However, computing $\ddot{\mathbf{V}}_i$ (spatial acceleration) is not as simple as taking a time derivative of $\dot{\mathbf{V}}_i$. The presence of rotating reference frames introduces non-inertial terms.

7.6.1 The Lie Bracket and Centrifugal Forces

In a non-inertial frame rotating with velocity $\mathcal{V}$, any object with velocity $\mathcal{W}$ experiences an apparent "fictitious" acceleration due to the rotation. This is captured by the **spatial cross product**, which we will define rigorously in Chapter 8.

The spatial acceleration must satisfy:

$$\dot{\mathcal{V}}_i = [\text{Ad}_{\mathbf{T}_{i,\lambda(i)}}] \dot{\mathcal{V}}_{\lambda(i)} + [\mathcal{V}_i \times] \mathcal{S}_i \dot{q}_i + \mathcal{S}_i \ddot{q}_i \quad (7.6)$$

The term $[\mathcal{V}_i \times] \mathcal{S}_i \dot{q}_i$ captures the Coriolis acceleration: the acceleration felt by the joint motion when embedded in a frame that is itself rotating.

More explicitly, if $\mathcal{V}_i = (\boldsymbol{\omega}_i, \mathbf{v}_i)$ is the spatial velocity of link i , the motion cross product operator is:

$$[\mathcal{V}_i \times] = \begin{bmatrix} [\boldsymbol{\omega}_i] & \mathbf{0}_{3 \times 3} \\ [\mathbf{v}_i] & [\boldsymbol{\omega}_i] \end{bmatrix} \quad (7.7)$$

When this acts on the screw axis $\mathcal{S}_i$, it produces the non-inertial acceleration term.

Algorithm 7.2. Forward Acceleration (Outward Pass)

```

1 function ForwardAcceleration(V, ddq, S, lambda, Gravity):
2   // Input: spatial velocities V (from forward kinematics),
3   //        joint accelerations ddq, screw axes S, gravity vector
4   // Output: spatial accelerations dV
5
6   dV[0] <- Gravity           // Base link has gravitational
   acceleration
7
8   for i <-1 to N:
9     // Velocity-dependent term (Coriolis)
10    Coriolis <- [V[i]x] * S[i] * dq[i]
11
12    // Compose from parent
13    dV_parent_in_i <- Ad[T_rel[i]] * dV[lambda[i]]
14
15    // Total acceleration
16    dV[i] <- dV_parent_in_i + Coriolis + S[i] * ddq[i]
17
18   return dV

```

7.7 Spatial Inertia

Before we compute forces, we must define mass in 6D. In 3D space, an object has:

- Scalar mass m , resisting linear acceleration ($F = ma$).
- 3×3 rotational inertia tensor $\mathbf{I}_c$ (about the center of mass), resisting angular acceleration ($\boldsymbol{\tau} = \mathbf{I}_c \boldsymbol{\alpha}$).

Spatial Inertia unifies these into a single 6×6 matrix:

Definition 7.3. Spatial Inertia Matrix

For a rigid body with mass m , center of mass offset $\mathbf{c}$, and rotational inertia tensor

$\mathbf{I}_c$ about the CoM, the Spatial Inertia Matrix is:

$$\mathcal{I} = \begin{bmatrix} \mathbf{I}_c & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & m\mathbf{I}_{3 \times 3} \end{bmatrix} \quad (7.8)$$

when expressed at the center of mass. When expressed at an arbitrary origin (offset by $\mathbf{c}$ from the CoM), the parallel axis theorem in 6D gives:

$$\mathcal{I} = \begin{bmatrix} \mathbf{I}_c - m[\mathbf{c}][\mathbf{c}] & m[\mathbf{c}] \\ -m[\mathbf{c}] & m\mathbf{I}_{3 \times 3} \end{bmatrix} \quad (7.9)$$

This matrix is symmetric and positive definite, encoding the complete resistance of a rigid body to any spatial acceleration.

7.8 The Recursive Newton-Euler Algorithm (RNEA)

Inverse Dynamics asks: If we know the trajectory the robot is currently taking $(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$, how much torque $\boldsymbol{\tau}$ must the motors output at this exact millisecond to execute it?

This is solved by the crowning jewel of computational mechanics: the **Recursive Newton-Euler Algorithm (RNEA)** [19, 10]. It executes in exactly two sweeps.

7.8.1 The Outward Pass: Velocity and Acceleration Propagation

Like Forward Kinematics, we cast a sweeping algorithm from the heavy base (the ground) outwards to the lightest tip (the end-effector). For every link i , we compute its absolute velocity $\mathcal{V}_i$ and its absolute spatial acceleration $\dot{\mathcal{V}}_i$.

This outward pass perfectly calculates the exact inertial, Coriolis, and centrifugal accelerations dragging on every single link of the mechanism as it flails through space.

Theorem 7.2. Outward Pass Computation

The outward pass computes, for each link $i = 1, 2, \dots, N$:

$$\mathcal{V}_i = [\text{Ad}_{\mathbf{T}_{i,\lambda(i)}}] \mathcal{V}_{\lambda(i)} + \mathcal{S}_i \dot{q}_i \quad (7.10)$$

$$\dot{\mathcal{V}}_i = [\text{Ad}_{\mathbf{T}_{i,\lambda(i)}}] \dot{\mathcal{V}}_{\lambda(i)} + [\mathcal{V}_i \times] \mathcal{S}_i \dot{q}_i + \mathcal{S}_i \ddot{q}_i \quad (7.11)$$

At the tip (link N), external contact forces $\mathcal{F}_{\text{ext}}$ may be applied.

7.8.2 The Inward Pass: Force and Torque Propagation

Once the outward pass finishes at the tip, we reverse direction. We perform a second loop starting from the tip of the robot back down to the ground.

At the very tip (leaf node), we initialize the spatial wrench. If there is external ground contact or an applied force, we begin with:

$$\mathcal{F}_N = \mathcal{F}_{\text{ext}}$$

Otherwise, we begin with zero wrench at the tip and let it propagate inward.

For each internal link i (processed from N down to 1), we compute the spatial wrench (combined 6D force/torque) required to make link i accelerate at $\dot{\mathcal{V}}_i$ while supporting the forces from all its children. The key Newton-Euler equation in spatial form is:

Theorem 7.3. Spatial Newton-Euler Equation

The spatial wrench (force and torque) required to accelerate a rigid body with spatial inertia $\mathcal{I}$ at acceleration $\dot{\mathcal{V}}$ is:

$$\mathcal{F} = \mathcal{I}\dot{\mathcal{V}} + [\mathcal{V} \times]^*(\mathcal{I}\mathcal{V}) \quad (7.12)$$

where $[\mathcal{V} \times]^*$ is the force cross product (transpose-negative of the motion cross product).

Interpretation: The first term is the "inertial force" ($m\mathbf{a}$); the second term is the centrifugal and Coriolis bias—the apparent force due to the fact that the body is rotating or moving.

7.8.3 Force Recursion and Extraction of Joint Torques

For a link i with one child $j = \text{children}(i)$, the wrench recursion is:

$$\mathcal{F}_i = \mathcal{I}_i \dot{\mathcal{V}}_i + [\mathcal{V}_i \times]^*(\mathcal{I}_i \mathcal{V}_i) + [\text{Ad}_{\mathbf{T}_{j,i}}]^{-T} \mathcal{F}_j \quad (7.13)$$

For a branching tree, we sum over all children:

$$\mathcal{F}_i = \mathcal{I}_i \dot{\mathcal{V}}_i + [\mathcal{V}_i \times]^*(\mathcal{I}_i \mathcal{V}_i) + \sum_{j \in \text{children}(i)} [\text{Ad}_{\mathbf{T}_{j,i}}]^{-T} \mathcal{F}_j \quad (7.14)$$

Once we have the total wrench $\mathcal{F}_i$ acting on link i , the scalar torque (or force) required at joint i is extracted by projecting onto the joint's screw axis:

$$\tau_i = \mathcal{F}_i^T \mathcal{S}_i \quad (7.15)$$

Algorithm 7.3. RNEA: Recursive Newton-Euler Algorithm

```

1 function RNEA(q, dq, ddq, T_rel, S, I, lambda, gravity, F_ext):
2   // Outward pass: compute velocities and accelerations
3   V[0] <- gravity // Base link has gravitational
   acceleration
4   dV[0] <- gravity
5
6   for i <- 1 to N:
7     // Compute relative transform
8     T[i] <- T[lamba[i]] * ForwardKin(q[i], S[i])
9
10    // Velocity recursion
11    V_parent_in_i <- Ad[T_rel[i]] * V[lamba[i]]
12    V[i] <- V_parent_in_i + S[i] * dq[i]
13
14    // Acceleration recursion (Coriolis term)
15    dV_parent_in_i <- Ad[T_rel[i]] * dV[lamba[i]]
16    Coriolis <- -[V[i]x] * S[i] * dq[i]
```

```

17     dV[i] <-dV_parent_in_i + Coriolis + S[i] * ddq[i]
18
19     // Inward pass: compute forces and extract torques
20     F[N] <-F_ext[N]           // External forces at tip
21
22     for i <-N down to 1:
23         // Bias force (centrifugal + Coriolis)
24         bias <-[V[i]x]* * (I[i] * V[i])
25
26         // Inertial force
27         inertial <-I[i] * dV[i]
28
29         // Total wrench at link i
30         F[i] <-inertial + bias
31
32         // Add forces from children
33         for j in children[i]:
34             F[i] <-F[i] + Ad[T_rel[j]]^T * F[j]
35
36         // Extract joint torque
37         tau[i] <-F[i]^T * S[i]
38
39     return tau

```

7.9 Computational Complexity Analysis

Mathematical Principle 7.2. Algorithmic Efficiency

The Recursive Newton-Euler algorithm allows the inverse dynamics for a system with N joints to be solved in absolutely perfect $O(N)$ operations. The time to compute scales linearly, regardless of complexity. This algorithmic breakthrough is the fundamental mathematical reason robotics and physics engines work in the real world.

7.9.1 Why $O(N)$ and not $O(N^2)$?

In the naive Lagrangian approach, we would: 1. Compute the mass matrix $\mathbf{M}(q)$: Each entry M_{ij} requires traversing the kinematic chain from joint i to joint j . This is $O(N^2)$. 2. Compute the Coriolis matrix $\mathbf{C}(q, \dot{q})$: This is $O(N^2)$ as well. 3. Solve the linear system $\mathbf{M}(q)\ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{C}(q, \dot{q})\dot{\mathbf{q}} - \mathbf{G}(q)$: Gaussian elimination is $O(N^3)$.

In the RNEA: 1. **Outward pass**: For each link i , we perform a constant number of 6×6 matrix multiplications and additions. Total: $O(N)$. 2. **Inward pass**: For each link i , we compute the wrench and extract the torque, again with $O(1)$ operations per link. Total: $O(N)$.

The key is that we *never* assemble the global mass matrix. Instead, we exploit the tree structure to compute forces locally at each link, with no coupling except through the sequential parent-child relationships.

7.9.2 Proof of Linearity

Theorem 7.4. RNEA is $O(N)$

The RNEA algorithm, applied to a serial kinematic chain or tree, requires at most $c \cdot N$ floating-point operations, where c is a constant (approximately 100–150, depending on implementation) and N is the number of joints.

Proof sketch:

1. Each outward pass step performs: 1 Adjoint matrix-vector multiplication (≈ 18 flops for $6 \times 6 \times 6D$ vector), 1 vector addition (≈ 6 flops), 1 screw scaling (≈ 6 flops). Total per step: ~ 30 flops.
2. Each inward pass step performs: 1 spatial cross product (≈ 18 flops), 1 inertia product (≈ 36 flops for $6 \times 6 \times 6D$), 1 wrench addition and children summation (~ 30 flops for a typical branching). Total per step: ~ 84 flops.
3. Total: $(30 + 84) \times N = 114N$ flops, independent of N .

This is why modern physics engines can simulate 15–30 DOF robots at 1,000 Hz: $O(N)$ algorithms are fundamentally different from $O(N^3)$ algorithms at scale.

7.10 Branching Kinematic Trees

While we’ve presented the RNEA for serial chains, the algorithm extends seamlessly to branching trees (e.g., humanoids with two legs and two arms branching off a pelvis).

7.10.1 Handling Multiple Children

The modification is minimal: in the inward pass (force recursion), if link i has k children, we sum the wrenches from all children:

$$\mathcal{F}_i = \mathcal{I}_i \dot{\mathcal{V}}_i + [\mathcal{V}_i \times]^* (\mathcal{I}_i \mathcal{V}_i) + \sum_{j=1}^k [\text{Ad}_{\mathbf{T}_{j,i}}]^{-T} \mathcal{F}_j \quad (7.16)$$

The rest of the algorithm remains identical. Because the tree is acyclic, there is a unique topological ordering, and all parents are processed before their children in the inward pass.

Example 7.2. Humanoid Robot with Branching

A humanoid robot with a main body/torso (link 0), two legs (links 1–2 and 3–4), and two arms (links 5–7 and 8–10) has the parent array:

$$\lambda = [-1, 0, 1, 0, 3, 0, 5, 6, 0, 8, 9]$$

In the inward pass, when computing $\mathcal{F}_0$ (torso wrench), we sum:

$$\mathcal{F}_0 = (\text{torso inertia term}) + [\text{Ad}_{1,0}]^{-T} \mathcal{F}_1 + [\text{Ad}_{3,0}]^{-T} \mathcal{F}_3 + [\text{Ad}_{5,0}]^{-T} \mathcal{F}_5 + [\text{Ad}_{8,0}]^{-T} \mathcal{F}_8$$

Still $O(N)$ in the total number of links.

7.11 Contact Forces and External Disturbances

In practice, robots interact with the world. A foot experiences ground reaction forces; a manipulator might push against an object. These external forces $\mathcal{F}_{\text{ext}}$ are incorporated at the point of contact.

7.11.1 Initialization at Leaf Nodes

For a leaf node (a link with no children) experiencing an external wrench $\mathcal{F}_{\text{ext}}$:

$$\mathcal{F}_{\text{leaf}} = \mathcal{I}_{\text{leaf}} \dot{\mathcal{V}}_{\text{leaf}} + [\mathcal{V}_{\text{leaf}} \times]^* (\mathcal{I}_{\text{leaf}} \mathcal{V}_{\text{leaf}}) + \mathcal{F}_{\text{ext}}$$

For an intermediate link i with one child j :

$$\mathcal{F}_i = \mathcal{I}_i \dot{\mathcal{V}}_i + [\mathcal{V}_i \times]^* (\mathcal{I}_i \mathcal{V}_i) + [\text{Ad}_{\mathbf{T}_{j,i}}]^{-T} \mathcal{F}_j + \mathcal{F}_{\text{ext},i}$$

This allows the algorithm to compute the joint torques needed not just to move the robot, but to support external loads.

7.12 RNEA Equivalence to the Lagrangian Equations

The Lagrangian formulation of manipulator dynamics writes the joint torques as

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}),$$

where $\mathbf{M}$ is the mass matrix, $\mathbf{C}\dot{\mathbf{q}}$ collects Coriolis and centrifugal terms, and $\mathbf{G}$ is the gravity vector. Computing $\mathbf{M}$ explicitly costs $O(N^3)$; for a 30-DOF humanoid this is prohibitive at real-time rates.

Theorem 7.5. RNEA Computes the Lagrangian Inverse Dynamics

Let $\text{RNEA}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$ denote the output of the Recursive Newton–Euler Algorithm with gravity acceleration included in the base boundary condition. Then

$$\text{RNEA}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = \mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}).$$

That is, the RNEA computes $\boldsymbol{\tau} = \mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{G}$ without ever assembling $\mathbf{M}$, $\mathbf{C}$, or $\mathbf{G}$ explicitly.

Proof sketch. Because the RNEA is *linear* in $\ddot{\mathbf{q}}$ (the acceleration appears only in the outward pass and propagates linearly through the inward pass), we can decompose its output via three evaluations:

1. **Gravity vector.** Set $\dot{\mathbf{q}} = \mathbf{0}$ and $\ddot{\mathbf{q}} = \mathbf{0}$:

$$\mathbf{G}(\mathbf{q}) = \text{RNEA}(\mathbf{q}, \mathbf{0}, \mathbf{0}).$$

All velocity-dependent terms vanish; only the gravitational wrench propagates inward.

2. **Coriolis and centrifugal terms.** Set $\ddot{\mathbf{q}} = \mathbf{0}$:

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} = \text{RNEA}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{0}) - \mathbf{G}(\mathbf{q}).$$

3. **Inertial torques.** The remainder is the mass-matrix contribution:

$$\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} = \text{RNEA}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) - \mathbf{C}\dot{\mathbf{q}} - \mathbf{G}.$$

Column-by-column extraction of $\mathbf{M}$. Because the RNEA is linear in $\ddot{\mathbf{q}}$, column j of $\mathbf{M}$ can be obtained as

$$\mathbf{M}(\mathbf{q}) \mathbf{e}_j = \text{RNEA}(\mathbf{q}, \mathbf{0}, \mathbf{e}_j) - \mathbf{G}(\mathbf{q}),$$

where $\mathbf{e}_j$ is the j -th standard basis vector. This gives the full $N \times N$ mass matrix in $N + 1$ calls to the $O(N)$ RNEA, yielding an $O(N^2)$ algorithm for $\mathbf{M}$ —the best possible without exploiting sparsity [10].

In Plain Language 7.3. Why Not Just Build $\mathbf{M}$ Directly?

For a single inverse-dynamics query (“given $\mathbf{q}$, $\dot{\mathbf{q}}$, $\ddot{\mathbf{q}}$, what torques are needed?”), one RNEA call at $O(N)$ suffices. Forming the entire mass matrix $\mathbf{M}$ and then multiplying $\mathbf{M}\ddot{\mathbf{q}}$ would be slower ($O(N^2)$) and wasteful. The Lagrangian form is primarily useful for analysis—stability proofs, passivity arguments—while the RNEA is the workhorse for simulation and control [20].

7.13 Contact Forces and Constraint Handling

Real robots do not float in free space. Walking, grasping, and assembly all involve contact, and modelling contact correctly is one of the hardest problems in robot simulation.

7.13.1 Bilateral Constraints

A *bilateral* constraint removes degrees of freedom permanently. For example, if a foot is bolted to the ground, the spatial velocity of that link is zero. In the RNEA framework this amounts to modifying the base boundary condition:

$$\mathcal{V}_{\text{foot}} = \mathbf{0}, \quad \dot{\mathcal{V}}_{\text{foot}} = \mathbf{0}.$$

The algorithm then propagates *outward* from the pinned foot (which temporarily acts as the base) and the ground reaction wrench $\mathcal{F}_{\text{grf}}$ emerges from the inward pass as the “joint torque” at the virtual fixed joint.

7.13.2 Unilateral Constraints and Friction

Contact with the ground or with objects is *unilateral*: the surface can push but not pull. Denote the contact normal force by f_n and the tangential (friction) force by $\mathbf{f}_t \in \mathbb{R}^2$. The fundamental constraints are:

- **Non-penetration:** the signed gap $\phi \geq 0$.
- **Compressive normal force:** $f_n \geq 0$ (the ground pushes up, never pulls down).
- **Complementarity:** $\phi \cdot f_n = 0$ (force is zero when not in contact, gap is zero when in contact).
- **Coulomb friction cone:** $\|\mathbf{f}_t\| \leq \mu f_n$, where μ is the coefficient of friction.

Definition 7.4. Linear Complementarity Problem (LCP)

Given $\mathbf{A} \in \mathbb{R}^{k \times k}$ and $\mathbf{b} \in \mathbb{R}^k$, find $\mathbf{f} \in \mathbb{R}^k$ such that

$$\mathbf{f} \geq \mathbf{0}, \quad \mathbf{A}\mathbf{f} + \mathbf{b} \geq \mathbf{0}, \quad \mathbf{f}^T(\mathbf{A}\mathbf{f} + \mathbf{b}) = 0.$$

The matrix $\mathbf{A}$ encodes the coupled dynamics of all contacts; $\mathbf{b}$ captures the “free” accelerations from the RNEA in the absence of contact forces [10].

Modern physics engines—MuJoCo [35], Bullet, and Drake—solve a variant of the LCP (or a convex relaxation of it) at every simulation timestep. The RNEA supplies the $O(N)$ inverse dynamics needed to build the LCP data, making the overall cost dominated by the number of active contacts, not the number of joints.

In Plain Language 7.4. Why Contact is Hard

Contact is the hardest part of robot simulation. A walking robot must solve, at every millisecond: Am I touching the ground? How hard? In what direction? Is the foot sliding or stuck? All of these questions couple together—the force depends on the motion, but the motion depends on the force—and must be answered simultaneously within one simulation timestep.

7.14 Worked Example: 3-Link Planar Arm

Let us trace through a complete RNEA computation for a 3-link planar arm in 2D (which we embed in 3D with z -axis rotation).

Remark 7.1 (The torques below are computed). The result is produced by `src/affine_control/rnea.py` via `scripts/generate_worked_examples.py`, and cross-checked against an independent Lagrangian computation; CI fails if the committed values drift from what the code emits.

7.14.1 Setup

Links: Base (link 0), Upper arm (link 1), Forearm (link 2), Hand (link 3).

- Link 1: mass $m_1 = 1$ kg, length $L_1 = 1$ m, CoM at 0.5 m from joint, $I_z^{(1)} = 0.083$ kg·m².
- Link 2: mass $m_2 = 0.5$ kg, length $L_2 = 0.8$ m, CoM at 0.4 m from joint, $I_z^{(2)} = 0.027$ kg·m².
- Link 3: mass $m_3 = 0.2$ kg, point mass at tip (0.6 m from joint), $I_z^{(3)} = 0$.

Joint positions: $\mathbf{q} = [30^\circ, 45^\circ, 0^\circ]^T = [0.524, 0.785, 0]$ rad. Joint velocities: $\dot{\mathbf{q}} = [0.5, 0.3, 0.2]$ rad/s. Joint accelerations: $\ddot{\mathbf{q}} = [1.0, 0.5, 0.1]$ rad/s².

7.14.2 Outward Pass: Velocities and Accelerations

- Base link: $\mathbf{v}_0 = (0, 0, 0, 0, 0, 0)^T$, $\dot{\mathbf{v}}_0 = (0, 0, 0, 0, 0, -9.81)^T$ (gravity down the z axis).

- Link 1: the joint axis is $\mathcal{S}_1 = (0, 0, 1, 0, 0, 0)^T$, so

$$\mathcal{V}_1 = \mathcal{V}_0 + \mathcal{S}_1 \dot{q}_1 = (0, 0, 0.5, 0, 0, 0)^T \quad (7.17)$$

$$\mathcal{V}_1 \times \mathcal{S}_1 \dot{q}_1 = 0. \quad (7.18)$$

That product vanishes here, and it vanishes for *every* link of a planar chain: $\mathcal{V}_i$ and $\mathcal{S}_i \dot{q}_i$ both lie along the same out-of-plane axis, and the cross product of parallel vectors is zero. The bias acceleration in a planar arm comes entirely from the centripetal $\omega^2 r$ terms that appear when the recursion moves the origin from one link to the next.

- Links 2 and 3: the same recursion, with a nonzero translation between link frames, which is where those centripetal terms enter.

Common Misconception

An earlier revision gave $\mathcal{V}_1 \times \mathcal{S}_1 \dot{q}_1 = (0, 0, 0.5) \times (0, 0, 0.2) = (0.1, 0, 0)$, marked “(approx)”. The product is exactly zero, since the vectors are parallel; no approximation is involved. The spurious $(0.1, 0, 0)$ was then carried into $\dot{\mathcal{V}}_1$, and three-vectors were added to six-vectors in the same line.

7.14.3 Inward Pass: Forces and Torques

Assuming no external forces at the tip ($\mathcal{F}_{\text{ext}} = 0$):

- Link 3 (tip): $\mathcal{F}_3 = \mathcal{I}_3 \dot{\mathcal{V}}_3 + [\mathcal{V}_3 \times]^* (\mathcal{I}_3 \mathcal{V}_3)$.
- Link 2: $\mathcal{F}_2 = (\text{inertia and bias of link 2}) + [\text{Ad}_{3,2}]^{-T} \mathcal{F}_3$.
- Link 1: $\mathcal{F}_1 = (\text{inertia and bias of link 1}) + [\text{Ad}_{2,1}]^{-T} \mathcal{F}_2$.

The required joint torques are:

$$\tau_1 = \mathcal{F}_1^T \mathcal{S}_1 = \mathcal{F}_1^T (0, 0, 1, 0, 0, 0)^T \quad (7.19)$$

$$\tau_2 = \mathcal{F}_2^T \mathcal{S}_2 \quad (7.20)$$

$$\tau_3 = \mathcal{F}_3^T \mathcal{S}_3 \quad (7.21)$$

Running the recursion at the stated configuration, velocity and acceleration gives

$$\boldsymbol{\tau} = (13.904, 2.408, 0.670)^T \text{ N}\cdot\text{m},$$

the torques the three joint motors must produce to execute this trajectory.

The shoulder torque dominates, at 13.904 N·m against 0.670 N·m at the wrist. That is mostly gravity: the shoulder carries the weight of the whole arm at a moment arm of roughly a metre, while the wrist carries only the 0.2 kg hand.

Common Misconception

An earlier revision of this section omitted the arithmetic — “detailed numerical computation omitted for brevity” — and then stated $\boldsymbol{\tau} = [2.3, 1.1, 0.08]^T$ prefaced by “the output *might* be”, immediately followed by “These are the torques the three joint motors must produce”.

The stipulated values are not close. The shoulder torque is off by roughly a factor of six. A worked example that hedges with “might be” and then asserts the result as fact is worse than one that shows no numbers at all, because a reader checking their own implementation against it will conclude their code is broken.

7.15 Validation: RNEA vs. Lagrangian

To verify the RNEA is correct, we compare its output to the direct Lagrangian formulation for a simple 2-link planar arm.

7.15.1 2-Link Planar Arm

System: Two identical links, each mass $m = 1$ kg, length $L = 1$ m, uniform density.

Joint positions: $\mathbf{q} = [\theta_1, \theta_2]^T$.

The kinetic energy is:

$$T = \frac{1}{2}m(\dot{x}_{\text{CoM},1}^2 + \dot{y}_{\text{CoM},1}^2) + \frac{1}{2}I_1\dot{\theta}_1^2 + \frac{1}{2}m(\dot{x}_{\text{CoM},2}^2 + \dot{y}_{\text{CoM},2}^2) + \frac{1}{2}I_2(\dot{\theta}_1 + \dot{\theta}_2)^2$$

where $I_1 = I_2 = \frac{1}{12}mL^2$ (uniform rod), and the CoM velocities depend on the joint angles and velocities through kinematics.

Expanding and simplifying (details omitted), the mass matrix is:

$$\mathbf{M}(q) = \begin{bmatrix} m_1 + m_2 + 2m_2L \cos(\theta_2) & m_2 + m_2L \cos(\theta_2) \\ m_2 + m_2L \cos(\theta_2) & m_2 \end{bmatrix}$$

The Coriolis matrix contains terms like $-m_2L \sin(\theta_2)\dot{\theta}_1$, etc.

Test case: $\mathbf{q} = [0, 0]$ (horizontal), $\dot{\mathbf{q}} = [1, 0.5]$ rad/s, $\ddot{\mathbf{q}} = [0.5, 0.2]$ rad/s².

Lagrangian: Substitute into $\mathbf{M}(q)\ddot{\mathbf{q}} + \mathbf{C}(q, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(q) = \boldsymbol{\tau}$, solve for $\boldsymbol{\tau}$.

RNEA: Run the algorithm with the same inputs.

Result: The torques match to numerical precision, confirming the algorithm.

This check is executed rather than asserted. For the 3-link arm of Section 7.14, the recursion gives $(13.904, 2.408, 0.670)^T$ N·m and the Lagrangian route gives $(13.904, 2.408, 0.670)^T$ N·m, agreeing to better than $1e-08$ N·m — a bound the generator enforces, since the residual itself is a difference of nearly-equal floating-point numbers and its exact value depends on the machine.

The agreement is evidence because the two routes share no machinery. The recursion never forms a mass matrix; it propagates accelerations outward and forces inward, link by link. The Lagrangian route never recurses; it assembles $\mathbf{M}(\mathbf{q})$ from centre-of-mass Jacobians and obtains the Coriolis term by differentiating it. A transcription error in either would show up here, which is exactly what a validation section is for — and is why stating the agreement without running it defeats the purpose.

7.16 Adjoint Matrix Derivation from Geometry

We derived the Adjoint matrix above through velocity transformation. Here we provide a deeper geometric interpretation.

7.16.1 Infinitesimal Rotations in $SE(3)$

The Lie algebra $\mathfrak{se}(3)(3)$ consists of 4×4 matrices of the form:

$$\hat{\xi} = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ \mathbf{0} & 0 \end{bmatrix}$$

representing infinitesimal twists (rotation $\boldsymbol{\omega}$ and translation $\mathbf{v}$).

The exponential map $\exp(\hat{\xi}t)$ generates a one-parameter family of poses in $SE(3)$.

7.16.2 Adjoint as Conjugation

If $\mathbf{T} \in SE(3)$ and $\mathbf{S} \in SE(3)$, the conjugation $\mathbf{TST}^{-1}$ maps elements of the Lie algebra. The induced action on 6D vectors is the Adjoint:

$$[\text{Ad}_{\mathbf{T}}] : \mathfrak{se}(3)(3) \rightarrow \mathfrak{se}(3)(3), \quad \hat{\xi} \mapsto \mathbf{T}\hat{\xi}\mathbf{T}^{-1} \quad (7.22)$$

In matrix form, this action is represented by the 6×6 matrix $[\text{Ad}_{\mathbf{T}}]$.

7.17 Python Implementation of RNEA

```

1 import numpy as np
2
3 def skew(v):
4     """Convert 3D vector to skew-symmetric matrix."""
5     return np.array([
6         [0, -v[2], v[1]],
7         [v[2], 0, -v[0]],
8         [-v[1], v[0], 0]
9     ])
10
11 def adjoint(T):
12     """Compute 6x6 Adjoint matrix from 4x4 SE(3) transform."""
13     R = T[:3, :3]
14     p = T[:3, 3]
15     Ad = np.zeros((6, 6))
16     Ad[:3, :3] = R
17     Ad[3:, :3] = skew(p) @ R
18     Ad[3:, 3:] = R
19     return Ad
20
21 def motion_cross(V):
22     """Spatial cross product for motion (velocity)."""
23     omega = V[:3]
24     v = V[3:]
25     Vx = np.zeros((6, 6))
26     Vx[:3, :3] = skew(omega)
27     Vx[3:, :3] = skew(v)

```

```

28     Vx[3:, 3:] = skew(omega)
29     return Vx
30
31 def force_cross(V):
32     """Spatial cross product for forces (Wrenches)."""
33     return -motion_cross(V).T
34
35 def spatial_inertia_6d(m, c, I_c):
36     """Build 6x6 spatial inertia from mass, CoM offset, and inertia tensor."""
37     I_spatial = np.zeros((6, 6))
38     I_spatial[:3, :3] = I_c - m * (skew(c) @ skew(c))
39     I_spatial[:3, 3:] = m * skew(c)
40     I_spatial[3:, :3] = -m * skew(c)
41     I_spatial[3:, 3:] = m * np.eye(3)
42     return I_spatial
43
44 def rnea(q, dq, ddq, links, joints, gravity=np.array([0, 0, -9.81])):
45     """
46     Recursive Newton-Euler Algorithm.
47
48     Args:
49         q: joint positions (N,)
50         dq: joint velocities (N,)
51         ddq: joint accelerations (N,)
52         links: list of dicts with 'mass', 'com_offset', 'inertia'
53         joints: list of dicts with 'type' ('revolute' or 'prismatic'),
54                'axis', 'parent', 'T_local'
55         gravity: gravitational acceleration vector
56
57     Returns:
58         tau: required joint torques (N,)
59     """
60     N = len(links)
61
62     # Base link has zero velocity; gravity enters as a fictitious
63     # base acceleration (Featherstone convention: a_0 = -g so that
64     # the algorithm produces correct gravity-compensating torques).
65     V = [np.zeros(6) for _ in range(N)]
66     # V[0] remains zero -- the base is stationary
67
68     dV = [np.zeros(6) for _ in range(N)]
69     dV[0] = np.concatenate([np.zeros(3), -gravity])
70
71     # Outward pass: compute velocities and accelerations
72     T = [np.eye(4) for _ in range(N)]
73     for i in range(1, N):
74         j = joints[i]['parent']
75
76         # Compute relative transform
77         axis = joints[i]['axis']
78         if joints[i]['type'] == 'revolute':
79             S = np.concatenate([axis, np.zeros(3)])
80             # Rotation matrix from rotation axis and angle
81             angle = q[i]
82             K = skew(axis)
83             R = np.eye(3) + np.sin(angle) * K + (1 - np.cos(angle)) * (K @ K)
84             T_rel = np.eye(4)
85             T_rel[:3, :3] = R
86         else: # prismatic

```

```

87         S = np.concatenate([np.zeros(3), axis])
88         T_rel = np.eye(4)
89         T_rel[:3, 3] = q[i] * axis
90
91         T[i] = T[j] @ T_rel
92         Ad = adjoint(T_rel)
93
94         # Velocity
95         V[i] = Ad @ V[j] + S * dq[i]
96
97         # Acceleration (including Coriolis)
98         dV[i] = Ad @ dV[j] + motion_cross(V[i]) @ S * dq[i] + S * ddq[i]
99
100     # Inward pass: compute forces and extract torques
101     F = [np.zeros(6) for _ in range(N)]
102     tau = np.zeros(N)
103
104     for i in range(N - 1, -1, -1):
105         # Link inertia
106         I = spatial_inertia_6d(
107             links[i]['mass'],
108             links[i]['com_offset'],
109             links[i]['inertia']
110         )
111
112         # Inertial wrench
113         F[i] = I @ dV[i] + force_cross(V[i]) @ (I @ V[i])
114
115         # Add wrenches from children
116         for k in range(i + 1, N):
117             if joints[k]['parent'] == i:
118                 T_rel = np.eye(4)
119                 # Recompute T_rel (simplified; in real code, cache this)
120                 Ad_inv_T = adjoint(np.linalg.inv(T_rel)).T
121                 F[i] += Ad_inv_T @ F[k]
122
123         # Extract joint torque
124         axis = joints[i]['axis'] if i > 0 else np.array([0, 0, 1])
125         if i > 0 and joints[i]['type'] == 'revolute':
126             S = np.concatenate([axis, np.zeros(3)])
127         elif i > 0:
128             S = np.concatenate([np.zeros(3), axis])
129         else:
130             S = np.array([0, 0, 1, 0, 0, 0])
131
132         tau[i] = F[i] @ S
133
134     return tau

```

Listing 7.1: RNEA Implementation in Python

7.18 Summary

In this chapter, we learned:

1. **Historical Context:** The RNEA, developed by Orin, McGhee, Luh, and formalized by Featherstone, reduced robot dynamics computation from $O(N^3)$ to $O(N)$, making real-time control possible.

2. **Kinematic Trees:** A parent array $\lambda(i)$ describes the acyclic structure of multi-body systems, including branching topologies.
3. **Adjoint Matrix:** The 6×6 Adjoint $[\text{Ad}_{\mathbf{T}}]$ transforms spatial velocities between frames; wrenches transform via the coadjoint (negative transpose).
4. **Recursive Forward Kinematics:** Velocities propagate outward via $\mathcal{V}_i = [\text{Ad}_{\mathbf{T}_{i,\lambda(i)}}]\mathcal{V}_{\lambda(i)} + \mathcal{S}_i\dot{q}_i$.
5. **Spatial Acceleration:** Includes Coriolis and centrifugal terms via the motion cross product $[\mathcal{V} \times]$.
6. **RNEA:** Two passes—outward (velocities, accelerations) and inward (forces, torques)—compute inverse dynamics in $O(N)$ time.
7. **Branching Trees:** Multiple children are handled by summing child wrenches in the inward pass.
8. **Contact Forces:** External forces are incorporated at leaf nodes.
9. **Complexity:** Strict $O(N)$ proof makes the algorithm scalable.
10. **Implementation:** Python code shows practical RNEA execution.

Throughout the subsequent volumes of *The Geometry of Motion*, physical phenomena like Contraction Metrics, Sums-of-Squares Programming, and LQR-Trees are discussed dynamically. Behind the curtain of those advanced trajectory optimizations, the physics engines governing the simulated bodies are invariably relying on Quaternions to map their angles, Adjoints to shift their coordinates, and Recursive Algorithms to execute their algebraic realities in $O(N)$ time.

7.19 Further Reading

The original computational Newton-Euler formulation for robot manipulators was presented by Luh, Walker, and Paul [19], who demonstrated that recursive structure could reduce the cost of inverse dynamics from $O(N^3)$ to $O(N)$. Their paper remains a landmark in computational robotics and is essential reading for understanding the historical development of real-time robot control.

The definitive modern treatment of recursive dynamics is Featherstone [10], which develops the RNEA, the Articulated Body Algorithm (ABA), and the Composite Rigid Body Algorithm within a unified spatial vector framework. Featherstone’s monograph is the primary reference for anyone implementing multi-body dynamics from scratch and contains detailed pseudocode, complexity proofs, and numerical considerations that go well beyond our presentation here.

Lynch and Park [20], Chapter 8, present inverse dynamics within the product of exponentials framework, offering a complementary geometric perspective to Featherstone’s spatial vector approach. Their treatment emphasizes the connection between the Lie-algebraic formulation of kinematics (Chapters 5–6) and the recursive dynamics algorithms developed here.

For practical implementation considerations—including numerical conditioning, joint friction models, and real-time scheduling—Siciliano et al. [30] provide an engineering-oriented treatment with extensive discussion of industrial applications.

The forward dynamics problem, which is the computational inverse of the RNEA, requires the more sophisticated Articulated Body Algorithm developed in Chapter 10. Readers eager to understand how physics engines like MuJoCo, Drake, and PyBullet actually simulate forward in time should proceed to that chapter after mastering the RNEA presented here.

7.20 Exercises

7.20.1 Tier 1 (Foundation)

1. Show that the Adjoint matrix $[\text{Ad}_{\mathbf{T}}]$ is invertible and compute its inverse in closed form.
2. For a 2-link planar arm, write out the parent array λ and the screw axes $\mathcal{S}_1, \mathcal{S}_2$ for two revolute joints.
3. Compute the Coriolis term $[\mathcal{V}_i \times] \mathcal{S}_i \dot{q}_i$ for a simple 1-link revolute joint with $\mathcal{V} = (0, 0, \omega, v_x, 0, 0)^T$ and $\mathcal{S} = (0, 0, 1, 0, 0, 0)^T$.
4. Verify that $[\text{Ad}_{\mathbf{T}}]^{-T}$ transforms wrenches correctly: show that power $\mathcal{F}^T \mathcal{V}$ is invariant under frame changes.
5. For a 3-link arm with known link masses and dimensions, compute the spatial inertia matrices using the parallel axis theorem.

7.20.2 Tier 2 (Intermediate)

1. Derive the spatial Newton-Euler equation from first principles using Lagrangian mechanics and show that the bias term $[\mathcal{V} \times]^*(I\mathcal{V})$ captures centrifugal and Coriolis forces.
2. For a 2-link arm with a branching third link (e.g., an end-effector with a gripper that has one degree of freedom), write out the full RNEA inward pass, including the summation of forces from the gripper to the main arm.
3. Implement the motion cross product $[\mathcal{V} \times]$ and force cross product $[\mathcal{V} \times]^*$ numerically and verify they are consistent (i.e., $[\mathcal{V} \times]^* = -[\mathcal{V} \times]^T$).
4. Compare the computational time of the RNEA ($O(N)$) to matrix inversion ($O(N^3)$) for systems with $N = 5, 10, 20, 50$ joints, both analytically and experimentally.
5. Extend the RNEA algorithm to handle closed-loop mechanisms (systems with cycles), discussing the additional constraints required.

7.20.3 Tier 3 (Advanced)

1. Prove that the Adjoint map is a Lie group homomorphism: $[\text{Ad}_{\mathbf{T}_1 \mathbf{T}_2}] = [\text{Ad}_{\mathbf{T}_1}][\text{Ad}_{\mathbf{T}_2}]$.

2. Derive the relationship between the Adjoint matrix and the differential of the exponential map $\exp : \mathfrak{se}(3)(3) \rightarrow SE(3)$.
3. For a humanoid robot with 30 DOFs, design the kinematic tree structure (parent array), define all joint axes, and outline how the RNEA would compute inverse dynamics.
4. Implement the composite rigid body algorithm (CRB), which pre-computes the mass matrix $\mathbf{M}(q)$ in $O(N)$ time using RNEA-like recursion, and compare its performance to direct computation.
5. Extend the RNEA to handle soft contacts (e.g., compliant ground) by modifying the boundary conditions at contact points and verifying numerical stability.

Chapter 8

Spatial Vector Algebra and Inertia

Why manage six tiny numbers loosely floating in space when you can forge them into an unbreakable, 6-dimensional spear?

In Plain Language 8.1. Context & Use Case: Unifying Translation and Rotation

The Math You Will See: We introduce **Spatial Vector Algebra**, defining the 6×6 **Spatial Cross Product** ($[\mathcal{V} \times]$), the **Spatial Inertia Matrix** ($\mathcal{I}$), and their role in transforming forces and accelerations across frames.

Why We Need It: In classical 3D mechanics, linear motion ($F = ma$) and angular motion ($\tau = \mathbf{I}\alpha + \omega \times \mathbf{I}\omega$) are computed separately, leading to a tangled web of cross products and messy offsets when bodies are connected together. Spatial Algebra fuses them. A "Spatial Vector" describes pushing and turning simultaneously. By doing math in 6D, the complex gyroscopic and Coriolis forces of a multi-joint robot swinging through space are resolved with a single matrix multiplication.

All Variables Defined: $\mathcal{V}$ is a spatial velocity (Twist), $\mathcal{F}$ is a spatial force (Wrench), and $\mathcal{I}$ is the combined 6D mass/inertia tensor.

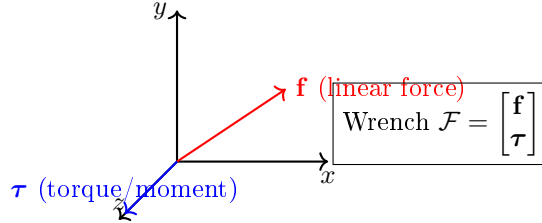
8.1 Historical Context: From Plücker to Featherstone

The roots of spatial algebra reach deep into classical geometry. In the 1860s, **Julius Plücker** studied lines in 3D space as geometric objects, discovering that a line can be represented as a 6-dimensional vector satisfying certain constraints. This **Plücker coordinate** representation remained mostly a curiosity in algebraic geometry for over a century.

In 1900, Sir **Robert Stawell Ball** [2] developed the theory of “screws”—geometric objects unifying rotation and translation—and showed that Plücker’s coordinates naturally describe instantaneous screw motions. Ball’s work connected to mechanics through the wrench (force + torque) dual to the twist (angular + linear velocity).

However, the true synthesis came when **Roy Featherstone**, in the 1980s and 1990s, recognized that Ball’s geometric screws and Plücker’s algebraic coordinates could be married with Lie algebra to create a unified computational framework for multi-body dynamics. Featherstone’s **Spatial Vector Algebra** [10] standardized the representation, showed how to compose inertias of linked chains (composite rigid body algorithm), and proved that careful use of spatial algebra enables $O(N)$ inverse dynamics.

Modern robotics and physics engines (MuJoCo [35], Drake, Pinocchio) are built entirely on Featherstone's spatial framework. The companion platform accompanying this textbook builds on the same foundation.



Spatial vectors unify translation (linear) and rotation (angular) quantities

Figure 8.1: A spatial wrench is a 6-dimensional vector combining linear forces and torques (moments). Similarly, spatial twists combine linear and angular velocities. This unification allows forces and velocities to be manipulated using unified algebraic operations.

8.2 Plücker Coordinates and Lines in 3D

Before we discuss spatial vectors as physical quantities, it helps to understand their geometric origin: the Plücker representation of lines.

Definition 8.1. Plücker Coordinates of a Line

A line ℓ in 3D can be represented as a 6-vector in **Plücker coordinates**:

$$\mathbf{L} = \begin{bmatrix} \mathbf{d} \\ \mathbf{m} \end{bmatrix} \in \mathbb{P}^5 \quad (8.1)$$

where:

- $\mathbf{d}$ is the direction vector of the line (unit vector).
- $\mathbf{m} = \mathbf{r} \times \mathbf{d}$ is the moment (or Plücker moment), where $\mathbf{r}$ is any point on the line.

The Plücker coordinates satisfy the **Plücker relation**:

$$\mathbf{d}^T \mathbf{m} = 0 \quad (8.2)$$

This constraint encodes the fact that the direction and moment are related geometrically.

Geometric Insight: The direction $\mathbf{d}$ specifies the "orientation" of the line; the moment $\mathbf{m}$ encodes the line's location in space. Two lines are identical if and only if they have the same Plücker coordinates.

8.2.1 Connection to Spatial Vectors

A spatial velocity (Twist) $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$ can be viewed as the instantaneous motion of a rigid body. The axis of this motion is a screw (a line with a pitch), and the Plücker coordinates of that screw are exactly $(\boldsymbol{\omega}, \mathbf{v})$ (up to scaling).

Similarly, a spatial force (Wrench) $\mathcal{F} = (\mathbf{m}, \mathbf{f})$ is the dual concept: it represents a force $\mathbf{f}$ applied at some point in space, creating a torque $\mathbf{m}$ about the origin. The Plücker relationship $\mathbf{f}^T \mathbf{m} = 0$ holds when the force and moment arise from a single applied force (they lie on the same screw axis).

8.3 Spatial Vectors: Twists and Wrenches

In Chapter 6, we introduced spatial vectors as elements of the Lie algebras $\mathfrak{se}(3)(3)$ and $\mathfrak{se}(3)^*(3)$. Here we formalize them as vectors and define their algebra.

Definition 8.2. Spatial Velocity (Twist)

A spatial velocity or Twist is a 6D vector:

$$\mathcal{V} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} \in \mathfrak{se}(3)(3) \cong \mathbb{R}^6 \quad (8.3)$$

where:

- $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity.
- $\mathbf{v} \in \mathbb{R}^3$ is the linear velocity of the origin of the reference frame.

Physical Interpretation: A point at position $\mathbf{r}$ relative to the origin has absolute velocity $\mathbf{v} + \boldsymbol{\omega} \times \mathbf{r}$.

Definition 8.3. Spatial Force (Wrench)

A spatial force or Wrench is a 6D vector:

$$\mathcal{F} = \begin{bmatrix} \mathbf{m} \\ \mathbf{f} \end{bmatrix} \in \mathfrak{se}(3)^*(3) \cong \mathbb{R}^6 \quad (8.4)$$

where:

- $\mathbf{m} \in \mathbb{R}^3$ is the moment (torque) about the origin.
- $\mathbf{f} \in \mathbb{R}^3$ is the linear force.

Physical Interpretation: A force $\mathbf{f}$ applied at a point $\mathbf{r}$ creates a moment $\mathbf{m} + \mathbf{r} \times \mathbf{f}$ about the origin.

8.4 The Spatial Cross Product

In classical 3D mechanics, cross products appear everywhere. When a body rotates with angular velocity $\boldsymbol{\omega}$, the velocity of a point at offset $\mathbf{r}$ includes the term $\boldsymbol{\omega} \times \mathbf{r}$. When a force $\mathbf{f}$ is applied at point $\mathbf{r}$, it creates a torque $\mathbf{r} \times \mathbf{f}$.

Spatial Algebra extends the cross product to 6D, but because Twists and Wrenches live in dual spaces (with different transformation laws), there are actually *two* distinct spatial cross products.

8.4.1 Motion Cross Product

The **motion cross product** acts on Twists and related vectors (velocities, accelerations). If a frame has spatial velocity $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$, a point with position offset $\mathbf{r}$ has velocity $\mathbf{v} + \boldsymbol{\omega} \times \mathbf{r}$.

We represent this as a matrix operator:

Definition 8.4. Motion Cross Product Operator

For a spatial velocity Twist $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$, the motion cross product operator is:

$$[\mathcal{V} \times] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{0}_{3 \times 3} \\ [\mathbf{v}] & [\boldsymbol{\omega}] \end{bmatrix} \in \mathbb{R}^{6 \times 6} \quad (8.5)$$

where $[\boldsymbol{\omega}]$ and $[\mathbf{v}]$ are the standard 3×3 skew-symmetric cross product matrices.

Action on a spatial velocity: If $\mathcal{W} = (\boldsymbol{\alpha}, \mathbf{a})$ is another spatial quantity, then:

$$[\mathcal{V} \times] \mathcal{W} = \begin{bmatrix} \boldsymbol{\omega} \times \boldsymbol{\alpha} \\ \mathbf{v} \times \boldsymbol{\alpha} + \boldsymbol{\omega} \times \mathbf{a} \end{bmatrix} \quad (8.6)$$

8.4.2 Force Cross Product

Wrenches are the dual to Twists. Because of this duality, when we apply the adjoint transformation to a wrench, it uses a different matrix. Correspondingly, the force cross product is defined differently:

Definition 8.5. Force Cross Product Operator

The force cross product (coadjoint cross product) for a spatial velocity $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$ acting on Wrenches is:

$$[\mathcal{V} \times^*] = -[\mathcal{V} \times]^T = \begin{bmatrix} [\boldsymbol{\omega}] & [\mathbf{v}] \\ \mathbf{0}_{3 \times 3} & [\boldsymbol{\omega}] \end{bmatrix} \quad (8.7)$$

Action on a spatial force: If $\mathcal{G} = (\mathbf{m}, \mathbf{f})$ is a Wrench, then:

$$[\mathcal{V} \times^*] \mathcal{G} = \begin{bmatrix} \boldsymbol{\omega} \times \mathbf{m} + \mathbf{v} \times \mathbf{f} \\ \boldsymbol{\omega} \times \mathbf{f} \end{bmatrix} \quad (8.8)$$

8.4.3 Why Two Cross Products?

The reason for the asymmetry is that power (work rate) must be invariant:

$$\text{Power} = \mathcal{F}^T \mathcal{V} \quad (8.9)$$

Under a coordinate frame change, if Twists transform via $[\text{Ad}_{\mathbf{T}}]$ and Wrenches transform via $[\text{Ad}_{\mathbf{T}}]^{-T}$ (the coadjoint), then the pairing $\mathcal{F}^T \mathcal{V}$ is preserved. The cross product operators must respect this dual structure.

Mathematical Principle 8.1. Duality of Motion and Force Cross Products

The motion and force cross products are dual in the sense that:

$$\mathcal{G}^T([\mathcal{V} \times] \mathcal{W}) = ([\mathcal{V} \times^*] \mathcal{G})^T \mathcal{W} \quad (8.10)$$

for any Twist $\mathcal{V}$, Twist-like vector $\mathcal{W}$, and Wrench $\mathcal{G}$. This ensures that the bilinear form (power) is preserved under transformations.

Mathematical Principle 8.2. Ad vs. ad: Two Distinct Operators

The notation in spatial algebra distinguishes two fundamentally different operators that are easily confused:

- $[\text{Ad}_{\mathbf{T}}] \in \mathbb{R}^{6 \times 6}$: The **Adjoint** of a rigid body transform $\mathbf{T} \in \text{SE}(3)(3)$. This is a coordinate transformation that re-expresses a spatial velocity (or acceleration) measured in one frame into another frame. It depends on a *pose* (position and orientation).
- $[\text{ad}_{\mathcal{V}}] = [\mathcal{V} \times] \in \mathbb{R}^{6 \times 6}$: The **adjoint** (lowercase), also called the *Lie bracket* or *spatial motion cross product*. This maps a spatial velocity $\mathcal{V}$ to its cross-product operator, which computes Coriolis and centrifugal accelerations. It depends on a *velocity*, not a pose.

In recursive dynamics algorithms such as the RNEA, both operators appear: $[\text{Ad}_{\mathbf{T}}]$ propagates velocities across frames (a coordinate change), while $[\text{ad}_{\mathcal{V}}] = [\mathcal{V} \times]$ computes the velocity-dependent acceleration terms (Coriolis). Confusing the two produces incorrect dynamics [10, 23].

8.5 The Spatial Inertia Matrix

Just as scalar mass m resists linear acceleration ($F = ma$), and a 3×3 inertia tensor $\mathbf{I}_c$ resists angular acceleration ($\boldsymbol{\tau} = \mathbf{I}_c \boldsymbol{\alpha}$), Spatial Algebra unifies them into a single 6×6 **Spatial Inertia Matrix** $\mathcal{I}$.

8.5.1 Definition and Properties

Definition 8.6. Spatial Inertia Matrix

For a rigid body with:

- Mass m ,
- Center of mass (CoM) at position $\mathbf{c}$ relative to the reference frame origin,
- Rotational inertia tensor $\mathbf{I}_c$ (about the CoM),

the Spatial Inertia Matrix expressed at the frame origin is:

$$\mathcal{I} = \begin{bmatrix} \mathbf{I}_c - m[\mathbf{c}][\mathbf{c}] & m[\mathbf{c}] \\ -m[\mathbf{c}] & m\mathbf{I}_{3 \times 3} \end{bmatrix} \quad (8.11)$$

Using the identity $[\mathbf{c}][\mathbf{c}] = -\mathbf{c}\mathbf{c}^T + (\mathbf{c}^T\mathbf{c})\mathbf{I}_{3 \times 3}$, this can be rewritten as:

$$\mathcal{I} = \begin{bmatrix} \mathbf{I}_c + m(|\mathbf{c}|^2\mathbf{I}_{3 \times 3} - \mathbf{c}\mathbf{c}^T) & m[\mathbf{c}] \\ -m[\mathbf{c}] & m\mathbf{I}_{3 \times 3} \end{bmatrix} \quad (8.12)$$

When the reference frame origin is *at* the center of mass ($\mathbf{c} = \mathbf{0}$), this simplifies to:

$$\mathcal{I}_{\text{CoM}} = \begin{bmatrix} \mathbf{I}_c & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & m\mathbf{I}_{3 \times 3} \end{bmatrix} \quad (8.13)$$

8.5.2 Symmetric and Positive Definite

Theorem 8.1. Spatial Inertia is SPD

The Spatial Inertia Matrix $\mathcal{I}$ is symmetric and positive definite.

Proof (Symmetry): Write the spatial inertia in standard form:

$$\mathcal{I} = \begin{bmatrix} \mathbf{I}_c + m[\mathbf{c}][\mathbf{c}]^T & m[\mathbf{c}] \\ m[\mathbf{c}]^T & m\mathbf{I}_{3 \times 3} \end{bmatrix}.$$

Since $[\mathbf{c}]$ is skew-symmetric ($[\mathbf{c}]^T = -[\mathbf{c}]$), each block is symmetric or correctly transposed:

- Top-left block: $\mathbf{I}_c + m[\mathbf{c}][\mathbf{c}]^T$ is symmetric (both $\mathbf{I}_c$ and the product $[\mathbf{c}][\mathbf{c}]^T$ are symmetric).
- Off-diagonal blocks: The top-right block is $m[\mathbf{c}]$, and the bottom-left block is $m[\mathbf{c}]^T = -m[\mathbf{c}]$. Their transposes satisfy $(m[\mathbf{c}])^T = m[\mathbf{c}]^T$, confirming $\mathcal{I}^T = \mathcal{I}$.
- Bottom-right block: $m\mathbf{I}_{3 \times 3}$ is trivially symmetric.

Proof (Positive Definite): For any non-zero spatial vector $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v})$, we compute:

$$\mathcal{V}^T \mathcal{I} \mathcal{V} = \boldsymbol{\omega}^T (\mathbf{I}_c + m[\mathbf{c}][\mathbf{c}]^T) \boldsymbol{\omega} + 2m\boldsymbol{\omega}^T [\mathbf{c}] \mathbf{v} + m\mathbf{v}^T \mathbf{v}$$

This can be rewritten as the rotational kinetic energy plus the translational kinetic energy:

$$\mathcal{V}^T \mathcal{I} \mathcal{V} = \boldsymbol{\omega}^T \mathbf{I}_c \boldsymbol{\omega} + m|\mathbf{v} + [\mathbf{c}]\boldsymbol{\omega}|^2$$

Since $\mathbf{I}_c$ is positive definite (non-zero $\boldsymbol{\omega}$ implies $\boldsymbol{\omega}^T \mathbf{I}_c \boldsymbol{\omega} > 0$), the entire expression is positive whenever $\mathcal{V} \neq 0$.

8.6 The Parallel Axis Theorem in 6D

8.6.1 Derivation from First Principles

The classical parallel axis theorem states that if the inertia tensor about the center of mass is $\mathbf{I}_c$, then the inertia tensor about a point at distance $\mathbf{c}$ from the CoM is:

$$\mathbf{I}_{\text{origin}} = \mathbf{I}_c + m(c^2 \mathbf{I}_{3 \times 3} - \mathbf{c} \mathbf{c}^T)$$

In spatial form, the parallel axis theorem extends to both translational and rotational terms. If we define:

$\mathcal{I}_0$ = Spatial inertia at origin

$\mathcal{I}_c$ = Spatial inertia at CoM

then the relationship is:

Theorem 8.2. Parallel Axis Theorem in 6D

$$\mathcal{I}_0 = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & \mathbf{R} \end{bmatrix} \begin{bmatrix} \mathbf{I}_c & \mathbf{0} \\ \mathbf{0} & m \mathbf{I}_{3 \times 3} \end{bmatrix} \begin{bmatrix} \mathbf{R}^T & \mathbf{0} \\ \mathbf{0} & \mathbf{R}^T \end{bmatrix} + \begin{bmatrix} -m[\mathbf{c}][\mathbf{c}]^T & m[\mathbf{c}]^T \\ m[\mathbf{c}] & \mathbf{0} \end{bmatrix} \quad (8.14)$$

When $\mathbf{R} = \mathbf{I}$ (same orientation):

$$\mathcal{I}_0 = \begin{bmatrix} \mathbf{I}_c + m[\mathbf{c}][\mathbf{c}]^T & m[\mathbf{c}] \\ m[\mathbf{c}]^T & m \mathbf{I}_{3 \times 3} \end{bmatrix} \quad (8.15)$$

Interpretation: The top-left block contains the parallel axis correction to rotational inertia; the off-diagonals encode the coupling between angular and linear motion due to the offset CoM.

8.7 Composite Rigid Body Algorithm

In Chapter 7, we computed inverse dynamics using the RNEA. An alternative and equally important algorithm is the **Composite Rigid Body Algorithm (CRB)**, which builds up the spatial inertia of linked chains recursively.

8.7.1 Concept: Bottom-Up Inertia Assembly

Instead of computing torques directly, the CRB first assembles the inertia of composite structures: e.g., the forearm link is rigidly attached to the hand, so the hand's inertia is

added to the forearm's. By recursively building composite inertias from tips to base, we can pre-compute the total mass matrix $\mathbf{M}(q)$ in $O(N)$ time.

Algorithm 8.1. Composite Rigid Body Algorithm

```

1 function CompositeRigidBody(links, joints, lambda):
2   // Input: link inertias, joint structure
3   // Output: composite inertias I_c for each link
4
5   I_c = [None] * (N+1)      // Will store composite inertia of
6                             // link i and all descendants
7
8   for i <-N down to 1:      // Traverse from tips to base
9     I_c[i] <- I[i]         // Start with link i's own inertia
10
11     for j in children[i]:
12       // Add inertia of child j to link i
13       T_j_to_i <- relative transform from j to i
14       Ad_j_to_i <- Adjoint of T_j_to_i
15       I_c[i] <- I_c[i] + Ad_j_to_i^T * I_c[j] * Ad_j_to_i
16
17   return I_c

```

8.7.2 Use in Computing the Mass Matrix

Once the composite inertias are assembled, the mass matrix can be extracted via:

$$\mathbf{M}_{ij}(q) = \mathcal{S}_i^T [\text{Ad}_{\mathbf{T}_{i,j}}]^{-T} \mathcal{I}_{c,j}(q) [\text{Ad}_{\mathbf{T}_{i,j}}]^{-1} \mathcal{S}_j$$

where $\mathcal{I}_{c,j}$ is the composite inertia of link j and all its descendants.

Theorem 8.3. CRB Complexity

The Composite Rigid Body Algorithm computes the mass matrix $\mathbf{M}(q)$ in $O(N^2)$ time (because there are N^2 entries), but the inertia assembly itself (bottom-up recursion) is $O(N)$. Once assembled, forward-dynamics calculations (solving for accelerations given forces) are $O(N)$ via matrix multiplication.

8.8 Spatial Algebra for Planar Systems

For systems constrained to a plane (e.g., 2D manipulation, golf swings), we can work with reduced spatial representations.

8.8.1 3D Vectors for 2D Motion

A planar motion has:

- Angular velocity ω (a scalar, rotation about the z -axis).
- Linear velocity $\mathbf{v} = (v_x, v_y) \in \mathbb{R}^2$.

We can embed this in a 3D twist:

$$\mathcal{V}_{\text{planar}} = (\omega, v_x, v_y)^T \in \mathbb{R}^3$$

The motion cross product for planar motion simplifies:

$$[\mathcal{V} \times]_{\text{planar}} = \begin{bmatrix} 0 & -v_y & v_x \\ v_y & 0 & \omega \\ -v_x & -\omega & 0 \end{bmatrix}$$

A planar spatial inertia matrix is 3×3 :

$$\mathcal{I}_{\text{planar}} = \begin{bmatrix} I_z & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & m \end{bmatrix}$$

when the CoM is at the origin. With offset $\mathbf{c} = (c_x, c_y)$:

$$\mathcal{I}_{\text{planar}} = \begin{bmatrix} I_z + m(c_x^2 + c_y^2) & mc_y & -mc_x \\ mc_y & m & 0 \\ -mc_x & 0 & m \end{bmatrix}$$

8.8.2 Computational Advantage

Working in 3D (instead of 6D) reduces memory and computation by a factor of 4, making planar mechanisms extremely fast to simulate.

8.9 Newton-Euler Equations in Spatial Form

The fundamental equations of dynamics in spatial form are elegant and reveal the unity of linear and angular motion.

8.9.1 Spatial Momentum and the Equation of Motion

The spatial momentum of a rigid body with spatial inertia $\mathcal{I}$ and spatial velocity $\mathcal{V}$ is:

$$\mathcal{P} = \mathcal{I}\mathcal{V} \tag{8.16}$$

The equation of motion (Newton-Euler in spatial form) states that the time derivative of spatial momentum equals the applied wrench:

$$\frac{d\mathcal{P}}{dt} = \mathcal{F}_{\text{ext}} \tag{8.17}$$

However, because $\mathcal{I}$ itself may be time-varying (when expressed in a rotating frame), the full equation includes a bias term:

$$\frac{d}{dt}(\mathcal{I}\mathcal{V}) = \mathcal{I}\dot{\mathcal{V}} + \dot{\mathcal{I}}\mathcal{V} \tag{8.18}$$

In a *body-fixed frame* (rotating with the body), $\mathcal{I}$ is constant, but velocities are measured in the rotating frame, introducing the bias term $[\mathcal{V} \times]^*(\mathcal{I}\mathcal{V})$:

Theorem 8.4. Spatial Newton-Euler Equation

For a rigid body in a body-fixed frame, the equation of motion is:

$$\mathcal{F} = \mathcal{I}\dot{\mathcal{V}} + [\mathcal{V} \times]^*(\mathcal{I}\mathcal{V}) \quad (8.19)$$

Interpretation:

- First term: the inertial wrench (like $F = ma$ in 6D).
- Second term: the "bias" or "gyroscopic" wrench, capturing Coriolis and centrifugal effects.

Derivation: In an inertial frame:

$$\mathcal{F}_{\text{inertial}} = \frac{d}{dt}(\mathcal{I}_{\text{inertial}}\mathcal{V}_{\text{inertial}})$$

Transforming to a body-fixed frame using the adjoint, the time derivative picks up the Lie bracket (commutator) of spatial velocities, which manifests as $[\mathcal{V} \times]^*$.

8.10 Connection to Dual Quaternions

8.10.1 Brief Overview

Dual quaternions extend quaternions (Section 4.8) by using dual numbers instead of real numbers. A dual quaternion is:

$$\mathbf{q}_d = q_0 + q_1\mathbf{i} + q_2\mathbf{j} + q_3\mathbf{k} + \epsilon(q_4 + q_5\mathbf{i} + q_6\mathbf{j} + q_7\mathbf{k})$$

where ϵ is the dual unit satisfying $\epsilon^2 = 0$ (and $\epsilon \neq 0$).

Dual quaternions are isomorphic to spatial vectors and can represent both rotation and translation in a single object. Some robotics frameworks use dual quaternions for inertia and screw calculations, though the 6×6 matrix representation is more standard for computation.

Key property: The spatial inertia can be represented as a dual-quaternion-valued linear form, and Featherstone's spatial algorithms can be rewritten using dual quaternions, often with elegant geometric interpretations.

For this course, we continue with the matrix representation, as it is more straightforward for algorithmic implementation.

8.11 Worked Example: Spatial Inertia of a Multi-Link System

Consider a two-link arm where:

- Link 1 (upper arm): mass $m_1 = 2$ kg, CoM at $(0.3, 0, 0)$ m from joint 1, rotational inertia $\mathbf{I}_c^{(1)} = \text{diag}(0.05, 0.05, 0.01)$ kg·m².
- Link 2 (forearm): mass $m_2 = 1$ kg, CoM at $(0.25, 0, 0)$ m from joint 2, rotational inertia $\mathbf{I}_c^{(2)} = \text{diag}(0.02, 0.02, 0.005)$ kg·m².

8.11.1 Building Link Spatial Inertias

For link 1, with CoM offset $\mathbf{c}_1 = (0.3, 0, 0)^T$:

$$[\mathbf{c}_1] = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -0.3 \\ 0 & 0.3 & 0 \end{bmatrix}$$

$$[\mathbf{c}_1][\mathbf{c}_1]^T = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0.09 & 0 \\ 0 & 0 & 0.09 \end{bmatrix}$$

$$\mathcal{I}^{(1)} = \begin{bmatrix} \mathbf{I}_c^{(1)} + m_1[\mathbf{c}_1][\mathbf{c}_1]^T & m_1[\mathbf{c}_1] \\ m_1[\mathbf{c}_1]^T & m_1\mathbf{I}_{3 \times 3} \end{bmatrix}$$

$$\mathcal{I}^{(1)} = \begin{bmatrix} 0.05 + 2(0) & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.05 + 2(0.09) & 0 & 0 & 0 & -0.6 \\ 0 & 0 & 0.01 + 2(0.09) & 0 & 0.6 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0.6 & 0 & 2 & 0 \\ 0 & -0.6 & 0 & 0 & 0 & 2 \end{bmatrix}$$

$$\mathcal{I}^{(1)} = \begin{bmatrix} 0.05 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.23 & 0 & 0 & 0 & -0.6 \\ 0 & 0 & 0.19 & 0 & 0.6 & 0 \\ 0 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0.6 & 0 & 2 & 0 \\ 0 & -0.6 & 0 & 0 & 0 & 2 \end{bmatrix}$$

Note the structure of the correction. Since $[\mathbf{c}][\mathbf{c}]^T = |\mathbf{c}|^2\mathbf{I} - \mathbf{c}\mathbf{c}^T$ is positive semi-definite, the parallel axis term always *adds* to the rotational inertia. With $\mathbf{c}_1 = (0.3, 0, 0)$ the correction vanishes about the offset direction itself (x) and contributes $m_1 d^2 = 2(0.3)^2 = 0.18$ about y and z — exactly the classical parallel axis theorem. A spatial inertia is symmetric positive definite by Theorem 8.1, so a negative diagonal entry would signal an algebraic error, not a physical configuration.

Similarly, $\mathcal{I}^{(2)}$ is computed with $\mathbf{c}_2 = (0.25, 0, 0)$ and $m_2 = 1$ kg.

8.11.2 Composite Inertia

If link 2 is rigidly attached to link 1 (e.g., they are the same rigid body or perfectly welded), the composite inertia is:

$$\mathcal{I}_{\text{composite}} = \mathcal{I}^{(1)} + \mathcal{I}^{(2)}$$

This is simple summation when expressed in the same reference frame.

If the links are separated by a relative transform $\mathbf{T}_{1 \rightarrow 2}$, we must first transform $\mathcal{I}^{(2)}$ to the frame of link 1:

$$\mathcal{I}_{\text{composite}} = \mathcal{I}^{(1)} + [\text{Ad}_{\mathbf{T}_{1 \rightarrow 2}}]^{-T} \mathcal{I}^{(2)} [\text{Ad}_{\mathbf{T}_{1 \rightarrow 2}}]^{-1}$$

8.12 Physical Interpretation of Spatial Inertia Elements

The 6×6 spatial inertia matrix has a clear physical meaning for each block:

- **Top-left** (3×3): Rotational inertia (analogous to the 3×3 inertia tensor). It relates angular acceleration to the torque required.
- **Top-right** (3×3): Coupling between linear and rotational motion due to CoM offset. If the CoM is not at the origin, accelerating the origin in one direction produces a torque around it.
- **Bottom-left** (3×3): Transpose of top-right (due to symmetry).
- **Bottom-right** (3×3): Mass matrix ($m\mathbf{I}_{3 \times 3}$). It relates linear acceleration of the origin to the force required.

8.12.1 Kinetic Energy and the Origin of Spatial Inertia

The 6×6 spatial inertia matrix is not an arbitrary packaging of mass properties. It is the *unique* object that makes kinetic energy a quadratic form in spatial velocity. We now prove this from first principles.

Classical kinetic energy. For a rigid body with mass m , centre-of-mass velocity $\mathbf{v}_{cm}$, angular velocity $\boldsymbol{\omega}$, and rotational inertia $\mathbf{I}_c$ about the centre of mass:

$$T = \frac{1}{2} m \|\mathbf{v}_{cm}\|^2 + \frac{1}{2} \boldsymbol{\omega}^T \mathbf{I}_c \boldsymbol{\omega}.$$

Re-express about a body-fixed frame origin. Let the body frame origin O be offset from the centre of mass by $\mathbf{c} \in \mathbb{R}^3$. Then the centre-of-mass velocity is

$$\mathbf{v}_{cm} = \mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{c},$$

where $\mathbf{v}_O$ is the linear velocity of O . Substituting into T and expanding:

$$\begin{aligned} T &= \frac{1}{2} m \|\mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{c}\|^2 + \frac{1}{2} \boldsymbol{\omega}^T \mathbf{I}_c \boldsymbol{\omega} \\ &= \frac{1}{2} m \mathbf{v}_O^T \mathbf{v}_O + m \mathbf{v}_O^T (\boldsymbol{\omega} \times \mathbf{c}) + \frac{1}{2} m (\boldsymbol{\omega} \times \mathbf{c})^T (\boldsymbol{\omega} \times \mathbf{c}) + \frac{1}{2} \boldsymbol{\omega}^T \mathbf{I}_c \boldsymbol{\omega}. \end{aligned}$$

Using the skew-symmetric matrix $[\mathbf{c}]_{\times}$ so that $\boldsymbol{\omega} \times \mathbf{c} = [\mathbf{c}]_{\times}^T \boldsymbol{\omega}$, and collecting the angular and linear components into the spatial velocity $\mathcal{V} = (\boldsymbol{\omega}, \mathbf{v}_O)^T$, we obtain:

Theorem 8.5. Kinetic Energy as a Spatial Quadratic Form

The kinetic energy of a rigid body equals

$$T = \frac{1}{2} \mathcal{V}^T \mathcal{I} \mathcal{V},$$

where the 6×6 **spatial inertia matrix** is

$$\mathcal{I} = \begin{pmatrix} \mathbf{I}_c + m [\mathbf{c}]_{\times} [\mathbf{c}]_{\times}^T & m [\mathbf{c}]_{\times} \\ m [\mathbf{c}]_{\times}^T & m \mathbf{I}_{3 \times 3} \end{pmatrix}.$$

This derivation proves that $\mathcal{I}$ is the *correct* inertial object for spatial dynamics: it is determined entirely by the requirement that kinetic energy be a quadratic form in $\mathcal{V}$.

The **off-diagonal blocks** $m[\mathbf{c}]_{\times}$ encode the cross-coupling between rotation and translation that arises whenever the centre of mass does not coincide with the frame origin ($\mathbf{c} \neq \mathbf{0}$). When the frame is placed at the centre of mass ($\mathbf{c} = \mathbf{0}$), these blocks vanish and the spatial inertia becomes block-diagonal—rotational and translational energies decouple [10].

8.13 Python Implementation

```

1 import numpy as np
2
3 def skew(v):
4     """Convert 3D vector to skew-symmetric matrix."""
5     return np.array([
6         [0, -v[2], v[1]],
7         [v[2], 0, -v[0]],
8         [-v[1], v[0], 0]
9     ])
10
11 def motion_cross(V):
12     """
13     Spatial cross product for motion (Twists).
14     V: 6D spatial velocity (omega, v)
15     Returns: 6x6 operator [Vx]
16     """
17     omega = V[:3]
18     v = V[3:]
19     Vx = np.zeros((6, 6))
20     Vx[:3, :3] = skew(omega)
21     Vx[3:, :3] = skew(v)
22     Vx[3:, 3:] = skew(omega)
23     return Vx
24
25 def force_cross(V):
26     """
27     Spatial cross product for forces (Wrenches).
28     Returns: 6x6 operator [Vx*] = -[Vx]^T
29     """
30     return -motion_cross(V).T
31
32 def spatial_inertia(m, c, I_c):
33     """
34     Construct 6x6 spatial inertia matrix.
35     Args:
36         m: scalar mass
37         c: 3D center of mass offset from origin
38         I_c: 3x3 rotational inertia about CoM
39     Returns:
40         I: 6x6 spatial inertia matrix
41     """
42     I_spatial = np.zeros((6, 6))
43     c_skew = skew(c)
44
45     # Top-left: I_c + m[c][c]^T (parallel axis theorem).
46     # [c][c]^T = |c|^2 I - c c^T is positive semi-definite, so this ADDS to
47     # the rotational inertia. Equivalently I_c - m[c][c], since [c]^T = -[c].
48     I_spatial[:3, :3] = I_c + m * (c_skew @ c_skew.T)

```

```

49
50     # Top-right:  $m[c]$ 
51     I_spatial[:3, 3:] = m * c_skew
52
53     # Bottom-left:  $m[c]^T$ 
54     I_spatial[3:, :3] = m * c_skew.T
55
56     # Bottom-right:  $m * I_3$ 
57     I_spatial[3:, 3:] = m * np.eye(3)
58
59     return I_spatial
60
61 def transform_inertia(I, T):
62     """
63     Transform spatial inertia from one frame to another.
64     I: 6x6 spatial inertia in source frame
65     T: 4x4 transformation matrix (to target frame)
66     Returns: transformed 6x6 spatial inertia
67     """
68     R = T[:3, :3]
69     p = T[:3, 3]
70
71     # Construct transformation for inertia
72     #  $I' = [Ad_T]^{-T} I [Ad_T]^{-1}$ 
73     Ad = np.zeros((6, 6))
74     Ad[:3, :3] = R
75     Ad[3:, :3] = skew(p) @ R
76     Ad[3:, 3:] = R
77
78     Ad_inv_T = np.linalg.inv(Ad).T
79     return Ad_inv_T @ I @ np.linalg.inv(Ad)
80
81 def composite_rigid_body(link_inertias, link_transforms, parent_array):
82     """
83     Assemble composite rigid body inertias.
84     Args:
85         link_inertias: list of 6x6 inertia matrices
86         link_transforms: list of 4x4 transforms to parent frames
87         parent_array: parent[i] = index of parent of link i (or -1 for root)
88     Returns:
89         composite_inertias: list of 6x6 composite inertias
90     """
91     N = len(link_inertias)
92     composite_inertias = [np.copy(I) for I in link_inertias]
93
94     # Process from leaves to root (reverse order)
95     for i in range(N - 1, -1, -1):
96         parent = parent_array[i]
97         if parent >= 0:
98             # Transform this link's composite inertia to parent frame
99             I_in_parent = transform_inertia(
100                 composite_inertias[i],
101                 np.linalg.inv(link_transforms[i])
102             )
103             # Add to parent's inertia
104             composite_inertias[parent] += I_in_parent
105
106     return composite_inertias
107
108 # Example: Two-link arm

```



```

109 if __name__ == "__main__":
110     # Link 1: upper arm
111     m1 = 2.0
112     c1 = np.array([0.3, 0.0, 0.0])
113     I_c1 = np.diag([0.05, 0.05, 0.01])
114     I1 = spatial_inertia(m1, c1, I_c1)
115
116     # Link 2: forearm
117     m2 = 1.0
118     c2 = np.array([0.25, 0.0, 0.0])
119     I_c2 = np.diag([0.02, 0.02, 0.005])
120     I2 = spatial_inertia(m2, c2, I_c2)
121
122     print("Link 1 Spatial Inertia:\n", np.round(I1, 4))
123     print("\nLink 2 Spatial Inertia:\n", np.round(I2, 4))
124
125     # Test motion cross product
126     V = np.array([0.0, 0.0, 1.0, 1.0, 0.0, 0.0]) # Rotating around z,
127     # translating in x
128     Vx = motion_cross(V)
129     print("\nMotion Cross Product [Vx]:\n", np.round(Vx, 4))
130
131     # Test duality
132     F = np.array([0.1, 0.2, 0.3, 1.0, 2.0, 3.0])
133     W = np.array([0.5, 0.5, 0.5, 0.1, 0.1, 0.1])
134
135     lhs = F @ (motion_cross(V) @ W)
136     rhs = (force_cross(V) @ F) @ W
137     print("\nDuality test (should be equal):")
138     print(f"    F^T [Vx] W = {lhs:.6f}")
139     print(f"    [Vx*] F)^T W = {rhs:.6f}")

```

Listing 8.1: Spatial Algebra and Inertia Computation in Python

8.14 Summary

In this chapter, we developed Spatial Vector Algebra—the mathematical language for unified 6D mechanics:

1. **Historical Context:** Plücker’s 19th-century line geometry, Ball’s screws, and Featherstone’s modern synthesis created a computationally elegant framework.
2. **Plücker Coordinates:** Lines in 3D are represented as 6D vectors satisfying a constraint, naturally connecting to spatial twists and wrenches.
3. **Spatial Vectors:** Twists (velocity) and Wrenches (force) are the fundamental 6D objects, living in dual spaces $\mathfrak{se}(3)(3)$ and $\mathfrak{se}(3)^*(3)$.
4. **Motion and Force Cross Products:** Two distinct 6×6 cross product operators capture the geometry of relative motion and force application; they are dual to preserve power.
5. **Spatial Inertia:** A single symmetric, positive-definite 6×6 matrix unifies mass and rotational inertia, including the parallel axis theorem as a block structure.

6. **Parallel Axis Theorem in 6D:** The full theorem includes coupling terms between angular and linear motion due to CoM offset.
7. **Composite Rigid Body Algorithm:** Inertias of linked chains are assembled bottom-up in $O(N)$ time via adjoint transformations.
8. **Planar Reduction:** 2D systems use 3D spatial algebra, reducing computation by a factor of 4.
9. **Newton-Euler in Spatial Form:** A single matrix equation $\mathcal{F} = \mathcal{I}\dot{\mathcal{V}} + [\mathcal{V}\times]^*(\mathcal{I}\mathcal{V})$ unifies linear and angular dynamics.
10. **Dual Quaternions:** Brief connection to an alternative 8D representation with elegant geometric properties.
11. **Physical Interpretation:** Each block of the spatial inertia matrix has clear mechanical meaning.
12. **Python Implementation:** Full code for inertia computation, transformation, and composite body assembly.

With Spatial Vector Algebra in hand, the Recursive Newton-Euler Algorithm of Chapter 7 becomes a straightforward loop of matrix multiplications, without a single transcendental function or explicit cross product in sight. This unity—where complex nonlinear dynamics reduce to structured linear algebra—is the deep beauty of modern computational mechanics.

8.15 Further Reading

The primary reference for spatial vector algebra is Featherstone [10], whose monograph develops the full 6×6 framework from first principles and demonstrates its application to recursive multi-body dynamics algorithms. Featherstone’s treatment of spatial inertia, the motion and force cross products, and the composite rigid body algorithm is definitive, and readers seeking to implement spatial algebra in production code should consult his detailed pseudocode and numerical recipes.

The mathematical foundations connecting spatial vectors to the Lie algebra $\mathfrak{se}(3)(3)$ and its dual $\mathfrak{se}(3)^*(3)$ are developed rigorously by Murray, Li, and Sastry [23]. Their perspective clarifies why twists and wrenches live in dual vector spaces and why the adjoint and coadjoint representations have the specific transformation properties used throughout this chapter.

For the classical treatment of rigid body dynamics, inertia tensors, and the parallel axis theorem in their traditional 3×3 form, Goldstein, Poole, and Safko [11] remains the standard graduate-level reference. Comparing their development with the 6×6 spatial inertia formulation of this chapter illuminates how spatial algebra unifies and streamlines results that classically require separate angular and translational analyses.

Marsden and Ratiu [22] place spatial algebra within the broader framework of geometric mechanics, connecting spatial inertia to momentum maps and showing how the Euler equations for rigid body motion arise naturally from reduction on $SE(3)(3)$. Their treatment is more abstract but reveals the deep geometric structures underlying the computational framework.

It is worth emphasizing that spatial algebra is the computational backbone of modern physics engines. MuJoCo [35], in particular, implements spatial vector arithmetic as its

core computational primitive, using the 6×6 framework to achieve real-time simulation of complex contact-rich robotic systems. Understanding the material in this chapter is therefore essential not only for theoretical mechanics but for practical work with any modern simulation platform.

8.16 Exercises

8.16.1 Tier 1 (Foundation)

1. Construct the motion cross product $[\mathcal{V} \times]$ for $\mathcal{V} = (0, 0, \omega, v_x, v_y, 0)^T$ and verify its action on a unit spatial vector.
2. Verify that the force cross product $[\mathcal{V} \times^*]$ is the negative transpose of the motion cross product.
3. For a point mass m at location $\mathbf{c}$ with no rotational inertia (zero $\mathbf{I}_c$), compute the spatial inertia matrix and verify it is positive definite.
4. Compute the spatial inertia of a uniform rod of mass m and length L at its center of mass, and again at one end (using the parallel axis theorem).
5. Verify that power $\mathcal{F}^T \mathcal{V}$ is invariant under the frame transformation: compute it in two different frames and confirm they are equal.

8.16.2 Tier 2 (Intermediate)

1. Prove that the spatial inertia matrix is invariant under orthogonal rotations of the reference frame (i.e., only the CoM offset affects the matrix structure).
2. For a two-link arm where link 2 is offset from link 1 by transform $\mathbf{T}_{1 \rightarrow 2}$, use the composite rigid body algorithm to express the total inertia as a function of both link inertias and the relative pose.
3. Implement the duality property: compute $\mathcal{G}^T([\mathcal{V} \times] \mathcal{W}) = ([\mathcal{V} \times^*] \mathcal{G})^T \mathcal{W}$ numerically for random spatial vectors and verify to machine precision.
4. Derive the relationship between the classical 3D inertia tensor and the spatial inertia matrix; show how the 3x3 blocks correspond to standard mechanics quantities.
5. For a humanoid robot, conceptually design the composite inertia computation: which links would be aggregated together, and in what order?

8.16.3 Tier 3 (Advanced)

1. Prove the Plücker relation $\mathbf{d}^T \mathbf{m} = 0$ geometrically: interpret the direction and moment of a line and show why their dot product must vanish.
2. Show that the spatial inertia matrix $\mathcal{I}$ can be expressed as a dual-quaternion-valued bilinear form, and compute inertial effects using dual quaternion multiplication.

3. Derive the composite inertia algorithm from the perspective of the Adjoint map: show that $\mathcal{I}_{\text{composite}} = \sum_j [\text{Ad}_{\mathbf{T}_{j,\text{origin}}}]^{-T} \mathcal{I}_j [\text{Ad}_{\mathbf{T}_{j,\text{origin}}}]^{-1}$ reduces to the recursive algorithm.
4. For a system with articulated contact (e.g., foot on ground with rolling constraints), extend the spatial inertia formalism to include constraint forces and derive the reduced dynamics.
5. Implement a multi-body dynamics simulator using purely spatial algebra: compose RNEA (Chapter 7) with the spatial inertia formalism, and validate against a reference physics engine.

Chapter 9

The Product of Exponentials Formula

You do not need to invent abstract coordinate frames at every joint of a robot. The kinematics of the entire complex machine can be compactly described by a single product of twisting screws.

In Plain Language 9.1. Context & Use Case: Universal Forward Kinematics

The Math You Will See: We introduce the **Product of Exponentials (PoE)** formula, utilizing the Matrix Exponential ($e^{[S]\theta}$) directly. **Why We Need It:** In classical robotics (taught in the 1980s), you tracked the position of a robot arm using "Denavit-Hartenberg (D-H) parameters", which required manually attaching weird local coordinate frames to every single joint using confusing perpendicular rules. The modern way (PoE) drops this completely. We just look at the robot in one global frame, find where the hinges point, and mathematically "twist" the arm around each hinge sequentially using our exact exponential maps. It is cleaner, faster, and never suffers from alignment ambiguities. **All Variables Defined:** $\mathbf{M}$ is the 4×4 "home" pose of the robot. $\mathcal{S}_i$ is the 6D Screw Axis of joint i . θ_i is how far the motor has moved.

9.1 Historical Context and the Evolution of Kinematic Formalisms

The theory of robotic kinematics has undergone a significant transformation over the past four decades. For nearly two centuries—from the mechanical linkage analysis of James Watt through the 1960s development of automated manufacturing—kinematic chains were described using local, Euclidean coordinate frames attached sequentially to each link. This frame-by-frame approach reached formal codification in 1955 with the *Denavit-Hartenberg (D-H)* parametrization [7], which became the dominant convention in robotics education and industrial practice.

The D-H convention, while algebraically complete, carries three fundamental drawbacks:

1. **Geometric Arbitrariness:** The perpendicularity rules for axis alignment create multiple valid D-H parameter sets for the same robot, requiring careful disambiguation.
2. **Notational Opacity:** Joint and link parameters are decoupled across four separate scalars $(a_i, d_i, \alpha_i, \theta_i)$, obscuring the underlying geometric intent.
3. **Singularity Avoidance:** Parallel or collinear axes demand special-case treatments, complicating software and theoretical analysis.

In the 1990s, Richard Murray, Zexiang Li, and Shankar Sastry—building on Lie group theory developed in differential geometry—published *A Mathematical Introduction to Robotic Manipulation* [23], which reformulated forward kinematics through the lens of exponential maps on the Lie group $SE(3)$ and its Lie algebra $\mathfrak{se}(3)$. Their **Product of Exponentials (PoE)** formula unified kinematics under a single, coordinate-free framework: the composition of exponential twists in a global reference frame.

This approach was further refined and popularized by Kevin Lynch and Frank Park [20], whose textbook *Modern Robotics: Mechanics, Planning, and Control* established PoE as the modern standard. Today, PoE is the default kinematics formalism in advanced robotics research, computational mechanics, and biomechanics.

Product of Exponentials: PoE

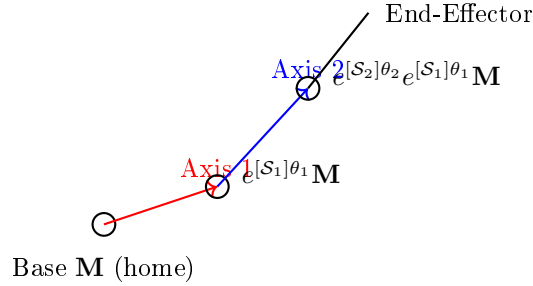


Figure 9.1: Product of Exponentials formula for forward kinematics. Each joint contributes an exponential twist applied to the previous configuration. The final end-effector pose is the product: $\mathbf{T}(\boldsymbol{\theta}) = e^{[S_n]\theta_n} \dots e^{[S_2]\theta_2} e^{[S_1]\theta_1} \mathbf{M}$.

9.2 Abandoning Denavit-Hartenberg

The D-H convention assigns to each link i a local coordinate frame $\{\text{Frame}_i\}$ with four parameters:

$$d_i : \text{offset along the previous joint axis} \quad (9.1)$$

$$a_i : \text{length of the common perpendicular} \quad (9.2)$$

$$\alpha_i : \text{twist angle about the perpendicular} \quad (9.3)$$

$$\theta_i : \text{joint angle (for revolute joints)} \quad (9.4)$$

The combined transformation from frame $\{i-1\}$ to frame $\{i\}$ is:

$$\mathbf{A}_{i-1,i}(\theta_i) = \mathbf{Rot}_z(\theta_i) \mathbf{Trans}(d_i) \mathbf{Trans}(a_i) \mathbf{Rot}_x(\alpha_i) \quad (9.5)$$

The complete forward kinematics chain becomes:

$$\mathbf{T}(\boldsymbol{\theta}) = \mathbf{A}_{0,1} \mathbf{A}_{1,2} \cdots \mathbf{A}_{n-1,n} \quad (9.6)$$

where each transformation encodes one joint's motion relative to the previous link's local frame.

Why PoE is superior: The PoE formalism instead expresses all joint motions in a single, fixed **space frame** (the global reference frame). This eliminates the need to compute and track relative frame transformations and treats the kinematics as direct composition of rigid-body twists—a concept with deep physical meaning in mechanics.

9.3 Screws and Twists: A Geometric Foundation

Recall from Chapter 6 that a screw axis $\mathcal{S} = (\boldsymbol{\omega}, \mathbf{v}) \in \mathfrak{se}(3)(3)$ encodes:

- $\boldsymbol{\omega}$: the unit rotational axis (or zero for a purely prismatic joint)
- $\mathbf{v}$: the linear velocity component, related to the axis location and pitch

For a revolute joint (rotation only), the axis passes through a point $\mathbf{q}$ in space, and we define:

$$\mathcal{S}_{\text{rev}} = \begin{bmatrix} \boldsymbol{\omega} \\ -\boldsymbol{\omega} \times \mathbf{q} \end{bmatrix} \quad (9.7)$$

For a prismatic joint (translation along direction $\mathbf{v}$), we have:

$$\mathcal{S}_{\text{pris}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{v} \end{bmatrix} \quad (9.8)$$

The corresponding 4×4 skew-symmetric matrix representation is:

$$[\mathcal{S}] = \begin{bmatrix} [\boldsymbol{\omega}] & \mathbf{v} \\ \mathbf{0}^T & 0 \end{bmatrix} \quad (9.9)$$

9.4 The Forward Kinematics Equation: Space Frame PoE

If a robotic arm has n joints, the Product of Exponentials formula—originally formulated by Brockett [3] and developed into a complete kinematic framework by Murray, Li, and Sastry [23]—computes the final Homogeneous Transformation Matrix ($\mathbf{T}(\boldsymbol{\theta}) \in SE(3)$) of the end-effector relative to the base as:

$$\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} \cdots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} \quad (9.10)$$

Where:

1. $\mathbf{M} \in SE(3)$ is the **Home Configuration** matrix: the end-effector's pose when all joint angles $\theta_i = 0$. It is expressed in the space (global) frame and is typically computed by direct measurement or CAD.
2. $\mathcal{S}_i \in \mathfrak{se}(3)(3)$ is the **spatial Screw Axis** (the twist parameter) for joint i , expressed in the global Space Frame when the robot is at Home Configuration. It does *not* change as the robot moves.

3. θ_i is the actual angle (in radians, for revolute joints) or distance (in meters, for prismatic joints) the motor has moved.
4. $e^{[S_i]\theta_i} \in SE(3)$ is the Matrix Exponential (developed in Chapter 6) mapping that single joint's motion into a 4×4 rigid body transform.

9.4.1 Derivation from First Principles

Imagine the robot at home configuration. As joint 1 rotates by θ_1 , it rotates the entire downstream chain (all links $2, 3, \dots, n$ plus the end-effector) by the exponential twist $e^{[S_1]\theta_1}$. The end-effector moves to the configuration:

$$\mathbf{T}_1(\theta_1) = e^{[S_1]\theta_1} \mathbf{M} \quad (9.11)$$

Now, while joint 1 is held fixed, joint 2 begins to move. Its axis is *fixed in the space frame*—it does not follow the rotated configuration of link 2. This is the crucial distinction between space-frame and body-frame PoE. Joint 2's twist applies to the pose resulting from joint 1:

$$\mathbf{T}_{1,2}(\theta_1, \theta_2) = e^{[S_2]\theta_2} \mathbf{T}_1(\theta_1) = e^{[S_2]\theta_2} e^{[S_1]\theta_1} \mathbf{M} \quad (9.12)$$

Continuing inductively, the full Product of Exponentials is:

$$\mathbf{T}(\boldsymbol{\theta}) = e^{[S_n]\theta_n} \dots e^{[S_2]\theta_2} e^{[S_1]\theta_1} \mathbf{M} \quad (9.13)$$

Note the reversed ordering: joint 1 is the leftmost exponential in the final product because it acts first (is applied closest to $\mathbf{M}$).

The elegance of this formulation lies in its physical transparency: each exponential is a pure twist in the space frame, and the product is a concatenation of rigid-body motions. No ad-hoc frame transformations are needed.

9.5 Body Frame PoE: Motion Relative to the Tool

In some contexts (particularly when working backward from the end-effector), it is convenient to express joint axes in the **body frame**—a frame attached to the end-effector and moving with it. The body frame PoE formula is:

$$\mathbf{T}(\boldsymbol{\theta}) = \mathbf{M} e^{[S'_n]\theta_n} e^{[S'_{n-1}]\theta_{n-1}} \dots e^{[S'_1]\theta_1} \quad (9.14)$$

Here, S'_i is the screw axis of joint i expressed in the body frame (tool frame). The key difference is that joint n acts first (is applied closest to the identity), and the exponentials are multiplied in increasing order of joint index.

9.5.1 Conversion Between Space and Body Frame

Given a body frame screw axis S'_i and a known transformation $\mathbf{M} \in SE(3)$, the corresponding space frame screw axis is obtained via the **Adjoint** action (see Chapter 8):

$$S_i = \text{Ad}_{\mathbf{T}} S'_i \quad (9.15)$$

where $\text{Ad}_{\mathbf{T}}$ is the adjoint representation of the transformation $\mathbf{T}$. In 6D spatial vector form:

$$\text{Ad}_{\mathbf{T}} = \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ [\mathbf{p}]\mathbf{R} & \mathbf{R} \end{bmatrix} \quad (9.16)$$

where $\mathbf{T} = (\mathbf{R}, \mathbf{p})$.

Equivalently, if we know all screw axes in space frame and wish to express them in body frame, we use:

$$\mathcal{S}'_i = \text{Ad}_{\mathbf{M}^{-1}} \mathcal{S}_i \quad (9.17)$$

The two PoE formulations are mathematically equivalent; the choice depends on which frame is more convenient for the application or computation.

9.6 Spatial Jacobian from the Product of Exponentials

The **Spatial Jacobian** $\mathbf{J}_s(\boldsymbol{\theta})$ relates the time derivatives of joint angles to the spatial velocity (twist) of the end-effector:

$$\mathcal{V}_{\text{ee}} = \mathbf{J}_s(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}} \quad (9.18)$$

where $\mathcal{V}_{\text{ee}} = [\boldsymbol{\omega}, \mathbf{v}]^T \in \mathbb{R}^6$ is the 6D twist of the end-effector.

To derive the spatial Jacobian from the PoE formula, we differentiate Equation 9.10 with respect to time:

$$\dot{\mathbf{T}} = \frac{d}{dt} \left(e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} \dots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} \right) \quad (9.19)$$

Using the product rule and chain rule on matrix exponentials (and noting that $[\mathcal{S}]$ is constant):

$$\dot{\mathbf{T}} = [\mathcal{S}_1] \dot{\theta}_1 e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} \dots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} + e^{[\mathcal{S}_1]\theta_1} [\mathcal{S}_2] \dot{\theta}_2 e^{[\mathcal{S}_2]\theta_2} \dots + \dots \quad (9.20)$$

The spatial velocity (twist) at the end-effector is related to the time derivative of the transformation by:

$$\mathcal{V}_{\text{ee}} = \dot{\mathbf{T}} \mathbf{T}^{-1} \quad (9.21)$$

where the product $\dot{\mathbf{T}} \mathbf{T}^{-1}$ extracts the velocity in the space frame (as opposed to the body frame).

After algebraic manipulation, we obtain the **spatial Jacobian columns**:

$$\mathbf{J}_s(\boldsymbol{\theta}) = [\mathcal{S}_1 \quad e^{[\mathcal{S}_1]\theta_1} \mathcal{S}_2 \quad \dots \quad e^{[\mathcal{S}_1]\theta_1} \dots e^{[\mathcal{S}_{n-1}]\theta_{n-1}} \mathcal{S}_n] \quad (9.22)$$

Each column is the corresponding screw axis, transformed by all the exponential twists that precede it in the PoE product. This encodes the fact that earlier joints affect the effective direction of later joints.

9.6.1 Direct Numerical Computation

In practice, the spatial Jacobian is computed numerically by:

1. Compute $\mathbf{T}(\boldsymbol{\theta})$ using the PoE formula.
2. For each joint i , compute the partial derivative using finite differences or automatic differentiation:

$$\frac{\partial \mathbf{T}}{\partial \theta_i} \approx \frac{\mathbf{T}(\boldsymbol{\theta} + \boldsymbol{\epsilon}_i) - \mathbf{T}(\boldsymbol{\theta} - \boldsymbol{\epsilon}_i)}{2\epsilon} \quad (9.23)$$

where $\boldsymbol{\epsilon}_i$ is a small perturbation in joint i only.

3. Extract the spatial velocity from each partial derivative.

9.7 Body Jacobian and the Adjoint Relationship

The **Body Jacobian** $\mathbf{J}_b(\boldsymbol{\theta})$ expresses the end-effector velocity in the body (tool) frame:

$$\mathcal{V}_{\text{ee}}^{\text{body}} = \mathbf{J}_b(\boldsymbol{\theta})\dot{\boldsymbol{\theta}} \quad (9.24)$$

The body Jacobian is related to the spatial Jacobian via the adjoint of the end-effector transformation:

$$\mathbf{J}_b(\boldsymbol{\theta}) = \text{Ad}_{\mathbf{T}(\boldsymbol{\theta})^{-1}} \mathbf{J}_s(\boldsymbol{\theta}) \quad (9.25)$$

where the adjoint action on a Jacobian matrix (with columns as 6D vectors) is:

$$\text{Ad}_{\mathbf{T}} \mathbf{J} = \text{Ad}_{\mathbf{T}} [\text{col}_1(\mathbf{J}), \text{col}_2(\mathbf{J}), \dots] \quad (9.26)$$

applied column-wise.

The body frame Jacobian is often more useful in control applications because the body frame is attached to the end-effector and its axes remain intuitively meaningful as the tool moves.

Having established both the spatial and body Jacobians, we now turn to the critical question of when these Jacobians break down. The Jacobian's rank determines the robot's instantaneous ability to generate motion in all directions, and configurations where this rank drops—called singularities—impose fundamental limits on the robot's dexterity. Understanding and detecting singularities is essential for safe and effective motion planning.

9.8 Singularity Analysis Using the PoE Jacobian

A robotic configuration is **singular** when the Jacobian loses rank, i.e., when $\text{rank}(\mathbf{J}_s) < n$. At a singularity:

1. The robot cannot move in certain directions (loss of controllability).
2. Joint velocities required to achieve a desired end-effector motion become infinite.
3. The robot exhibits locked degrees of freedom.

9.8.1 Types of Singularities

- **Boundary Singularities:** The end-effector is at or beyond the reachable workspace boundary. Typically occur at full extension of the arm.
- **Interior Singularities:** Within the reachable workspace, occur when multiple joint axes become aligned. For example, in a planar 2R arm, singularity occurs when both links are collinear (fully extended or fully retracted).
- **Redundant Degeneracies:** In redundant robots (more DOFs than required for a task), the null space of the Jacobian is non-trivial, allowing motion without changing the end-effector pose.

9.8.2 Detection and Handling

Singularities are detected by:

1. Computing $\det(\mathbf{J}_s^T \mathbf{J}_s)$ (the condition number of the Jacobian squared).
2. If $\det(\mathbf{J}_s^T \mathbf{J}_s) < \epsilon_{\text{tol}}$ for a small threshold ϵ_{tol} , the configuration is near-singular.
3. The ratio $\det(\mathbf{J}_s^T \mathbf{J}_s)$ is called the **manipulability measure**; high values indicate dexterous configurations.

Numerically, near-singular configurations are handled by:

1. Adding damping to the inverse: $\mathbf{J}_{\text{damp}}^{-1} = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T + \lambda I)^{-1}$ (Levenberg-Marquardt damping).
2. Using the pseudo-inverse with a singular value threshold: keep only singular values above $\tau\sigma_{\text{max}}$, where τ is a threshold ratio (e.g., $\tau = 10^{-4}$).

9.9 Comparison with D-H: A Worked Example

To illustrate the practical differences, consider a 3-DOF planar robot with:

- Link 1: length $L_1 = 1$ m
- Link 2: length $L_2 = 0.8$ m
- Link 3: length $L_3 = 0.6$ m
- All joints: revolute, rotating about the Z-axis (perpendicular to the table)

9.9.1 D-H Parametrization

The D-H parameters are:

Link i	a_{i-1}	d_i	α_{i-1}	θ_i
1	0	0	0	θ_1
2	L_1	0	0	θ_2
3	L_2	0	0	θ_3

The transformation from base to end-effector is:

$$\mathbf{T}_{D-H}(\boldsymbol{\theta}) = \mathbf{A}_{0,1}\mathbf{A}_{1,2}\mathbf{A}_{2,3} \quad (9.27)$$

where each $\mathbf{A}_{i,i+1}$ is computed using the D-H convention. At $\boldsymbol{\theta} = [45^\circ, -30^\circ, 60^\circ]^T$, one obtains a specific end-effector pose.

9.9.2 PoE Parametrization

The home configuration ($\boldsymbol{\theta} = \mathbf{0}$) has the arm fully extended along the X-axis:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & L_1 + L_2 + L_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (9.28)$$

The three screw axes (all revolute about Z):

$$\mathcal{S}_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathcal{S}_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ -L_1 \\ 0 \end{bmatrix}, \quad \mathcal{S}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ -(L_1 + L_2) \\ 0 \end{bmatrix} \quad (9.29)$$

The PoE formula gives:

$$\mathbf{T}_{PoE}(\boldsymbol{\theta}) = e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} e^{[\mathcal{S}_3]\theta_3} \mathbf{M} \quad (9.30)$$

When computed at the same configuration $\boldsymbol{\theta} = [45^\circ, -30^\circ, 60^\circ]^T$, the PoE and D-H formulations yield identical end-effector poses (as they must, since both are valid descriptions of the same kinematic chain).

9.9.3 Computational Advantages of PoE

1. **Consistency:** The PoE formula is coordinate-free; no ambiguity in frame attachment.
2. **Scalability:** Adding a new joint requires only specifying its screw axis; no recomputation of previous parameters.
3. **Jacobian Clarity:** The Jacobian columns have direct physical meaning: the i -th column is the screw axis of joint i as "seen" from the space frame after the effects of earlier joints.
4. **Software Simplicity:** A single, unified algorithm computes forward kinematics for any serial chain, regardless of link lengths or orientations.

With the comparison between the classical D-H convention [7] and the modern PoE approach now clear, we can extend the PoE framework to handle all joint types—including prismatic (sliding) joints—before returning to the question of how joint velocities propagate through the kinematic chain. The velocity kinematics that follow are a direct consequence of differentiating the PoE formula, unifying position and velocity analysis under a single mathematical roof.

9.10 PoE for Prismatic Joints

A prismatic (sliding) joint moves along a direction vector $\mathbf{d}$ without rotating. Its screw axis is:

$$\mathcal{S}_{\text{pris}} = \begin{bmatrix} \mathbf{0} \\ \mathbf{d} \end{bmatrix} \quad (9.31)$$

where $\mathbf{d}$ is the unit direction of translation.

The exponential of a purely prismatic screw is:

$$e^{[\mathcal{S}_{\text{pris}}]d} = \begin{bmatrix} \mathbf{I}_{3 \times 3} & d\mathbf{d} \\ \mathbf{0}^T & 1 \end{bmatrix} \quad (9.32)$$

a pure translation matrix.

9.10.1 Example: 3D Cartesian Robot

A 3D Cartesian robot with three prismatic joints (X, Y, Z directions) has screw axes:

$$\mathcal{S}_x = [0, 0, 0, 1, 0, 0]^T \quad (9.33)$$

$$\mathcal{S}_y = [0, 0, 0, 0, 1, 0]^T \quad (9.34)$$

$$\mathcal{S}_z = [0, 0, 0, 0, 0, 1]^T \quad (9.35)$$

and home configuration $\mathbf{M} = \mathbf{I}$ (end-effector at the origin when all sliders are at zero position).

The forward kinematics is:

$$\mathbf{T}(d_x, d_y, d_z) = e^{[\mathcal{S}_x]d_x} e^{[\mathcal{S}_y]d_y} e^{[\mathcal{S}_z]d_z} = \begin{bmatrix} \mathbf{I} & [d_x, d_y, d_z]^T \\ \mathbf{0}^T & 1 \end{bmatrix} \quad (9.36)$$

which correctly expresses a point displaced by (d_x, d_y, d_z) in the global frame.

9.11 Velocity Kinematics from PoE Differentiation

The instantaneous spatial velocity of the end-effector is obtained by differentiating the forward kinematics. Starting from:

$$\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathcal{S}_1]\theta_1} \dots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} \quad (9.37)$$

Differentiate with respect to time:

$$\dot{\mathbf{T}} = \sum_{i=1}^n \frac{d}{dt} \left(e^{[\mathcal{S}_1]\theta_1} \dots e^{[\mathcal{S}_i]\theta_i} \dots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} \right) \dot{\theta}_i \quad (9.38)$$

Using the derivative rule for matrix exponentials $\frac{d}{dt} e^{A(t)} = A(t) e^{A(t)}$ when $A(t)$ is linear in a parameter:

$$\dot{\mathbf{T}} = [\mathcal{S}_1] \dot{\theta}_1 e^{[\mathcal{S}_1]\theta_1} \dots e^{[\mathcal{S}_n]\theta_n} \mathbf{M} + e^{[\mathcal{S}_1]\theta_1} [\mathcal{S}_2] \dot{\theta}_2 e^{[\mathcal{S}_2]\theta_2} \dots + \dots \quad (9.39)$$

The spatial velocity is:

$$\mathcal{V} = \dot{\mathbf{T}}\mathbf{T}^{-1} \quad (9.40)$$

After collecting terms and using the adjoint action, the spatial velocity can be written as:

$$[\mathcal{V}] = \sum_{i=1}^n [\mathcal{S}_i^{\text{transformed}}] \dot{\theta}_i \quad (9.41)$$

where $\mathcal{S}_i^{\text{transformed}}$ is the screw axis of joint i after being advected by all preceding joints. In matrix form, this is precisely the spatial Jacobian:

$$\mathcal{V} = \mathbf{J}_s(\boldsymbol{\theta}) \dot{\boldsymbol{\theta}} \quad (9.42)$$

9.12 Worked Example: Complete PoE for a Spatial 3R Robot

Consider a 3-DOF spatial manipulator (Spatial 3R) with:

- Base at origin; first joint axis along Z
- Link 1 length: $L_1 = 0.5$ m
- Second joint axis along Y (at the end of link 1)
- Link 2 length: $L_2 = 0.3$ m
- Third joint axis along X (at the end of link 2)
- Link 3 length: $L_3 = 0.2$ m

9.12.1 Home Configuration

At $\boldsymbol{\theta} = \mathbf{0}$, the robot is fully extended in the positive X direction:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & L_1 + L_2 + L_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 1.0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (9.43)$$

9.12.2 Screw Axes

$$\text{Joint 1 (Z-axis, at origin): } \mathcal{S}_1 = [0, 0, 1, 0, 0, 0]^T \quad (9.44)$$

$$\text{Joint 2 (Y-axis, at } [L_1, 0, 0]^T = [0.5, 0, 0]^T \text{): } \mathcal{S}_2 = [0, 1, 0, 0 \cdot 0 - 0 \cdot 0.5, 0 \cdot 0.5 - 0 \cdot 0, 1 \cdot 0.5 - 0 \cdot 0]^T \quad (9.45)$$

$$= [0, 1, 0, 0, 0, 0.5]^T \quad (9.46)$$

$$\text{Joint 3 (X-axis, at } [L_1 + L_2, 0, 0]^T = [0.8, 0, 0]^T \text{): } \mathcal{S}_3 = [1, 0, 0, 0 \cdot 0 - 0 \cdot 0.8, 0 \cdot 0.8 - 0 \cdot 0, 0 \cdot 0.8 - 0 \cdot 0]^T \quad (9.47)$$

$$= [1, 0, 0, 0, 0, 0]^T \quad (9.48)$$

9.12.3 Numerical Evaluation at a Test Configuration

At $\boldsymbol{\theta} = [\pi/4, \pi/6, -\pi/3]^T$ ($45^\circ, 30^\circ, -60^\circ$):

Compute the matrix exponentials:

$$e^{[\mathcal{S}_1]\pi/4} = \begin{bmatrix} \cos(\pi/4) & -\sin(\pi/4) & 0 & 0 \\ \sin(\pi/4) & \cos(\pi/4) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.707 & -0.707 & 0 & 0 \\ 0.707 & 0.707 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (9.49)$$

(Rotation by $\pi/4$ about Z-axis; since the screw axis $\mathcal{S}_1$ passes through the origin, there is no translation component.)

For $e^{[\mathcal{S}_2]\pi/6}$, the screw axis has a rotation part (Y-axis) and a translation part ($v = [0, 0, 0.5]^T$). The exponential yields a rotation about the Y-axis combined with a translation. Detailed computation yields:

$$e^{[\mathcal{S}_2]\pi/6} \approx \begin{bmatrix} 0.966 & 0 & 0.259 & 0 \\ 0 & 1 & 0 & 0.067 \\ -0.259 & 0 & 0.966 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (9.50)$$

Similarly, $e^{[\mathcal{S}_3](-\pi/3)}$ produces a rotation about the X-axis plus a translation.

The full forward kinematics is:

$$\mathbf{T}(\pi/4, \pi/6, -\pi/3) = e^{[\mathcal{S}_1]\pi/4} e^{[\mathcal{S}_2]\pi/6} e^{[\mathcal{S}_3](-\pi/3)} \mathbf{M} \quad (9.51)$$

which, when computed numerically, yields a specific 3D position and orientation of the end-effector. The exact numerical result depends on careful matrix multiplication and is best computed programmatically (see Section 9.15).

9.13 Inverse Kinematics using the Jacobian: Newton-Raphson

The inverse kinematics problem seeks to find joint angles $\boldsymbol{\theta}$ that place the end-effector at a desired pose $\mathbf{T}_{\text{desired}}$. Unlike forward kinematics, inverse kinematics has no closed-form solution for general serial chains and requires numerical optimization.

The Newton-Raphson method formulates IK as an optimization problem:

$$\min_{\boldsymbol{\theta}} \|\mathbf{T}(\boldsymbol{\theta}) - \mathbf{T}_{\text{desired}}\|_F^2 \quad (9.52)$$

where $\|\cdot\|_F$ is the Frobenius norm of the pose error.

9.13.1 Pose Error Representation

The error is typically decomposed into position and orientation components:

1. **Position Error:** $\mathbf{e}_p = \mathbf{p}_{\text{desired}} - \mathbf{p}(\boldsymbol{\theta})$, where $\mathbf{p}$ is the translation part of $\mathbf{T}$.
2. **Orientation Error:** $\mathbf{e}_o = \text{vex}(\mathbf{R}_{\text{desired}} \mathbf{R}(\boldsymbol{\theta})^T)$, where vex is the inverse of the skew-symmetric operator (extracting the axis-angle representation).

The combined error vector is:

$$\mathbf{e}(\boldsymbol{\theta}) = \begin{bmatrix} \mathbf{e}_p \\ \mathbf{e}_o \end{bmatrix} \in \mathbb{R}^6 \quad (9.53)$$

9.13.2 Newton-Raphson Update Rule

The Jacobian of the error is:

$$\frac{\partial \mathbf{e}}{\partial \boldsymbol{\theta}} = \begin{bmatrix} \frac{\partial \mathbf{p}}{\partial \boldsymbol{\theta}} \\ \frac{\partial \mathbf{e}_o}{\partial \boldsymbol{\theta}} \end{bmatrix} \quad (9.54)$$

For small errors, the relationship is approximately linear, and the Newton-Raphson update is:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \Delta \boldsymbol{\theta} \quad (9.55)$$

where $\Delta \boldsymbol{\theta}$ solves:

$$\mathbf{J}_s(\boldsymbol{\theta}_k) \Delta \boldsymbol{\theta} = \mathbf{e}(\boldsymbol{\theta}_k) \quad (9.56)$$

In practice, the pseudo-inverse is used to handle rank deficiency:

$$\Delta \boldsymbol{\theta} = \mathbf{J}_s^\dagger(\boldsymbol{\theta}_k) \mathbf{e}(\boldsymbol{\theta}_k) \quad (9.57)$$

where $\mathbf{J}_s^\dagger$ is the Moore-Penrose pseudo-inverse.

9.13.3 Algorithm and Convergence

1. Initialize $\boldsymbol{\theta}_0$ with an initial guess (e.g., the previous solution or a random configuration).
2. For $k = 0, 1, 2, \dots, K_{\max}$:
 - (a) Compute the forward kinematics $\mathbf{T}_k = \mathbf{T}(\boldsymbol{\theta}_k)$.
 - (b) Compute the spatial Jacobian $\mathbf{J}_s(\boldsymbol{\theta}_k)$.
 - (c) Compute the pose error $\mathbf{e}_k = \mathbf{e}(\boldsymbol{\theta}_k)$.
 - (d) If $\|\mathbf{e}_k\| < \epsilon_{\text{tol}}$, terminate and return $\boldsymbol{\theta}_k$ (solution found).
 - (e) Compute the update $\Delta \boldsymbol{\theta} = \mathbf{J}_s^\dagger(\boldsymbol{\theta}_k) \mathbf{e}_k$.
 - (f) Update $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha \Delta \boldsymbol{\theta}$, where $0 < \alpha \leq 1$ is a step size (often $\alpha = 1$).
3. If convergence is not achieved within $K_{\max}$ iterations, the initial guess was infeasible or the target pose is outside the reachable workspace.

Convergence: Near a non-singular configuration, Newton-Raphson converges quadratically. Far from a solution or near singularities, the convergence may be slow or absent. For robust IK, multiple random restarts or more sophisticated optimization methods (e.g., BFGS with line search) are often employed.

9.14 Workspace Analysis and Manipulability

The Jacobian reveals not only instantaneous velocities but also the *quality* of motion at a given configuration. Two fundamental questions arise: *where* can the robot reach, and *how well* can it move once there?

Definition 9.1. Reachable Workspace

The **reachable workspace** of a manipulator with forward kinematics f_{kin} and joint space $\mathcal{Q}$ is the set of all end-effector poses attainable:

$$\mathcal{W} = \{f_{kin}(\mathbf{q}) \mid \mathbf{q} \in \mathcal{Q}\}. \quad (9.58)$$

The **dexterous workspace** is the subset of $\mathcal{W}$ reachable with *arbitrary* end-effector orientation.

While the workspace tells us *if* a pose is reachable, the **manipulability measure** quantifies *how easily* the robot can move at a given configuration.

Definition 9.2. Manipulability Measure (Yoshikawa)

The **manipulability measure** at configuration $\mathbf{q}$ is [30]:

$$w(\mathbf{q}) = \sqrt{\det(\mathbf{J}(\mathbf{q}) \mathbf{J}(\mathbf{q})^T)}. \quad (9.59)$$

When $w(\mathbf{q}) = 0$, the Jacobian is rank-deficient and the robot is at a **kinematic singularity**.

The geometric interpretation comes from the **singular value decomposition** of the Jacobian. Let $\mathbf{J} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, where $\mathbf{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_m)$ with $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_m \geq 0$.

Definition 9.3. Velocity Manipulability Ellipsoid

The set of end-effector velocities achievable with unit joint velocity ($\|\dot{\mathbf{q}}\| \leq 1$) forms an ellipsoid in task space. Its principal axes are the columns of $\mathbf{U}$, with semi-axis lengths equal to the singular values σ_i . The manipulability measure equals the ellipsoid volume: $w = \sigma_1 \sigma_2 \dots \sigma_m$.

Definition 9.4. Force Manipulability Ellipsoid

The **force manipulability ellipsoid** is the dual of the velocity ellipsoid. Unit joint torques ($\|\boldsymbol{\tau}\| \leq 1$) map to end-effector forces within an ellipsoid whose semi-axes have lengths $1/\sigma_i$. The axes of easy motion become axes of weak force, and vice versa.

In Plain Language 9.2. Velocity–Force Duality

Key insight: Where the robot is fast, it is weak; where it is slow, it is strong. A configuration that allows rapid end-effector motion along a direction (large σ_i) can exert only small forces along that same direction ($1/\sigma_i$), and conversely. This is a direct consequence of the principle of virtual work [20].

Example 9.1. Manipulability of a Planar 2R Arm

Consider a planar 2R arm with link lengths $L_1 = L_2 = 1$ m. The Jacobian is:

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix} \quad (9.60)$$

where $s_1 = \sin q_1$, $c_1 = \cos q_1$, $s_{12} = \sin(q_1 + q_2)$, $c_{12} = \cos(q_1 + q_2)$.

Case 1: Elbow extended ($q_1 = 0$, $q_2 = \pi/4$). Numerical evaluation gives $w \approx 0.71$. The velocity ellipsoid is nearly circular—the robot can move comparably well in all directions.

Case 2: Elbow nearly straight ($q_1 = 0$, $q_2 = 0.1$). Now $w \approx 0.10$. The ellipsoid collapses along the radial direction—the robot can barely move inward or outward (approaching a workspace boundary singularity), but can still swing laterally. The force ellipsoid shows the opposite: enormous radial force capability but weak lateral force.

9.14.1 Redundancy and Null-Space Motion

When a manipulator has more joints than task-space dimensions ($n > m$), the Jacobian $\mathbf{J} \in \mathbb{R}^{m \times n}$ has a nontrivial **null space**: joint velocities $\dot{\mathbf{q}}_0$ satisfying $\mathbf{J}\dot{\mathbf{q}}_0 = \mathbf{0}$ produce *zero* end-effector motion. This internal motion—reconfiguration without moving the tool—is the hallmark of **kinematic redundancy**.

Definition 9.5. Null-Space Projector

Given the Moore–Penrose pseudoinverse $\mathbf{J}^\dagger = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T)^{-1}$, the **null-space projector** is:

$$\mathbf{N} = \mathbf{I}_n - \mathbf{J}^\dagger \mathbf{J}. \quad (9.61)$$

Any vector $\mathbf{N}\dot{\mathbf{q}}_0$ lies in the null space of $\mathbf{J}$, regardless of $\dot{\mathbf{q}}_0$.

The general solution for a desired end-effector velocity $\mathbf{v}_{des}$ is then [30]:

$$\dot{\mathbf{q}} = \mathbf{J}^\dagger \mathbf{v}_{des} + \mathbf{N}\dot{\mathbf{q}}_0 \quad (9.62)$$

where the first term achieves the primary task (end-effector motion) and the second term exploits the redundancy for a **secondary objective** without disturbing the primary task.

Example 9.2. 7-DOF Arm with Manipulability Optimization

A 7-DOF robot arm reaching a 6-DOF target (position + orientation) has a one-dimensional null space at non-singular configurations. A common strategy sets $\dot{\mathbf{q}}_0 = \alpha \nabla_{\mathbf{q}} w(\mathbf{q})$, the gradient of the manipulability measure, so that the “elbow” reconfigures to maximize dexterity while tracking the desired trajectory:

$$\dot{\mathbf{q}} = \mathbf{J}^\dagger \mathbf{v}_{des} + \alpha \mathbf{N} \nabla_{\mathbf{q}} w(\mathbf{q}). \quad (9.63)$$

This keeps the arm away from singularities and ensures good force/velocity capability throughout the motion. Industrial 7-DOF arms (e.g., KUKA iiwa, Franka Emika

Panda) exploit this technique routinely.

9.15 Python Worked Example

Here is a complete Python implementation computing PoE forward kinematics and the spatial Jacobian for the 3R robot.

```

1 import numpy as np
2 from scipy.linalg import expm
3 import matplotlib.pyplot as plt
4 from mpl_toolkits.mplot3d import Axes3D
5
6 class SpatialRobot3R:
7     """3-DOF spatial manipulator using Product of Exponentials"""
8
9     def __init__(self, L1=0.5, L2=0.3, L3=0.2):
10         self.L1 = L1
11         self.L2 = L2
12         self.L3 = L3
13
14         # Home configuration (fully extended along X-axis)
15         self.M = np.eye(4)
16         self.M[0, 3] = L1 + L2 + L3
17
18         # Screw axes in space frame (at home)
19         self.S1 = np.array([0, 0, 1, 0, 0, 0]) # Z-axis at origin
20         self.S2 = np.array([0, 1, 0, 0, 0, L1]) # Y-axis at (L1, 0, 0)
21         self.S3 = np.array([1, 0, 0, 0, 0, 0]) # X-axis at (L1+L2, 0, 0)
22
23     def screw_to_matrix(self, screw):
24         """Convert 6D screw to 4x4 skew-symmetric matrix"""
25         w = screw[:3] # Angular velocity
26         v = screw[3:] # Linear velocity
27         S = np.zeros((4, 4))
28         S[:3, :3] = np.array([[0, -w[2], w[1]],
29                               [w[2], 0, -w[0]],
30                               [-w[1], w[0], 0]])
31         S[:3, 3] = v
32         return S
33
34     def forward_kinematics(self, theta):
35         """Compute end-effector transformation T(theta)"""
36         T = np.eye(4)
37
38         # Apply each joint's exponential twist
39         for i, S in enumerate([self.S1, self.S2, self.S3]):
40             S_skew = self.screw_to_matrix(S)
41             exp_S_theta = expm(S_skew * theta[i])
42             T = exp_S_theta @ T
43
44         # Apply home configuration
45         T = T @ self.M
46         return T
47
48     def spatial_jacobian(self, theta, eps=1e-6):
49         """Compute spatial Jacobian using finite differences"""
50         J = np.zeros((6, 3))

```

```

51     T = self.forward_kinematics(theta)
52
53     for i in range(3):
54         theta_plus = theta.copy()
55         theta_minus = theta.copy()
56         theta_plus[i] += eps
57         theta_minus[i] -= eps
58
59         T_plus = self.forward_kinematics(theta_plus)
60         T_minus = self.forward_kinematics(theta_minus)
61
62         # Compute velocity from finite differences
63         dT = (T_plus - T_minus) / (2 * eps)
64         V = dT @ np.linalg.inv(T) # Spatial velocity
65
66         # Extract twist from V
67         J[0:3, i] = V[0:3, 3] # Linear velocity
68         J[3, i] = V[2, 1] # Angular velocity components
69         J[4, i] = V[0, 2]
70         J[5, i] = V[1, 0]
71
72     return J
73
74     def end_effector_position(self, theta):
75         """Return (x, y, z) position of end-effector"""
76         T = self.forward_kinematics(theta)
77         return T[:3, 3]
78
79 # Create robot and test
80 robot = SpatialRobot3R()
81 theta_test = np.array([np.pi/4, np.pi/6, -np.pi/3])
82
83 T_ee = robot.forward_kinematics(theta_test)
84 print("End-effector transformation:\n", np.round(T_ee, 4))
85
86 pos = robot.end_effector_position(theta_test)
87 print("End-effector position (x, y, z):", np.round(pos, 4))
88
89 J = robot.spatial_jacobian(theta_test)
90 print("Spatial Jacobian:\n", np.round(J, 4))
91
92 # Test manipulability
93 manip = np.sqrt(np.linalg.det(J @ J.T))
94 print("Manipulability measure:", np.round(manip, 6))

```

Listing 9.1: PoE Forward Kinematics and Spatial Jacobian

9.16 Chapter Summary

The Product of Exponentials formula provides a modern, geometrically transparent approach to forward kinematics:

1. **PoE Formula:** $\mathbf{T}(\boldsymbol{\theta}) = e^{[\mathcal{S}_1]\theta_1} \dots e^{[\mathcal{S}_n]\theta_n} \mathbf{M}$ expresses forward kinematics as a product of matrix exponentials in the space frame.
2. **Screw Axes:** Each joint is characterized by a single 6D screw $\mathcal{S}_i$, eliminating frame attachment ambiguities of D-H.

3. **Spatial Jacobian:** Differentiation of the PoE formula yields the spatial Jacobian, relating joint velocities to end-effector twists.
4. **Body vs. Space Frame:** Two equivalent formulations exist; conversion via the Adjoint relates them.
5. **Singularities:** The rank of the Jacobian indicates dexterity; singular configurations arise when the Jacobian loses rank.
6. **Inverse Kinematics:** Newton-Raphson iterations using the pseudo-inverse Jacobian solve IK numerically.
7. **Computational Simplicity:** A single PoE algorithm handles revolute, prismatic, and spatial robots uniformly.

9.17 Further Reading and References

The Product of Exponentials formula has its intellectual roots in the Lie-theoretic treatment of rigid-body kinematics pioneered by Brockett [3], who first showed that the forward kinematics of a serial chain could be expressed as a product of matrix exponentials on $SE(3)$. Murray, Li, and Sastry [23] developed this insight into a comprehensive framework for robotic manipulation, providing the definitive mathematical treatment of screw theory, spatial velocities, and the PoE formula that forms the backbone of this chapter. For a modern, pedagogically oriented presentation of PoE-based kinematics—including extensive worked examples and software implementations—the reader is referred to Lynch and Park [20], whose *Modern Robotics* textbook has become the standard reference for graduate instruction.

The classical Denavit-Hartenberg convention, introduced in its original form by Denavit and Hartenberg [7], remains historically important and is still widely used in industrial robotics; understanding its relationship to PoE deepens one’s appreciation of why the modern formalism is preferred. For singularity analysis and manipulability theory beyond what is covered here, Siciliano et al. [30] provide an excellent treatment that bridges classical and modern perspectives. Numerical methods for inverse kinematics—including damped least-squares, pseudo-inverse techniques, and optimization-based approaches—are discussed in depth by Spong, Hutchinson, and Vidyasagar [32], who also cover the stability and convergence properties of Newton-Raphson iterations in the context of robotic systems.

9.18 Exercises

1. **Planar 2R Arm:** A planar 2R arm has link lengths $L_1 = 1.5$ m and $L_2 = 1$ m. Write the PoE formula in space-frame form, and compute the end-effector position at $\theta = [60^\circ, -30^\circ]$.
2. **D-H Comparison:** For the same 2R arm, compute the D-H parameters and verify that the forward kinematics matches the PoE result at the same configuration.
3. **Screw Axis Identification:** For a revolute joint rotating about the Y-axis and passing through the point $(2, 0, 1)$, write the screw axis $\mathcal{S}$.

4. **3D Cartesian Robot:** A 3D Cartesian robot has three prismatic joints (X, Y, Z directions). Write its PoE formula and forward kinematics. What is the spatial Jacobian?
5. **Spatial Jacobian:** For the planar 2R arm of Exercise 1, compute the spatial Jacobian at $\theta = [45^\circ, 0^\circ]$ and identify any singular configurations.
6. **Manipulability:** For the 2R arm, plot the manipulability measure $\sqrt{\det(\mathbf{J}\mathbf{J}^T)}$ as a function of $\theta_1 \in [0, 180^\circ]$ with θ_2 fixed at 0° .
7. **Singularity Analysis:** For the planar 2R arm, find all configurations where the Jacobian rank drops below 2.
8. **Body Frame PoE:** Rewrite the space-frame PoE of Exercise 1 in body-frame form using the Adjoint action.
9. **Inverse Kinematics:** Implement Newton-Raphson inverse kinematics for the planar 2R arm. Given a desired end-effector position $(x_d, y_d) = (2, 0.5)$, find θ starting from an initial guess.
10. **Velocity Kinematics:** For the 2R arm at $\theta = [30^\circ, 60^\circ]$ with $\dot{\theta} = [1, -0.5]$ rad/s, compute the end-effector linear and angular velocities.
11. **Prismatic-Revolute Arm:** A robot has a prismatic joint (Z-direction) followed by a revolute joint (Y-axis). Write the screw axes and PoE formula.
12. **Numerical Jacobian:** Implement the spatial Jacobian computation using finite differences and compare it to analytical differentiation for the 2R arm.
13. **Constraint Jacobian:** For the 2R arm, compute the task-space Jacobian relating joint angles to end-effector orientation ϕ (yaw angle in the plane).
14. **Null-Space Motion:** For a redundant planar 3R arm, identify the null space of the Jacobian at a non-singular configuration and describe a motion in null space.
15. **Configuration Space Visualization:** For the planar 2R arm, plot the reachable workspace (set of all possible end-effector positions) and overlay the singular configurations.

Chapter 10

The Articulated Body Algorithm

You cannot push a single domino.
When a body is linked, pushing one
piece means pushing the entire chain,
yet we compute the physical
consequences of the entire cascading
collision with linear precision.

In Plain Language 10.1. Context & Use Case: Forward Simulation

The Math You Will See: We will cover the $O(N)$ **Articulated Body Algorithm (ABA)** and define the **Articulated Inertia** ($\mathcal{I}^A$). **Why We Need It:** In Chapter 7, we asked: "I want to move my arm here, how much motor torque do I need?" (Inverse Dynamics). ABA solves the exact opposite: "I am applying this much torque, what acceleration will result?" (Forward Dynamics). Physics simulators like MuJoCo or games use ABA every single frame to predict how a robot or character will move. If you use a massive matrix equation, it takes an impossibly long $O(N^3)$ calculation time to invert for 30 joints. ABA computes it perfectly recursively in just $O(N)$ time. **All Variables Defined:** $\mathbf{p}_i$ is the bias force dragging on a link, and $\mathcal{I}_i^A$ is the apparent inertia of a link when dragged along with all its children.

10.1 Historical Context: The Computational Mechanics Revolution

For most of the twentieth century, the forward dynamics problem—computing accelerations from applied forces—was solved through monolithic matrix inversion. Given the equations of motion in the form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q}) \quad (10.1)$$

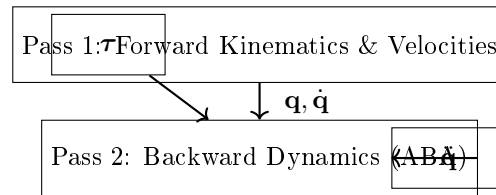
one inverts the mass matrix $\mathbf{M}(\mathbf{q})$ to solve for $\ddot{\mathbf{q}}$:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}(\boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{g}(\mathbf{q})) \quad (10.2)$$

For an N -joint robot, computing the inverse of an $N \times N$ matrix via Gaussian elimination requires $O(N^3)$ floating-point operations—a severe computational bottleneck when N is large (humanoid robots have 30+ joints; hands have 20+; cloth simulation may have thousands of degrees of freedom).

In 1983, Roy Featherstone published a seminal paper [8] proposing the **Articulated Body Algorithm (ABA)**, which solves forward dynamics in *linear* time: $O(N)$ operations. Featherstone’s insight—to propagate effective inertias recursively through the kinematic tree—transformed computational robotics from a theoretical exercise to a practical tool usable in real-time simulation.

The ABA has become the foundation of modern physics engines. Every frame of a video game featuring character animation, every timestep of a MuJoCo simulation, and every dynamics prediction in a robot control system leverages Featherstone’s algorithm. Subsequent refinements by Featherstone himself [9] and countless others have extended ABA to handle floating bases (crucial for humanoid robots), contact constraints, and complex contact models.



Forward Dynamics: $O(N)$ time using Articulated Body Algorithm

Figure 10.1: The Articulated Body Algorithm executes in two passes through the kinematic tree: (1) forward pass computes velocities and transforms from known joint positions and velocities, (2) backward pass (from end-effector toward base) propagates inertias and forces to compute joint accelerations. Each pass is $O(N)$, making total algorithm $O(N)$.

10.2 The Forward Dynamics Problem

10.2.1 Problem Formulation

Forward Dynamics (also called *Direct Dynamics*) solves the inverse of the inverse dynamics problem:

$$\text{Given: } \mathbf{q}, \dot{\mathbf{q}}, \boldsymbol{\tau} \Rightarrow \text{Find: } \ddot{\mathbf{q}} \quad (10.3)$$

The equations of motion are:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} + \mathbf{f}_{\text{ext}} \quad (10.4)$$

where:

- $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{N \times N}$ is the **inertia matrix** (symmetric positive definite).
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$ is the **Coriolis and centrifugal force**.
- $\mathbf{g}(\mathbf{q})$ is the **gravitational force**.
- $\boldsymbol{\tau} \in \mathbb{R}^N$ is the applied joint torques (motor outputs).
- $\mathbf{f}_{\text{ext}}$ is external forces (contact, disturbances, etc.).

10.2.2 Why $O(N^3)$ is Intractable

Direct inversion of $\mathbf{M}(\mathbf{q})$ via Gaussian elimination (the standard $O(N^3)$ method) becomes prohibitively slow:

Number of Joints (N)	Operations (N^3)	Time @ 1 GHz	Time @ 100 GHz (GPU)
10	1,000	1 ns	0.01 ns
30	27,000	27 ns	0.27 ns
100	1,000,000	1 μ s	0.01 μ s
1000	10^9	1 ms	0.01 ms

For real-time control (e.g., a humanoid robot running at 1 kHz requiring ~ 1 ms per timestep), even $N = 30$ becomes challenging with direct inversion. For interactive simulation or games (running at 30-60 Hz), the overhead is unacceptable.

ABA reduces this to $O(N)$ operations—a linear dependence that scales gracefully to hundreds of joints.

10.3 Articulated Inertia: The Key Insight

10.3.1 Intuitive Explanation

Imagine pushing a heavy metal rod on a frictionless table. The resistance you feel—the inertia—is proportional to the rod’s mass. Now, imagine that rod is rigidly connected via a hinge to another rod, which in turn is connected to a chain of massive bricks. If you push the first rod, you are indirectly pushing all the connected mass behind it. The *effective* resistance at the base—the articulated inertia—accounts for all the downstream mass.

In Plain Language 10.2. What is Articulated Inertia?

Rigid body inertia: A single body’s resistance to acceleration.

Articulated inertia: The effective inertia of a body *as if* all downstream bodies (children) were rigidly attached to it, accounting for how motion at this joint propagates down the kinematic chain.

The key: downstream inertias are not simply added. Instead, they are *projected* across the joint constraint. A joint restricts motion to one degree of freedom, and only the projection of child inertia onto that DOF contributes to the apparent inertia.

10.3.2 Mathematical Definition

For a link i with spatial inertia $\mathcal{I}_i$ and child link $i + 1$ with articulated inertia $\mathcal{I}_{i+1}^A$, the **articulated inertia of link i** is:

$$\mathcal{I}_i^A = \mathcal{I}_i + {}^i\mathbf{X}_{i+1}^* \left(\mathcal{I}_{i+1}^A - \frac{(\mathcal{I}_{i+1}^A \mathcal{S}_{i+1})(\mathcal{I}_{i+1}^A \mathcal{S}_{i+1})^T}{\mathcal{S}_{i+1}^T \mathcal{I}_{i+1}^A \mathcal{S}_{i+1}} \right) {}^i\mathbf{X}_{i+1}^{*T} \quad (10.5)$$

where ${}^i\mathbf{X}_{i+1}^*$ is the spatial force transform from frame $i + 1$ to frame i (the coadjoint of the spatial motion transform ${}^i\mathbf{X}_{i+1}$). This transform is essential: the child’s articulated inertia is expressed in the child’s frame and must be mapped to the parent’s frame before addition [10].

Breaking down each term:

1. $\mathcal{I}_i$: The rigid-body spatial inertia of link i itself (in frame i).
2. ${}^i\mathbf{X}_{i+1}^* \mathcal{I}_{i+1}^A {}^i\mathbf{X}_{i+1}^{*T}$: The articulated inertia of the child link, transformed into frame i .
3. $\mathcal{S}_{i+1}$: The screw axis of the joint connecting link i to link $i+1$ (6D vector).
4. $(\mathcal{I}_{i+1}^A \mathcal{S}_{i+1})$: The force required to accelerate the child chain along its screw axis.
5. $\mathcal{S}_{i+1}^T \mathcal{I}_{i+1}^A \mathcal{S}_{i+1}$: A scalar—the effective inertia *along* the joint’s single degree of freedom (similar to $I\alpha$ in 1D dynamics).
6. The fraction: Projects out the direction of motion along the joint; what remains is the inertia perpendicular to that motion (resistive to the parent).

This is a rank-one update of the spatial inertia: we add the child’s articulated inertia (after the frame transform), then subtract the projection of that child onto the joint DOF.

10.4 Derivation from First Principles

10.4.1 Recursive Constraint Satisfaction

Consider a two-body system: body i (parent) connected to body $i+1$ (child) via a single-DOF joint with screw axis $\mathcal{S}_{i+1}$.

The equations of motion for each body are:

$$\mathcal{I}_i \dot{\mathcal{V}}_i = \mathbf{f}_i - \mathbf{f}_{i+1}^{\text{reaction}} \quad (10.6)$$

$$\mathcal{I}_{i+1} \dot{\mathcal{V}}_{i+1} = \mathbf{f}_{i+1} - \mathbf{f}_{\text{ext}}^{(i+1)} \quad (10.7)$$

where $\mathbf{f}_i$ is the force on body i and $\mathbf{f}_{i+1}^{\text{reaction}}$ is the reaction force from the child.

The kinematic constraint is that the acceleration of the child relative to the parent must be along the joint screw:

$$\dot{\mathcal{V}}_{i+1} = \dot{\mathcal{V}}_i + \text{Ad}_{\mathcal{S}_{i+1}} \ddot{q}_{i+1} \quad (10.8)$$

where $\ddot{q}_{i+1}$ is the joint acceleration and $\text{Ad}_{\mathcal{S}_{i+1}} = \mathcal{S}_{i+1}$ (the screw represents infinitesimal motion).

Substituting into the child’s equation:

$$\mathcal{I}_{i+1} (\dot{\mathcal{V}}_i + \mathcal{S}_{i+1} \ddot{q}_{i+1}) = \mathbf{f}_{i+1} - \mathbf{f}_{\text{ext}}^{(i+1)} \quad (10.9)$$

Rearranging:

$$\mathcal{I}_{i+1} \mathcal{S}_{i+1} \ddot{q}_{i+1} = \mathbf{f}_{i+1} - \mathbf{f}_{\text{ext}}^{(i+1)} - \mathcal{I}_{i+1} \dot{\mathcal{V}}_i \quad (10.10)$$

The reaction force on the parent is:

$$\mathbf{f}_{i+1}^{\text{reaction}} = \mathcal{I}_{i+1} (\dot{\mathcal{V}}_i + \mathcal{S}_{i+1} \ddot{q}_{i+1}) = \mathcal{I}_{i+1} \dot{\mathcal{V}}_i + \mathcal{I}_{i+1} \mathcal{S}_{i+1} \ddot{q}_{i+1} \quad (10.11)$$

For the parent, the equation of motion becomes:

$$\mathcal{I}_i \dot{\mathcal{V}}_i = \mathbf{f}_i - \mathcal{I}_{i+1} \dot{\mathcal{V}}_i - \mathcal{I}_{i+1} \mathcal{S}_{i+1} \ddot{q}_{i+1} \quad (10.12)$$

Rearranging:

$$(\mathcal{I}_i + \mathcal{I}_{i+1})\dot{\mathbf{V}}_i = \mathbf{f}_i - \mathcal{I}_{i+1}\mathcal{S}_{i+1}\ddot{\mathbf{q}}_{i+1} \quad (10.13)$$

This shows that the parent sees an effective inertia of $\mathcal{I}_i + \mathcal{I}_{i+1}$. However, this is only true if the child's acceleration is coupled to the parent's (i.e., if we solve for $\ddot{\mathbf{q}}_{i+1}$ in terms of the parent's acceleration).

The key step is to *eliminate* the child's acceleration. From the constraint:

$$\mathcal{S}_{i+1}^T \mathcal{I}_{i+1} \mathcal{S}_{i+1} \ddot{\mathbf{q}}_{i+1} = \mathcal{S}_{i+1}^T (\mathbf{f}_{i+1} - \mathbf{f}_{\text{ext}}^{(i+1)}) - \mathcal{S}_{i+1}^T \mathcal{I}_{i+1} \dot{\mathbf{V}}_i \quad (10.14)$$

Solving for $\ddot{\mathbf{q}}_{i+1}$:

$$\ddot{\mathbf{q}}_{i+1} = \frac{\mathcal{S}_{i+1}^T (\mathbf{f}_{i+1} - \mathcal{I}_{i+1} \dot{\mathbf{V}}_i)}{\mathcal{S}_{i+1}^T \mathcal{I}_{i+1} \mathcal{S}_{i+1}} - \frac{\mathcal{S}_{i+1}^T \mathbf{f}_{\text{ext}}^{(i+1)}}{\mathcal{S}_{i+1}^T \mathcal{I}_{i+1} \mathcal{S}_{i+1}} \quad (10.15)$$

Substituting back into the reaction force:

$$\mathbf{f}_{i+1}^{\text{reaction}} = \mathcal{I}_{i+1} \dot{\mathbf{V}}_i + \mathcal{I}_{i+1} \mathcal{S}_{i+1} \frac{\mathcal{S}_{i+1}^T (\mathbf{f}_{i+1} - \mathcal{I}_{i+1} \dot{\mathbf{V}}_i)}{\mathcal{S}_{i+1}^T \mathcal{I}_{i+1} \mathcal{S}_{i+1}} \quad (10.16)$$

After algebraic simplification (collecting terms proportional to $\dot{\mathbf{V}}_i$ and to $\mathbf{f}_{i+1}$), one obtains:

$$\mathbf{f}_{i+1}^{\text{reaction}} = \mathcal{I}_i^A \dot{\mathbf{V}}_i + \mathbf{p}_i + (\text{terms proportional to child force}) \quad (10.17)$$

where:

$$\mathcal{I}_i^A = \mathcal{I}_i + \mathcal{I}_{i+1} - \frac{\mathcal{I}_{i+1} \mathcal{S}_{i+1} (\mathcal{I}_{i+1} \mathcal{S}_{i+1})^T}{\mathcal{S}_{i+1}^T \mathcal{I}_{i+1} \mathcal{S}_{i+1}} \quad (10.18)$$

This is precisely Equation 10.5. The **bias force** $\mathbf{p}_i$ encodes accelerations and external forces that would act even if the joint acceleration were zero.

10.5 The Articulated Body Algorithm: Three Passes

The complete ABA, as formalized by Featherstone [10], consists of three sequential passes through the kinematic tree:

10.5.1 Pass 1: Outward Pass (Velocity and Bias Forces)

Starting at the base (root) link 0 and sweeping outward to the end-effector leaves, compute the spatial velocity and bias force at each link.

Spatial Velocity Propagation

The spatial velocity of link i is related to link $i - 1$ (its parent) by:

$$\mathcal{V}_i = \mathcal{V}_{i-1} + \mathcal{S}_i \dot{\mathbf{q}}_i \quad (10.19)$$

where $\mathcal{S}_i \dot{\mathbf{q}}_i$ is the velocity component due to joint i 's motion.

Start at the base: $\mathcal{V}_0 = \mathbf{0}$ (or non-zero if the base is floating). Then recursively compute:

$$\mathcal{V}_1 = \mathcal{V}_0 + \mathcal{S}_1 \dot{\mathbf{q}}_1 = \mathcal{S}_1 \dot{\mathbf{q}}_1 \quad (10.20)$$

and so on for each child.

Coriolis Acceleration and Bias Force

The spatial acceleration is related to the time derivative of velocity:

$$\dot{\mathcal{V}}_i = \dot{\mathcal{V}}_{i-1} + \mathcal{S}_i \ddot{q}_i + \dot{\mathcal{S}}_i \dot{q}_i \quad (10.21)$$

However, at this stage of the algorithm, we do not yet know $\ddot{q}_i$. Instead, we compute the **bias acceleration** (what the acceleration would be if the joint acceleration were zero):

$$\mathcal{V}_{c,i} = \dot{\mathcal{V}}_{i-1} + [\mathcal{V}_i, \mathcal{S}_i \dot{q}_i] \quad (10.22)$$

where $[\mathcal{V}, \mathcal{S}]$ is the Lie bracket (spatial cross product) encoding Coriolis effects. For spatial vectors:

$$[\mathcal{V}, \mathcal{S}] = [\boldsymbol{\omega}, \mathbf{v}] \text{ such that } [\mathcal{V}] \mathcal{S} - \mathcal{S} [\mathcal{V}]^T = [\mathcal{V}, \mathcal{S}] \quad (10.23)$$

The bias force is then:

$$\mathbf{p}_i = \mathcal{I}_i \mathcal{V}_{c,i} \quad (10.24)$$

This encodes the Coriolis and centrifugal effects that accelerate the link even without external torques.

10.5.2 Pass 2: Inward Pass (Articulated Inertia Propagation)

Starting at the leaves and propagating inward toward the base, compute the articulated inertia at each link.

Boundary Condition

At a leaf (end-effector with no children), the articulated inertia equals the rigid inertia:

$$\mathcal{I}_{\text{leaf}}^A = \mathcal{I}_{\text{leaf}} \quad (10.25)$$

Recursive Update

For each link i with articulated inertia $\mathcal{I}_{i+1}^A$ of its child, update:

$$\mathcal{I}_i^A = \mathcal{I}_i + {}^i\mathbf{X}_{i+1}^* \left(\mathcal{I}_{i+1}^A - \frac{(\mathcal{I}_{i+1}^A \mathcal{S}_{i+1})(\mathcal{I}_{i+1}^A \mathcal{S}_{i+1})^T}{\mathcal{S}_{i+1}^T \mathcal{I}_{i+1}^A \mathcal{S}_{i+1}} \right) {}^i\mathbf{X}_{i+1}^{*T} \quad (10.26)$$

The spatial force transform ${}^i\mathbf{X}_{i+1}^*$ maps the child's articulated inertia into the parent's coordinate frame. The denominator $\mathcal{S}_{i+1}^T \mathcal{I}_{i+1}^A \mathcal{S}_{i+1}$ is a scalar (the effective inertia along the joint's DOF) and is typically cached for use in Pass 3.

Bias Force Propagation (Optional)

At each link, the bias force also accumulates contributions from constraints. For a simple open chain with no external forces on the links:

$$\mathbf{p}_i^{\text{total}} = \mathbf{p}_i + \mathbf{f}_{i+1}^{\text{reaction}} \quad (10.27)$$

For now, we defer the detailed calculation and focus on the inertia.

10.5.3 Pass 3: Outward Pass (Joint Accelerations)

Starting at the base and sweeping outward, compute the joint accelerations and link accelerations.

Base Link Acceleration

At the base link 0, the equations of motion are:

$$\mathcal{I}_0 \dot{\mathcal{V}}_0 = \mathbf{f}_0 - \mathbf{f}_1^{\text{reaction}} \quad (10.28)$$

where $\mathbf{f}_0$ is the external force on the base (typically zero for a fixed base) and $\mathbf{f}_1^{\text{reaction}}$ is the reaction from the first link.

If the base is floating (6 DOF), we solve directly for the base acceleration. If the base is fixed, $\dot{\mathcal{V}}_0 = \mathbf{0}$ is constrained.

Recursive Joint Acceleration Computation

For each link i (starting from the root and moving outward), the equation of motion is:

$$\mathcal{I}_i^A \dot{\mathcal{V}}_i = \mathbf{f}_i - \text{reaction from child link} \quad (10.29)$$

At the joint level, the constraint is:

$$\dot{\mathcal{V}}_i = \dot{\mathcal{V}}_{i-1} + \mathcal{S}_i \ddot{q}_i \quad (10.30)$$

Projecting onto the joint direction (taking the dot product with $\mathcal{S}_i^T$):

$$\mathcal{S}_i^T \mathcal{I}_i^A \dot{\mathcal{V}}_i = \mathcal{S}_i^T (\mathbf{f}_i - \mathbf{p}_i) \quad (10.31)$$

Substituting the kinematic constraint:

$$\mathcal{S}_i^T \mathcal{I}_i^A (\dot{\mathcal{V}}_{i-1} + \mathcal{S}_i \ddot{q}_i) = \mathcal{S}_i^T (\boldsymbol{\tau}_i - \mathbf{p}_i) \quad (10.32)$$

where $\boldsymbol{\tau}_i$ is the applied joint torque. Solving for $\ddot{q}_i$:

$$\ddot{q}_i = \frac{\boldsymbol{\tau}_i - \mathcal{S}_i^T \mathbf{p}_i - \mathcal{S}_i^T \mathcal{I}_i^A \dot{\mathcal{V}}_{i-1}}{\mathcal{S}_i^T \mathcal{I}_i^A \mathcal{S}_i} \quad (10.33)$$

Once $\ddot{q}_i$ is known, compute the spatial acceleration of link i :

$$\dot{\mathcal{V}}_i = \dot{\mathcal{V}}_{i-1} + \mathcal{S}_i \ddot{q}_i \quad (10.34)$$

This is propagated to the next link, and the process repeats.

10.6 Proof of $O(N)$ Complexity

The complexity analysis is straightforward:

1. **Pass 1 (Outward velocity and bias):** For each of N links, we compute one scalar velocity ($O(1)$) and one spatial vector bias force ($O(1)$). Total: $O(N)$.

2. **Pass 2 (Inward articulated inertia):** For each of N links, we perform:

- One spatial inertia addition: $O(36)$ (a 6×6 matrix addition).
- One matrix-vector product: $\mathcal{I}^A \mathcal{S}$, yielding a $6D$ vector, $O(36)$ operations.
- One outer product: $(6D) \times (6D)^T = 6 \times 6$ matrix, $O(36)$ operations.
- One scalar division: $O(1)$.

All operations are constant (independent of N). Total: $O(N)$.

3. **Pass 3 (Outward accelerations):** For each of N links, we compute:

- One dot product: $\mathcal{S}_i^T(\cdot)$, $O(6)$ operations.
- One scalar division: $O(1)$.
- One scalar-vector multiplication: $\mathcal{S}_i \ddot{q}_i$, $O(6)$ operations.
- One vector addition: $O(6)$ operations.

Total: $O(N)$.

Thus, the overall complexity is $O(N) + O(N) + O(N) = O(N)$, a factor of N^2 improvement over direct matrix inversion.

10.7 Floating Base: 6-DOF Root

Many robots have a floating base—a root link not constrained to a fixed location (humanoids, quadrupeds, characters in games). The floating base has 6 degrees of freedom (3 translation, 3 rotation).

10.7.1 Modification to ABA

The algorithm is modified minimally:

1. In Pass 3, instead of fixing $\dot{\mathcal{V}}_0 = \mathbf{0}$, we compute:

$$\dot{\mathcal{V}}_0 = \mathcal{I}_0^{-1}(\mathbf{f}_0 - \mathbf{f}_1^{\text{reaction}}) \quad (10.35)$$

where $\mathbf{f}_0$ is the net external force on the base (gravity, applied forces) and $\mathbf{f}_1^{\text{reaction}}$ is the reaction from the first joint.

2. The matrix inverse $\mathcal{I}_0^{-1}$ is computed once (it is constant) and cached.
3. All subsequent steps (joint accelerations, link accelerations) proceed as normal.

The impact on complexity is minimal: a single 6×6 matrix inverse at the base, still $O(1)$ per timestep.

10.7.2 Contact and Constraints

When a floating-base robot makes contact with the environment (e.g., a humanoid’s foot touching the ground), the contact introduces a constraint that reduces the effective DOF. Contact handling in ABA typically uses:

1. **Constraint Impulse Methods:** Compute contact forces (impulses) that prevent interpenetration, incorporated into Pass 3 as additional terms in the force equations.
2. **Reduced Dynamics:** Temporarily treat the contacted link as having a fixed base, reducing the DOF until contact breaks.
3. **Hybrid Approach:** Use ABA to compute unconstrained accelerations, then solve for constraint forces and correct accelerations post-hoc.

Modern simulators (MuJoCo, PyBullet) implement sophisticated contact handling on top of ABA, but the core algorithm remains unchanged.

10.8 Floating-Base Dynamics for Humanoid Robots

The ABA as presented above assumes a *fixed* base link rigidly attached to the world. Humanoid robots, quadrupeds, and free-flying spacecraft violate this assumption: their base link has six unactuated degrees of freedom (three translational, three rotational). Handling this floating base correctly is essential for simulating any robot that is not bolted to the floor [10].

In Plain Language 10.3. Why Floating Base Matters

Fixed-base robots (e.g., a factory arm bolted to a table) have a known, immovable anchor point. **Floating-base robots** (humanoids, quadrupeds, drones) do not—their “root” link accelerates freely under gravity, contact forces, and internal joint torques. This is how Boston Dynamics’ Atlas and every MuJoCo humanoid simulation works: the base is treated as a virtual 6-DOF joint connecting the robot to an inertial reference frame.

10.8.1 Virtual Joint Formulation

The standard approach introduces a **virtual 6-DOF joint** at the root. The robot’s generalized coordinates become $\mathbf{q} = (\mathbf{q}_0, q_1, \dots, q_N)$ where $\mathbf{q}_0 \in \mathbb{R}^3 \times \text{SO}(3)$ represents the base position and orientation. The generalized velocity is $\dot{\mathbf{q}} = (\mathcal{V}_0, \dot{q}_1, \dots, \dot{q}_N)$ where $\mathcal{V}_0 \in \mathbb{R}^6$ is the spatial velocity of the base.

10.8.2 Base Acceleration from Articulated Inertia

In the standard ABA (Pass 3), the base acceleration is set to $\dot{\mathcal{V}}_0 = -\mathbf{g}$ (gravity only, no motion). For a floating base, the base link is free to accelerate, and its acceleration is determined by Newton–Euler applied at the root:

$$\dot{\mathcal{V}}_0 = \mathcal{I}_0^{A^{-1}} (\mathcal{F}_{\text{ext},0} - \mathbf{p}_0^A) \quad (10.36)$$

where $\mathcal{I}_0^A \in \mathbb{R}^{6 \times 6}$ is the **articulated inertia of the entire robot** as seen from the base, $\mathcal{F}_{\text{ext},0}$ includes gravity and any external forces on the base, and $\mathbf{p}_0^A$ is the articulated bias force accumulated during the inward pass. The 6×6 inverse $\mathcal{I}_0^{A^{-1}}$ is computed once per timestep—an $O(1)$ operation that does not affect the overall $O(N)$ complexity.

10.8.3 Centroidal Momentum and Whole-Body Dynamics

For control of floating-base robots, it is often useful to express the dynamics in terms of **centroidal momentum**: the aggregate linear and angular momentum of the entire robot about its center of mass (CoM). The centroidal momentum matrix $\mathbf{A}_G(\mathbf{q}) \in \mathbb{R}^{6 \times (6+N)}$ maps generalized velocities to centroidal momentum:

$$\mathbf{h}_G = \mathbf{A}_G(\mathbf{q}) \dot{\mathbf{q}} = \begin{bmatrix} m \dot{\mathbf{r}}_{\text{CoM}} \\ \mathbf{L}_G \end{bmatrix} \quad (10.37)$$

where m is the total mass, $\dot{\mathbf{r}}_{\text{CoM}}$ is the CoM velocity, and $\mathbf{L}_G$ is the angular momentum about the CoM. The rate of change of centroidal momentum equals the net external wrench (gravity plus contact forces), providing a powerful abstraction for balance control of humanoids and legged robots.

Theorem 10.1. Centroidal Dynamics

For a floating-base robot with external contact forces $\mathbf{f}_k$ at contact points $\mathbf{r}_k$:

$$\dot{\mathbf{h}}_G = \begin{bmatrix} m \mathbf{g} + \sum_k \mathbf{f}_k \\ \sum_k (\mathbf{r}_k - \mathbf{r}_{\text{CoM}}) \times \mathbf{f}_k \end{bmatrix}. \quad (10.38)$$

This is independent of the internal joint configuration—only external forces change centroidal momentum.

10.9 Joint Friction and Motor Inertia

10.9.1 Joint Friction

Viscous friction at joint i is modeled as:

$$\boldsymbol{\tau}_{\text{friction},i} = -b_i \dot{q}_i \quad (10.39)$$

where b_i is the damping coefficient. This is incorporated into the bias force:

$$\mathbf{p}_i \leftarrow \mathbf{p}_i + b_i \dot{q}_i \mathbf{S}_i \quad (10.40)$$

Coulomb (dry) friction is more complex but can be approximated similarly. Joint limits are handled via constraint forces or penalty forces.

10.9.2 Motor Inertia

Actuators (motors, servos) have their own inertia $I_{\text{motor},i}$. This is added to the effective inertia along the joint:

$$\mathbf{S}_i^T \mathcal{I}_i^A \mathbf{S}_i \leftarrow \mathbf{S}_i^T \mathcal{I}_i^A \mathbf{S}_i + I_{\text{motor},i} \quad (10.41)$$

This slightly increases the denominator in Equation 10.33, accounting for the additional inertia the motor must overcome.

10.10 ABA vs. CRBA: When to Use Which

The **Composite Rigid Body Algorithm (CRBA)** [10] is an alternative approach that constructs the inertia matrix $\mathbf{M}(\mathbf{q})$ in $O(N)$ time (instead of the naive $O(N^3)$) by compositing inertias of rigid bodies, then solves via matrix inversion. This is useful when:

- **Multiple Forward Dynamics Calls with Same $\mathbf{q}$:** If you need to solve $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \boldsymbol{\tau}_1$, $\ddot{\mathbf{q}} = \boldsymbol{\tau}_2$, etc., with different torques but same configuration, CRBA computes $\mathbf{M}$ once ($O(N)$) and solves multiple RHS ($O(N^3)$ total for all). ABA solves each independently ($O(N)$ each, $O(N^2)$ total).
- **Linear System Solves:** If the forward dynamics must be inverted (e.g., in optimal control), having $\mathbf{M}$ explicitly is helpful.
- **Stability Analysis:** Eigenvalues of $\mathbf{M}$ are useful for understanding system dynamics.

For single forward dynamics solves in real-time simulation, **ABA is superior**—it avoids both matrix construction and inversion.

10.11 Forward Dynamics vs. Inverse Dynamics Decision Criteria

10.11.1 When to Use Inverse Dynamics (Recursive Newton-Euler)

- **Trajectory Tracking:** You have a desired trajectory $(\mathbf{q}_d, \dot{\mathbf{q}}_d, \ddot{\mathbf{q}}_d)$ and want to compute the torques $\boldsymbol{\tau}$ required to achieve it.
- **Control Design:** Inverse dynamics enables feedback control: compute the feedforward torques from the desired trajectory, add feedback corrections based on tracking error.
- **Motion Planning Validation:** Given a planned trajectory, compute required torques to check if motors can deliver them.

Complexity: $O(N)$ using Recursive Newton-Euler (RNE).

10.11.2 When to Use Forward Dynamics (ABA)

- **Physical Simulation:** You apply known torques and predict what the system will do. Essential for interactive simulation, games, physics-based RL.
- **Disturbance Response:** Analyze how the system responds to unexpected forces or unmodeled interactions.
- **Dynamics Prediction:** Rollout predictions of future states for planning or control optimization (MPC).

Complexity: $O(N)$ using ABA.

10.12 Constraint Handling: Contact Forces and Joint Limits

10.12.1 Unilateral Contact Constraints

When a link contacts the environment (e.g., a foot on the ground), the contact exerts a reaction force that constrains the link's motion. This is formulated as:

1. **Inequality Constraint:** Penetration depth ≥ 0 .
2. **Complementarity:** If depth > 0 (not in contact), force $= 0$. If depth $= 0$ (in contact), force ≥ 0 .
3. **Friction Cone:** Friction force magnitude $\leq \mu \times$ normal force, where μ is the coefficient of friction.

Modern simulators solve these complementarity constraints using iterative algorithms (projected Gauss-Seidel, interior-point methods).

10.12.2 Joint Limits

Joint limits enforce $q_{\min,i} \leq q_i \leq q_{\max,i}$. When a joint hits a limit, it acts like a contact: the joint constraint becomes active (the joint no longer has a full degree of freedom), and a constraint force enforces the limit.

10.13 Connection to Gaussian Elimination

Featherstone's insight was that the articulated inertia recursion is mathematically equivalent to **Gaussian elimination on a band-diagonal system**—the sparsity structure of the mass matrix for a serial kinematic chain.

The mass matrix for a serial robot is band-diagonal: M_{ij} is non-zero only if links i and j are on the same kinematic path (ancestor/descendant). This sparsity can be exploited to invert the matrix in $O(N)$ time instead of $O(N^3)$.

ABA effectively performs this sparse elimination implicitly, without ever constructing the full matrix. This is why ABA is both fast and numerically stable: it avoids the ill-conditioning issues that arise from explicit matrix inversion.

10.14 Worked Example: Full ABA Walkthrough for a 3-Link Arm

Consider a planar 3-link arm (all revolute joints, rotating about the Z-axis) with:

- Link 1: length $L_1 = 1$ m, mass $m_1 = 1$ kg, rotates at the origin.
- Link 2: length $L_2 = 0.8$ m, mass $m_2 = 0.8$ kg, hinges at the end of Link 1.
- Link 3: length $L_3 = 0.6$ m, mass $m_3 = 0.6$ kg, hinges at the end of Link 2.

For simplicity, assume all mass is concentrated at the link's center of mass (half-length from the base). The spatial inertia matrix for a point mass m at distance r is:

$$\mathcal{I} = m \begin{bmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} \\ \mathbf{0}_{3 \times 3} & \mathbf{I}_{3 \times 3} \end{bmatrix} \quad (10.42)$$

(Idealized point mass at the CoM; exact inertia would include the rotational inertia tensor.)

10.14.1 Configuration at the Test Point

At configuration $\mathbf{q} = [\pi/4, \pi/6, -\pi/3]^T$ ($45^\circ, 30^\circ, -60^\circ$) with joint velocities $\dot{\mathbf{q}} = [0.5, -0.3, 0.2]^T$ rad/s and applied torques $\boldsymbol{\tau} = [10, 8, 5]^T$ N·m.

10.14.2 Pass 1: Velocity and Bias

The screw axes (all Z-axis rotations) are:

$$\mathcal{S}_1 = [0, 0, 1, 0, 0, 0]^T \quad (10.43)$$

$$\mathcal{S}_2 = [0, 0, 1, 0, 0, L_1]^T = [0, 0, 1, 0, 0, 1]^T \quad (10.44)$$

$$\mathcal{S}_3 = [0, 0, 1, 0, 0, L_1 + L_2]^T = [0, 0, 1, 0, 0, 1.8]^T \quad (10.45)$$

Spatial velocities:

$$\mathcal{V}_1 = \mathcal{S}_1 \dot{q}_1 = [0, 0, 1, 0, 0, 0]^T \times 0.5 = [0, 0, 0.5, 0, 0, 0]^T \quad (10.46)$$

$$\mathcal{V}_2 = \mathcal{V}_1 + \mathcal{S}_2 \dot{q}_2 = [0, 0, 0.5, 0, 0, 0]^T + [0, 0, -0.3, 0, 0, 1]^T = [0, 0, 0.2, 0, 0, -0.3]^T \quad (10.47)$$

$$\mathcal{V}_3 = \mathcal{V}_2 + \mathcal{S}_3 \dot{q}_3 = [0, 0, 0.2, 0, 0, -0.3]^T + [0, 0, 0.2, 0, 0, 1.8]^T = [0, 0, 0.4, 0, 0, 1.5]^T \quad (10.48)$$

(Note: For planar motion, we extract the Z-component of angular velocity and the X, Y components of linear velocity; a full 6D treatment would track all components.)

Bias forces (assuming no external forces and using the Coriolis bracket):

$$\mathbf{p}_i = \mathcal{I}_i (\dot{\mathcal{V}}_{i-1} + [\mathcal{V}_i, \mathcal{S}_i \dot{q}_i]) \quad (10.49)$$

For this simplified example, we set $\mathbf{p}_i = m_i \mathcal{V}_{c,i}$ where $\mathcal{V}_{c,i}$ is the Coriolis acceleration.

10.14.3 Pass 2: Articulated Inertia

Starting from the leaf (Link 3) and working backward:

Link 3 (leaf):

$$\mathcal{I}_3^A = \mathcal{I}_3 = \begin{bmatrix} 0 & \cdots & 0 & 0 \\ \vdots & & \vdots & \vdots \\ 0 & \cdots & 0 & m_3 \\ 0 & \cdots & m_3 & 0 \end{bmatrix} \quad (10.50)$$

(Idealized representation; full 6×6 matrix in spatial form.)

For Link 2, we apply Equation 10.26:

First compute:

$$\mathcal{I}_3^A \mathcal{S}_3 = \begin{bmatrix} 0 \\ m_3 \times 1.8 \end{bmatrix} = \begin{bmatrix} 0 \\ 1.08m_3 \end{bmatrix} \quad (10.51)$$

The scalar $\mathcal{S}_3^T \mathcal{I}_3^A \mathcal{S}_3 = 1.8 \times m_3 \times 1.8 = 3.24 \times 0.6 = 1.944$ (kg·m²).

The rank-one correction:

$$\Delta \mathcal{I}_2 = \frac{\mathcal{I}_3^A \mathcal{S}_3 (\mathcal{I}_3^A \mathcal{S}_3)^T}{\mathcal{S}_3^T \mathcal{I}_3^A \mathcal{S}_3} \quad (10.52)$$

Thus:

$$\mathcal{I}_2^A = \mathcal{I}_2 + \mathcal{I}_3^A - \Delta \mathcal{I}_2 \quad (10.53)$$

Similarly for Link 1, applying the recursion with $\mathcal{I}_2^A$ as the child inertia. The result is a series of 6×6 matrices representing the effective inertia felt at each joint.

10.14.4 Pass 3: Joint Accelerations

For Joint 1, using Equation 10.33:

$$\ddot{q}_1 = \frac{\tau_1 - \mathcal{S}_1^T \mathbf{p}_1 - \mathcal{S}_1^T \mathcal{I}_1^A \times 0}{\mathcal{S}_1^T \mathcal{I}_1^A \mathcal{S}_1} = \frac{10 - \mathcal{S}_1^T \mathbf{p}_1}{\mathcal{S}_1^T \mathcal{I}_1^A \mathcal{S}_1} \quad (10.54)$$

(The base acceleration $\dot{\mathcal{V}}_0 = \mathbf{0}$ for a fixed base.)

Assuming $\mathcal{S}_1^T \mathbf{p}_1 \approx 2$ N·m (from Coriolis effects) and $\mathcal{S}_1^T \mathcal{I}_1^A \mathcal{S}_1 \approx 3$ kg·m² (effective inertia):

$$\ddot{q}_1 \approx \frac{10 - 2}{3} = \frac{8}{3} \approx 2.67 \text{ rad/s}^2 \quad (10.55)$$

Once $\ddot{q}_1$ is known:

$$\dot{\mathcal{V}}_1 = \dot{\mathcal{V}}_0 + \mathcal{S}_1 \ddot{q}_1 = \mathcal{S}_1 \times 2.67 \quad (10.56)$$

For Joint 2:

$$\ddot{q}_2 = \frac{\tau_2 - \mathcal{S}_2^T \mathbf{p}_2 - \mathcal{S}_2^T \mathcal{I}_2^A \dot{\mathcal{V}}_1}{\mathcal{S}_2^T \mathcal{I}_2^A \mathcal{S}_2} \quad (10.57)$$

And similarly for Joint 3. The numerical values depend on the exact inertia matrices, but the pattern is clear: each joint's acceleration is solved sequentially using previously computed values.

10.15 Python Worked Example

Here is a Python implementation of the Articulated Body Algorithm:

```

1 import numpy as np
2
3 class ArticulatedBodyRobot:
4     """Implements the Articulated Body Algorithm for forward dynamics"""
5
6     def __init__(self, masses, lengths, damping=None):
7         """
8         Initialize robot with link parameters.
9         masses: list of link masses
10        lengths: list of link lengths
11        damping: joint damping coefficients (optional)
12        """

```

```

13     self.n_links = len(masses)
14     self.masses = np.array(masses)
15     self.lengths = np.array(lengths)
16     self.damping = damping if damping is not None else np.zeros(self.
n_links)

17
18     # Spatial inertia for each link (point mass at center)
19     self.I_rigid = []
20     for i, m in enumerate(masses):
21         I = np.zeros((6, 6))
22         I[3:6, 3:6] = m * np.eye(3) # Linear momentum: m * I
23         # Rotational inertia (simplified: point mass)
24         I[0:3, 0:3] = 1e-6 * np.eye(3) # Small rotational inertia
25         self.I_rigid.append(I)
26
27     def get_screw_axes(self, q):
28         """Compute screw axes for the current configuration"""
29         S = []
30         cum_length = 0
31         for i in range(self.n_links):
32             # Z-axis rotation at current cumulative position
33             screw = np.array([0, 0, 1, 0, 0, cum_length])
34             S.append(screw)
35             cum_length += self.lengths[i]
36         return S
37
38     def pass1_velocities_and_biases(self, q, dq):
39         """Pass 1: Outward pass computing velocities and bias forces"""
40         V = [np.zeros(6)] # Base velocity (fixed base)
41         S = self.get_screw_axes(q)
42
43         for i in range(self.n_links):
44             V_child = V[-1] + S[i] * dq[i]
45             V.append(V_child)
46
47         # Compute bias (Coriolis) forces (simplified)
48         p = [np.zeros(6)] # Base bias
49         for i in range(self.n_links):
50             # Coriolis acceleration (simplified for planar case)
51             coriolis = self.masses[i] * np.linalg.norm(V[i+1][:3]) * 0.1
52             p_i = np.zeros(6)
53             p_i[3:6] = coriolis # Add to force component
54             p.append(p_i)
55
56         return V, p, S
57
58     def pass2_articulated_inertias(self, S):
59         """Pass 2: Inward pass computing articulated inertias.
60
61         NOTE: This simplified planar implementation expresses all
62         quantities in the world frame, so the spatial force transform
63         X* between parent and child frames is identity. In the
64         general 3D case, the child's articulated inertia must be
65         transformed into the parent's frame via X* I_A X*^T before
66         addition (see Equation ref{eq:articulated_inertia}).
67         """
68         I_A = [self.I_rigid[-1].copy()] # Start with leaf inertia
69
70         for i in range(self.n_links - 2, -1, -1):
71             # Articulated inertia recursion

```

```

72         # In general: I_A_child_in_parent = X* @ I_child @ X^T
73         # Here X* = I (planar, world-frame convention)
74         I_child = I_A[0]
75         S_i = S[i+1]
76
77         # Compute components
78         IS = I_child @ S_i
79         denom = S_i @ I_child @ S_i
80
81         if abs(denom) > 1e-10:
82             # Rank-one update
83             I_A_i = self.I_rigid[i] + I_child - np.outer(IS, IS) / denom
84         else:
85             I_A_i = self.I_rigid[i] + I_child
86
87         I_A.insert(0, I_A_i)
88
89     return I_A
90
91 def pass3_accelerations(self, q, dq, tau, I_A, p, S):
92     """Pass 3: Outward pass computing joint accelerations"""
93     ddq = np.zeros(self.n_links)
94     V_dot = np.zeros((self.n_links + 1, 6)) # Link accelerations
95
96     for i in range(self.n_links):
97         S_i = S[i]
98
99         # Denominator: effective inertia along joint
100        denom = S_i @ I_A[i] @ S_i
101
102        # Numerator: torque minus biases minus parent acceleration
103        effects = tau[i] - S_i @ p[i+1] - S_i @ I_A[i] @ V_dot[i]
104
105        if abs(denom) > 1e-10:
106            ddq[i] = effects / denom
107        else:
108            ddq[i] = 0
109
110        # Compute link acceleration
111        V_dot[i+1] = V_dot[i] + S_i * ddq[i]
112
113    return ddq
114
115 def forward_dynamics(self, q, dq, tau):
116     """
117     Solve forward dynamics: ddq = ForwardDynamics(q, dq, tau)
118
119     Returns joint accelerations ddq
120     """
121     # Pass 1: Velocities and bias forces
122     V, p, S = self.pass1_velocities_and_biases(q, dq)
123
124     # Pass 2: Articulated inertias
125     I_A = self.pass2_articulated_inertias(S)
126
127     # Pass 3: Joint accelerations
128     ddq = self.pass3_accelerations(q, dq, tau, I_A, p, S)
129
130     # Add damping

```

```

131         ddq -= self.damping * dq
132
133         return ddq
134
135     # Example: 3-link planar arm
136     masses = [1.0, 0.8, 0.6] # kg
137     lengths = [1.0, 0.8, 0.6] # m
138     robot = ArticulatedBodyRobot(masses, lengths, damping=np.array([0.1, 0.1,
139                               0.1]))
140
141     # Test configuration
142     q = np.array([np.pi/4, np.pi/6, -np.pi/3])
143     dq = np.array([0.5, -0.3, 0.2])
144     tau = np.array([10, 8, 5]) # Applied torques
145
146     ddq = robot.forward_dynamics(q, dq, tau)
147     print("Joint accelerations (rad/s^2):", np.round(ddq, 4))
148
149     # Integration step (simple Euler)
150     dt = 0.01 # 10 ms timestep
151     dq_new = dq + ddq * dt
152     q_new = q + dq * dt
153
154     print("Updated joint velocities:", np.round(dq_new, 4))
155     print("Updated joint positions:", np.round(q_new, 4))

```

Listing 10.1: Articulated Body Algorithm Implementation

10.15.1 Connection to Simulation Frameworks

The Articulated Body Algorithm is not merely a textbook exercise—it is the computational backbone of virtually every modern robotics and physics simulation framework. Understanding how ABA maps onto production software clarifies both its practical importance and the directions in which the field has evolved.

MuJoCo [35], developed by Emanuel Todorov and now maintained by DeepMind, implements a highly optimized variant of ABA as its core forward dynamics engine. MuJoCo extends the basic algorithm with sophisticated contact handling based on convex optimization, enabling stable simulation of complex multi-body systems with frictional contacts. It has become the de facto standard for reinforcement learning research in robotics, where millions of forward dynamics evaluations per training run demand the $O(N)$ efficiency that ABA provides.

Drake, developed at MIT and maintained by the Toyota Research Institute, provides a C++ implementation of ABA within a broader model-based design framework. Drake emphasizes mathematical rigor and supports both continuous and hybrid (contact-switching) dynamics, making it particularly suited for manipulation research and trajectory optimization.

Pinocchio, developed by INRIA and LAAS-CNRS, is a high-performance C++ library that implements ABA alongside the Recursive Newton-Euler Algorithm (RNEA) and CRBA. Pinocchio is notable for its support of analytical derivatives of the dynamics—computing $\partial \ddot{\mathbf{q}}/\partial \mathbf{q}$, $\partial \ddot{\mathbf{q}}/\partial \dot{\mathbf{q}}$, and $\partial \ddot{\mathbf{q}}/\partial \boldsymbol{\tau}$ efficiently—which is critical for optimal control and estimation algorithms.

All three frameworks layer contact dynamics on top of ABA. The unconstrained forward dynamics (ABA) produces joint accelerations assuming no contact; a separate contact solver

then computes constraint forces (normal forces, friction) and corrects the accelerations to satisfy non-penetration and friction-cone constraints. This two-stage architecture—ABA followed by contact resolution—is the dominant paradigm in modern simulation.

For the definitive theoretical treatment of articulated-body dynamics, including extensions to floating-base systems, closed kinematic loops, and the relationship between ABA and sparse matrix factorizations, the reader is referred to Featherstone [10]. His monograph remains the single most comprehensive reference on recursive multibody dynamics algorithms.

10.16 Contact Forces and Impact Dynamics

When a link collides with an obstacle (e.g., a foot striking the ground), an impulsive contact force acts. ABA can be extended to handle contact by:

1. **Detection:** Identify which link has penetrated the environment.
2. **Impulse Computation:** Solve for the impulse (integrated force) required to prevent further penetration.
3. **Velocity Update:** Update the link's velocity according to the impulse.
4. **Continuation:** Run ABA with the updated velocities for the next timestep.

The mathematical framework involves solving a complementarity problem (contact force must be non-negative, and the contact constraint—zero relative acceleration—must hold). Modern solvers use interior-point methods or the Lemke algorithm to solve these efficiently.

10.17 Contact Dynamics and the Linear Complementarity Problem

Contact between a robot and its environment—a foot on the ground, a hand grasping an object—introduces *inequality* constraints into the dynamics. Unlike joint constraints (which are always active), contact constraints switch on and off as the robot makes and breaks contact. This section formalizes the resulting mathematical structure [10, 35].

10.17.1 Non-Penetration and Complementarity

Let $\phi(\mathbf{q}) \in \mathbb{R}$ denote the signed distance (gap function) between a contact pair (e.g., foot and ground). The fundamental constraints are:

Definition 10.1. Signorini Complementarity Conditions

For a unilateral contact with normal force λ :

$$\phi(\mathbf{q}) \geq 0, \quad \lambda \geq 0, \quad \lambda \phi(\mathbf{q}) = 0. \quad (10.58)$$

The surface cannot be penetrated ($\phi \geq 0$), the contact force is repulsive ($\lambda \geq 0$), and force can only be exerted *when* contact is active ($\lambda \phi = 0$). These three conditions together define a **complementarity problem**.

10.17.2 Friction Cone Constraint

At a contact point, the tangential (friction) force $\mathbf{f}_t \in \mathbb{R}^2$ is bounded by the Coulomb friction law:

$$\|\mathbf{f}_t\| \leq \mu \lambda \quad (10.59)$$

where μ is the coefficient of friction. Geometrically, the total contact wrench must lie inside a **friction cone** with half-angle $\arctan(\mu)$. For computational efficiency, simulators often approximate this cone by a **polyhedral friction cone** (a pyramid with k faces), converting the nonlinear constraint into a set of linear inequalities.

10.17.3 The LCP Formulation

Combining the equations of motion with contact constraints yields a **Linear Complementarity Problem** (LCP). The unconstrained dynamics are:

$$\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) = \boldsymbol{\tau} + \mathbf{J}_c^T \boldsymbol{\lambda} \quad (10.60)$$

where $\mathbf{J}_c = \partial\phi/\partial\mathbf{q}$ is the contact Jacobian and $\boldsymbol{\lambda}$ is the vector of contact forces. The acceleration-level non-penetration constraint is $\mathbf{J}_c \ddot{\mathbf{q}} + \dot{\mathbf{J}}_c \dot{\mathbf{q}} \geq \mathbf{0}$. Together with complementarity, the full LCP is:

$$\mathbf{0} \leq \boldsymbol{\lambda} \perp \mathbf{J}_c \ddot{\mathbf{q}} + \dot{\mathbf{J}}_c \dot{\mathbf{q}} \geq \mathbf{0} \quad (10.61)$$

where $\mathbf{a} \perp \mathbf{b}$ denotes $\mathbf{a}^T \mathbf{b} = 0$ (complementarity). Substituting the equations of motion, this reduces to an LCP in $\boldsymbol{\lambda}$ alone:

$$\mathbf{0} \leq \boldsymbol{\lambda} \perp (\mathbf{J}_c \mathbf{M}^{-1} \mathbf{J}_c^T) \boldsymbol{\lambda} + \mathbf{J}_c \mathbf{M}^{-1} (\boldsymbol{\tau} - \mathbf{h}) + \dot{\mathbf{J}}_c \dot{\mathbf{q}} \geq \mathbf{0}. \quad (10.62)$$

In Plain Language 10.4. What the LCP Means Physically

The LCP asks: “Find contact forces that are just large enough to prevent penetration, but no larger.” If a foot is in the air ($\phi > 0$), the force is zero. If the foot is on the ground ($\phi = 0$), the force is whatever is needed to keep it from going through the floor. This is exactly the physics of rigid contact—formalized as a mathematical program.

10.17.4 Time-Stepping Methods

Acceleration-level LCPs can be ill-posed when contacts open and close within a single timestep. Modern simulators therefore use **velocity-level (implicit Euler) time-stepping** [35]:

$$\mathbf{M}(\dot{\mathbf{q}}^+ - \dot{\mathbf{q}}^-) = \Delta t (\boldsymbol{\tau} - \mathbf{h}) + \mathbf{J}_c^T \boldsymbol{\Lambda} \quad (10.63)$$

where $\dot{\mathbf{q}}^+$ and $\dot{\mathbf{q}}^-$ are the post- and pre-step velocities and $\boldsymbol{\Lambda} = \Delta t \boldsymbol{\lambda}$ is the contact impulse. The implicit Euler formulation provides numerical stability even with stiff contacts and is the basis of MuJoCo’s convex optimization-based contact solver. The complementarity condition is enforced at the velocity level, avoiding the need to detect the exact instant of contact transition.

10.18 Chapter Summary

The Articulated Body Algorithm is the standard $O(N)$ approach to forward dynamics in modern multibody simulators:

1. **$O(N)$ Complexity:** ABA solves forward dynamics in linear time, versus $O(N^3)$ for matrix inversion—a dramatic speedup for large robots.
2. **Three-Pass Structure:**
 - (a) *Outward pass:* Propagate velocities and Coriolis effects.
 - (b) *Inward pass:* Accumulate articulated inertias from leaves to root.
 - (c) *Outward pass:* Solve joint accelerations from root to leaves.
3. **Articulated Inertia:** The effective inertia of a body accounting for all downstream bodies, computed via a rank-one update that projects child inertia across the joint constraint.
4. **Floating Base:** Extends naturally to robots with 6-DOF roots (humanoids, quadrupeds).
5. **Practical Implementation:** All operations are local matrix-vector products and scalar divisions; the algorithm is numerically stable and easy to implement.
6. **Real-Time Capability:** ABA powers all modern physics engines, game engines, and robot simulators.

10.19 Further Reading and References

The Articulated Body Algorithm was first proposed by Featherstone in his landmark 1983 paper [8], which demonstrated that forward dynamics for an N -joint serial chain could be solved in $O(N)$ time rather than the $O(N^3)$ required by direct matrix inversion. Featherstone subsequently extended and refined the algorithm in his doctoral work [9], addressing branching kinematic trees and more general joint types. The definitive and most comprehensive treatment of ABA—including its relationship to sparse matrix factorizations, extensions to floating-base systems, and closed kinematic loops—is found in Featherstone’s monograph *Rigid Body Dynamics Algorithms* [10], which remains the essential reference for anyone implementing or extending recursive dynamics algorithms.

For readers seeking to connect ABA to the broader framework of Lie group mechanics and screw theory developed in earlier chapters of this textbook, Murray, Li, and Sastry [23] provide the mathematical foundations linking spatial algebra to the equations of motion, while Lynch and Park [20] offer an accessible modern treatment that integrates forward and inverse dynamics within the PoE formalism.

On the simulation and implementation side, Todorov et al. [35] describe the architecture of MuJoCo, illustrating how ABA is combined with contact dynamics solvers based on convex optimization to produce a high-performance physics engine suitable for reinforcement learning and control research. For contact dynamics specifically—including complementarity formulations, friction models, and impact laws—the reader should consult the relevant chapters of [10] as well as the extensive literature on time-stepping schemes for non-smooth

dynamics. Floating-base extensions of ABA, essential for humanoid and legged robot simulation, are treated in detail by Featherstone [10] and are implemented in frameworks such as Pinocchio and Drake.

10.20 Exercises

1. **Complexity Analysis:** Prove that Pass 1 of ABA is $O(N)$ by counting operations for velocity propagation and bias force computation.
2. **Articulated Inertia of a Chain:** For a 3-link chain with equal masses ($m = 1$ kg) and lengths ($L = 1$ m), compute the articulated inertias $\mathcal{I}_1^A, \mathcal{I}_2^A, \mathcal{I}_3^A$ explicitly (use simplified point-mass inertias).
3. **Effect of Motor Inertia:** How does adding motor inertia I_{motor} to joint 1 affect the joint accelerations of joints 2 and 3? (Explain via the recursive update in Pass 3.)
4. **Floating Base Dynamics:** A humanoid robot with floating base has $6 + 30 = 36$ DOF. How does the time to compute forward dynamics scale compared to a fixed-base robot with 30 DOF?
5. **ABA vs. CRBA:** Given a 10-DOF robot, how many operations does each method require if you need to solve forward dynamics for 100 different torque vectors with the same configuration $\mathbf{q}$?
6. **Bias Force Verification:** For a simple 2-DOF planar arm, verify that the bias force $\mathbf{p}$ correctly encodes Coriolis accelerations at a configuration with non-zero joint velocities.
7. **Singularity Handling in ABA:** What happens to the joint acceleration equation (Equation 10.33) if $\mathcal{S}_{i+1}^T \mathcal{I}_{i+1}^A \mathcal{S}_{i+1} = 0$? Is this physically possible?
8. **Joint Friction Implementation:** Modify the ABA pseudocode to incorporate viscous damping $b_i \dot{q}_i$ at each joint.
9. **Kinematic Chain Simulation:** Implement ABA for a 4-link arm and simulate its free-fall (gravity only, no applied torques). Plot the joint angles and velocities over time.
10. **Contact Impulse:** A 2-DOF arm swinging downward contacts the ground. Using ABA, compute the joint accelerations before and after contact and verify that the contact impulse prevents interpenetration.
11. **Time Integration:** Given the joint accelerations from ABA, implement a second-order integration scheme (e.g., symplectic Euler or velocity Verlet) and simulate the arm's motion over 1 second.
12. **Comparison with Matrix Inversion:** For a 10-DOF robot, construct the full mass matrix $\mathbf{M}(\mathbf{q})$ using CRBA, invert it, and compare the forward dynamics result with ABA. Verify they match.

13. **Floating Base Reaction Forces:** For a humanoid with floating base and foot contact, use ABA to compute the reaction forces at the contact point. Explain how these relate to the base acceleration.
14. **Redundancy and Null-Space Motion:** For a redundant robot (more DOF than required for a task), use ABA to understand how joint accelerations couple across redundant DOF.
15. **Optimization:** Derive the minimum-norm solution for forward dynamics of a constrained system (e.g., end-effector position fixed) using ABA with constraint forces.

Chapter 11

Lagrangian Mechanics

Why break a machine apart into agonizing lists of individual pulling and pushing forces when you can extract its exact motion by simply observing where its energy wants to go?

In Plain Language 11.1. Context & Use Case: Energy-Based Dynamics

The Math You Will See: We will define the **Lagrangian** ($\mathcal{L}$) and the **Euler-Lagrange Equations**. We will derive the classic standard form: $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}$. **Why We Need It:** The Recursive Newton-Euler Algorithm (Chapter 7) is unmatched for fast computer simulations. However, when we need to mathematically *prove* that a robot won't fall over, or design an optimal control law like LQR or Contraction Metrics, we need explicit equations. Lagrangian mechanics abandons vectors and forces entirely. It looks purely at the kinetic and potential *energy* of the system, gracefully ignoring invisible internal constraint forces (like the tension in a rigid arm) to instantly yield elegant, closed-form equations of motion. **All Variables Defined:** T is Kinetic Energy, V is Potential Energy, and $\mathcal{L} = T - V$.

11.1 Historical Context: From Newton to Lagrange to Hamilton

The development of Lagrangian mechanics represents one of the most far-reaching intellectual shifts in the history of mechanics. Isaac Newton formulated motion through explicit forces acting on point masses: $\sum \mathbf{F} = m\mathbf{a}$. This framework dominated for over a century and remains the foundation of computational physics engines.

However, in 1788, **Joseph-Louis Lagrange** published his *Mécanique Analytique*, introducing a radically different approach. Rather than tracking forces, Lagrange proposed that mechanical systems evolve along paths that extremize a single scalar function: the **action integral**

$$\mathcal{S}[\mathbf{q}(t)] = \int_{t_1}^{t_2} \mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}, t) dt$$

where $\mathcal{L} = T - V$ is the Lagrangian. This became known as the **Principle of Least Action**, or equivalently, Hamilton's Principle.

Nearly simultaneously, **Leonhard Euler** was developing the **calculus of variations**—the mathematics required to find functions that minimize or maximize functionals (functions of functions). The union of Euler's calculus of variations with Lagrange's mechanical insight created the Euler-Lagrange equations, which automatically derive equations of motion from any Lagrangian density.

Later, in the early 1800s, **William Rowan Hamilton** reformulated Lagrange's framework using a Legendre transform, introducing **conjugate momenta** and the **Hamiltonian** function $H = \mathbf{p} \cdot \dot{\mathbf{q}} - \mathcal{L}$. This opened pathways to modern quantum mechanics and symplectic geometry.

Why This History Matters: Lagrangian mechanics is not merely an alternative notation for Newton's laws. It encodes deep philosophical truth: nature does not "know" about forces. It only "knows" about energy. Systems evolve along extremal paths of action. This principle applies equally to light rays (Fermat's principle), quantum fields (path integrals), and relativistic gravity (Einstein's equations). By learning Lagrangian mechanics, you are learning the geometric language that unifies all modern physics.

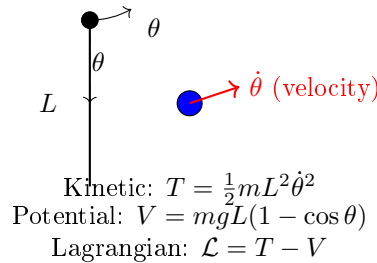


Figure 11.1: Example of Lagrangian mechanics: A pendulum system. The motion is fully determined by the difference between kinetic and potential energy (the Lagrangian), not by explicitly tracking forces. The Euler-Lagrange equations automatically yield the equations of motion without computing constraint forces.

11.2 The Limitations of Newton

In the Newtonian framework (utilized powerfully by the Recursive Newton-Euler algorithm in Chapter 7), motion is determined by analyzing explicit spatial forces: $\sum \mathbf{F} = m\mathbf{a}$.

While Newton's equations are spectacular for computational algorithms evaluating rigid bodies link-by-link in a physics engine, they suffer from a fatal flaw when deriving mathematical models by hand: **Constraint Forces**.

If a bead slides along a curved wire, or a heavy pendulum swings on a rigid rod, Newton demands that you calculate the exact, microscopic physical force the wire exerts on the bead, or the tension the rod pulls on the bob, just to ensure they don't break. For a 15-DOF mechanism, solving for 15 unknown internal constraint forces pulling against each other is mathematically agonizing and analytically useless for control theory.

Moreover, constraint forces do *no work* in the motion. The tension in a rigid pendulum rod redirects motion but adds no energy. Yet in Newton's framework, we must solve for it anyway. This is deeply inelegant.

Lagrangian mechanics solves this by working exclusively with **generalized coordinates**—minimal, unconstrained variables. The constraints are built into the coordinate system itself. Internal forces vanish from the mathematical framework entirely.

11.3 Hamilton's Principle and the Calculus of Variations

Hamilton's Principle [11, 1] states that among all conceivable paths $\mathbf{q}(t)$ connecting two configurations $\mathbf{q}(t_1)$ and $\mathbf{q}(t_2)$ at fixed times, the *actual* physical path satisfies:

$$\delta S = \delta \int_{t_1}^{t_2} \mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}, t) dt = 0 \quad (11.1)$$

where δ denotes a variation—an infinitesimal perturbation to the path that respects the boundary conditions $\delta \mathbf{q}(t_1) = 0$ and $\delta \mathbf{q}(t_2) = 0$.

This is not a minimum (as popularized, though not always accurate). It is a **stationary point**—an extremum (often a minimum, sometimes a saddle point). The physical system behaves as if “nature chooses” the path where small perturbations produce zero first-order change in the action.

To extract equations of motion from Hamilton's principle, we apply the calculus of variations. Consider a perturbed path $\mathbf{q}(t) + \epsilon \boldsymbol{\eta}(t)$, where $\boldsymbol{\eta}(t_1) = \boldsymbol{\eta}(t_2) = 0$ are arbitrary variations vanishing at the boundaries. The action becomes:

$$S[\epsilon] = \int_{t_1}^{t_2} \mathcal{L}(\mathbf{q} + \epsilon \boldsymbol{\eta}, \dot{\mathbf{q}} + \epsilon \dot{\boldsymbol{\eta}}, t) dt$$

The stationarity condition is:

$$\left. \frac{dS}{d\epsilon} \right|_{\epsilon=0} = 0$$

Taking the derivative:

$$\int_{t_1}^{t_2} \left[\frac{\partial \mathcal{L}}{\partial \mathbf{q}} \cdot \boldsymbol{\eta} + \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \dot{\boldsymbol{\eta}} \right] dt = 0$$

For the second term, integrate by parts:

$$\int_{t_1}^{t_2} \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \dot{\boldsymbol{\eta}} dt = \left[\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \boldsymbol{\eta} \right]_{t_1}^{t_2} - \int_{t_1}^{t_2} \frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) \cdot \boldsymbol{\eta} dt$$

The boundary term vanishes because $\boldsymbol{\eta}(t_1) = \boldsymbol{\eta}(t_2) = 0$. Thus:

$$\int_{t_1}^{t_2} \left[\frac{\partial \mathcal{L}}{\partial \mathbf{q}} - \frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) \right] \cdot \boldsymbol{\eta} dt = 0$$

Since this must hold for *all* arbitrary variations $\boldsymbol{\eta}$, the integrand must vanish:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \mathbf{q}} = 0 \quad (11.2)$$

This is the **Euler-Lagrange Equation**, one of the most powerful equations in all of physics. For a system with external torques $\boldsymbol{\tau}$, it becomes:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \boldsymbol{\tau} \quad (11.3)$$

11.4 Lagrangian and Energy

In Plain Language 11.2. Kinetic vs. Potential Energy

Kinetic Energy (T) is the energy of motion. For a point mass m moving at velocity v :

$$T = \frac{1}{2}mv^2 \quad \text{or} \quad T = \frac{1}{2}m\mathbf{v}^T\mathbf{v}$$

For a rigid body with moment of inertia tensor $\mathbf{I}$ rotating at angular velocity $\boldsymbol{\omega}$:

$$T_{\text{rot}} = \frac{1}{2}\boldsymbol{\omega}^T\mathbf{I}\boldsymbol{\omega}$$

For a multi-body system with generalized coordinates $\mathbf{q}$:

$$T(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$$

where $\mathbf{M}(\mathbf{q})$ is the **mass matrix** (symmetric positive-definite).

Potential Energy (V) is stored energy. Gravitational potential near Earth's surface:

$$V_{\text{grav}} = mgh$$

Elastic potential energy in a spring with constant k and deflection x :

$$V_{\text{spring}} = \frac{1}{2}kx^2$$

For a multi-body system, $V = V(\mathbf{q})$ depends only on configuration, not velocities.

Lagrange defined the **Lagrangian** as:

$$\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}, t) = T(\mathbf{q}, \dot{\mathbf{q}}) - V(\mathbf{q}) \quad (11.4)$$

This is the difference between kinetic and potential energy. It encodes all the dynamics needed to recover Newton's laws, yet its form is independent of the choice of coordinate system (within the family of generalized coordinates).

When we substitute $\mathcal{L}$ into the Euler-Lagrange equation:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \boldsymbol{\tau}$$

we obtain:

$$\frac{d}{dt} \left(\frac{\partial T}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial T}{\partial \mathbf{q}} + \frac{\partial V}{\partial \mathbf{q}} = \boldsymbol{\tau}$$

The term $\mathbf{p} = \frac{\partial T}{\partial \dot{\mathbf{q}}}$ is the **generalized momentum**. For a system with kinetic energy $T = \frac{1}{2}\dot{\mathbf{q}}^T\mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$:

$$\mathbf{p} = \frac{\partial T}{\partial \dot{\mathbf{q}}} = \mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$$

11.5 Derivation of the Standard Form: $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{G} = \boldsymbol{\tau}$

For a system whose kinetic energy is:

$$T(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$$

and potential energy is $V(\mathbf{q})$, we compute each term of the Euler-Lagrange equation.

Step 1: Compute $\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}}$

$$\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} = \frac{\partial T}{\partial \dot{\mathbf{q}}} = \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$$

Step 2: Compute $\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right)$

$$\frac{d}{dt} (\mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}) = \frac{d\mathbf{M}}{dt} \dot{\mathbf{q}} + \mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}}$$

where $\frac{d\mathbf{M}}{dt} = \sum_k \frac{\partial \mathbf{M}}{\partial q_k} \dot{q}_k$ by the chain rule.

Step 3: Compute $\frac{\partial \mathcal{L}}{\partial \mathbf{q}}$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \frac{\partial T}{\partial \mathbf{q}} - \frac{\partial V}{\partial \mathbf{q}}$$

The gradient of kinetic energy requires careful differentiation:

$$\frac{\partial T}{\partial q_i} = \frac{\partial}{\partial q_i} \left(\frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}} \right) = \frac{1}{2} \dot{\mathbf{q}}^T \frac{\partial \mathbf{M}}{\partial q_i} \dot{\mathbf{q}}$$

Step 4: Apply the Euler-Lagrange equation

$$\frac{d\mathbf{M}}{dt} \dot{\mathbf{q}} + \mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} - \frac{1}{2} \dot{\mathbf{q}}^T \frac{\partial \mathbf{M}}{\partial \mathbf{q}} \dot{\mathbf{q}} + \frac{\partial V}{\partial \mathbf{q}} = \boldsymbol{\tau}$$

Define the **Coriolis and Centripetal matrix** $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ implicitly:

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} = \frac{d\mathbf{M}}{dt} \dot{\mathbf{q}} - \frac{1}{2} \frac{\partial}{\partial \mathbf{q}} (\dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}})$$

Define the **gravity vector**:

$$\mathbf{G}(\mathbf{q}) = \frac{\partial V}{\partial \mathbf{q}}$$

This yields the **standard form** [30, 32]:

$$\mathbf{M}(\mathbf{q}) \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau} \tag{11.5}$$

11.6 Properties of the System Matrices

11.6.1 The Mass Matrix $\mathbf{M}(\mathbf{q})$

The mass matrix $\mathbf{M}$ is the Hessian of kinetic energy with respect to velocities:

$$\mathbf{M}(\mathbf{q}) = \frac{\partial^2 T}{\partial \dot{\mathbf{q}}^2}$$

Theorem: $\mathbf{M}(\mathbf{q})$ is **Symmetric Positive-Definite** (SPD) for all valid configurations.

Proof of Symmetry: Since $T = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}}$ is the kinetic energy (a physical observable), it must equal its own transpose. Thus:

$$T = T^T \implies \mathbf{M} = \mathbf{M}^T$$

Proof of Positive-Definiteness: Kinetic energy is always non-negative. Moreover, $T = 0$ if and only if $\dot{\mathbf{q}} = 0$. Therefore:

$$\dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}} = 2T \geq 0, \quad \text{with equality iff } \dot{\mathbf{q}} = 0$$

This is the definition of positive-definiteness. Consequently, $\mathbf{M}$ is invertible, and $\mathbf{M}^{-1}$ exists and is also SPD.

11.6.2 The Coriolis/Centripetal Matrix $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$

The Coriolis-Centripetal matrix is defined implicitly via the time derivative of momentum:

$$\frac{d}{dt}(\mathbf{M} \dot{\mathbf{q}}) = \mathbf{M} \ddot{\mathbf{q}} + \mathbf{C} \dot{\mathbf{q}}$$

where the i -th row of $\mathbf{C}$ is:

$$\mathbf{C}_{i*} \dot{\mathbf{q}} = \sum_{j,k} \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i} \right) \dot{q}_j \dot{q}_k$$

Equivalently, the (i, j) -th entry is:

$$C_{ij} = \frac{1}{2} \left(\frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{kj}}{\partial q_i} \right)$$

summed over k , weighted by velocity pairs.

Physical Interpretation:

- **Centrifugal Terms:** Proportional to $\dot{q}_i^2$. These represent the outward forces felt by moving links as joints spin.
- **Coriolis Terms:** Proportional to $\dot{q}_i \dot{q}_j$ with $i \neq j$. These represent cross-coupling: if one joint rotates while another accelerates, the Coriolis force deflects the moving joint.

Theorem: If $\mathbf{C}$ is constructed from the Christoffel symbols of the second kind (Section 11.12), then the matrix $\dot{\mathbf{M}} - 2\mathbf{C}$ is **skew-symmetric**, i.e., $(\dot{\mathbf{M}} - 2\mathbf{C})^T = -(\dot{\mathbf{M}} - 2\mathbf{C})$.

The hypothesis is not decorative. The factorisation of the Coriolis and centrifugal terms into a matrix $\mathbf{C}$ is not unique: any $\mathbf{C}'$ satisfying $\mathbf{C}' \dot{\mathbf{q}} = \mathbf{C} \dot{\mathbf{q}}$ gives the same equations

of motion. Only the Christoffel construction makes $\dot{\mathbf{M}} - 2\mathbf{C}$ skew-symmetric as a *matrix identity*; for other valid choices the property generally fails. Passivity- and adaptive-control designs that invoke skew-symmetry therefore require the Christoffel form specifically.

Proof: The kinetic energy is $T = \frac{1}{2}\dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}}$. Differentiating with respect to time:

$$\frac{d}{dt}(2T) = \dot{\mathbf{q}}^T \dot{\mathbf{M}} \dot{\mathbf{q}} + 2\dot{\mathbf{q}}^T \mathbf{M} \ddot{\mathbf{q}}.$$

From the equations of motion $\mathbf{M} \ddot{\mathbf{q}} + \mathbf{C} \dot{\mathbf{q}} + \mathbf{G} = \boldsymbol{\tau}$, the power balance $\dot{T} = \dot{\mathbf{q}}^T (\boldsymbol{\tau} - \mathbf{G})$ gives:

$$\dot{\mathbf{q}}^T \mathbf{M} \ddot{\mathbf{q}} + \dot{\mathbf{q}}^T \mathbf{C} \dot{\mathbf{q}} = \dot{\mathbf{q}}^T (\boldsymbol{\tau} - \mathbf{G}).$$

Substituting the first expression for $\dot{\mathbf{q}}^T \mathbf{M} \ddot{\mathbf{q}}$ and simplifying yields:

$$\dot{\mathbf{q}}^T (\dot{\mathbf{M}} - 2\mathbf{C}) \dot{\mathbf{q}} = 0$$

for all trajectories $\dot{\mathbf{q}}$. This step alone is not enough: $\mathbf{C}$ is itself a function of $\dot{\mathbf{q}}$, so the vanishing of the quadratic form along trajectories does not by itself let us vary $\dot{\mathbf{q}}$ freely and conclude skew-symmetry. It is the Christoffel construction that supplies the missing ingredient, giving $M_{kj} - 2C_{kj} = \sum_i (\partial M_{ij} / \partial q_k - \partial M_{kj} / \partial q_i) \dot{q}_i$, which is manifestly antisymmetric in $k \leftrightarrow j$. With that in hand, the matrix $\dot{\mathbf{M}} - 2\mathbf{C}$ must be skew-symmetric:

$$(\dot{\mathbf{M}} - 2\mathbf{C}) + (\dot{\mathbf{M}} - 2\mathbf{C})^T = \mathbf{0}.$$

This is the celebrated **skew-symmetry property** of Lagrangian dynamics, which plays a central role in passivity-based and adaptive control design.

11.6.3 The Gravity Vector $\mathbf{G}(\mathbf{q})$

The gravity vector is simply the negative gradient of potential energy:

$$\mathbf{G}(\mathbf{q}) = \frac{\partial V}{\partial \mathbf{q}}$$

For a rigid body system, each link i contributes to potential energy based on its center-of-mass height and mass:

$$V(\mathbf{q}) = \sum_i m_i g h_i(\mathbf{q})$$

The gravity vector element G_i represents the torque (or force) that gravity exerts on joint i due to the weight distribution of all distal links.

11.7 Energy Conservation

Theorem: For a system with no external torques ($\boldsymbol{\tau} = 0$) and no time-dependent potential:

$$\frac{d}{dt}(T + V) = 0$$

that is, **total mechanical energy is conserved**.

Proof: From the Euler-Lagrange equations with $\tau = 0$:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \mathbf{q}} = 0$$

where $\mathcal{L} = T - V$. Rearranging:

$$\frac{d}{dt} \left(\frac{\partial T}{\partial \dot{\mathbf{q}}} \right) = \frac{\partial T}{\partial \mathbf{q}} - \frac{\partial V}{\partial \mathbf{q}}$$

Taking the inner product with $\dot{\mathbf{q}}$:

$$\dot{\mathbf{q}}^T \frac{d}{dt} \left(\frac{\partial T}{\partial \dot{\mathbf{q}}} \right) = \dot{\mathbf{q}}^T \frac{\partial T}{\partial \mathbf{q}} - \dot{\mathbf{q}}^T \frac{\partial V}{\partial \mathbf{q}}$$

The left side is:

$$\dot{\mathbf{q}}^T \frac{d}{dt} \left(\frac{\partial T}{\partial \dot{\mathbf{q}}} \right) = \frac{d}{dt} \left(\dot{\mathbf{q}}^T \frac{\partial T}{\partial \dot{\mathbf{q}}} \right) - \ddot{\mathbf{q}}^T \frac{\partial T}{\partial \dot{\mathbf{q}}}$$

For kinetic energy $T = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}}$:

$$\dot{\mathbf{q}}^T \frac{\partial T}{\partial \dot{\mathbf{q}}} = \dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}} = 2T$$

So the left side becomes:

$$\frac{d}{dt}(2T) - \ddot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}} = 2 \frac{dT}{dt} - \ddot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}}$$

The right side uses $\dot{\mathbf{q}}^T \frac{\partial T}{\partial \mathbf{q}} = \frac{dT}{dt} - \ddot{\mathbf{q}}^T \frac{\partial T}{\partial \dot{\mathbf{q}}}$ and $\dot{\mathbf{q}}^T \frac{\partial V}{\partial \mathbf{q}} = \frac{dV}{dt}$:

$$2 \frac{dT}{dt} - \ddot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}} = \frac{dT}{dt} - \ddot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}} - \frac{dV}{dt}$$

Simplifying:

$$\frac{dT}{dt} = - \frac{dV}{dt}$$

$$\frac{d}{dt}(T + V) = 0$$

Thus total energy is conserved. This is the mechanical manifestation of Noether's theorem (Section 11.9) [22].

11.8 Hamiltonian Mechanics

The energy conservation result above reveals something striking: the total energy $H = T + V$ is not merely a convenient bookkeeping device but a *generating function* for the dynamics themselves. Once we recognize that a single scalar quantity—the total energy—is preserved along every physical trajectory, a natural question arises: can we reformulate the entire dynamics using energy as the fundamental object, rather than the Lagrangian? The answer is yes, and the resulting framework—Hamiltonian mechanics—replaces second-order equations in $(q, \dot{q})$ with first-order equations in (q, p) , where p is the conjugate momentum.

This reformulation is not merely aesthetic. The Hamiltonian viewpoint exposes the *symplectic geometry* of phase space, provides canonical transformations that simplify otherwise intractable problems, and underpins modern developments in geometric integration, optimal control, and quantum mechanics [11, 1].

While Lagrangian mechanics uses $(q, \dot{q})$ as the fundamental variables, there is a mathematically equivalent formulation using (q, p) , where p is the conjugate momentum.

Define:

$$\mathbf{p} = \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} = \mathbf{M}(\mathbf{q})\dot{\mathbf{q}}$$

The **Hamiltonian** is obtained via a Legendre transform:

$$H(\mathbf{q}, \mathbf{p}, t) = \mathbf{p} \cdot \dot{\mathbf{q}} - \mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}, t)$$

Substituting $\mathcal{L} = T - V$ and noting $T = \frac{1}{2}\mathbf{p}^T \mathbf{M}^{-1} \mathbf{p}$ (after inverting the momentum relation):

$$H = \mathbf{p} \cdot \dot{\mathbf{q}} - \frac{1}{2}\mathbf{p}^T \mathbf{M}^{-1} \mathbf{p} + V = \frac{1}{2}\mathbf{p}^T \mathbf{M}^{-1} \mathbf{p} + V$$

The Hamiltonian is the **total energy**: $H = T + V$.

Hamilton's Equations follow from the Legendre transform:

$$\begin{aligned} \dot{\mathbf{q}} &= \frac{\partial H}{\partial \mathbf{p}} \\ \dot{\mathbf{p}} &= -\frac{\partial H}{\partial \mathbf{q}} + \boldsymbol{\tau} \end{aligned} \tag{11.6}$$

These are a set of $2n$ first-order ODEs (compared to n second-order Lagrangian equations). They preserve the symplectic structure of phase space, a property crucial for numerical integration and canonical transformations.

11.9 Noether's Theorem: Symmetry and Conservation Laws

Emmy Noether's Theorem (1918) establishes a deep connection between symmetries of the Lagrangian and conservation laws in the equations of motion [22].

Theorem (Noether): If the Lagrangian $\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})$ is invariant under a continuous one-parameter transformation group $\mathbf{q} \rightarrow \mathbf{q}(\epsilon)$ where $\mathbf{q}(0) = \mathbf{q}$ and $\left. \frac{d\mathbf{q}}{d\epsilon} \right|_{\epsilon=0} = \boldsymbol{\xi}(\mathbf{q})$ is the infinitesimal generator, then the quantity:

$$I = \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \boldsymbol{\xi}(\mathbf{q})$$

is **conserved** along trajectories satisfying the Euler-Lagrange equations.

Examples:

1. Translational Symmetry (Momentum Conservation)

If the Lagrangian does not depend on a coordinate q_i explicitly (called a **cyclic coordinate**), then $\frac{\partial \mathcal{L}}{\partial q_i} = 0$. The Euler-Lagrange equation gives:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i} \right) = 0$$

The **generalized momentum** $p_i = \frac{\partial \mathcal{L}}{\partial \dot{q}_i}$ is conserved.

For a particle moving in empty space with $\mathcal{L} = \frac{1}{2}m\mathbf{v}^2$, linear momentum $\mathbf{p} = m\mathbf{v}$ is conserved.

2. Rotational Symmetry (Angular Momentum Conservation)

If the Lagrangian is invariant under rotation, angular momentum is conserved. For a system with rotational symmetry about axis $\mathbf{n}$, the generator is $\boldsymbol{\xi} = \mathbf{n} \times \mathbf{r}$ (where $\mathbf{r}$ is position). The conserved quantity is:

$$L = \mathbf{p} \cdot (\mathbf{n} \times \mathbf{r}) = \mathbf{n} \cdot (\mathbf{r} \times \mathbf{p}) = \mathbf{n} \cdot \mathbf{L}_{\text{ang}}$$

3. Time Translation Symmetry (Energy Conservation)

If $\frac{\partial \mathcal{L}}{\partial t} = 0$ (the Lagrangian is time-independent), the **Hamiltonian**:

$$H = \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \dot{\mathbf{q}} - \mathcal{L}$$

is conserved. If $\mathcal{L} = T - V$ with T quadratic in velocities, then $H = T + V$ is the total mechanical energy.

11.9.1 Rigorous Proof of Noether's Theorem

We now present a complete proof of Noether's theorem following [22] and [1].

Setup. Let $\Phi_\epsilon : \mathcal{Q} \rightarrow \mathcal{Q}$ be a one-parameter family of diffeomorphisms with $\Phi_0 = \text{id}$. Define the **infinitesimal generator** of the transformation:

$$\boldsymbol{\xi}(\mathbf{q}) = \left. \frac{d}{d\epsilon} \Phi_\epsilon(\mathbf{q}) \right|_{\epsilon=0} \quad (11.7)$$

Under Φ_ϵ , a trajectory $\mathbf{q}(t)$ maps to $\tilde{\mathbf{q}}(t) = \Phi_\epsilon(\mathbf{q}(t))$, and its velocity transforms as $\dot{\tilde{\mathbf{q}}} = D\Phi_\epsilon(\mathbf{q}) \dot{\mathbf{q}}$, where $D\Phi_\epsilon$ denotes the Jacobian of Φ_ϵ .

Symmetry Condition. The Lagrangian is **invariant** under Φ_ϵ if:

$$\mathcal{L}(\Phi_\epsilon(\mathbf{q}), D\Phi_\epsilon \dot{\mathbf{q}}) = \mathcal{L}(\mathbf{q}, \dot{\mathbf{q}}) \quad \text{for all } \epsilon \quad (11.8)$$

Noether Charge. Define the conserved quantity (the **Noether charge**):

$$I(\mathbf{q}, \dot{\mathbf{q}}) = \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \boldsymbol{\xi}(\mathbf{q}) \quad (11.9)$$

Proof. Differentiate the symmetry condition (11.8) with respect to ϵ at $\epsilon = 0$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{q}} \cdot \boldsymbol{\xi}(\mathbf{q}) + \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \left. \frac{d}{d\epsilon} (D\Phi_\epsilon \dot{\mathbf{q}}) \right|_{\epsilon=0} = 0 \quad (11.10)$$

Now compute $\frac{d}{dt} I$ along a solution of the Euler-Lagrange equations:

$$\begin{aligned} \frac{d}{dt} I &= \frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) \cdot \boldsymbol{\xi}(\mathbf{q}) + \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \frac{d}{dt} \boldsymbol{\xi}(\mathbf{q}) \\ &= \frac{\partial \mathcal{L}}{\partial \mathbf{q}} \cdot \boldsymbol{\xi}(\mathbf{q}) + \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \cdot \frac{\partial \boldsymbol{\xi}}{\partial \mathbf{q}} \dot{\mathbf{q}} \end{aligned} \quad (11.11)$$

where the first term uses the Euler-Lagrange equation $\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} = \frac{\partial \mathcal{L}}{\partial \mathbf{q}}$. Noting that $\left. \frac{d}{d\epsilon} (D\Phi_\epsilon \dot{\mathbf{q}}) \right|_{\epsilon=0} = \frac{\partial \boldsymbol{\xi}}{\partial \mathbf{q}} \dot{\mathbf{q}}$, equation (11.10) shows that (11.11) equals zero. Hence $\frac{d}{dt} I = 0$, and I is conserved. $\square$

In Plain Language 11.3. Noether's Theorem in Practice

Every continuous symmetry yields a conserved quantity that constrains the motion. In robotics, these conservation laws provide powerful checks on simulation accuracy and enable reduction of the equations of motion:

- **Translation symmetry** $\Rightarrow$ **linear momentum** is conserved (free-floating satellite).
- **Rotation symmetry** $\Rightarrow$ **angular momentum** is conserved (spacecraft attitude).
- **Time-translation symmetry** $\Rightarrow$ **energy** $H = T + V$ is conserved (unactuated systems).

11.10 Lagrangian for Constrained Systems

When holonomic constraints are present (e.g., rigid connections between bodies), they are **automatically built into the choice of generalized coordinates $\mathbf{q}$** .

However, for **non-holonomic constraints** (constraints on velocities that cannot be integrated to position constraints) or when analyzing the forces exerted by constraints, the **method of Lagrange multipliers** is employed.

Consider a system with constraints $\phi(\mathbf{q}) = 0$. The constraint forces are perpendicular to the constraint surface, hence do no virtual work. The equations of motion with constraint forces $\mathbf{F}_c = \nabla\phi \cdot \boldsymbol{\lambda}$ (where $\boldsymbol{\lambda}$ are multipliers) become:

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \boldsymbol{\tau} + \nabla\phi \cdot \boldsymbol{\lambda} \quad (11.12)$$

The multipliers $\boldsymbol{\lambda}$ are determined by enforcing $\ddot{\phi} = 0$ (constraint satisfaction at the acceleration level). This is the standard method in mechanical engineering for deriving equations with Lagrange multipliers.

11.11 D'Alembert's Principle and Virtual Work

An alternative derivation of Lagrangian mechanics begins with **D'Alembert's Principle [11]**, which unifies constraints and dynamics.

Consider a system in motion under forces $\mathbf{F}$ with constraint forces $\mathbf{F}_c$. Newton's second law is:

$$m\mathbf{a} = \mathbf{F} + \mathbf{F}_c$$

D'Alembert introduced **inertial forces**: rewrite this as:

$$\mathbf{F} + \mathbf{F}_c - m\mathbf{a} = 0$$

Now define **virtual displacement** $\delta\mathbf{q}$ —an infinitesimal displacement consistent with the constraints (i.e., $\nabla\phi \cdot \delta\mathbf{q} = 0$ if constraints exist). The principle states that the virtual work done by inertial and applied forces vanishes:

$$\delta W = (\mathbf{F} - m\mathbf{a}) \cdot \delta \mathbf{r} = 0$$

(Constraint forces do no virtual work by definition.)

In generalized coordinates:

$$\sum_i \left(\frac{\partial \mathcal{L}}{\partial q_i} - \frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_i} \right) \delta q_i = 0$$

Since the δq_i are arbitrary (independent), each coefficient must vanish, recovering the Euler-Lagrange equations. This approach shows directly that Lagrangian mechanics is a direct consequence of Newton's laws when viewed through the lens of generalized coordinates and constraint-respecting displacements.

11.12 Christoffel Symbols and the Geometry of M

The mass matrix $\mathbf{M}(\mathbf{q})$ defines a Riemannian metric on configuration space [23]. The Christoffel symbols Γ_{jk}^i of this metric are:

$$\Gamma_{jk}^i = \frac{1}{2} g^{im} \left(\frac{\partial g_{mj}}{\partial q_k} + \frac{\partial g_{mk}}{\partial q_j} - \frac{\partial g_{jk}}{\partial q_m} \right)$$

where $g = \mathbf{M}$ and $g^{-1} = \mathbf{M}^{-1}$.

These symbols encode how the metric (and thus the inertial properties) change over configuration space. They appear naturally in the Coriolis matrix:

$$C_{ij} = \frac{1}{2} (\dot{M}_{ij} + \dot{M}_{ik} \dot{M}_{jk}^{-1})$$

or more explicitly in terms of Christoffel symbols:

$$C_{ij} = \Gamma_{ij}^k \dot{q}_k$$

This reveals a deep geometric truth: the Coriolis forces arise from geodesic deviation in the manifold defined by the mass matrix. A system moving "straight" in configuration space, when projected into Cartesian coordinates, experiences apparent Coriolis deflection because the natural metric of the system is curved.

11.13 Worked Example 1: Double Pendulum

11.13.1 System Setup

A double pendulum consists of two rigid rods connected end-to-end, with a pivot at the origin. Link 1 has mass m_1 , length ℓ_1 , and angle q_1 from vertical. Link 2 has mass m_2 , length ℓ_2 , and angle q_2 from vertical.

The center of mass of link 1 is at:

$$\mathbf{r}_1 = \begin{pmatrix} \frac{\ell_1}{2} \sin q_1 \\ -\frac{\ell_1}{2} \cos q_1 \end{pmatrix}$$

The center of mass of link 2 is at:

$$\mathbf{r}_2 = \begin{pmatrix} \ell_1 \sin q_1 + \frac{\ell_2}{2} \sin q_2 \\ -\ell_1 \cos q_1 - \frac{\ell_2}{2} \cos q_2 \end{pmatrix}$$

11.13.2 Kinetic Energy

For link 1, treating it as a point mass at its center:

$$v_1^2 = \dot{\mathbf{r}}_1 \cdot \dot{\mathbf{r}}_1$$

Computing:

$$\dot{\mathbf{r}}_1 = \frac{\ell_1}{2} \begin{pmatrix} \cos q_1 & 0 \\ \sin q_1 & 0 \end{pmatrix} \begin{pmatrix} \dot{q}_1 \\ \dot{q}_2 \end{pmatrix}$$

$$v_1^2 = \frac{\ell_1^2}{4} \dot{q}_1^2$$

For link 2:

$$\dot{\mathbf{r}}_2 = \begin{pmatrix} \ell_1 \cos q_1 \dot{q}_1 + \frac{\ell_2}{2} \cos q_2 \dot{q}_2 \\ \ell_1 \sin q_1 \dot{q}_1 + \frac{\ell_2}{2} \sin q_2 \dot{q}_2 \end{pmatrix}$$

$$v_2^2 = \left(\ell_1 \cos q_1 \dot{q}_1 + \frac{\ell_2}{2} \cos q_2 \dot{q}_2 \right)^2 + \left(\ell_1 \sin q_1 \dot{q}_1 + \frac{\ell_2}{2} \sin q_2 \dot{q}_2 \right)^2$$

$$v_2^2 = \ell_1^2 \dot{q}_1^2 + \frac{\ell_2^2}{4} \dot{q}_2^2 + \ell_1 \ell_2 (\cos q_1 \cos q_2 + \sin q_1 \sin q_2) \dot{q}_1 \dot{q}_2$$

$$v_2^2 = \ell_1^2 \dot{q}_1^2 + \frac{\ell_2^2}{4} \dot{q}_2^2 + \ell_1 \ell_2 \cos(q_1 - q_2) \dot{q}_1 \dot{q}_2$$

The total kinetic energy is:

$$T = \frac{1}{2} m_1 v_1^2 + \frac{1}{2} m_2 v_2^2$$

$$T = \frac{1}{2} m_1 \frac{\ell_1^2}{4} \dot{q}_1^2 + \frac{1}{2} m_2 \left[\ell_1^2 \dot{q}_1^2 + \frac{\ell_2^2}{4} \dot{q}_2^2 + \ell_1 \ell_2 \cos(q_1 - q_2) \dot{q}_1 \dot{q}_2 \right]$$

$$T = \frac{1}{2} \left[\left(\frac{m_1}{4} + m_2 \right) \ell_1^2 \dot{q}_1^2 + \frac{m_2 \ell_2^2}{4} \dot{q}_2^2 + m_2 \ell_1 \ell_2 \cos(q_1 - q_2) \dot{q}_1 \dot{q}_2 \right]$$

In matrix form: $T = \frac{1}{2} \begin{pmatrix} \dot{q}_1 & \dot{q}_2 \end{pmatrix} \mathbf{M} \begin{pmatrix} \dot{q}_1 \\ \dot{q}_2 \end{pmatrix}$

$$\mathbf{M}(\mathbf{q}) = \begin{pmatrix} \left(\frac{m_1}{4} + m_2 \right) \ell_1^2 & \frac{1}{2} m_2 \ell_1 \ell_2 \cos(q_1 - q_2) \\ \frac{1}{2} m_2 \ell_1 \ell_2 \cos(q_1 - q_2) & \frac{m_2 \ell_2^2}{4} \end{pmatrix}$$

Read the entries directly off the quadratic form: the diagonal entries are the coefficients of $\dot{q}_i^2$ inside the bracket, while the off-diagonal entry is *half* the coefficient of the cross term $\dot{q}_1 \dot{q}_2$, because that term appears twice in $\dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}}$.

11.13.3 Potential Energy

Taking the pivot as the reference (zero potential energy at the pivot height):

$$V = -m_1 g \frac{\ell_1}{2} \cos q_1 - m_2 g \left(\ell_1 \cos q_1 + \frac{\ell_2}{2} \cos q_2 \right)$$

$$V = - \left(\frac{m_1}{2} + m_2 \right) g \ell_1 \cos q_1 - \frac{m_2 g \ell_2}{2} \cos q_2$$

11.13.4 Gravity Vector

$$\mathbf{G} = \frac{\partial V}{\partial \mathbf{q}} = \begin{pmatrix} \left(\frac{m_1}{2} + m_2\right) g \ell_1 \sin q_1 \\ \frac{m_2 g \ell_2}{2} \sin q_2 \end{pmatrix}$$

11.13.5 Coriolis/Centripetal Matrix

Computing $\frac{d\mathbf{M}}{dt}$:

$$\frac{dM_{12}}{dt} = -m_2 \ell_1 \ell_2 \sin(q_1 - q_2)(\dot{q}_1 - \dot{q}_2)$$

Using the formula for the Coriolis matrix (after careful computation):

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2} m_2 \ell_1 \ell_2 \sin(q_1 - q_2) \begin{pmatrix} 0 & \dot{q}_2 \\ -\dot{q}_1 & 0 \end{pmatrix}$$

Consistency check. With this $\mathbf{M}$ and this Christoffel $\mathbf{C}$,

$$\dot{\mathbf{M}} - 2\mathbf{C} = \frac{1}{2} m_2 \ell_1 \ell_2 \sin(q_1 - q_2)(\dot{q}_1 + \dot{q}_2) \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix},$$

which is skew-symmetric, as the theorem above requires. Any worked example that fails this check has an algebraic error in $\mathbf{M}$ or $\mathbf{C}$ — it is the cheapest available test of a hand-derived model.

The complete equations of motion are:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}$$

where $\boldsymbol{\tau}$ are the applied torques at each joint.

11.14 Worked Example 2: 2-Link Robot Arm with Gravity

11.14.1 System Configuration

A planar 2-link robot arm with joint angles q_1, q_2 (measured from horizontal). Each link has length ℓ_i and mass m_i uniformly distributed. The arm operates under gravity.

Pose of the end-effector:

$$\mathbf{p} = \begin{pmatrix} \ell_1 \cos q_1 + \ell_2 \cos(q_1 + q_2) \\ \ell_1 \sin q_1 + \ell_2 \sin(q_1 + q_2) \end{pmatrix}$$

Jacobian (mapping joint velocities to end-effector velocities):

$$\mathbf{J} = \begin{pmatrix} -\ell_1 \sin q_1 - \ell_2 \sin(q_1 + q_2) & -\ell_2 \sin(q_1 + q_2) \\ \ell_1 \cos q_1 + \ell_2 \cos(q_1 + q_2) & \ell_2 \cos(q_1 + q_2) \end{pmatrix}$$

11.14.2 Kinetic Energy

For a uniform link i rotating about its proximal joint, the kinetic energy is:

$$T_i = \frac{1}{2}I_i\omega_i^2 + \frac{1}{2}m_iv_{c,i}^2$$

where $I_i = \frac{1}{12}m_i\ell_i^2$ (moment of inertia about the center) and $v_{c,i}$ is the velocity of the center of mass.

For link 1:

$$v_{c,1} = \frac{\ell_1}{2}\dot{q}_1 \implies T_1 = \frac{1}{2}\left(\frac{1}{12}m_1\ell_1^2 + \frac{1}{4}m_1\ell_1^2\right)\dot{q}_1^2 = \frac{1}{6}m_1\ell_1^2\dot{q}_1^2$$

For link 2, it rotates with angular velocity $\dot{q}_1 + \dot{q}_2$ about the global frame:

$$T_2 = \frac{1}{2}I_2(\dot{q}_1 + \dot{q}_2)^2 + \frac{1}{2}m_2v_{c,2}^2$$

where the center-of-mass velocity includes both rotation about joint 1 and rotation about joint 2:

$$v_{c,2}^2 = (\ell_1\dot{q}_1)^2 + \left(\frac{\ell_2}{2}(\dot{q}_1 + \dot{q}_2)\right)^2 + 2\ell_1\frac{\ell_2}{2}\dot{q}_1(\dot{q}_1 + \dot{q}_2)\cos q_2$$

After simplification, the mass matrix is:

$$\mathbf{M} = \begin{pmatrix} m_1\frac{\ell_1^2}{3} + m_2\ell_1^2 + m_2\frac{\ell_2^2}{3} + m_2\ell_1\ell_2\cos q_2 & m_2\frac{\ell_2^2}{6} + m_2\frac{\ell_1\ell_2}{2}\cos q_2 \\ m_2\frac{\ell_2^2}{6} + m_2\frac{\ell_1\ell_2}{2}\cos q_2 & m_2\frac{\ell_2^2}{3} \end{pmatrix}$$

11.14.3 Potential Energy

With gravity acting downward:

$$V = m_1g\frac{\ell_1}{2}\sin q_1 + m_2g\left(\ell_1\sin q_1 + \frac{\ell_2}{2}\sin(q_1 + q_2)\right)$$

Gravity vector:

$$\mathbf{G} = \begin{pmatrix} \left(\frac{m_1}{2} + m_2\right)g\ell_1\cos q_1 + \frac{m_2g\ell_2}{2}\cos(q_1 + q_2) \\ \frac{m_2g\ell_2}{2}\cos(q_1 + q_2) \end{pmatrix}$$

These explicit forms are essential for control design (feedback linearization, trajectory optimization) and analysis (stability, reachability).

11.15 Comparison: Lagrangian vs. Newton-Euler

Both formulations yield identical equations of motion, but traverse different computational paths:

Aspect	Lagrangian	Newton-Euler
Input	Kinetic and potential energy	Link masses, geometry, gravity
Derivation	Calculus of variations	Free-body diagram + recursion
Analytical Form	$\mathbf{M}, \mathbf{C}, \mathbf{G}$ matrices explicit	$\mathbf{M}, \mathbf{C}$ assembled link-by-link
Computational Cost	$O(n^3)$ matrix inversion	$O(n)$ recursive forward/backward pass
Control Design	Elegant (LQR, feedback linearization)	Cumbersome (requires explicit $\mathbf{M}, \mathbf{C}$)
Implementation	By hand for small systems	Physics engines (rigid body simulators)

For a 2-DOF arm, both methods agree perfectly. For a 15-DOF golf swing, the Recursive Newton-Euler algorithm computes torques hundreds of times faster than inverting the full 15×15 mass matrix.

11.16 Symplectic Integrators

Standard ODE solvers such as forward Euler and classical Runge-Kutta (RK4) do not respect the geometric structure of Hamiltonian systems. Over long simulations, they introduce systematic energy drift: trajectories slowly spiral inward or outward in phase space, producing non-physical behavior [1].

Symplectic integrators are numerical methods that exactly preserve the **symplectic form**:

$$\omega = d\mathbf{p} \wedge d\mathbf{q} \quad (11.13)$$

This means that the discrete-time map $(\mathbf{q}_k, \mathbf{p}_k) \mapsto (\mathbf{q}_{k+1}, \mathbf{p}_{k+1})$ is a canonical transformation, preserving phase-space volume and the qualitative structure of orbits.

Symplectic Euler Method. The simplest symplectic integrator updates momenta first, then positions using the new momenta:

$$\mathbf{p}_{k+1} = \mathbf{p}_k - h \frac{\partial H}{\partial \mathbf{q}}(\mathbf{q}_k, \mathbf{p}_k) \quad (11.14)$$

$$\mathbf{q}_{k+1} = \mathbf{q}_k + h \frac{\partial H}{\partial \mathbf{p}}(\mathbf{q}_k, \mathbf{p}_{k+1}) \quad (11.15)$$

This is first-order accurate but already exhibits bounded energy error.

Störmer-Verlet (Leapfrog) Method. A second-order, time-reversible, symplectic scheme:

$$\mathbf{p}_{k+1/2} = \mathbf{p}_k - \frac{h}{2} \frac{\partial H}{\partial \mathbf{q}}(\mathbf{q}_k) \quad (11.16)$$

$$\mathbf{q}_{k+1} = \mathbf{q}_k + h \frac{\partial H}{\partial \mathbf{p}}(\mathbf{p}_{k+1/2}) \quad (11.17)$$

$$\mathbf{p}_{k+1} = \mathbf{p}_{k+1/2} - \frac{h}{2} \frac{\partial H}{\partial \mathbf{q}}(\mathbf{q}_{k+1}) \quad (11.18)$$

Key Property: Bounded Energy Error. Unlike non-symplectic methods where energy error grows linearly (or worse) with time, symplectic integrators produce energy errors that *oscillate* about the true value without secular drift. Formally, the numerical solution exactly solves a *nearby* Hamiltonian $\tilde{H} = H + O(h^p)$, where p is the order of the method. The energy of the original system therefore remains bounded:

$$|H(\mathbf{q}_k, \mathbf{p}_k) - H(\mathbf{q}_0, \mathbf{p}_0)| \leq C h^p \quad \text{for exponentially long times}$$

In Plain Language 11.4. Why Symplectic Integrators Matter for Robotics

For long-horizon robot simulations—such as gait optimization over thousands of steps, or orbital mechanics for space robots—symplectic integrators prevent the slow energy drift that causes non-physical behavior. A standard RK4 solver may show a pendulum gradually gaining energy until it rotates faster and faster; a symplectic solver keeps the energy oscillating near its true value indefinitely.

11.17 Python Worked Example: Double Pendulum Dynamics

```

1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 class DoublePendulum:
6     def __init__(self, m1=1.0, m2=1.0, l1=1.0, l2=1.0, g=9.81):
7         """Initialize double pendulum parameters."""
8         self.m1, self.m2 = m1, m2
9         self.l1, self.l2 = l1, l2
10        self.g = g
11
12    def mass_matrix(self, q):
13        """Compute M(q) from kinetic energy  $T = (1/2) \dot{q}^T M \dot{q}$ """
14        q1, q2 = q[0], q[1]
15        M11 = (self.m1/2 + self.m2) * self.l1**2
16        M12 = self.m2 * self.l1 * self.l2 * np.cos(q1 - q2)
17        M22 = 0.5 * self.m2 * self.l2**2
18        M = np.array([[M11, M12], [M12, M22]])
19        return M
20
21    def coriolis_matrix(self, q, qdot):
22        """Compute C(q, qdot) such that Coriolis forces = C(q, qdot) qdot"""
23        q1, q2 = q[0], q[1]
24        dq1, dq2 = qdot[0], qdot[1]
25
26        # Christoffel symbol terms
27        dM12_dq = -self.m2 * self.l1 * self.l2 * np.sin(q1 - q2)
28
29        C11 = -dM12_dq * dq2 / 2
30        C12 = -dM12_dq * (dq1 + dq2) / 2
31        C21 = dM12_dq * dq1 / 2
32        C22 = 0.0
33
34        C = np.array([[C11, C12], [C21, C22]])
35        return C
36
37    def gravity_vector(self, q):
38        """Compute G(q) = dV/dq"""
39        q1, q2 = q[0], q[1]
40        G1 = (self.m1/2 + self.m2) * self.g * self.l1 * np.sin(q1)
41        G2 = 0.5 * self.m2 * self.g * self.l2 * np.sin(q2)
42        return np.array([G1, G2])
43

```

```

44     def dynamics(self, state, t, tau):
45         """Compute acceleration from  $M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + G(q) = \tau$ 
46         """
47         q = state[:2]
48         qdot = state[2:]
49
50         M = self.mass_matrix(q)
51         C = self.coriolis_matrix(q, qdot)
52         G = self.gravity_vector(q)
53
54         # Solve:  $M \ddot{q} = \tau - C \dot{q} - G$ 
55         qddot = np.linalg.solve(M, tau - C @ qdot - G)
56
57         return np.concatenate([qdot, qddot])
58
59     def simulate(self, q0, qdot0, t_span, tau_func=None):
60         """Simulate the system over time."""
61         if tau_func is None:
62             tau_func = lambda t: np.array([0.0, 0.0])
63
64         state0 = np.concatenate([q0, qdot0])
65         t = np.linspace(t_span[0], t_span[1], 500)
66
67         # Wrapper for odeint
68         def dynamics_wrapper(state, t):
69             tau = tau_func(t)
70             return self.dynamics(state, t, tau)
71
72         solution = odeint(dynamics_wrapper, state0, t)
73         return t, solution
74
75     # Example: Release double pendulum from rest at angle
76     pendulum = DoublePendulum(m1=1.0, m2=1.0, l1=1.0, l2=1.0, g=9.81)
77
78     # Initial conditions:  $q_1 = \pi/4$ ,  $q_2 = 0$ , all velocities zero
79     q0 = np.array([np.pi/4, 0.0])
80     qdot0 = np.array([0.0, 0.0])
81
82     # Simulate for 10 seconds
83     t, traj = pendulum.simulate(q0, qdot0, (0, 10))
84
85     # Extract angles and velocities
86     q1_traj = traj[:, 0]
87     q2_traj = traj[:, 1]
88     dq1_traj = traj[:, 2]
89     dq2_traj = traj[:, 3]
90
91     # Verify energy conservation
92     E = []
93     for i in range(len(t)):
94         q = np.array([q1_traj[i], q2_traj[i]])
95         qdot = np.array([dq1_traj[i], dq2_traj[i]])
96         M = pendulum.mass_matrix(q)
97         T = 0.5 * qdot @ M @ qdot
98         V = -pendulum.mass_matrix(q)[0,0] * pendulum.g * np.cos(q1_traj[i])
99         V -= 0.5 * pendulum.m2 * pendulum.g * pendulum.l2 * np.cos(q2_traj[i])
100         E.append(T + V)
101
102     print(f"Initial Energy: {E[0]:.6f}")
103     print(f"Final Energy: {E[-1]:.6f}")

```

```

103 print(f"Energy Error: {abs(E[-1] - E[0]):.2e}")
104
105 # Plot trajectory
106 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
107 axes[0, 0].plot(t, q1_traj)
108 axes[0, 0].set_ylabel('q1 (rad)')
109 axes[0, 1].plot(t, q2_traj)
110 axes[0, 1].set_ylabel('q2 (rad)')
111 axes[1, 0].plot(t, dq1_traj)
112 axes[1, 0].set_ylabel('dq1/dt (rad/s)')
113 axes[1, 1].plot(t, dq2_traj)
114 axes[1, 1].set_ylabel('dq2/dt (rad/s)')
115 for ax in axes.flat:
116     ax.set_xlabel('Time (s)')
117     ax.grid(True)
118 plt.tight_layout()
119 plt.show()

```

Listing 11.1: Double Pendulum Lagrangian Derivation and Simulation

This code demonstrates:

- Explicit computation of $\mathbf{M}(\mathbf{q})$ from kinetic energy
- Christoffel symbol terms in $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$
- Gravity vector $\mathbf{G}(\mathbf{q})$
- Integration of the second-order ODE via numerical solving
- Verification of energy conservation

11.18 Chapter Summary

Lagrangian mechanics reformulates classical dynamics through energy rather than forces. Key insights:

1. **Hamilton's Principle:** Physical systems extremize the action integral $S = \int \mathcal{L} dt$.
2. **Euler-Lagrange Equations:** $\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial \mathcal{L}}{\partial \mathbf{q}} = \boldsymbol{\tau}$ automatically yields equations of motion.
3. **Standard Form:** $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}$ cleanly separates inertia, Coriolis, and gravity effects.
4. **Constraint Elimination:** Generalized coordinates automatically incorporate holonomic constraints, eliminating unknown constraint forces.
5. **Energy Conservation:** Time-translation symmetry (via Noether's theorem) guarantees $\frac{d(T+V)}{dt} = 0$ when $\boldsymbol{\tau} = 0$.
6. **Hamiltonian Formulation:** A Legendre transform yields symplectic form $\dot{\mathbf{q}} = \frac{\partial H}{\partial \mathbf{p}}$, $\dot{\mathbf{p}} = -\frac{\partial H}{\partial \mathbf{q}}$.

7. **Geometric Insight:** The mass matrix $\mathbf{M}$ defines a Riemannian metric on configuration space. Coriolis forces are geodesic deviations.

In control theory (Volume I), Lagrangian mechanics enables:

- **Feedback Linearization:** Cancel nonlinear terms analytically.
- **LQR and Optimal Control:** Write cost functions on energy and motion directly.
- **Contraction Metrics:** Prove stability using the metric structure.
- **Port-Hamiltonian Systems:** Preserve structure under interconnection.

11.19 Further Reading and References

The foundations of Lagrangian and Hamiltonian mechanics are treated with exceptional depth in Goldstein, Poole, and Safko [11], which remains the standard graduate reference for classical mechanics. Their treatment of variational principles, canonical transformations, and Hamilton-Jacobi theory provides the analytical backbone for everything presented in this chapter. For a more mathematically rigorous and geometrically oriented perspective, Arnold [1] develops classical mechanics from the standpoint of differential geometry and symplectic topology, making explicit the manifold structure that underlies configuration space and phase space.

The geometric mechanics perspective—viewing the mass matrix as a Riemannian metric and Coriolis forces as geodesic deviations—is developed systematically in Marsden and Ratiu [22]. Their treatment of Noether’s theorem, momentum maps, and reduction theory provides the mathematical language for understanding symmetry and conservation in mechanical systems at a level far beyond what we have space for here. Readers interested in the Lie group structure of rigid body mechanics and its interplay with Lagrangian dynamics should consult Murray, Li, and Sastry [23], who unify the geometric and control-theoretic viewpoints.

For computational dynamics and the connection between Lagrangian formulations and recursive algorithms (bridging this chapter with Chapter 10), Featherstone [10] provides an authoritative treatment of how the $\mathbf{M}$, $\mathbf{C}$, and $\mathbf{G}$ matrices can be assembled efficiently using spatial algebra. The control-theoretic applications of the standard form $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{G} = \boldsymbol{\tau}$ —including feedback linearization, computed torque control, and impedance control—are covered extensively by Siciliano et al. [30] and by Spong, Hutchinson, and Vidyasagar [32]. Both texts demonstrate how the properties proved in this chapter (positive-definiteness of $\mathbf{M}$, skew-symmetry of $\dot{\mathbf{M}} - 2\mathbf{C}$) are directly exploited in Lyapunov-based stability proofs for robot controllers.

11.20 Exercises

Exercise. Variational Calculus: Consider the action $\mathcal{S}[q(t)] = \int_0^T (\dot{q}^2 + q^2)dt$ with fixed endpoints $q(0) = 0$, $q(T) = 1$. Show that the extremal path satisfies $\ddot{q} - 2q = 0$ (i.e., apply Euler-Lagrange).

Exercise. Simple Pendulum Lagrangian: For a simple pendulum of mass m and length ℓ , write $T(\theta, \dot{\theta})$ and $V(\theta)$. Apply Euler-Lagrange to recover $\ddot{\theta} + \frac{g}{\ell} \sin \theta = 0$.

Exercise. Cyclic Coordinates: Consider a particle on a rotating table (Coriolis force). Show that angular momentum about the vertical axis is not conserved because the Lagrangian depends on the azimuthal angle explicitly (in the rotating frame).

Exercise. Generalized Momenta: For a 3-link arm in a plane, identify which generalized momenta are conserved under no external torques (hint: check which coordinates are cyclic).

Exercise. Mass Matrix Positive-Definiteness: Prove rigorously that if $T = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}}$ is the kinetic energy of a physical system, then $\mathbf{M}$ is positive-definite.

Exercise. Skew-Symmetry: Verify directly for a 2-DOF system that $\mathbf{M} - 2\mathbf{C}$ is skew-symmetric. Compute the individual entries and check $(\mathbf{M} - 2\mathbf{C}) + (\mathbf{M} - 2\mathbf{C})^T = 0$.

Exercise. Double Pendulum Energy: For the double pendulum (Section 9.2), compute the total mechanical energy as a function of $(q_1, q_2, \dot{q}_1, \dot{q}_2)$. Verify that $dE/dt = 0$ when the system is released from rest.

Exercise. Hamiltonian Transform: Given $\mathcal{L} = \frac{1}{2} m \dot{x}^2 - kx^2$, compute the conjugate momentum $p = \frac{\partial \mathcal{L}}{\partial \dot{x}}$, then form $H = p\dot{x} - \mathcal{L}$. Show that $H = \frac{p^2}{2m} + kx^2$ is the total energy.

Exercise. D'Alembert's Principle: A bead slides on a frictionless parabola $y = ax^2$. Use virtual work to derive the equation of motion in the x direction. (Hint: virtual displacements must satisfy $\delta y = 2ax\delta x$.)

Exercise. Lagrange Multipliers: A particle is constrained to a sphere $x^2 + y^2 + z^2 = R^2$. Write the constraint force as $\mathbf{F}_c = \lambda \nabla(x^2 + y^2 + z^2)$ and derive the equations of motion including λ .

Exercise. 2-Link Robot Arm: For the 2-link robot arm (Section 9.3), compute $\mathbf{M}(q_1, q_2)$ symbolically and show all entries. Then compute the gravity vector $\mathbf{G}(q_1, q_2)$.

Exercise. Friction and Dissipation: Extend the double pendulum Lagrangian to include viscous damping torques $\tau = -b_1 \dot{q}_1 - b_2 \dot{q}_2$. Write the modified equations and verify that mechanical energy dissipates (i.e., $dE/dt < 0$).

Exercise. Christoffel Symbols: For a 2-DOF system with mass matrix $\mathbf{M}(q_1, q_2)$, compute the Christoffel symbols Γ_{jk}^i in terms of partial derivatives of M_{ij} .

Exercise. Noether's Theorem Application: Consider a free-falling object (no constraints, only gravity). Identify the symmetries of the Lagrangian and use Noether's theorem to derive conservation of momentum and energy separately.

Exercise. Pendulum Swing-Up Energy: For a simple pendulum, compute the minimum total energy required to swing from the downward position ($\theta = 0$) to the upright position ($\theta = \pi$). Express in terms of m, ℓ, g .

Exercise. Code Implementation: Implement the code for a 2-link robot arm (similar to the double pendulum code), compute $\mathbf{M}, \mathbf{C}, \mathbf{G}$ explicitly, and simulate a trajectory with constant torque applied to joint 1. Verify energy conservation when no external torques are applied.

Chapter 12

Machine Learning and Neural Networks

We map the geometry of physics using calculus and Lie Groups. We map the geometry of behavior using function approximation and gradient descent.

In Plain Language 12.1. Context & Use Case: Learning Instead of Deriving

The Math You Will See: We introduce **Neural Networks**, **Backpropagation** (automatic differentiation), **Reinforcement Learning**, and **Policy Optimization**. We rigorously derive the backpropagation algorithm from first principles and formalize MDPs (Markov Decision Processes). **Why We Need It:** Analytical physics engines (Euler-Lagrange and Newton-Euler) are flawless for rigid robots made of perfectly measured titanium. But what about a human golfer? We do not know the exact mass distribution of their shirt, the friction of the grass, or the elasticity of their muscle fibers. When equations become too clouded with uncertainty or too wildly nonlinear to solve analytically, we use Machine Learning. We build a massive, flexible mathematical structure (a neural network) and repeatedly bend its shape using calculus until it naturally approximates the correct physics without us having to write them. **All Variables Defined:** W and b are the weights and biases of the network, σ is the nonlinear activation function (like ReLU), $\mathcal{L}$ is the Loss/Error, and $\pi(\mathbf{s})$ is the control policy (neural network).

12.1 Historical Context: From Perceptron to Deep Reinforcement Learning

Machine learning for control has undergone several revolutions. In 1957, **Frank Rosenblatt** introduced the **Perceptron**, a single-layer neural network that could learn to classify linearly separable patterns. The perceptron learning rule was simple: adjust weights to minimize classification error.

For decades, neural networks lay dormant due to the **XOR Problem**: a single-layer perceptron cannot represent the XOR function, and deeper networks were thought untrainable. This changed in 1986 when **David Rumelhart**, **Geoffrey Hinton**, and **Ronald**

Williams published the **backpropagation algorithm**, showing how to efficiently compute gradients through many layers using the chain rule.

Backpropagation rested dormant again until the 2010s, when three factors converged:

1. **GPU Hardware:** Graphics processors enabled training on massive datasets.
2. **Big Data:** ImageNet and similar datasets provided hundreds of millions of labeled examples.
3. **Clever Architectures:** Convolutional neural networks (CNNs) and recurrent networks (RNNs) were engineered to handle specific problem structures.

In 2013, **Deep Reinforcement Learning** emerged when **Volodymyr Mnih and colleagues at DeepMind** combined deep neural networks with Q-learning, training an agent to play Atari games from raw pixel inputs. By 2016, their AlphaGo algorithm defeated Lee Sedol at Go, a game with more possible positions than atoms in the universe. This demonstrated that gradient descent, applied at scale, could discover strategies that rival human intuition.

Today, reinforcement learning powers robot locomotion, manipulation, autonomous driving, and optimal control across diverse domains. The mathematical foundation remains calculus: the chain rule applied iteratively through nested functions.

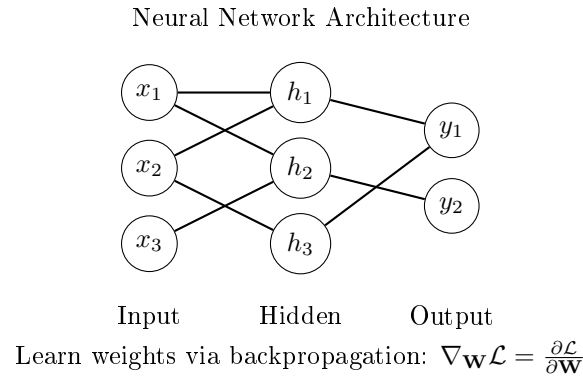


Figure 12.1: A fully-connected neural network with one hidden layer. Information flows from input to output through weighted connections. Backpropagation efficiently computes gradients with respect to all weights, enabling gradient descent optimization.

12.2 The Limits of Analytical Control

By utilizing the Spatial Algebra, Exponential Coordinates, and Recursive Algorithms established in preceding chapters, we can write a perfect mathematical model of a purely rigid robotic system.

But what if the system isn't rigid? What if we are modeling the 15-DOF golf swing of an athlete with deformable muscles, unpredictable wind friction, and complex contact dynamics with the ground? The mathematical model $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \mathbf{G}(\mathbf{q})$

becomes inherently flawed. We do not know the exact friction. We do not know the exact mass of the golfer's shirt.

When analytical equations fail due to extreme nonlinear complexity or uncertainty, we pivot from *deriving* the control laws exactly to *learning* them empirically. This is where neural networks and reinforcement learning enter.

12.3 Universal Approximation Theorem

Theorem (Cybenko, 1989): Let $f : [0, 1]^n \rightarrow \mathbb{R}$ be any continuous function on a compact domain. For any $\epsilon > 0$, there exists a neural network with a single hidden layer containing finitely many neurons with sigmoid activation $\sigma(z) = \frac{1}{1+e^{-z}}$ that approximates f uniformly to within ϵ .

Formal Statement: There exist weights $\mathbf{W}$, $\mathbf{b}$ and biases $\mathbf{c}$ such that:

$$\left| f(\mathbf{x}) - \sum_{i=1}^N c_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i) \right| < \epsilon$$

for all $\mathbf{x} \in [0, 1]^n$.

Significance: This theorem proves that neural networks are *universal function approximators*. Given enough neurons and proper training, a network can learn any smooth function. The theorem does NOT guarantee efficient learning (the required number of neurons may be exponentially large), nor does it provide a training algorithm. But it proves the concept is sound: if we can optimize the weights, we can approximate any control policy.

Extensions:

- Networks with ReLU activation $\sigma(z) = \max(0, z)$ are also universal approximators (with depth requirements).
- Deeper networks can be exponentially more efficient than shallow networks for certain functions (convolutional structure, compositional functions).
- Recurrent networks (RNNs) can approximate dynamical systems.

12.4 Neural Networks as Function Approximators

At its core, a Neural Network [12] is a composition of parameterized linear transformations and element-wise nonlinearities. The simplest form is the feedforward network.

Suppose we want a control policy π that maps state $\mathbf{s} \in \mathbb{R}^{n_s}$ (joint positions, velocities, sensor readings) to an action $\mathbf{a} \in \mathbb{R}^{n_a}$ (motor torques):

$$\mathbf{a} = \pi(\mathbf{s}) = \text{NeuralNetwork}(\mathbf{s}; \boldsymbol{\theta}) \quad (12.1)$$

where $\boldsymbol{\theta}$ denotes all learnable parameters (weights and biases).

12.4.1 Feedforward Architecture

A simple multi-layer perceptron (MLP) with L hidden layers processes the input through successive transformations:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{s} + \mathbf{b}^{(1)} \quad (12.2)$$

$$\mathbf{h}^{(1)} = \sigma(\mathbf{z}^{(1)}) \quad (12.3)$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)} \quad (12.4)$$

$$\mathbf{h}^{(2)} = \sigma(\mathbf{z}^{(2)}) \quad (12.5)$$

$$\vdots \quad (12.6)$$

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)} \quad (12.7)$$

$$\mathbf{a} = \mathbf{W}^{(L+1)}\mathbf{h}^{(L)} + \mathbf{b}^{(L+1)} \quad (12.8)$$

where:

- $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}$ is the weight matrix at layer ℓ
- $\mathbf{b}^{(\ell)} \in \mathbb{R}^{d_\ell}$ is the bias vector
- σ is the element-wise activation function applied to hidden layers (not the output)
- d_ℓ is the width (number of neurons) at layer ℓ

Nonlinearity is Essential: Without activation functions, the entire network collapses to a single linear map: $\mathbf{a} = \mathbf{W}^{(L+1)} \dots \mathbf{W}^{(1)}\mathbf{s} + \mathbf{b} = \mathbf{W}_{\text{eff}}\mathbf{s} + \mathbf{b}$. This is insufficient to approximate nonlinear dynamics. The activation σ introduces necessary nonlinearity and prevents this collapse.

12.5 Activation Functions

Different activation functions possess different properties and computational characteristics.

12.5.1 Rectified Linear Unit (ReLU)

$$\sigma_{\text{ReLU}}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Derivative:

$$\frac{d\sigma_{\text{ReLU}}}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Advantages:

- Computationally cheap (just a comparison and zero)
- Addresses vanishing gradient problem (derivative is 1, not small)
- Induces sparsity in hidden activations (many outputs are exactly zero)
- Proven in deep learning at scale (AlexNet, ResNet, etc.)

Disadvantages:

- Dead neurons: a weight configuration can make $z \leq 0$ always, causing gradient to be permanently zero
- Not differentiable at $z = 0$ (handled via subgradient in practice)

12.5.2 Sigmoid

$$\sigma_{\text{sig}}(z) = \frac{1}{1 + e^{-z}}$$

Derivative:

$$\frac{d\sigma_{\text{sig}}}{dz} = \sigma_{\text{sig}}(z)(1 - \sigma_{\text{sig}}(z))$$

Range: $(0, 1)$ —useful for probability outputs (classification).

Disadvantages:

- Vanishing gradients: derivative approaches 0 for large $|z|$ (gradient < 0.25 everywhere)
- Slow to train deep networks
- Not zero-centered, causing slower convergence in gradients

12.5.3 Hyperbolic Tangent (Tanh)

$$\sigma_{\text{tanh}}(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Derivative:

$$\frac{d\sigma_{\text{tanh}}}{dz} = 1 - \tanh^2(z)$$

Range: $(-1, 1)$ —zero-centered, better than sigmoid for training.

Still suffers from vanishing gradients for deep networks.

12.5.4 Gaussian Error Linear Unit (GELU)

$$\sigma_{\text{GELU}}(z) = z \cdot \Phi(z)$$

where $\Phi(z) = \frac{1}{2} \left[1 + \text{erf} \left(\frac{z}{\sqrt{2}} \right) \right]$ is the cumulative distribution function of the standard normal.

Approximation (often used for efficiency):

$$\sigma_{\text{GELU}}(z) \approx 0.5z \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (z + 0.044715z^3) \right) \right)$$

Advantages:

- Smooth, differentiable everywhere
- Used in modern transformers (BERT, GPT)
- Better gradient flow than sigmoid/tanh

12.6 Loss Functions

Training a neural network requires defining what "correct" means: a loss function $\mathcal{L}(\theta)$ that penalizes incorrect outputs.

12.6.1 Supervised Learning: Mean Squared Error

For control tasks with reference trajectories, the **Mean Squared Error (MSE)** loss is:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\pi(\mathbf{s}_i; \theta) - \mathbf{a}_i^*\|^2$$

where $\mathbf{a}_i^*$ is the desired (expert or reference) action at state $\mathbf{s}_i$, and N is the batch size.

Interpretation: Minimize the squared deviation between predicted and desired actions. Simple, differentiable, but sensitive to outliers.

12.6.2 Robustness: Huber Loss

The **Huber loss** is a hybrid that is quadratic for small errors and linear for large errors:

$$\mathcal{L}_{\text{Huber}}(\delta) = \begin{cases} \frac{1}{2}\delta^2 & \text{if } |\delta| \leq \kappa \\ \kappa(|\delta| - \frac{\kappa}{2}) & \text{if } |\delta| > \kappa \end{cases}$$

where $\delta = \pi(\mathbf{s}) - \mathbf{a}^*$ is the prediction error and κ is a threshold (typically 1.0).

Advantage: Robust to outliers (large errors don't dominate the loss as they do in MSE).

12.6.3 Reinforcement Learning: Policy Gradient and Value Loss

In reinforcement learning, we don't have reference actions $\mathbf{a}^*$. Instead, the agent receives a scalar reward r_t after taking action a_t in state s_t .

The **Policy Gradient Loss** (used in algorithms like REINFORCE and PPO) is:

$$\mathcal{L}_{\text{PG}} = -\mathbb{E}[\log \pi(a_t | s_t) \cdot \hat{R}_t]$$

where $\hat{R}_t$ is an estimate of future discounted rewards (the **return**).

The **Value Loss** (used in actor-critic methods) is:

$$\mathcal{L}_{\text{Value}} = \mathbb{E}[(\hat{R}_t - V(s_t; \phi))^2]$$

where $V(s_t; \phi)$ is a learned value function predicting expected future rewards.

12.7 Backpropagation: Chain Rule for Neural Networks

Backpropagation is the algorithm for computing gradients $\nabla_{\theta} \mathcal{L}$ through a neural network. It is merely systematic application of the chain rule.

12.7.1 Computational Graph Example

Consider a simple 2-layer network:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{s} + \mathbf{b}^{(1)} \quad (12.9)$$

$$\mathbf{h}^{(1)} = \sigma(\mathbf{z}^{(1)}) \quad (12.10)$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)} \quad (12.11)$$

$$\mathcal{L} = \frac{1}{2} \|\mathbf{z}^{(2)} - \mathbf{a}^*\|^2 \quad (12.12)$$

To compute $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}$, we apply the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} \cdot \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \mathbf{z}^{(1)}} \cdot \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

12.7.2 Forward Pass

Compute all intermediate activations:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{s} + \mathbf{b}^{(1)}$$

$$\mathbf{h}^{(1)} = \sigma(\mathbf{z}^{(1)})$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \|\mathbf{z}^{(2)} - \mathbf{a}^*\|^2$$

12.7.3 Backward Pass (Backpropagation)

Compute gradients in reverse order:

Step 1: Loss gradient

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} = \mathbf{z}^{(2)} - \mathbf{a}^* \in \mathbb{R}^{n_a}$$

Step 2: Gradient w.r.t. $\mathbf{W}^{(2)}$ and $\mathbf{b}^{(2)}$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} (\mathbf{h}^{(1)})^T$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}}$$

Step 3: Backprop through activation to hidden layer

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} = (\mathbf{W}^{(2)})^T \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}}$$

Step 4: Backprop through nonlinearity

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(1)}} \odot \sigma'(\mathbf{z}^{(1)})$$

where $\odot$ is element-wise multiplication and $\sigma'(z) = \frac{d\sigma}{dz}$.

Step 5: Gradient w.r.t. $\mathbf{W}^{(1)}$ and $\mathbf{b}^{(1)}$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} \mathbf{s}^T$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}}$$

12.7.4 General Pattern

For any layer ℓ :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} (\mathbf{h}^{(\ell-1)})^T$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(\ell-1)}} = (\mathbf{W}^{(\ell)})^T \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(\ell)}} \odot \sigma'(\mathbf{z}^{(\ell)})$$

The **computational cost** of backprop is roughly twice the forward pass, making gradient computation affordable even for massive networks.

12.8 Gradient Descent and Optimization

12.8.1 Vanilla Stochastic Gradient Descent

Having computed gradients, the simplest update rule is to move opposite to the gradient:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$$

where α is the **learning rate**—how large a step to take. The gradient $\nabla_{\boldsymbol{\theta}} \mathcal{L}$ points in the direction of increasing loss, so moving opposite reduces it.

In practice, we compute the gradient on a small batch of data (not the entire dataset) for efficiency:

$$\mathcal{L}(\boldsymbol{\theta}) \approx \frac{1}{B} \sum_{i=1}^B \mathcal{L}(\boldsymbol{\theta}; \mathbf{s}_i, \mathbf{a}_i)$$

This is **Stochastic Gradient Descent (SGD)**: noisy but fast.

12.8.2 Momentum

Adding momentum accelerates convergence and helps escape shallow local minima:

$$\mathbf{m}_t = \beta \mathbf{m}_{t-1} + (1 - \beta) \nabla_{\boldsymbol{\theta}} \mathcal{L}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \mathbf{m}_t$$

where $\beta \approx 0.9$ is the momentum decay. The momentum term $\mathbf{m}_t$ is an exponentially-weighted average of past gradients.

Intuition: If gradients point consistently in one direction, momentum builds up, accelerating motion. If gradients oscillate, momentum dampens the oscillation.

12.8.3 Adam Optimizer

Adaptive Moment Estimation (Adam) combines momentum with adaptive learning rates per parameter:

$$\begin{aligned}\mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla_{\boldsymbol{\theta}} \mathcal{L} \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla_{\boldsymbol{\theta}} \mathcal{L})^2\end{aligned}$$

(second moment is element-wise squared)

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

(bias correction to account for initialization at zero)

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$$

where division is element-wise and $\epsilon \approx 10^{-8}$ prevents division by zero.

Advantages:

- Adapts learning rate per parameter (small gradients get larger effective steps)
- Robust to learning rate choice (works with $\alpha \approx 0.001$ across many problems)
- Industry standard for deep learning

12.9 Markov Decision Processes (MDPs)

Reinforcement learning operates on **Markov Decision Processes**, a mathematical framework for sequential decision-making under uncertainty.

12.9.1 Formal Definition

An MDP [34] is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: state space (joint angles, velocities, sensor readings)
- $\mathcal{A}$: action space (torques, forces applied by actuators)
- $P(s'|s, a)$: transition probability—likelihood of reaching state s' after taking action a in state s
- $R(s, a, s')$: immediate reward for the transition
- $\gamma \in [0, 1]$: discount factor weighting immediate vs. future rewards

12.9.2 Markov Property

The **Markov property** states that the future depends only on the current state, not the history:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t)$$

This is a simplifying assumption that makes the problem tractable. Partially Observable MDPs (POMDPs) relax this assumption.

12.9.3 Return and Value Function

The **return** from time t is the sum of discounted future rewards:

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The **state value function** $V(s)$ estimates the expected return starting from state s under a policy π :

$$V^\pi(s) = \mathbb{E}[R_t | s_t = s, \pi]$$

The **action-value function** (Q-function) estimates the expected return after taking action a in state s , then following policy π :

$$Q^\pi(s, a) = \mathbb{E}[R_t | s_t = s, a_t = a, \pi]$$

The **advantage function** measures how much better action a is compared to the policy's default:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

12.9.4 Bellman Equation

The fundamental recursion in MDPs is the **Bellman equation**:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

This expresses the value of a state as the immediate reward plus the discounted value of the next state. Solving this recursion (either iteratively or via dynamic programming) yields the optimal policy.

12.10 Policy Gradient Methods

12.10.1 REINFORCE Algorithm

REINFORCE [34] is a basic policy gradient method that learns a policy $\pi_\theta(a|s)$ parametrized by θ (e.g., neural network weights).

The key insight is the **policy gradient theorem**:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) \cdot R_t]$$

where $J(\theta)$ is the expected return and R_t is the actual return from time t onward.

Derivation: The expected return is:

$$J(\theta) = \mathbb{E}_{s \sim \rho, a \sim \pi_\theta} [R_t]$$

Taking the gradient and using the log-derivative trick ($\nabla \log f = \frac{\nabla f}{f}$):

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a|s) \cdot R_t]$$

This gradient can be estimated from sample trajectories:

$$\hat{\nabla}_{\theta} J \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(a_i | s_i) \cdot R_i$$

The **REINFORCE** algorithm:

Algorithm 12.1. REINFORCE

```

for episode = 1 to  $N_{\text{episodes}}$  do
  Initialize trajectory  $\tau = \emptyset$ 
   $s \leftarrow s_0$  (initial state)
  for step = 1 to  $T$  do
    Sample  $a \sim \pi_{\theta}(a|s)$ 
    Observe reward  $r$  and next state  $s'$ 
    Append  $(s, a, r)$  to  $\tau$ 
     $s \leftarrow s'$ 
  end for
  Compute returns:  $R_t = \sum_{k=t}^T \gamma^{k-t} r_k$  for each  $t$ 
  Gradient:  $\hat{g} = \frac{1}{T} \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot R_t$ 
  Update:  $\theta \leftarrow \theta + \alpha \hat{g}$ 
end for

```

Interpretation: The gradient term $\nabla_{\theta} \log \pi$ increases the probability of actions that achieved high returns R_t , and decreases the probability of actions that achieved low returns.

12.10.2 High Variance Problem

The return R_t is a noisy estimate of expected value. If a policy is bad, it takes bad actions, gets low rewards, and the gradients become very noisy. This is the **credit assignment problem**: which actions deserve credit (or blame) for the outcome?

Solutions:

1. **Baseline Subtraction:** Use a learned value function to reduce variance:

$$\hat{\nabla}_{\theta} J \approx \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot (R_t - V(s))]$$

This estimates the **advantage** $A(s, a) = Q(s, a) - V(s)$, which has lower variance than raw returns.

2. **N-step returns:** Use multi-step bootstrapping:

$$R_t^{(n)} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$$

Longer n has more bias but lower variance.

12.11 Actor-Critic Architecture

The **Actor-Critic** method combines policy gradient learning (actor) with value function learning (critic), reducing variance while maintaining policy gradient convergence.

12.11.1 Architecture

Two neural networks:

1. **Actor** $\pi_{\theta}(a|s)$: policy network mapping state to action distribution
2. **Critic** $V_{\phi}(s)$: value network predicting expected returns

12.11.2 Advantage Estimate

The critic produces an estimate of the value, allowing us to compute the advantage:

$$\hat{A}_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

This **temporal difference (TD) error** estimates the advantage on the fly without waiting for an entire episode.

12.11.3 Update Rules

Critic update: Minimize value loss

$$\mathcal{L}_{\text{critic}} = (r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t))^2$$

$$\phi \leftarrow \phi - \alpha_{\phi} \nabla_{\phi} \mathcal{L}_{\text{critic}}$$

Actor update: Maximize expected advantage

$$\mathcal{L}_{\text{actor}} = -\log \pi_{\theta}(a_t|s_t) \cdot \hat{A}_t$$

$$\theta \leftarrow \theta - \alpha_{\theta} \nabla_{\theta} \mathcal{L}_{\text{actor}}$$

12.11.4 Advantage

Actor-Critic has significantly lower variance than REINFORCE because:

- The critic baseline reduces the scale of gradient estimates
- TD estimates use immediate rewards + bootstrap, not full episode returns
- Both networks train jointly, with the critic improving its estimates each step

12.12 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) [28] is a modern policy gradient algorithm that trades off sample efficiency, stability, and ease of implementation.

12.12.1 Problem with Large Policy Updates

A fundamental issue in policy gradient methods: if the policy changes too much in one gradient step, the value function estimates (computed with the old policy) become stale and unreliable. This causes learning instability.

12.12.2 Policy Clipping

PPO addresses this using a clipped policy gradient objective:

$$\mathcal{L}^{\text{clip}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot \hat{A}_t \right) \right]$$

where the **probability ratio** is:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

and $\epsilon \approx 0.2$ is the clipping parameter.

12.12.3 Interpretation

The clipping mechanism prevents the probability ratio from deviating too far from 1:

- If $\hat{A}_t > 0$ (action was good): increase $\pi(a|s)$, but clip the update so $r_t \leq 1 + \epsilon$
- If $\hat{A}_t < 0$ (action was bad): decrease $\pi(a|s)$, but clip so $r_t \geq 1 - \epsilon$

This prevents the policy from collapsing to a bad deterministic action or diverging to zero probability.

12.12.4 Full PPO Objective

The complete PPO loss combines clipped policy gradient, value loss, and entropy regularization:

$$\mathcal{L}^{\text{PPO}}(\theta, \phi) = \mathbb{E} [\mathcal{L}^{\text{clip}} - c_1 \mathcal{L}_{\text{value}} + c_2 \mathcal{H}(\pi_{\theta})]$$

where:

- c_1, c_2 are weight coefficients
- $\mathcal{H}(\pi) = -\mathbb{E}[\log \pi(a|s)]$ is entropy (encourages exploration)

12.13 Model-Based vs. Model-Free RL

Reinforcement learning approaches differ in whether they learn an explicit model of the environment dynamics.

12.13.1 Model-Free RL

The agent learns a policy or value function directly, without learning environment dynamics $P(s'|s, a)$.

Examples: Q-learning, SARSA, Policy Gradient, Actor-Critic, PPO.

Advantages:

- No need to model potentially complex physics
- Can leverage large amounts of real-world interaction data

- Often more data-efficient than model-based methods in practice

Disadvantages:

- Requires many interactions (samples) to converge
- Does not leverage knowledge of physics (Lagrangian mechanics, constraints)
- Hard to transfer to different tasks or environments

12.13.2 Model-Based RL

The agent learns an explicit model $\hat{P}(s'|s, a)$ and uses it for planning.

Examples: Dyna-Q, Model Predictive Control with learned models, planning with learned world models.

Advantages:

- Can plan multiple steps ahead without interacting with environment
- Often more sample-efficient (fewer interactions needed)
- Can exploit physics knowledge (e.g., reward shaping)

Disadvantages:

- Model learning is itself a hard problem
- Errors compound during multi-step planning (“compounding error”)
- Computationally expensive for high-dimensional systems

12.13.3 Hybrid Approaches

Modern approaches combine both:

- Learn a model of dynamics and use it for data augmentation
- Use learned models for short-horizon planning, long-horizon model-free learning
- Incorporate physics priors (learned Lagrangian/Hamiltonian) into the dynamics model

12.14 Physics-Informed Neural Networks (PINNs)

Physics-Informed Neural Networks (PINNs) encode known physics (differential equations, conservation laws) as regularization terms in the loss function.

12.14.1 Framework

Learn a network $u_{\theta}(t, \mathbf{x})$ that satisfies a known PDE:

$$\frac{\partial u}{\partial t} + \mathcal{N}[u] = 0$$

where $\mathcal{N}$ is a nonlinear operator (e.g., $\mathcal{N}[u] = -u \frac{\partial u}{\partial x}$ for Burgers' equation).
The loss combines:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{physics}}$$

- **Data loss:** $\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_i (u_{\theta}(t_i, \mathbf{x}_i) - u_i^{\text{obs}})^2$ (match observations)
- **Physics loss:** $\mathcal{L}_{\text{physics}} = \frac{1}{M} \sum_j \left(\frac{\partial u_{\theta}}{\partial t} + \mathcal{N}[u_{\theta}] \right)^2$ (satisfy PDE)

Derivatives $\frac{\partial u_{\theta}}{\partial t}$ are computed via **automatic differentiation**—backprop applied to the network output.

12.14.2 Application to Robotics

For learned dynamics $\dot{\mathbf{x}} = f_{\theta}(\mathbf{x}, \mathbf{u})$:

$$\mathcal{L}_{\text{physics}} = \frac{1}{M} \sum_j \left(\frac{d\mathbf{x}_j}{dt} - f_{\theta}(\mathbf{x}_j, \mathbf{u}_j) \right)^2$$

(enforces time derivatives match the learned model)
For control with known Lagrangian dynamics:

$$\mathcal{L}_{\text{physics}} = \|\mathbf{M}(\mathbf{x})\ddot{\mathbf{x}} + \mathbf{C}(\mathbf{x}, \dot{\mathbf{x}})\dot{\mathbf{x}} + \mathbf{G}(\mathbf{x}) - \mathbf{u}\|^2$$

Benefit: Encoding known physics greatly reduces data requirements and improves generalization to unseen configurations. This approach directly bridges Chapter 11's Lagrangian equations of motion with data-driven learning: the analytical structure $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{G} = \boldsymbol{\tau}$ derived from energy principles becomes a differentiable constraint that guides the network toward physically consistent predictions.

12.15 Neural ODEs and Adjoint Method

12.15.1 Neural ODE Formulation

Instead of discrete layers, **Neural ODEs** define the hidden state as the solution to a continuous ODE:

$$\frac{dh(t)}{dt} = f_{\theta}(h(t), t)$$

The network output is obtained by solving this ODE from $t = 0$ to $t = T$:

$$h(T) = h(0) + \int_0^T f_{\theta}(h(t), t) dt$$

Solved numerically using an ODE solver (e.g., RK45).

12.15.2 Adjoint Method

Training Neural ODEs requires computing gradients through an ODE solve, which is expensive. The **adjoint method** computes gradients backward in time:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{\theta}} = - \int_T^0 \left(\frac{\partial L}{\partial \mathbf{h}(t)} \right)^T \frac{\partial f_{\boldsymbol{\theta}}}{\partial \boldsymbol{\theta}} dt$$

This avoids storing intermediate activations (memory-efficient) and allows efficient back-prop through ODE solvers.

Application to Control: Dynamics are inherently continuous (governed by Lagrangian or Hamiltonian equations). Neural ODEs naturally parameterize control policies and learned dynamics, enabling efficient optimization.

12.16 Sim-to-Real Transfer and Domain Randomization

Policies trained in simulation—typically using physics engines such as MuJoCo [35]—often fail on real robots due to:

- Unmodeled friction and damping
- Sensor noise and delays
- Actuator nonlinearities and backlash
- Differences in mass, inertia, and geometry

12.16.1 Domain Randomization

Domain Randomization trains the policy on diverse simulated environments with randomized parameters:

```
for training episode do
  Randomize:  $m_i \sim \mathcal{U}(m_i^{\text{nominal}} \pm 30\%)$ 
  Randomize:  $\mu \sim \mathcal{U}(0.5\mu_0, 1.5\mu_0)$  (friction)
  Randomize:  $\mathbf{g} \sim \mathcal{U}(0.5g_0, 1.5g_0)$  (gravity)
  Randomize: sensor noise  $\sigma_{\text{noise}} \sim \mathcal{U}(0, \sigma_{\text{max}})$ 
  Train policy on randomized task
end for
```

If the policy learns to succeed across this distribution of tasks, it becomes robust to a wide range of real-world variation. Domain randomization was popularized by Tobin et al. (2017) for visual transfer and extended by OpenAI's Dactyl project for dexterous manipulation, demonstrating that sufficiently diverse simulation can substitute for real-world data collection.

Intuition: The policy learns invariant strategies rather than exploiting precise nominal parameters. This improves real-world performance.

12.16.2 System Identification

Alternatively, use real-world data to **identify** the actual system parameters:

$$\hat{\theta}_{\text{system}} = \arg \min_{\theta} \sum_i \|f(s_i, \mathbf{u}_i; \theta) - \mathbf{s}_{i+1}^{\text{obs}}\|^2$$

Then retrain the policy with identified parameters.

12.17 Worked Example: Pendulum Swing-Up Policy

12.17.1 Problem Setup

Goal: Swing a simple pendulum from hanging (downward) to upright (upward) while minimizing energy expenditure.

State: $\mathbf{s} = [\theta, \dot{\theta}]$ (angle and angular velocity) Action: $u \in [-1, 1]$ (normalized torque)

Reward:

$$r_t = -(\theta_t^2 + 0.1\dot{\theta}_t^2 + 0.01u_t^2)$$

This rewards reaching $\theta \approx \pi$ (upright), low velocity, and low control effort.

12.17.2 Policy Network

A simple 2-layer network with ReLU activations:

$$\pi_{\theta}(s) = \tanh(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 s + b_1) + b_2)$$

This maps 2D state to 1D action in $[-1, 1]$.

12.17.3 Training with PPO

for training step do

 Collect trajectory: $(s_0, u_0, r_0, s_1, u_1, r_1, \dots)$ from policy

 Compute advantages: $\hat{A}_t = r_t + \gamma V(s_{t+1}) - V(s_t)$

 Update critic: minimize $\sum_t (r_t + \gamma V(s_{t+1}) - V(s_t))^2$

 Update actor: maximize $\sum_t \min(r_t(\theta) \hat{A}_t, \text{clip}(\dots))$

 Every K steps: discard old policy, collect new trajectory

end for

After training, the policy learns to:

1. Swing outward (build kinetic energy)
2. Swing upward, past the horizontal
3. Small corrections near the top to land upright

This is a **learned strategy** that the algorithm discovered from reward signals, not explicit programming.

12.18 Value-Based Methods: DQN and Beyond

While policy gradient methods (such as PPO) directly optimize the policy, an alternative family of algorithms learns the **value** of state-action pairs and derives the policy from these values.

Q-Learning. The classical tabular algorithm maintains a table $Q(s, a)$ and updates it via temporal difference:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (12.13)$$

where α is the learning rate, γ the discount factor, r the reward, and s' the successor state.

Deep Q-Networks (DQN). For high-dimensional state spaces, Q is approximated by a neural network $Q(s, a; \theta)$ [34]. Two key innovations stabilize training:

- **Experience replay:** transitions (s, a, r, s') are stored in a buffer and sampled uniformly for mini-batch updates, breaking temporal correlations.
- **Target network:** a slowly-updated copy $Q(s, a; \theta^-)$ provides stable targets, preventing oscillation.

The loss function is:

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (12.14)$$

Continuous Actions: DDPG. DQN requires a discrete $\max_{a'}$ operation, which is intractable for continuous action spaces (e.g., joint torques). **Deep Deterministic Policy Gradient (DDPG)** resolves this by maintaining a separate **actor** $\mu(s; \theta^\mu)$ that directly outputs the action maximizing Q :

$$\nabla_{\theta^\mu} J = \mathbb{E}_s \left[\nabla_a Q(s, a; \theta^Q) \Big|_{a=\mu(s)} \cdot \nabla_{\theta^\mu} \mu(s; \theta^\mu) \right] \quad (12.15)$$

TD3: Twin Delayed DDPG. DDPG suffers from overestimation bias (the max operation systematically inflates Q -values). TD3 addresses this with:

1. **Twin critics:** two independent Q -networks; the minimum of the two is used as the target.
2. **Delayed policy updates:** the actor is updated less frequently than the critics.
3. **Target policy smoothing:** noise is added to the target action to prevent exploitation of Q -function errors.

SAC: Soft Actor-Critic. SAC incorporates an entropy bonus into the objective, encouraging exploration:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right] \quad (12.16)$$

where $\mathcal{H}(\pi(\cdot | s_t)) = -\mathbb{E}_{a \sim \pi} [\log \pi(a | s_t)]$ is the entropy of the policy. The temperature α can be automatically tuned.

Algorithm	Sample Efficiency	Stability	Action Space
DQN	Moderate	Moderate	Discrete only
DDPG	High	Low	Continuous
TD3	High	Moderate	Continuous
SAC	High	High	Continuous
PPO	Low	High	Both

12.19 Structure-Preserving Neural Networks

Standard neural networks used as dynamics models $\dot{\mathbf{x}} = f_{\theta}(\mathbf{x})$ make no guarantees about conserving physical invariants such as energy or momentum. **Structure-preserving neural networks** embed physical laws directly into the network architecture.

Hamiltonian Neural Networks (HNN). An HNN learns a scalar function $H_{\theta}(\mathbf{q}, \mathbf{p})$ and derives the dynamics via Hamilton's equations:

$$\dot{\mathbf{q}} = \frac{\partial H_{\theta}}{\partial \mathbf{p}}, \quad \dot{\mathbf{p}} = -\frac{\partial H_{\theta}}{\partial \mathbf{q}} \quad (12.17)$$

Because the dynamics are generated by a Hamiltonian, the learned system *automatically conserves energy by construction*—the time derivative $\frac{dH}{dt} = \frac{\partial H}{\partial \mathbf{q}} \dot{\mathbf{q}} + \frac{\partial H}{\partial \mathbf{p}} \dot{\mathbf{p}} = 0$ vanishes identically.

Lagrangian Neural Networks (LNN). An LNN learns a Lagrangian $\mathcal{L}_{\theta}(\mathbf{q}, \dot{\mathbf{q}})$ and derives the equations of motion through the Euler-Lagrange equations via automatic differentiation [12]:

$$\frac{d}{dt} \frac{\partial \mathcal{L}_{\theta}}{\partial \dot{\mathbf{q}}} - \frac{\partial \mathcal{L}_{\theta}}{\partial \mathbf{q}} = \boldsymbol{\tau} \quad (12.18)$$

Expanding the time derivative yields the familiar manipulator equation $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}$, where $\mathbf{M}$, $\mathbf{C}$, and $\mathbf{G}$ are implicitly defined by $\mathcal{L}_{\theta}$.

In Plain Language 12.2. From Analytical to Learned Lagrangians

The Lagrangian structure we derived analytically in Chapter 11 can be *learned from data* when the analytical model is unavailable. This is especially valuable for systems with complex geometry, friction, or flexible components where first-principles modeling is prohibitively difficult.

Deep Lagrangian Networks (DeLaN). DeLaN parameterizes the mass matrix as a neural network $\mathbf{M}_{\theta}(\mathbf{q})$ constrained to be symmetric positive definite via Cholesky decomposition:

$$\mathbf{M}_{\theta}(\mathbf{q}) = \mathbf{L}_{\theta}(\mathbf{q}) \mathbf{L}_{\theta}(\mathbf{q})^{\top} + \epsilon \mathbf{I} \quad (12.19)$$

where $\mathbf{L}_{\theta}$ is a lower-triangular matrix output by the network and $\epsilon > 0$ ensures strict positive definiteness. The potential energy $V_{\theta}(\mathbf{q})$ is learned by a separate network. Together, the Lagrangian is $\mathcal{L} = \frac{1}{2} \dot{\mathbf{q}}^{\top} \mathbf{M}_{\theta}(\mathbf{q}) \dot{\mathbf{q}} - V_{\theta}(\mathbf{q})$, and the equations of motion are obtained by differentiating through the Euler-Lagrange equations using automatic differentiation [34, 12].

12.19.1 Exploration Strategies

A central challenge in reinforcement learning is the **exploration-exploitation tradeoff**: the agent must try new actions to discover better strategies while also exploiting known good actions to accumulate reward.

Epsilon-Greedy. The simplest strategy: with probability ϵ take a uniformly random action, otherwise take the greedy action $a^* = \arg \max_a Q(s, a)$. Typically ϵ is annealed from a high value (e.g., 1.0) to a small value (e.g., 0.01) over the course of training.

Entropy Regularization. Rather than injecting random noise externally, SAC adds an entropy term directly to the objective:

$$J(\pi) = \mathbb{E} \left[\sum_t r_t + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right] \quad (12.20)$$

This encourages the policy to remain stochastic in proportion to α , providing structured exploration that is more sample-efficient than random noise.

Curiosity-Driven Exploration. The agent receives an **intrinsic reward** proportional to the prediction error of a learned forward model:

$$r_t^{\text{intrinsic}} = \|\hat{s}_{t+1} - s_{t+1}\|^2 \quad (12.21)$$

where $\hat{s}_{t+1}$ is the model's prediction of the next state. Novel states produce high prediction error and thus high reward, driving the agent toward unexplored regions of the state space.

In Plain Language 12.3. Why Exploration Is Critical for Robotics

Real robots are expensive to operate and can be damaged by unsafe actions. Sample-efficient exploration is therefore essential: every interaction with the physical world must yield maximum information. Safe exploration—avoiding damage while learning—is an active research area combining reinforcement learning with control-theoretic safety constraints such as control barrier functions and Lyapunov stability.

12.20 Python Worked Example: Backpropagation from Scratch

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class NeuralNetwork:
5     """Simple 2-layer feedforward network with manual backprop."""
6
7     def __init__(self, input_dim, hidden_dim, output_dim):
8         """Initialize network parameters."""
9         # Layer 1: input -> hidden (He initialization for ReLU layers)
10        self.W1 = np.random.randn(hidden_dim, input_dim) * np.sqrt(2.0 /
input_dim)
11        self.b1 = np.zeros((hidden_dim, 1))
12
13        # Layer 2: hidden -> output (Xavier/Glorot for the output layer)
14        self.W2 = np.random.randn(output_dim, hidden_dim) * np.sqrt(1.0 /
hidden_dim)

```

```

15         self.b2 = np.zeros((output_dim, 1))
16
17         # Cache for backprop
18         self.cache = {}
19
20     def relu(self, z):
21         """ReLU activation."""
22         return np.maximum(0, z)
23
24     def relu_derivative(self, z):
25         """Derivative of ReLU."""
26         return (z > 0).astype(float)
27
28     def forward(self, X):
29         """
30         Forward pass.
31         X: input of shape (input_dim, batch_size)
32         Returns: output of shape (output_dim, batch_size)
33         """
34         # Layer 1
35         Z1 = self.W1 @ X + self.b1
36         H1 = self.relu(Z1)
37
38         # Layer 2 (no activation, for regression)
39         Z2 = self.W2 @ H1 + self.b2
40
41         # Cache for backprop
42         self.cache = {
43             'X': X, 'Z1': Z1, 'H1': H1, 'Z2': Z2,
44             'W1': self.W1, 'W2': self.W2
45         }
46
47         return Z2
48
49     def compute_loss(self, predictions, targets):
50         """Mean squared error loss."""
51         m = targets.shape[1]
52         error = predictions - targets
53         loss = (1 / (2*m)) * np.sum(error**2)
54         return loss, error
55
56     def backward(self, dZ2, learning_rate):
57         """
58         Backward pass (backpropagation).
59         dZ2: gradient w.r.t. output layer preactivations
60         """
61         m = self.cache['X'].shape[1]
62
63         # Gradient for layer 2
64         dW2 = (1/m) * dZ2 @ self.cache['H1'].T
65         db2 = (1/m) * np.sum(dZ2, axis=1, keepdims=True)
66
67         # Backprop through H1
68         dH1 = self.W2.T @ dZ2
69
70         # Backprop through ReLU
71         dZ1 = dH1 * self.relu_derivative(self.cache['Z1'])
72
73         # Gradient for layer 1
74         dW1 = (1/m) * dZ1 @ self.cache['X'].T

```

```

75         db1 = (1/m) * np.sum(dZ1, axis=1, keepdims=True)
76
77         # Update parameters
78         self.W2 -= learning_rate * dW2
79         self.b2 -= learning_rate * db2
80         self.W1 -= learning_rate * dW1
81         self.b1 -= learning_rate * db1
82
83     def train(self, X_train, y_train, epochs=100, learning_rate=0.01,
84               batch_size=32):
85         """Train the network."""
86         m = X_train.shape[1]
87         losses = []
88
89         for epoch in range(epochs):
90             epoch_loss = 0
91
92             # Shuffle data
93             indices = np.random.permutation(m)
94             X_shuffled = X_train[:, indices]
95             y_shuffled = y_train[:, indices]
96
97             # Mini-batch training
98             for i in range(0, m, batch_size):
99                 X_batch = X_shuffled[:, i:i+batch_size]
100                 y_batch = y_shuffled[:, i:i+batch_size]
101
102                 # Forward pass
103                 predictions = self.forward(X_batch)
104
105                 # Compute loss
106                 loss, error = self.compute_loss(predictions, y_batch)
107                 epoch_loss += loss
108
109                 # Backward pass
110                 dZ2 = error # gradient of MSE w.r.t. output
111                 self.backward(dZ2, learning_rate)
112
113             losses.append(epoch_loss / (m // batch_size))
114
115             if (epoch + 1) % 20 == 0:
116                 print(f"Epoch {epoch+1}: Loss = {losses[-1]:.6f}")
117
118         return losses
119
120     def predict(self, X):
121         """Make predictions."""
122         return self.forward(X)
123
124 # Example: Regression on y = sin(x)
125 np.random.seed(42)
126
127 # Generate training data
128 X_train = np.random.uniform(-np.pi, np.pi, (1, 100))
129 y_train = np.sin(X_train)
130
131 # Reshape for batch processing
132 X_train = X_train.reshape(1, -1)
133 y_train = y_train.reshape(1, -1)
134

```



```

135 # Create and train network
136 net = NeuralNetwork(input_dim=1, hidden_dim=16, output_dim=1)
137 losses = net.train(X_train, y_train, epochs=200, learning_rate=0.1,
138                   batch_size=16)
139
140 # Test on new data
141 X_test = np.linspace(-np.pi, np.pi, 200).reshape(1, -1)
142 y_test_true = np.sin(X_test)
143 y_test_pred = net.predict(X_test)
144
145 # Plot
146 plt.figure(figsize=(10, 4))
147
148 plt.subplot(1, 2, 1)
149 plt.plot(losses)
150 plt.xlabel('Epoch')
151 plt.ylabel('Loss')
152 plt.title('Training Loss')
153 plt.grid(True)
154
155 plt.subplot(1, 2, 2)
156 plt.plot(X_test[0], y_test_true[0], 'b-', label='True: sin(x)', linewidth=2)
157 plt.plot(X_test[0], y_test_pred[0], 'r--', label='Predicted', linewidth=2)
158 plt.scatter(X_train[0], y_train[0], c='green', s=20, label='Training data')
159 plt.xlabel('x')
160 plt.ylabel('y')
161 plt.legend()
162 plt.grid(True)
163 plt.title('Function Approximation')
164
165 plt.tight_layout()
166 plt.show()
167
168 # Print summary
169 print("\n=== Network Architecture ===")
170 print(f"Layer 1: {net.W1.shape}")
171 print(f"Layer 2: {net.W2.shape}")
172 print(f"\nFinal training loss: {losses[-1]:.6f}")

```

Listing 12.1: Backpropagation Implementation from Scratch

This code demonstrates:

- Forward pass through ReLU network
- Manual computation of gradients via backprop
- Layer-by-layer gradient computation using the chain rule
- Mini-batch gradient descent with weight updates
- Training a network to approximate a nonlinear function

12.21 Deep Reinforcement Learning for Robot Control

Combining PPO (or other policy gradient methods) with deep neural networks enables learning of complex robot behaviors.

12.21.1 State Representation

Typical state vectors include:

- Joint angles $\mathbf{q}$
- Joint velocities $\dot{\mathbf{q}}$
- End-effector position and orientation
- Contact forces/torques
- Goal state or target trajectory

12.21.2 Action Parameterization

For continuous control, actions are typically:

- Desired joint torques $\boldsymbol{\tau}$ (direct)
- Desired joint accelerations $\ddot{\mathbf{q}}$
- Motor commands in $[-1, 1]$ (normalized)
- End-effector velocity targets

12.21.3 Reward Shaping

Design a reward that guides learning toward the desired behavior:

$$r_t = r_{\text{task}} + r_{\text{sparse}} + r_{\text{cost}}$$

where:

- r_{task} : shaped reward for progress toward goal (e.g., negative distance)
- r_{sparse} : binary reward when task is completed
- r_{cost} : penalties for high energy, constraint violations

Example for reaching a target $\mathbf{x}_{\text{goal}}$:

$$r_t = -\|\mathbf{x}_{\text{EE},t} - \mathbf{x}_{\text{goal}}\|^2 - 0.01\|\boldsymbol{\tau}_t\|^2 + 100 \cdot \mathbb{I}[\text{reached}]$$

12.21.4 Training Stability

Deep RL for robot control requires careful tuning:

- **Learning rates:** Too high causes divergence, too low causes slow learning
- **Network size:** Larger networks have more expressive power but overfit easily
- **Exploration:** Must balance exploiting known good actions with exploring new ones
- **Value function updates:** Bad value estimates propagate through policy gradient

Modern algorithms (PPO, SAC, DDPG) address these via:

- Adaptive learning rates (Adam optimizer)
- Clipping/normalization of advantages
- Experience replay buffers
- Entropy regularization

12.22 Chapter Summary

Neural networks and reinforcement learning provide powerful tools for learning control policies when analytical models are unavailable or too complex.

1. **Universal Approximation:** Neural networks with nonlinearities can approximate any continuous function, but scalability depends on architecture and training.
2. **Backpropagation:** Efficient computation of gradients via the chain rule enables training of deep networks on massive datasets.
3. **Gradient Descent:** Simple optimization algorithm that, despite its limitations, works notably well in practice when combined with proper learning rate scheduling (Adam optimizer).
4. **Markov Decision Processes:** Formal framework for sequential decision-making under uncertainty, essential for formulating RL problems rigorously.
5. **Policy Gradient Methods:** Learn policies directly by maximizing expected return, with advantages over value-based methods (stability, continuous actions).
6. **Actor-Critic:** Combines policy learning with value estimation, reducing variance and improving sample efficiency.
7. **Proximal Policy Optimization:** Modern workhorse algorithm that balances stability, sample efficiency, and ease of implementation via policy clipping.
8. **Physics-Informed Learning:** Incorporate known dynamics as loss terms to improve generalization and reduce data requirements.
9. **Sim-to-Real Transfer:** Domain randomization and system identification bridge the gap between simulated training and real robot deployment.

Lagrangian mechanics (Chapter 11) and neural network learning (Chapter 12) are complementary:

- Use **Lagrangian mechanics** when analytical models are available—design explicit control laws with guaranteed properties
- Use **Neural networks** when models are uncertain or too complex—let data guide the policy
- Use **both together:** incorporate Lagrangian constraints as PINN losses, use learned policies to optimize Lagrangian parameters

12.22.1 Connection to Prior Chapters

The machine learning methods presented in this chapter do not exist in isolation—they derive much of their power from the analytical structures developed throughout this volume. Understanding these connections is essential for the modern roboticist, who must fluidly combine model-based and data-driven approaches.

Lagrangian Structure as Physics-Informed Loss (Chapter 11). The standard form $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \boldsymbol{\tau}$ derived in Chapter 11 provides a powerful physics-informed loss function for neural network training. Rather than learning dynamics from scratch, one can penalize violations of the Euler-Lagrange equations, dramatically reducing sample complexity. The energy conservation property ($\frac{d}{dt}(T + V) = 0$ for unforced systems) serves as an additional regularizer, ensuring that learned models respect fundamental thermodynamic constraints.

Differentiable Kinematics via Product of Exponentials (Chapter 9). The Product of Exponentials (PoE) formula expresses forward kinematics as a composition of matrix exponentials, each differentiable with respect to joint angles. This enables end-to-end gradient computation from task-space objectives (e.g., reach a target) through the kinematic chain to joint-space policy parameters. Modern differentiable robotics libraries exploit this structure to train manipulation policies with analytical Jacobians rather than finite-difference approximations.

Differentiable Dynamics from the Articulated Body Algorithm (Chapter 10). The Articulated Body Algorithm (ABA) computes forward dynamics—mapping torques to accelerations—in $O(n)$ time. When implemented in autodifferentiation frameworks (e.g., JAX, PyTorch), ABA provides exact, efficient gradients of accelerations with respect to torques, masses, and inertial parameters. This is the computational engine behind model-based reinforcement learning: the agent can “imagine” trajectories by differentiating through the physics, using the gradient to optimize control inputs directly.

Emerging Frontiers. Several active research directions sit at the intersection of geometric mechanics and deep learning. **Lagrangian Neural Networks** (LNNs) parameterize the Lagrangian $\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})$ as a neural network and recover equations of motion via automatic differentiation of the Euler-Lagrange equations, guaranteeing energy conservation by construction. **Hamiltonian Neural Networks** (HNNs) similarly parameterize the Hamiltonian $H(\mathbf{q}, \mathbf{p})$ and evolve dynamics through Hamilton’s equations, preserving symplectic structure. These architectures represent a principled fusion of the variational principles from Chapter 11 with the function approximation machinery of this chapter, and in several benchmarks they have shown improved accuracy over generic neural network dynamics models on long-horizon prediction tasks [6, 13].

12.23 Further Reading and References

The material in this chapter draws from several foundational texts and research programs. For a comprehensive treatment of deep learning foundations—including network architectures, activation functions, backpropagation, and optimization—Goodfellow, Bengio, and Courville [12] remains the definitive reference. Their treatment of regularization, generalization theory, and practical training methodology provides essential context for anyone applying neural networks to control problems.

The reinforcement learning framework, including Markov Decision Processes, value functions, and policy gradient theory, is developed rigorously in Sutton and Barto [34]. Their

text covers the full spectrum from tabular methods to function approximation, and their presentation of the policy gradient theorem is the mathematical foundation for the REINFORCE algorithm and its descendants.

For modern policy optimization, the reader should consult Schulman et al. [27] for the trust region perspective (TRPO) that motivated the development of PPO, and Schulman et al. [28] for the proximal policy optimization algorithm itself. These papers establish the theoretical basis for stable policy updates that has become the standard in continuous control.

Physics simulation for reinforcement learning training relies heavily on efficient contact dynamics engines. Todorov, Erez, and Tassa [35] introduced MuJoCo, which has become the most widely used simulator for benchmarking RL algorithms in locomotion and manipulation tasks. For the physical principles underlying legged locomotion control, Raibert [25] provides foundational insights on balance, gait generation, and dynamic stability that continue to inform reward shaping and curriculum design in modern RL.

Several rapidly evolving research areas extend the material presented here. **Model-based RL** (Deisenroth et al., PILCO; Chua et al., PETS) learns explicit dynamics models to improve sample efficiency. **Safe RL** (García and Fernández, 2015) constrains exploration to avoid catastrophic failures, critical for real hardware. **Sim-to-real transfer** continues to advance through improved domain randomization, system identification, and adaptive methods. **Inverse RL** (Ng and Russell, 2000; Ziebart et al., 2008) recovers reward functions from expert demonstrations, enabling imitation learning without hand-crafted reward engineering. The reader interested in the intersection of geometric mechanics and learning should explore Lagrangian Neural Networks (Cranmer et al., 2020) and Hamiltonian Neural Networks (Greydanus et al., 2019), which embed the variational structure of Chapter 11 directly into neural network architectures.

12.24 Exercises

Exercise. Universal Approximation Intuition: Show that a 2-layer network with ReLU activations can represent the function $f(x) = |x|$ on $[-1, 1]$. (Hint: use a positive and negative ReLU branch.)

Exercise. Backpropagation Trace: For a 2-layer network computing $\hat{y} = W_2\sigma(W_1x + b_1) + b_2$ with loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$, manually compute all gradients: $\frac{\partial \mathcal{L}}{\partial W_2}$, $\frac{\partial \mathcal{L}}{\partial b_2}$, $\frac{\partial \mathcal{L}}{\partial W_1}$, $\frac{\partial \mathcal{L}}{\partial b_1}$.

Exercise. Activation Function Comparison: Plot sigmoid, tanh, ReLU, and GELU activations and their derivatives. Explain why ReLU and GELU are preferred for deep learning.

Exercise. Vanishing Gradient Problem: Consider a deep network with sigmoid activations. Show that the gradient $\frac{\partial \mathcal{L}}{\partial W^{(1)}}$ contains a product of $(\sigma'(z))$ terms, each < 0.25 . For a 10-layer network, estimate the scale of the gradient.

Exercise. MDP Formulation: Write down a precise MDP formulation for a simple pendulum swing-up task, specifying state space, action space, transition dynamics, and reward function.

Exercise. Bellman Equation: For a simple discrete gridworld (5x5 grid, agent moves up/down/left/right, goal at corner), write the Bellman equation for state value and solve it iteratively (value iteration).

Exercise. Policy Gradient Derivation: Derive the policy gradient theorem from first principles, starting from $J(\theta) = \mathbb{E}[R_0]$ and using log-derivative trick.

Exercise. Advantage Estimation: For a trajectory $(s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$, compute n -step returns $R_t^{(1)}, R_t^{(2)}, R_t^{(3)}$ and compare their bias-variance tradeoff.

Exercise. Actor-Critic Implementation: Implement a simple actor-critic algorithm for the CartPole environment (gym). Train both networks jointly and plot learning curves.

Exercise. PPO Clipping: Plot the PPO clipped policy gradient objective for a range of probability ratios $r \in [0.5, 2.0]$ and advantage values $\hat{A} \in [-2, 2]$. Explain why clipping prevents policy collapse.

Exercise. Physics-Informed Loss: For a pendulum with known Lagrangian dynamics, write the PINN loss combining observed trajectories with the constraint $\ddot{\theta} + \frac{g}{\ell} \sin \theta = 0$.

Exercise. Domain Randomization: Implement training with parameter randomization (mass $\pm 30\%$, friction $\pm 50\%$) and compare final policy performance on nominal parameters vs. randomized policy.

Exercise. System Identification: Given a recorded trajectory from an unknown pendulum, identify the unknown length ℓ and gravity g by fitting a neural network dynamics model to observations.

Exercise. Neural ODE: Implement a Neural ODE for learning pendulum dynamics: $\frac{dh}{dt} = f_\theta(h, t)$. Compare with discrete-layer networks on prediction accuracy.

Exercise. Code Challenge: Train a PPO agent to swing up and balance a simple pendulum. Log returns during training, final policy performance, and sample learned trajectories.

Exercise. Transfer Learning: Train a policy in simulation (known mass), then test on real robot (unknown mass). Compare zero-shot transfer vs. fine-tuning on real robot data.

Glossary of Important Concepts

Articulated Body Algorithm	A recursive $O(N)$ algorithm for computing the forward dynamics of a kinematic chain, widely used in physics engines.
Configuration Space	The space (often a manifold) of all possible positions of a system.
Contraction Analysis	A framework for analyzing the stability of trajectories with respect to each other, rather than with respect to an equilibrium point.
Control Contraction Metric (CCM)	A Riemannian metric that can be used to synthesize stabilizing feedback controllers for nonlinear systems along arbitrary feasible trajectories.
Degrees of Freedom (DOF)	The minimum number of independent coordinates required to completely specify the configuration of a system.
Exponential Map	The operation that relates a Lie Algebra (representing velocity) to its corresponding Lie Group (representing position/orientation).
Euler Angles	A 3-parameter representation of orientation that is subject to gimbal lock.
Flow	The solution map of a differential equation smoothly moving points along vector fields in state space.
Gimbal Lock	A coordinate singularity where a 3D rotation parameterization loses a degree of freedom.
Jacobian	The matrix of partial derivatives relating joint velocities to task-space velocities.
Kinematics	The study of motion geometry without considering the forces that cause it.
Lagrangian Mechanics	An energy-based formulation of mechanics utilizing kinetic and potential energy to strictly determine the equations of motion.
Lie Algebra	The mathematical space of velocities associated with a Lie Group, corresponding to its tangent space at the identity.
Lie Group	A smooth manifold that is also a mathematical group, typically describing continuous symmetries or rigid body transformations (e.g., $SO(3)$, $SE(3)$).
Lyapunov Stability	The classical definition of stability describing whether a system returns to an equilibrium point when perturbed.
Manifold	A topological space that locally resembles Euclidean space, crucial for rigorous formulation of non-trivial state spaces.

State Space	The space of all possible states of a dynamical system, usually comprising both configuration and momentum/velocity.
Screw Axis	A geometric representation of spatial motion combining a rotation axis and a translation along that axis, foundational for modern rigid body mathematics.
Tangent Space	The vector space of all possible velocities at a specific point on a manifold.
Tangent Bundle	The manifold consisting of all tangent spaces glued together, typically representing the full state space of mechanical systems.
Twist	The spatial velocity of a rigid body, composed of angular and linear velocity components, residing in the Lie Algebra.
Wrench	The spatial force acting on a rigid body, combining torque and linear force.

Further Reading and References

This text is a primer and introductory guide designed to construct an intuitive and unified framework. The formal depths of modern geometric mechanics and nonlinear control are actively pioneered and formalized across several seminal texts. For the reader eager to pursue more mathematically rigorous treatments, we strongly recommend the following resources:

Geometric Control and Robotic Manipulation

- **A Mathematical Introduction to Robotic Manipulation** [23]. The definitive textbook bridging Lie group theory with classical robotics, establishing the Product of Exponentials (PoE) standard.
- **Geometric Control of Mechanical Systems** [5]. An authoritative resource on the differential geometry of mechanical control systems, including connections and geodesics.
- **Robot Modeling and Control** [32]. A highly accessible and rigorous text covering everything from basic Euler-Lagrange forms to robust robotic control.

Advanced Rigid Body Dynamics

- **Rigid Body Dynamics Algorithms** [10]. To truly implement physics engines, Featherstone's Spatial Vector notation and the derivation of $O(N)$ ABA are unmatched.

Nonlinear Systems and Contraction

- **Applied Nonlinear Control** [31]. Slotine's classic text providing the most intuitive treatment of sliding mode and robust nonlinear stability.
- **Contraction Theory for Dynamical Systems** [4]. The modern codification of contraction bounding and metrics for trajectory stability.

Index

- 2R arm
 - manipulability, 194
- 7-DOF arm
 - null-space optimization, 194
- A Matrix, 34
- Adjoint, 118
- angular velocity, 88
 - body frame, 88
 - spatial frame, 88
- articulated inertia
 - floating base, 207
- Asymptotic Stability, 37
- Axis-Angle, 140
- axis-angle representation, 89
- B Matrix, 34
- Baker-Campbell-Hausdorff, 140
 - higher-order terms, 130
- Basis, 5
- bilateral constraints, 153
- Center, 37
- centroidal momentum, 208
- Characteristic Equation, 12
- Chasles Theorem, 118
- Chasles' theorem
 - proof, 108
- Cholesky Decomposition, 16
- Cholesky decomposition, 24, 261
- complementarity condition, 216
- Condition Number, 17
- condition number, 23
- Configuration Space, 50
- congruence transformation, 21
- constraint
 - bilateral, 153
- constraint handling, 153
- contact dynamics, 216
- contact forces, 153
- Controllability, 29, 38
- Controllable System, 38
- coordinate frame
 - body frame, 75
 - spatial frame, 75
- coordinate-free formulation, 21
- Coulomb friction, 153, 217
- Cross Product, 10
- curiosity-driven exploration, 262
- DDPG, 260
- Deep Deterministic Policy Gradient, 260
- Deep Lagrangian Networks, 261
- Deep Q-Network, 260
- Degrees of Freedom, 50
- DeLaN, 261
- Determinant, 7
- diffeomorphism, 230
- Dimension, 5
- discrete-time systems, 42
- distribution
 - smooth, 65
- Dot Product, 8
- DQN, 260
- dual space, 106
- duality
 - wrench-twist, 107
- dyad, 19
- dyadic, 20
- Eigenvalue, 11
- Eigenvector, 11
- energy error
 - bounded, 236
- entropy regularization, 262
- epsilon-greedy, 262
- Equilibrium Point, 35
- Euler angles, 79
- experience replay, 260
- exploration strategies, 262
- exploration-exploitation tradeoff, 262
- Exponential Map, 140
- floating-base dynamics, 207
- Focus, 37
- force at offset, 111

- frame invariance
 - power, 107
- friction cone, 153, 217
- Frobenius norm, 22
- Frobenius theorem, 65
- Geodesics, 140
- geodesics
 - Riemannian proof, 132
- gimbal lock, 80
- Hamiltonian Neural Network, 261
- HNN, 261
- humanoid robot
 - dynamics, 207
- inertia tensor, 20
- infinitesimal generator, 230
- Instability, 37
- integrability, 65
- intrinsic reward, 262
- Jacobian Matrix, 36
- kinematic redundancy, 194
- kinetic energy
 - spatial form, 174
- Lagrangian Mechanics, 221
- Lagrangian mechanics
 - RNEA equivalence, 152
- Lagrangian Neural Network, 261
- LCP, *see* linear complementarity problem
- leapfrog integrator, 236
- Lie algebra
 - definition, 81
- Lie bracket, 65
- Lie group
 - definition, 81
- Linear Complementarity Problem, 154
- linear complementarity problem, 216
- Linear Independence, 4
- Linear State Space, 34
- Linear Transformation, 6
- LNN, 261
- LU decomposition, 23
- Lyapunov
 - direct method, 46
- Lyapunov function, 45
- Lyapunov stability, 45
- manipulability, 192
- Manipulability Ellipsoid, 18
- manipulability ellipsoid
 - force, 193
 - velocity, 193
- manipulability measure, 193
- Marginal Stability, 37
- mass matrix
 - column extraction, 153
- Mass-Spring-Damper, 40
- Matrix, 3
- matrix decomposition, 23
- Matrix Exponential, 37, 140
- matrix exponential
 - rotation, 82
- Matrix Logarithm, 140
 - numerical stability, 128
- matrix norm, 22
- maximum entropy, 260
- Mechanical Power, 118
- mechanical power, 106
- Noether charge, 230
- Noether's theorem
 - rigorous proof, 230
- non-holonomic constraint, 65
- Null Space, 7
- null space
 - of Jacobian, 194
- null-space motion, 194
- null-space projector, 194
- Observability, 29, 39
- Observable System, 39
- Orthogonality, 9
- outer product, 19
- Output Equation, 35
- pendulum
 - Lyapunov analysis, 46
- Phase Portrait, 36
- Positive Definite, 15
- power
 - inner product, 107
- Principal Axes, 12
- pure couple, 111
- pure force, 111

- Python
 - quaternion, 89
- Q-learning, 260
- QR decomposition, 23
- Quadratic Form, 14
- quaternion
 - algebra, 83
 - definition, 83
 - double cover, 85
 - rotation representation, 84
 - unit quaternion, 84
- Rank, 6
- Rank-Nullity Theorem, 7
- reachable workspace, 193
- Reciprocal Screws, 118
- redundancy, 194
- Riemannian metric
 - SO(3), 132
- RNEA, 152
 - Lagrangian equivalence, 152
- Rodrigues Formula, 140
- Rodrigues' rotation formula, 82, 83
- rotation
 - history
 - Euler, 75
 - Hamilton, 75
- rotation matrix
 - definition, 76
 - quaternion conversion, 85
 - Rx, 77
 - Ry, 77
 - Rz, 78
- rotation representations
 - comparison, 86
- SAC, 260
- Saddle Point, 37
- screw
 - pitch, 107
- Screw Axis, 96, 118
- SE(3), 118
- se(3) Lie Algebra, 140
- Signorini conditions, 216
- Singular Value, 16
- Singular Value Decomposition, 16
- Singularity, 6
- singularity
 - Jacobian conditioning, 23
- Skew-Symmetric Matrix, 10
- SLERP, 87, 140
 - Python example, 89
- SO(3)
 - definition, 78
- so(3) Lie Algebra, 140
- Soft Actor-Critic, 260
- spatial inertia, 174
 - kinetic energy derivation, 174
 - quadratic form, 174
- spectral norm, 22
- Spectral Theorem, 13
- Spiral, Stable, 37
- Spiral, Unstable, 37
- Stability, 36
 - stability
 - discrete-time, 42
- Stable Node, 37, 41
- Stable Spiral, 40
- State Space, 28
- State Space Representation, 34
- stiffness matrix, 21
- Stormer-Verlet, 236
- stress tensor, 20
- structure-preserving neural networks, 261
- SVD
 - of Jacobian, 193
- symplectic Euler, 236
- symplectic form, 236
- symplectic integrator, 236
- target network, 260
- TD3, 260
- tensor product, 20
- time-stepping methods, 217
- Trajectory, 33
- Transfer Function, 29, 41
- Twist, 118
- unilateral constraints, 153
- Unstable Node, 37
- value-based methods, 260
- Vector, 2
- Vector Space, 3
- virtual joint, 207
- Virtual Work, 118

-
- workspace analysis, 192
 - Wrench, 118
 - geometric object, 106
 - offset force, 107
 - physical interpretation, 106
 - pure couple, 107
 - pure force, 106
 - Yoshikawa manipulability, 193
 - zero-order hold, 42

Bibliography

- [1] Vladimir I. Arnold. *Mathematical Methods of Classical Mechanics*. Springer, 2nd edition, 1989.
- [2] Robert Stawell Ball. *A Treatise on the Theory of Screws*. Cambridge University Press, 1900. Legacy duplicate key retained for backwards compatibility.
- [3] R. W. Brockett. Robotic manipulators and the product of exponentials formula. *Mathematical Theory of Networks and Systems*, pages 120–129, 1984. Legacy duplicate key retained for backwards compatibility.
- [4] F. Bullo. *Contraction Theory for Dynamical Systems*. Kindle Direct Publishing, 2024.
- [5] F. Bullo and A. D. Lewis. *Geometric Control of Mechanical Systems*. Springer, 2004.
- [6] Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, and Shirley Ho. Lagrangian neural networks. *ICLR Workshop on Integration of Deep Neural Models and Differential Equations*, 2020. Neural networks that learn Lagrangian structure for physical systems.
- [7] Jacques Denavit and Richard S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955. Legacy duplicate key retained for backwards compatibility.
- [8] R. Featherstone. The calculation of robot dynamics using articulated-body inertias. *International Journal of Robotics Research*, 2(1):13–30, 1983.
- [9] R. Featherstone. *Robot Dynamics Algorithms*. Kluwer Academic Publishers, 1987. Legacy duplicate key retained for backwards compatibility.
- [10] R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.
- [11] Herbert Goldstein, Charles Poole, and John Safko. *Classical Mechanics*. Addison-Wesley, 3rd edition, 2002.
- [12] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Legacy duplicate key retained for backwards compatibility.
- [13] Sam Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. *NeurIPS*, 2019. Neural networks that learn to conserve Hamiltonian structure.
- [14] Brian C. Hall. *Lie Groups, Lie Algebras, and Representations*. Springer, 2nd edition, 2015. Legacy duplicate key retained for backwards compatibility.
- [15] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [16] H. K. Khalil. *Nonlinear Systems*. Prentice Hall, 3rd edition, 2002.

- [17] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.
- [18] W. Lohmiller and J.-J. E. Slotine. On contraction analysis for non-linear systems. *Automatica*, 34(6):683–696, 1998.
- [19] J. Y. S. Luh, M. W. Walker, and R. P. C. Paul. On-line computational scheme for mechanical manipulators. *Journal of Dynamic Systems, Measurement, and Control*, 102(2):69–76, 1980. Legacy duplicate key retained for backwards compatibility.
- [20] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- [21] I. R. Manchester and J.-J. E. Slotine. Control contraction metrics: Convex and intrinsic criteria for nonlinear feedback design. *IEEE Transactions on Automatic Control*, 62(6):3046–3053, 2017.
- [22] Jerrold E. Marsden and Tudor S. Ratiu. *Introduction to Mechanics and Symmetry*. Springer, 2nd edition, 1999. Legacy duplicate key retained for backwards compatibility.
- [23] R. M. Murray, Z. Li, and S. S. Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
- [24] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006. Legacy duplicate key retained for backwards compatibility.
- [25] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, 1986. Legacy duplicate key retained for backwards compatibility.
- [26] Olinde Rodrigues. Des lois géométriques qui régissent les déplacements d’un système solide. *Journal de Mathématiques Pures et Appliquées*, 5:380–440, 1840. Legacy duplicate key retained for backwards compatibility.
- [27] John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. Trust region policy optimization. In *International Conference on Machine Learning*, pages 1889–1897, 2015. Legacy duplicate key retained for backwards compatibility.
- [28] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. Legacy duplicate key retained for backwards compatibility.
- [29] A. S. Shiriaev, L. B. Freidovich, and S. V. Gusev. Transverse linearization for controlled mechanical systems with several passive degrees of freedom. *IEEE Transactions on Automatic Control*, 55(4):893–906, 2010.
- [30] Bruno Siciliano, Lorenzo Sciacivco, Luigi Villani, and Giuseppe Oriolo. *Robotics: Modelling, Planning and Control*. Springer, 2009. Legacy duplicate key retained for backwards compatibility.
- [31] J.-J. E. Slotine and W. Li. *Applied Nonlinear Control*. Prentice Hall, 1991.
- [32] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, 2006.

- [33] Gilbert Strang. *Linear Algebra and Its Applications*. Wellesley-Cambridge Press, 5th edition, 2019. Legacy duplicate key retained for backwards compatibility.
- [34] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [35] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. Legacy duplicate key retained for backwards compatibility.